



andreyzaostrovnykh
13 авг в 14:05

SPI-контроллер на FPGA: переносим на Zynq 7000. Часть 1

Средний 204 мин 10K

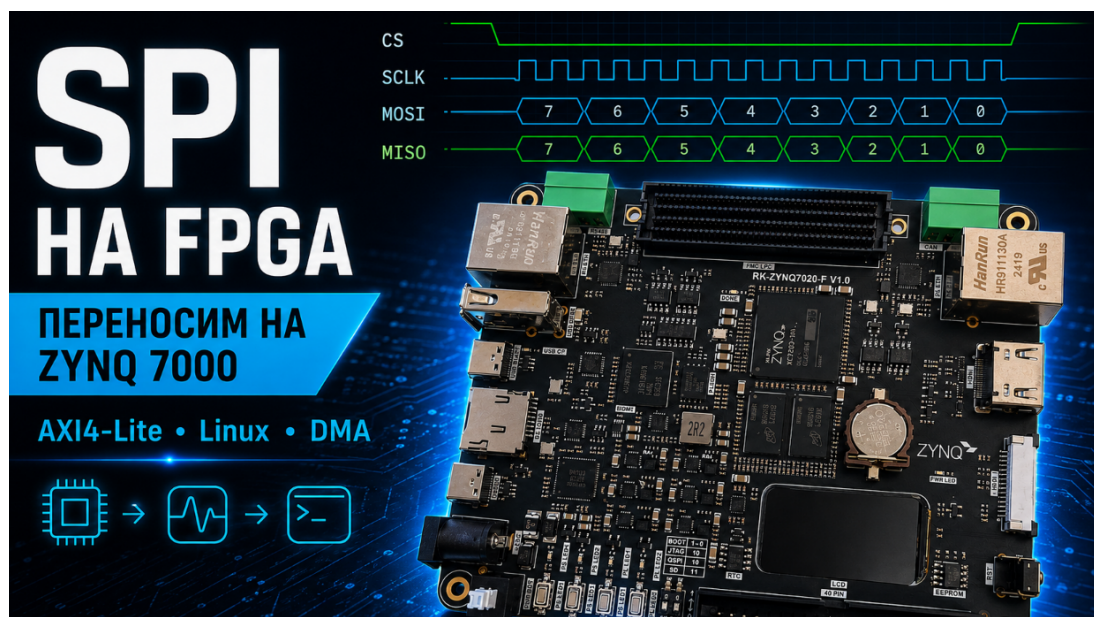
Блог компании Beget, FPGA*, Производство и разработка электроники*, Электроника для начинающих, DIY или Сделай

Тutorial

Продолжаем разработку SPI IP Core на Verilog. Следующий логичный этап - перенести разработанный SPI IP Core на Verilog на Xilinx Zynq-7020, который установлен на плате TZT RK-ZYNQ7020-F rev.1.1. Рядом с программируемой логикой в Zynq живёт ARM-процессор, между ними проложена шина AXI, на процессоре крутится Linux, а на плате есть дисплей, которому можно скормить готовые кадры, с часами на дисплее ST7789, которые рисуются потоком через AXI DMA. Читать статью можно и без Altera-части, всё существенное мы повторим по дороге; но если есть лишний вечер, начните с неё — увидите, откуда взялись решения, за которые нам сейчас придётся платить.

Главная ценность статьи — не описание того, как «должно быть», а разбор того, почему с первого раза не получилось. Каждая нетривиальная ошибка рассказана целиком: что мы видели своими глазами, какая гипотеза казалась логичной, как проверяли, какая улика перевернула картину, где оказалась настоящая причина и какой вывод мы из этого сделали.

Всем, кому интересно продолжение - добро пожаловать под кат! =)



Дисклеймер. Перед началом повествования, хотелось бы заранее оговориться, что основная цель, которую я преследую при написании этой статьи — рассказать о своем опыте. Я не являюсь профессиональным разработчиком под ПЛИС на языке Verilog и могу допускать какие-либо ошибки в использовании терминологии, использовать не самые оптимальные пути решения задач, etc. Но отмечу, что любая конструктивная и аргументированная критика только приветствуется. Что ж, поехали...

Репозиторий

Весь список материалов, исходный код для проектов и подробную документацию вы можете найти по ссылке в репозитории: https://github.com/megaloid/SPI-Master-Controller/tree/master/spi_xilinx

Содержание

Часть 0. Введение (главы 1–4)

Чем эта история вообще интересна, спросит уважаемый читатель. Чем «портировать RTL» отличается от «получить работающее устройство». Коротко напомним, что такое система на кристалле и почему на Zynq ваша логика превращается из самостоятельного устройства в периферийный блок с адресом в карте памяти. Здесь же — плата TZT RK-ZYNQ7020-F, набор инструментов и договорённость о том, как читать разборы ошибок.

1. О чём статья: от SPI на Cyclone IV к Zynq, Linux и дисплею.
2. Что уже было на Altera и что меняется при переходе к SoC.
3. Плата, Vivado, Buildroot, JTAG: карта инструментов.
4. Как читать: теория, лаборатория и кладбище ошибок.

Часть I. Зачем переносить и что такое миграция IP (главы 5–7)

Что такое vendor lock в языке описания аппаратуры, почему поведенческий Verilog переносится, а вендорские мегафункции нет, и как выглядит аудит зависимостей, когда поиск заканчивается пустым результатом — и почему этот пустой результат надо задокументировать.

1. Vendor-neutral RTL против вендорской обвязки.
2. Аудит зависимостей и доказательство эквивалентности.
3. Дерево модулей: что копируем, что добавляем, что выбрасываем.

Часть II. Архитектура ядра SPI (главы 8–11)

Устройство ядра с акцентом на то, что делает его переносимым: единый тактовый домен, SCLK как разрешающий сигнал, а не второй клок, дисциплина сброса и синхронизаторы. Затем — карта регистров и шесть унаследованных дефектов, каждый как отдельная детективная история.

1. Один тактовый домен и SCLK как enable.
2. Модули `spi_reg_if`, FIFO, `spi_engine`, `spi_master_top`.
3. Карта регистров от CONTROL до ERROR_CLR.
4. Дефекты A-1...A-6, в первую очередь предел `CLK_DIV` и мёртвый флаг переполнения.

Часть III. От параллельной шины к AXI4-Lite (главы 12–15)

Почему процессору недостаточно «дёргнуть провод», как работает рукопожатие VALID/READY, как выглядит запись регистра по такту и какие четыре осознанных отклонения от спецификации мы допустили — вместе с описанием цены каждого.

1. Зачем системе на кристалле нужна шина.
2. AXI4-Lite за сорок минут.

3. Обёртка `spi_axi4lite` и отклонения D-1...D-4.
4. Два верхних уровня: автономный selftest и продуктовый Block Design.

Часть IV. Vivado, constraints, распиновка (главы 16–19)

Что переносится из QSF и SDC, а что пишется заново; почему SCLK нельзя объявлять производным тактовым сигналом; почему MISO идёт через два триггера; и самая физическая глава серии — напряжение банка ввода-вывода, которое легко превращает ошибку в дым.

1. Из Quartus в Vivado: QSF и SDC против XDC.
2. Дефекты C-2 и C-3: SCLK и MISO.
3. Распиновка, банк 13 и напряжение VCCO.
4. Преобразователи уровней: почему логика молчит до `ps7_init`.

Часть V. Первый bring-up на железе до Linux (главы 20–22)

Лабораторная работа: как доказать, что SPI жив, не имея ни дисплея, ни операционной системы. Светодиоды, встроенный самотест, петля MOSI–MISO и логический анализатор внутри кристалла. И типичные провалы первого включения, которые почти всегда лежат снаружи ядра.

1. Шаги bring-up как лабораторная работа.
2. Selftest, светодиоды и ILA: доказательства без дисплея.
3. Типичные провалы первых измерений.

Часть VI. Block Design с процессорной системой (главы 23–25)

Откуда берутся адреса и почему в них нет магии; как прерывание из логики доходит до ядра Linux и почему по пути легко потеряться; и три ограничения железа, которые определяют облик драйвера ещё до первой строки на C.

1. Порт GP0, Address Editor и адрес `0x40000000`.
2. `IRQ_F2P`, `xlconcat` и «драйвер не видит прерывание».
3. Ограничения F-1...F-3 и что они запрещают софту.

Глава 1. О чём статья: от SPI на Cyclone IV к Zynq + Linux + дисплей

1.1. Продолжение истории

У этой серии есть предыстория, и без неё половина решений выглядит произвольной. В списке моих статей уже есть материал про то как с нуля был построен SPI master на Altera Cyclone IV — с техническим заданием, резкой устройства на модули, конечным автоматом протокола, охотой за пинами в Quartus и осциллографом в финале, где из абстрактного Verilog получились настоящие пачки импульсов на ножках кристалла. Если вы пришли мимо той серии, достаточно вспомнить, что SPI — простой последовательный интерфейс, которым микросхемы общаются между собой на плате: линия тактов SCLK, линия данных «туда» MOSI, линия «обратно» MISO и линия выбора устройства CS. Наш проект — ведущий (master), та сторона, которая дёргает такты и командует обменом.

Эта статья — следующий акт того же сюжета. Мы не пишем ещё один учебный SPI с нуля: мы берём уже работающее, отлаженное и проверенное тестами ядро и переносим его на систему-на-кристалле Xilinx Zynq-7020. Рядом с программируемой логикой там живёт настоящий ARM-процессор, между ними проложена шина AXI, на процессоре крутится Linux, а на плате есть дисплей, которому можно скормить готовый кадр. Финал истории — часы на дисплее ST7789, которые рисуются потоком через DMA и не рассыпаются на артефакты. Читать серию можно и без Altera-статьи, всё существенное мы повторим по дороге; но если есть лишний вечер, начните с неё — увидите, откуда взялись решения, за которые нам сейчас придётся платить.

1.2. Зачем переносить IP и чем это отличается от «переписать код»

Первый честный вопрос: **а зачем?** Ядро работает на Cyclone IV, тесты зелёные, осциллограф показывает то что нужно — зачем тащить его в чужую экосистему? Причин три, и все встречаются в работе чаще, чем хотелось бы. Кристалл кончился или подорожал: линейку сняли с производства, поставщик поднял цену, заказчик потребовал микросхему из другого списка — проект не переписывают, проект переносят. Задача выросла: пока хватало «голой» логики, всё было хорошо, а как только понадобились сеть, файловая система или человеческий интерфейс, дешевле взять кристалл со встроенным процессором, чем городить внешний микроконтроллер и мост к нему. И третья: вы не автор ядра — чужой IP-блок (IP, intellectual property core, — готовый кусок логики, который переиспользуют как библиотеку) приезжает к вам вместе с задачей «интегрировать до пятницы». Есть и четвёртая, педагогическая, ради которой написан этот текст: перенос — единственная ситуация, в которой видно, что в проекте было настоящей инженерией, а что случайно подошедшей особенностью инструмента. Пока код живёт на одной платформе, вы не отличаете «я так спроектировал» от «Quartus так умеет».

Дальше начинается главное недоразумение новичка. Когда студенту говорят «перенеси проект с Altera на Xilinx», он слышит «перепиши код»: сидишь и меняешь одни конструкции языка на другие, как при переводе с диалекта на диалект. Так вот, кода мы почти не тронем. RTL (register-transfer level — тот уровень описания, на котором вы пишете `always @(posedge clk)` и думаете регистрами и проводами) в нашем ядре оказался практически нейтральным к вендору: на момент переноса из шести файлов четыре переехали байт-в-байт, а в двух добавлен один-единственный атрибут.

А вот устройство при этом не заработало — ещё много недель, потому что «портировать RTL» и «получить работающее устройство» — две разные профессии, и вторая гораздо насыщеннее. Между ними лежит всё то, что обычно называют скучным словом «обвязка»: шина, по которой процессор доберётся до ваших регистров; файл ограничений, где написано, какой сигнал на какой физической ножке; конфигурация процессорной подсистемы; цепочка загрузки от подачи питания до приглашения командной строки; драйвер и описание железа для операционной системы; протокол дисплея; поток данных через DMA. И отдельным сортом боли — кэш самого синтезатора, который однажды честно соберёт вам старую версию логики и не скажет об этом ни слова.

1.3. Что такое SoC

Слово SoC (system on chip, система на кристалле) встретится в серии сотню раз, поэтому договоримся о значении сразу. Обычная ПЛИС — поле из десятков тысяч одинаковых кубиков: таблица истинности плюс триггер, и так тысячи раз. Когда вы пишете Verilog и нажимаете «собрать», инструмент решает, какой кубик что делает и какими проводами их соединить. Никакой программы внутри нет, нет процессора, нет понятия «выполнить инструкцию»: есть только схема, работающая вся сразу, каждый такт. Zynq устроен иначе — на одном кристалле уживаются две принципиально разные половины. Одна — та самая ПЛИС, поле кубиков; в терминологии Xilinx это PL, programmable logic. Вторая — полноценный компьютер, вылитый в кремнии на заводе и неизменяемый: два ядра ARM Cortex-A9, контроллер оперативной памяти DDR, Ethernet, USB, UART, контроллер SD-карты и логика начальной загрузки. Эта половина называется PS, processing system, и на ней запускается Linux — самая обычная операционная система, та же, что на

ноутбуке, только собранная под ARM.

Бытовая аналогия довольно точная: раньше у вас был цех со станками (PL), где вы сами всё расставили и сами всё запускали. Теперь к цеху пристроен офис с менеджером и бухгалтерией (PS). Станки по-прежнему ваши, но у здания появился распорядок дня, пропускная система и человек, решающий, когда цех включается. Новая работа — не в станках, а в коридоре между цехом и офисом. Коридор называется AXI: стандартная шина, по которой процессор обращается к вашей логике так же, как к любой другой периферии, — пишет и читает по адресам. Нужная нам разновидность называется AXI4-Lite: упрощённый вариант без пакетных передач, ровно «прочитай 32 бита по адресу» и «запиши 32 бита по адресу». Ей посвящена часть III.

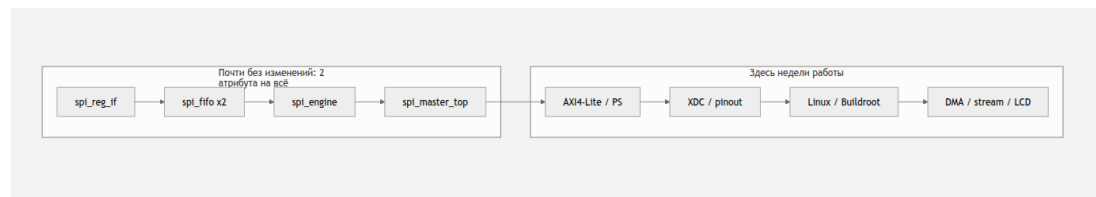
Отсюда следует смена роли разработчика — пожалуй, самая важная мысль главы. На Cyclone IV вы были одновременно всем: процессора не было, значит, писать в регистры приходилось вам самим (сначала руками через отладочную шину, потом крошечным автоматом-секвенсором из ПЗУ), а чтобы понять, что творится внутри кристалла, вы выводили состояние на LCD-панель, потому что больше некуда. Процессор, периферия, отладчик и интерфейс пользователя — всё это были вы. На Zynq процессор уже есть, и он главный: ваша логика перестаёт быть устройством и становится периферийным блоком, таким же, как встроенный UART или таймер. У блока появляется адрес в карте памяти процессора (у нас `0x4000_0000`), линия прерывания и драйвер, который им управляет. Вы больше не дёргаете `wr_en` руками — вы описываете контракт, а дёргаете софт. Звучит как понижение в должности, но это повышение требований: пока вы были единственным хозяином кристалла, неточность в контракте регистров стоила вам вечера отладки, а теперь ваш контракт читает чужой код, написанный другим человеком по документации, и всякая неточность превращается в баг, который ловят на другом конце системы — в драйвере, в приложении, в картинке на экране. Отсюда и вся драма серии: она разворачивается не внутри SPI-движка, а на границе с платформой.

1.4. Сквозной тезис и почему он неочевиден

Теперь можно сформулировать тезис, который держит на себе все десять частей.

RTL ядра SPI почти vendor-neutral. Настоящая работа — AXI, PS, constraints, Linux boot chain, DMA и периферия платы.

В цифрах: четыре модуля — `spi_reg_if`, `spi_fifo` в двух экземплярах, протокольный движок `spi_engine` и интеграционный уровень `spi_master_top` — переехали без единой правки логики, а во всём существующем RTL на момент переноса изменились ровно два атрибута `ASYNC_REG`: по одному на цепочку синхронизатора в `spi_engine.v` и в `spi_reset_sync.v`. Две строки на весь перенос. Много инженерного времени ушло на то, что вокруг: обёртку `spi_axi4lite`, файл ограничений XDC, блок-дизайн процессорной системы, сборку Linux в Buildroot, драйвер, дисплей и потоковую передачу через DMA.

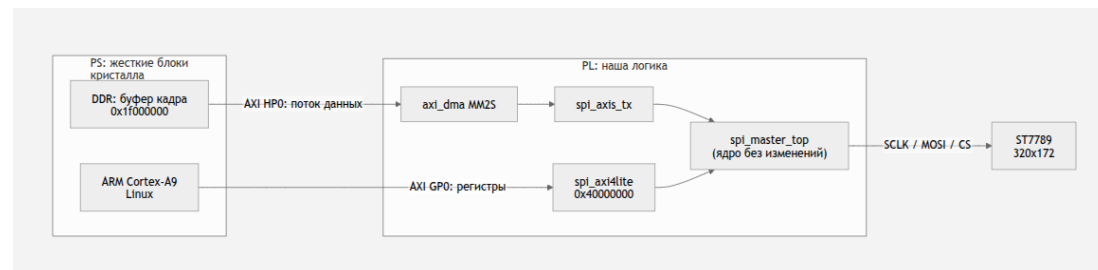


Почему тезис неочевиден? Он противоречит тому, как устроено обучение: в университете вас оценивают по коду, а код видимый и измеримый, его можно показать. Обязка невидима — в ней нет красивых алгоритмов, её нельзя запустить в симуляторе как самостоятельную вещь, и в резюме про неё не напишешь, поэтому начинающий инженер планирует проект по объёму кода и промахивается в сроках втрое. Вторая причина: у обявки нет собственной теории, которую учат

один раз. Она состоит из сотен мелких договорённостей конкретной платформы — этот банк выводов питается отсюда, этот файл инициализации привязан к плате, а не к кристаллу, этот драйвер ищет вот такое имя в описании железа. Каждая по отдельности тривиальна, а нарушение любой даёт мёртвую плату и ноль подсказок.

1.5. Что в итоге получилось на плате

Целевое железо — плата TZT RK-ZYNQ7020-F v1.1 с кристаллом XC7Z020 (корпус c1g484, speed grade -2), инструмент Vivado 2025.2, среда сборки Linux — Buildroot. На этой плате живёт тот же самый SPI master: все четыре режима CPOL/CPHA, оба порядка бит, слова 8/16/24/32 бита, два FIFO, sticky-флаги ошибок и прерывания, без единого изменения в поведении. Но добраться до него теперь можно тремя способами, и каждый — отдельная часть статьи. Из процессора по AXI4-Lite, обычными чтениями и записями по адресам (части III и VI). Из Linux — сначала утилитой `devmem` из командной строки, потом драйвером `spi-zynq-fpga`, который регистрируется в стандартной подсистеме SPI ядра (часть VII). И, наконец, потоком: процессор кладёт в память готовый кадр, говорит контроллеру прямого доступа к памяти «отправь вот это», и данные текут в SPI сами, без участия процессора (часть IX).



Финальный критерий успеха нарочно бытовой: на дисплее идут часы. Не «тест прошёл», не «запас по времени положительный», а цифры, которые видно глазами и которые обновляются без мусора, без двоения и без артефактов. Такой критерий невозможно подделать самообманом: пока картинка кривая, вы не закончили, сколько бы зелёных галочек ни было в тестах.

Кое-чего мы честно не измеряли, и лучше сказать это здесь, а не прятать в сноску. Мы не сравнивали площадь ядра на Cyclone IV и на Zynq «один к одному». Мы не фиксируем итоговое число LUT для полной сборки с DMA — оно зависит от глубины FIFO и от наличия встроенного анализатора, смотрите отчёт своей сборки. И мы не измеряли частоту кадров часов: она заведомо избыточна для задачи.

1.6. Паспорт IP: регистр по смещению 0x3C

Одна маленькая деталь заслуживает отдельного разговора: она объясняет саму логику работы с SoC. По смещению `0x3C` в AXI-обёртке живёт регистр только для чтения, `ID_VERSION`. В классической PIO-сборке он читается как `0x53500100`, в сборке с потоком и DMA — как `0x53500200`. Старшая половина `0x5350` — два ASCII-символа, 'S' и 'P'; младшая — версия в формате «старшая.младшая», `01.00` и `02.00` соответственно.

Зачем логике паспорт? Затем, что ваш блок больше не единственный житель системы, а софт не может посмотреть на него глазами. Драйвер получает из описания железа адрес и должен убедиться, что по этому адресу действительно он, а не сосед, не пустое место и не прошлогодняя версия битстрима. Поэтому первое, что делает драйвер при запуске, — читает `0x3C`. Если старшие 16 бит не равны `0x5350`, он отказывается привязываться к устройству и возвращает ошибку `-ENODEV`: либо в описании железа неверный адрес, либо в кристалл залита не та логика. Совпала старшая версия, отличается младшая — предупреждение в журнале, но не отказ.

Отдельно ценно, что старые битстримы, собранные до появления этого регистра, вернут по адресу `0x3C` ноль, и ноль — тоже ответ: «прошивка старая». Это спасает от потерь времени на дебаг, когда вы отлаживаете софт против логики, которой в кристалле давно нет; в части IX увидите, во что превращается отсутствие такой проверки. Практический вывод, который стоит унести сразу: **первая команда после прошивки — прочитать идентификатор**. Строчка `devmem 0x4000003C 32` экономит часы, потому что отвечает на вопрос «а ту ли логику я вообще отлаживаю».

Глава 2. Что уже было на Altera и что меняется при SoC

2.1. Что умело ядро на Cyclone IV

Начнём с инвентаризации результатов работы прошлой части: перенос — это в первую очередь понимание того, что именно вы держите в руках. К моменту переноса SPI master на плате ALINX с Cyclone IV умел всё, что должен уметь «взрослый» периферийный блок. Он генерировал такты SCLK целочисленным делением системного клока, выдавал данные на MOSI, сэмплировал MISO, опускал и поднимал линию выбора устройства с программируемыми паузами до первого фронта, после последнего и между словами пакета. Он поддерживал все четыре сочетания полярности и фазы такта, оба порядка бит, слова длиной 8, 16, 24 и 32 бита — причём всё это переключалось на лету записью в регистр, а не параметром синтеза. Он буферизовал данные в двух FIFO по восемь слов, вёл sticky-флаги ошибок (флаги, которые взводятся и висят, пока хост их явно не квитирует) и поднимал прерывание по маске.

Внутри каждый модуль отвечал за своё: `spi_reg_if` разговаривал с хостом и не знал ничего про фронты, `spi_fifo` был универсальным складом слов и не знал ни про хост, ни про SPI, `spi_engine` держал в себе весь протокол, а `spi_master_top` только соединял их проводами. Эта дисциплина границ и оказалась главным подарком для переноса, хотя писалась совсем не ради него. Решающим же оказалось другое: в ядре не было ни одного вендорского примитива — ни готового PLL из библиотеки Altera, ни `scfifo`, ни блочной памяти через мегафункцию, ни специальных буферов ввода-вывода. FIFO написан руками как обычный массив регистров, делитель — как обычный счётчик, тактовый домен ровно один. Часть I посвящена аудиту, который это подтвердил, и там же объясняется, почему пустой результат поиска — хорошая новость, а не повод для скуки.

2.2. Параллельная шина: правильное решение и его цена

Теперь самое интересное — интерфейс с хостом. На Cyclone IV ядро управлялось собственной параллельной memory-mapped шиной. Memory-mapped означает буквально следующее: с точки зрения управляющей стороны контроллер выглядит как кусок памяти. Хочешь настроить делитель — «запиши» число по адресу 2, хочешь узнать состояние — «прочитай» адрес 1. Никаких команд и протокола: адрес, данные и строб. Строб здесь ключевое слово. Шина состояла из сигналов `wr_en`, `rd_en`, `addr`, `wr_data` и `rd_data`, и оба разрешающих сигнала были **импульсами ровно на один такт**. Чтение было комбинационным: данные появлялись на `rd_data` в тот же такт, без задержки. А чтение регистра принятых данных имело побочный эффект — слово извлекалось из FIFO насовсем.

Почему тогда это было правильным решением? Потому что вокруг не было системы, к которой стоило бы прибиваться. Взять стандартную шину Altera Avalon-MM значило намертво привязать ядро к экосистеме одного вендора ради удобства, которое в том проекте никому не было нужно: процессора-то не было. Расчёт был другой — интерфейс делаем нарочито простым и нейтральным, а мост на любую системную шину (Avalon, Wishbone, AXI) пишется тонкой прослойкой за вечер, когда понадобится. Ставка оправдалась полностью: цена моста на AXI4-Lite

по итоговому отчёту размещения составила 12 таблиц истинности и 79 триггеров.

Но была и вторая, менее приятная сторона. Но при переходе на Zynq меняется способ доступа к регистрам. В исходном ядре `spi_reg_if` использует простой внутренний memory-mapped интерфейс: `addr`, `wr_data`, `rd_data` и одноканальные стробы `wr_en` и `rd_en`. Сам по себе этот интерфейс не привязан ни к Altera, ни к физическим выводам FPGA и поэтому сохраняется без изменений. На Zynq хостом становится Cortex-A9 в Processing System. Процессор не управляет внутренними сигналами PL напрямую: доступ к пользовательской периферии выполняется через системную шину AXI. Поэтому поверх существующего интерфейса появляется `spi_axi4lite`, который преобразует транзакции AXI4-Lite в те же обращения, которые раньше ожидал `spi_reg_if`. Здесь важно сохранить исходный контракт интерфейса. `wr_en` и `rd_en` должны быть импульсами ровно на один такт, `rd_data` формируется комбинационно, а чтение `RX_DATA` имеет побочный эффект и извлекает слово из RX FIFO. Если AXI-обертка, например, удержит `rd_en` два такта, произойдут два рор из FIFO вместо одного и одно принятое слово будет потеряно. Отдельно при аудите старого Quartus-проекта обнаружился дефект A-6: в одной из промежуточных ревизий top-level 65 портам был задан `IO_STANDARD`, но не был задан `set_location_assignment`. Эта ревизия не предназначалась для реальной аппаратной сборки, поэтому дефект не являлся причиной перехода на AXI, но показал, что незавершенные top-level и constraints нельзя принимать за валидную распиновку платы.

2.3. Наблюдаемость: зачем ядру был нужен экран

Осталась третья часть наследства, о которой обычно забывают, — способность видеть, что происходит. Кристалл непрозрачен: внутри работающей ПЛИС нет ни печати в консоль, ни отладчика с точками останова, есть только те сигналы, которые вы сами вывели наружу. На Altera проблему решал LCD-дашборд: панель 480×272 с параллельным RGB-интерфейсом, на которую выводились состояние контроллера, шестнадцатеричные значения регистров и живые координаты касания. Рядом жил автомат `demo_init`, проигрывавший начальную конфигурацию из ПЗУ вместо процессора, и модуль `spi_probe`, самостоятельно опрашивавший контроллер сенсора ХРТ2046 — настоящее внешнее SPI-устройство со своим даташитом и своим характером. Это была не декорация, а приборный щиток; без него отладка свелась бы к угадыванию. Запомните эту роль — сейчас её придётся восстанавливать другими средствами.

2.4. Три кучки: байт-в-байт, обёртка и то, что остаётся

Разложим наследство на три кучки. В первую попадает почти всё ядро: файл описаний `spi_defs.vh` с картой регистров и битовыми полями, буфер `spi_fifo`, регистровый интерфейс `spi_reg_if` и интеграционный уровень `spi_master_top` перенесены байт-в-байт. Это не фигура речи, а проверяемое утверждение: совпадение контрольных сумм проверяет отдельный скрипт. Движок `spi_engine` и синхронизатор сброса `spi_reset_sync` отличаются ровно одним атрибутом каждый.

Одна оговорка, чтобы вы не поймали статью на противоречии, сверив суммы самостоятельно. Всё сказанное описывает состояние на момент переноса. Позже, уже в части IX, потоковый режим и DMA всё-таки заставили нас изменить `spi_fifo`, `spi_engine` и `spi_master_top`, поэтому сегодня совпадают только `spi_defs.vh` и `spi_reg_if`. Это не провал доказательства, а его работа: скрипт сравнения честно показывает, где мы отошли от оригинала и по какой причине. Регистровый интерфейс мы при этом не тронули сознательно — на его неизменности держится весь аргумент об эквивалентности.

Про атрибут стоит сказать пару слов — он единственный представитель категории «пришлось изменить». `ASYNC_REG` — подсказка синтезатору Vivado: «эти два триггера образуют цепочку синхронизатора, не растаскивай их по разным углам кристалла, не переставляй местами и не превращай в сдвиговый регистр в памяти». Синхронизатор нужен там, где в вашу логику приходит сигнал, не связанный с вашим тактовым сигналом, — у нас это входная линия `MISO` и кнопка

сброса. Такой сигнал может измениться ровно в момент защёлкивания, и триггер окажется в метастабильном состоянии: не ноль и не единица, а неопределённость, рассасывающаяся за непредсказуемое время. Два последовательных триггера дают ей время рассосаться, но только если стоят рядом. Без атрибута инструмент об их особой роли не догадывается: видит обычную пару регистров и вправе оптимизировать её как угодно. Вот и весь перенос ядра — две строки.

Вторая кучка — то, что остаётся внутри, но получает новую упаковку. Хост-интерфейс никуда не делся: `spi_reg_if` по-прежнему ждёт свои одноктактовые стробы, четырёхбитный адрес и 32 бита данных, просто теперь их формирует не человек и не автомат из ПЗУ, а новый модуль `spi_axi4lite` — переводчик между шиной AXI4-Lite и внутренним контрактом ядра. Ключевое слово здесь «над»: обёртка добавлена **поверх** неизменённого `spi_master_top`, а не вместо него, и внутренний интерфейс сохранён до последнего провода — именно поэтому мы вправе говорить, что ядро то же самое. Обёртка не совсем тривиальна: в ней крошечный автомат, обслуживающий одну транзакцию за раз, чтения и записи не перекрываются. Это не лень, а способ сделать свойство «ровно один строб на одно обращение» доказуемым по построению. Она же приносит четыре осознанных отклонения от учебного AXI4-Lite, пронумерованные D-1...D-4: игнорируется побайтовое разрешение записи, чтение и запись не перекрываются, все ответы всегда «успешно», а окно адресов в 64 байта зеркалируется, если отобразить его на больший диапазон. Каждое обосновано контрактом регистров и проверяется тестами. Сюда же попадает файл ограничений: временные требования пришлось не переносить, а переосмыслить — например, объявление `SCLK` производным тактовым сигналом в XDC не переехало, оно семантически неверно, потому что коэффициент деления программируется во время работы, а сам `SCLK` не тактирует ни одного триггера. Подробности — в части IV.

Третья кучка. В перенос **не вошли** подсистема `rtl/lcd/*` из девяти модулей, автомат начальной инициализации `demo_init`, опросчик `spi_probe` и топ-уровень под панель 480×272. Причина предельно понятная — на целевой плате нет ни параллельной RGB-панели 480×272, ни сенсорного контроллера XPT2046, к которому `spi_probe` привязан жёстко. Переносить драйвер несуществующей панели так же не требовалось, на плате есть свой дисплей и устройство SPI для экспериментов.

А вот функцию этой подсистемы выбрасывать нельзя. Наблюдаемость восстановлена в три приёма по нарастающей: сначала внутренний мастер `spi_selftest`, который сам конфигурирует ядро и гоняет транзакции сразу после подачи питания, плюс два светодиода (один мигает от обычного счётчика и доказывает, что такт живой и сброс снят, второй загорается, когда на `MISO` пришло что-то осмысленное); затем ILA — логический анализатор, синтезируемый прямо внутри кристалла и показывающий внутренние сигналы через JTAG; и наконец, в частях VIII–IX, дисплей ST7789 по SPI, та самая приборная панель, только на другом железе и другом протоколе. Правило отсюда: переносится не подсистема, а её роль. Ответ на вопрос «а как я увижу, что оно работает?» обязан существовать на каждом этапе, и придумывать его надо заранее, а не когда прижмёт.

2.5. Сначала перенос, потом bugfix

Осталось последнее правило переноса, неочевидное настолько, что его приходится обосновывать. При аудите исходного ядра нашлись шесть дефектов, помеченных A-1...A-6. Два проявляются только при экзотических значениях параметров, один непринципиальный (запись нуля в делитель молча игнорируется, и хост об этом не узнаёт), один касается неполной распиновки. А два действительно неприятные. A-3: при делителе, равном единице, приём портится — слово сдвинуто на бит, отправленный `0x3C` читается как `0x1E`, причём передача остаётся корректной. A-5: бит 9 регистра состояния, который по документации сообщает о переполнении приёмного FIFO, не выставляется никогда ни при каких условиях.

Ни один из шести дефектов **не исправлен**, и это сознательная инженерная позиция, а не халатность. Задача переноса — получить версию, про которую можно доказуемо сказать «она

ведёт себя точно так же»: совпадением контрольных сумм, прогоном одних и тех же тестов, теми же осциллограммами. Как только вы одновременно с переносом «заодно» чините баг, это утверждение рассыпается: если новая версия ведёт себя иначе, вы уже не отличите «мы правильно исправили баг» от «мы что-то сломали при переносе». Два наложенных друг на друга изменения невозможно развести по последствиям — а именно умение разводить причины и следствия отличает инженерию от шаманства.

Поэтому дефекты не чинятся молча, а фиксируются, причём в самом жёстком виде: тест в симуляции **утверждает дефектное значение как ожидаемое**. Тест на A-3 требует, чтобы при делителе, равном единице, приём давал именно `0x1E`. Выглядит дико, пока не поймёшь замысел: если кто-то завтра изменит глубину синхронизатора или момент сэмплирования, тест упадёт и потребует объяснений вместо того, чтобы молча дать PASS, пока поведение железа меняется незаметно для всех.

Практическое следствие: там, где документация исходного проекта говорит «делитель не меньше двух рекомендуется», правильное чтение — «не меньше двух обязательно». Это не совет по улучшению запаса, а граница корректности, и из неё прямо следует максимальная частота SPI: 8.33 МГц при системном такте 50 МГц, а не 12.5 и не 25. Разбор механизма — в части II.

2.6. Сводка: слой за слоем

Всё сказанное удобно держать перед глазами картинкой и таблицей. Картинка — про смену роли из главы 1.



Таблица ниже — справочная карточка, к которой стоит возвращаться из следующих частей.

Слой	Altera-проект	Zynq-проект	Что это означает
Протокол SPI	<code>spi_engine</code>	тот же файл, +ASYNC_REG	движок не переписываем вообще
Регистры и FIFO	<code>spi_reg_if</code> , <code>spi_fifo</code>	байт-идентичны на момент переноса	семантика регистров сохранена
Хост-интерфейс	параллельная шина на выводах	<code>spi_axi4lite</code>	главная архитектурная добавка
Топ-уровень	<code>spi_master_fpga_top</code> , LCD-топ	<code>spi_zynq_top</code> либо PS Block Design	отладочная шина ликвидирована
Ограничения	QSF + SDC	XDC	распиновка пишется с нуля
Софт	нет ОС, «голый» хост	FSBL, U-Boot, Linux, драйвер	появляется цепочка загрузки
Наблюдаемость	RGB LCD + <code>spi_probe</code>	selftest, LED, ILA, затем ST7789	роль та же, средства другие

Поток данных	только PIO через регистры	PIO и AXI DMA через AXI-Stream	новый класс задач и новый класс ошибок
--------------	---------------------------	--------------------------------	--

Короткий вывод главы: ядро пережило переезд практически без потерь, потому что раньше кто-то (то есть я) отказался от удобных вендорских мегафункций и написал FIFO руками. Всё, что было привязано к конкретной плате — шина на выводах, панель, опросчик сенсора, — переезд не пережило, и это нормально: переносится логика, а не окружение.

Глава 3. Плата TZT RK-ZYNQ7020-F, Vivado, Buildroot, JTAG

3.1. Паспорт железа и инструментов

Прежде чем говорить про архитектуру, зафиксируем, на чём всё это работает.

Параметр	Значение
Плата	TZT RK-ZYNQ7020-F v1.1
SoC	Xilinx Zynq-7000, XC7Z020, корпус c1g484 , speed grade -2
Такт программируемой логики	50 МГц, осциллятор на выводе W17 (вход MRCC)
Такт от процессорной системы	FCLK_CLK0 ; 50 МГц в PL-варианте, чаще 100 МГц в сборке с DMA
Инструмент синтеза	Vivado 2025.2
Сборка Linux	Buildroot (ядро, U-Boot и корневая ФС собираются из исходников)
Отладка логики	JTAG: Hardware Manager, XSCT, встроенный анализатор ILA
Дисплей на плате	IPS-модуль на ST7789V, физически 172×320, интерфейс SPI
Исходная платформа-референс	Altera Cyclone IV E EP4CE6F17C8, плата ALINX, Quartus

Пара пояснений к тому, что здесь не очевидно. Speed grade -2 — «средний по скорости» кристалл, и наших 50 МГц ему хватает с огромным запасом: полная сборка показала запас по установке около 14.6 нс при периоде 20 нс, а предельная частота логики оказалась 258 МГц. Наиболее уязвимое место в этом проекте — Buildroot — система сборки, которая по одному конфигурационному файлу собирает из исходников всё сразу: компилятор для целевой архитектуры, загрузчик U-Boot, ядро Linux, минимальную корневую файловую систему, а на выходе выдаёт образ SD-карты. Альтернатива — взять чужой готовый образ, но тогда вы не управляете ни версией ядра, ни набором драйверов, а именно они нам и понадобятся. Цена — время: первая сборка занимает десятки минут.

3.2. PS и PL: два мира и граница между ними

Вернёмся к архитектуре Zynq подробнее — от неё зависит понимание всех последующих ошибок.

PS (processing system) — готовая, вылитая в кремнии микропроцессорная система: два ядра Cortex-A9, кэши, контроллер DDR-памяти, контроллер прерываний, Ethernet, USB, SD, UART и десятки других блоков. Изменить в ней нельзя ничего, можно только сконфигурировать: какие выводы MIO кому отданы, на какой частоте работает память, какие такты уходят в логику. PL (programmable logic) — обычная ПЛИС семейства 7-series, в нашем случае с 53 200 таблицами истинности и 106 400 триггерами; SPI-ядро с AXI-обёрткой занимает около 405 и 406 соответственно, то есть меньше процента.

Между половинами проложено несколько мостов, и они не равноправны. Порты общего назначения GP — для управления: процессор обращается через них к регистрам блоков в логике. Порты высокой производительности HP — для данных: через них блок в логике сам ходит в оперативную память, минуя процессор. Прерывания идут в обратную сторону по шине IRQ_F2P, «fabric to processor». И наконец, процессорная система раздаёт в логику тактовые сигналы FCLK_CLK0...3. В финальном дизайне задействованы почти все мосты сразу: регистры SPI и GPIO на GP0, контроллер DMA читает кадр из памяти через HP0, прерывание уходит в IRQ_F2P, тактируется всё от FCLK_CLK0. Поэтому финальная часть серии и самая насыщенная ошибками: там впервые работают одновременно все каналы связи.

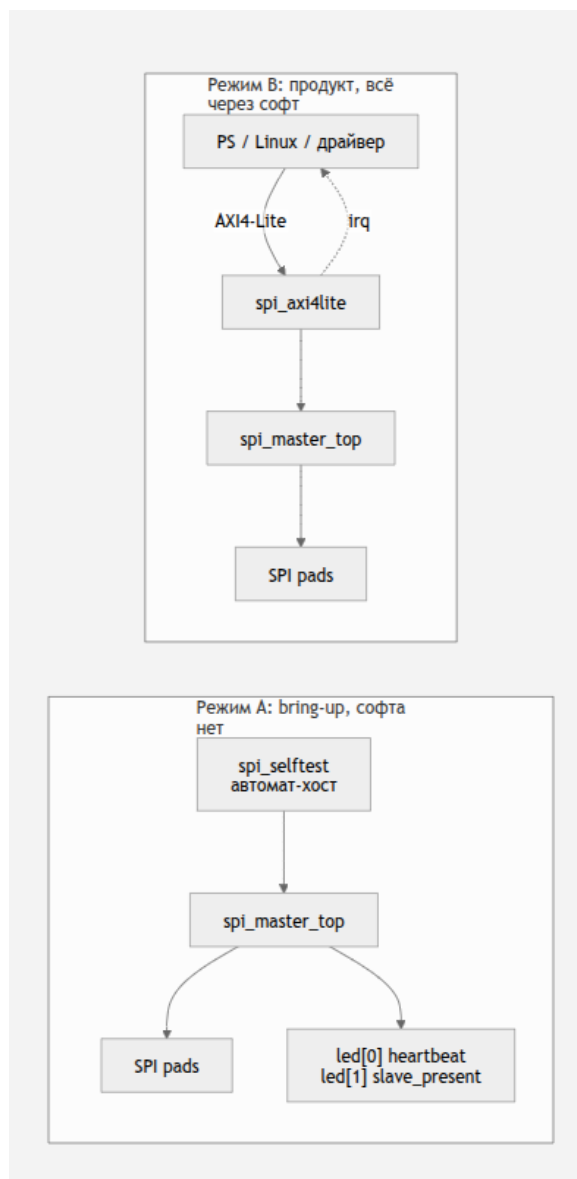
Ошибки на этой границе обидны по одной причине: обе стороны выглядят исправными. Возьмите сценарий из части VII. Ethernet на плате поднимается, `ethtool` радостно сообщает `Link detected: yes`, интерфейс в системе есть — а пинг не идёт и счётчик ошибок приёма растёт. Логика подсказывает искать в сети: кабель, настройки, адрес. На самом деле описание железа для Linux досталось от чужой платы и перепрограммирует электрический стандарт выводов процессорной системы поверх того, что уже правильно настроил загрузчик: служебный канал управления физическим уровнем продолжает работать, отсюда «Link Up», а канал данных превращается в мусор. Или сценарий из части IX: вы правите файл `.v`, собираете новый битстрим, заливаете и получаете ровно то же поведение, что до правки, при зелёной симуляции нового кода.

Общее у этих историй одно: сигнал об ошибке приходит очень далеко от места ошибки. Логика не знает, что процессор перепрограммировал выводы; софт не знает, что в кристалле старая логика; ни один инструмент не жалуется, потому что каждый по-своему прав. Единственная защита — привычка не доверять косвенным признакам и проверять контракт напрямую: прочитать идентификатор, сравнить время изменения файлов, посмотреть реальный электрический стандарт вывода, а не тот, который вы имели в виду.

3.3. Два режима жизни одного IP

Проект существует в двух ипостасях, и для новичка два разных верхних уровня выглядят избыточностью — разберём, почему это не так. Первый режим — автономный, без процессора: верхний уровень `spi_zynq_top`, и внутри него, кроме ядра, живёт `spi_selftest` — маленький аппаратный автомат, который сам пишет в регистры по той самой параллельной шине, настраивает делитель и длину слова, включает контроллер и непрерывно гоняет транзакции. Linux не нужен, софта нет никакого: загрузили битстрим — на ножках сразу появилась жизнь. Индикация двумя светодиодами, диагностика через ILA.

Второй режим — продуктовый. Здесь написанного руками верхнего уровня нет вовсе: есть блок-дизайн, схема из готовых блоков, где `spi_axi4lite` инициализируется как обычный модуль рядом с процессорной системой, контроллером DMA и блоком GPIO. Адрес регистров `0x4000_0000`, прерывание заведено в `IRQ_F2P`, управление — сначала утилитой `devmem`, потом драйвером `spi-zynq-fpga`.



Общее у двух картинок — `spi_master_top`: один и тот же файл, те же провода, то же поведение. Меняется только то, кто исполняет роль хоста.

3.4. Почему два топа, а не один универсальный

Соблазн сделать один верхний уровень с переключателем режимов возникает немедленно, и аргумент за него понятен: один файл проще сопровождать, нет риска, что версии разъедутся. Аргументов против оказалось три, все практические.

Первый — независимость bring-up от софта. Автономный режим проверяет железо, когда всего остального ещё не существует: нет сконфигурированной процессорной системы, загрузчика, ядра Linux, драйвера. Если для проверки ножки `SCLK` нужно, чтобы сначала загрузился Linux, первая же ошибка в цепочке загрузки блокирует всю аппаратную отладку. Второй — чистота продуктового битстрима: `spi_selftest` пишет в те же регистры, что и процессор, и в продуктовой сборке он не просто не нужен, а опасен, потому что два источника записи в один регистровый файл — это гонка, которую придётся или арбитровать, или запрещать параметром. Третий — ограничения физически разные: у автономного варианта свои выводы, свой осциллятор на W17, свой файл ограничений, а у варианта с процессорной системой такт приходит из PS, часть выводов уходит в MIO и часть ограничений генерируется блок-дизайном автоматически. Слить это в один файл значит написать в нём условную логику, а условная логика в ограничениях — верный способ

однажды собрать кривой битстрим.

Цена названа честно: два маршрута сборки, обе надо периодически прогонять, и есть риск, что они разойдутся. Риск уменьшен тем, что расходиться почти нечему: оба инстанцируют один и тот же неизменённый `spi_master_top`, различие живёт в тонком слое над ним. Плюс скрипт `build.tcl` в автономном режиме **отказывается** генерировать битстрим, если в файле ограничений остался хотя бы один маркер `TBD` — незаполненное назначение вывода. Это сделано именно ошибкой, а не предупреждением: битстрим с неверной распиновкой способен физически повредить плату. В исходниках можете изучить это самостоятельно.

3.5. Первая ловушка: level shifters и ps7_init

А теперь затравка на будущее — история из частей IV и V, идеально иллюстрирующая всё сказанное про границу PS/PL. Автономный дизайн процессору логически не нужен: он тактируется от собственного осциллятора, ему не нужна ни память, ни сеть, ни загрузчик. Казалось бы, залил битстрим по JTAG — и работай. На Zynq-7000 это не совсем так: программируемая логика **электрически** зависит от процессорной системы. Между половинами кристалла стоят согласователи уровней (level shifters), и включаются они программно, кодом инициализации при штатной загрузке. Пока они выключены, ваша логика может честно работать внутри и не иметь связи с внешним миром.

Vivado предупреждает так, что предупреждение легко пропустить: проверка правил проектирования выдаёт единственное замечание `ZPS7-1 Warning: PS7 block required` — «в дизайне Zynq должен присутствовать блок PS7, иначе не будет правильной конфигурации по умолчанию». Для PL-only дизайна оно ожидаемо и дефектом переноса не является, но именно оно — главный подозреваемый в ситуации «загрузил битстрим, на плате мёртвая тишина». Варианты решения разобраны: дать плате штатно загрузиться с SD-карты и только потом залить логику по JTAG; выполнить `ps7_init` из XSCT перед загрузкой; либо сразу перейти к блок-дизайну с процессорной системой, где вопрос снимается по построению.

Ловушка простая, а ценна порядком подозрений, который задаёт. Когда после прошивки нет никаких признаков жизни, неправильный первый вопрос — «что не так в моём конечном автомате»; правильный — «а включена ли вообще та половина кристалла, в которой он живёт». Та же логика позже приведёт нас к `ps7_init` от чужой платы, к электрическому стандарту выводов Ethernet и к устаревшему кэшу синтезатора: одна привычка — куча сохраненного времени и нервов. Кстати, о физическом риске: в списке незакрытых вопросов проекта числится напряжение питания банка 13. Его 3.3 В — заводское значение по умолчанию, переключаемое переключками на 1.8 или 2.5 В, и если объявить в файле ограничений стандарт `LVCMS33`, а банк переключён, испортить можно уже не настроение, а железо. Это единственный пункт серии, где ошибка стоит денег, и в части IV останавливаемся на нём отдельно.

3.6. Порядок первого включения

Полная процедура из тринадцати шагов расписана в части bringup. Часть V проходит её с комментариями. Здесь только скелет, потому что его логика важнее деталей: шаги упорядочены **по цене проверки**, а не по интересности, и каждый следующий имеет смысл лишь после предыдущего.

1. **Сверка распиновки до подачи питания.** Собрать проект и посмотреть отчёт о размещении выводов: все ли назначены явно и совпадают ли с документом распиновки. Тридцать секунд здесь против недели с анализатором потом — именно на этом шаге исходный Altera-проект дал кучу проблем.
2. **Такт и сброс.** Загрузить битстрим и посмотреть на светодиод heartbeat: он мигает примерно 1.5 раза в секунду от обычного 25-разрядного счётчика. Это доказывает, что такт живой, сброс снят и логика в кристалле, — независимо от того, работает ли SPI.

3. **Уровни покоя мультиметром.** Две минуты и один прибор закрывают класс отказов «не тот вывод, нет питания банка, вывод занят кем-то ещё»: линия выбора в покое около 3.3 В, SCLK при нулевой полярности около нуля, MISO подтянут к 3.3 В.
4. **Осциллограф на SCLK, затем на CS и MOSI.** Пачки импульсов 1 МГц, линия выбора обрамляет каждый кадр, данные меняются от кадра к кадру.
5. **Петля MOSI → MISO джампером.** Единственная проверка, покрывающая весь тракт без внешнего оборудования: сдвиг наружу, выходной вывод, провод, входной вывод, синхронизатор, момент сэмплирования, сборка слова обратно.

Обратите внимание на приём из шага 4, который сам по себе стоит главы: у `spi_selftest` есть режим непрерывной работы, включённый по умолчанию. В периодическом режиме активность занимает около 10 мкс на каждые 84 мс — скажность 0.012 %, и анализатор в режиме свободного запуска не поймает её никогда: линия выглядит мёртвой при абсолютно живом дизайне. Это второй урок, за который исходный проект заплатил кучей времени.

3.7. Где что лежит

Наконец, карта репозитория — пригодится каждый раз, когда в тексте встретится путь.

```
SPI/
├ docs/                ← статья про Altera (Cyclone IV / ALINX)
├ rtl/                 ← исходное ядро; эталон для сравнения
├ sim/, quartus/       ← тесты и проект Altera-версии
└ spi_xilinx/          ← всё, о чём эта серия
  ├── rtl/              ← ядро (перенос: 4 файла байт-в-байт) +
  |                     spi_axi4lite, spi_axis_tx,
  |                     spi_zynq_top, spi_selftest
  ├── constraints/      ← XDC: выводы платы и обоснованные исключения
  ├── sim/              ← 26 тестов AXI-обвязки, свип делителя
  ├── scripts/          ← build.tcl, проверка эквивалентности, сверка выводов
  ├── linux/            ← драйвер spi-zynq-fpga и Device Tree
  ├── buildroot-external/ ← конфигурация сборки образа
  ├── userspace/        ← утилиты, в том числе часы на ST7789
  ├── docs/             ← инженерные отчёты (migration, bringup, ...)
  └ docs/article/       ← эта серия (вы здесь)
```

Объяснить с  SourceCraft

Разделение на «статью» и «отчёты» намеренное, и им стоит пользоваться. Статья упрощает: объясняет, рассказывает, иногда округляет ради понятности. Отчёты фиксируют доказательства: контрольные суммы файлов, номера строк в исходниках, выводы инструментов, точные числа из Vivado. Если утверждение статьи покажется сомнительным — не спорьте с текстом, откройте `migration.md` или сам код в `../rtl/spi_engine.v`. Источник истины всегда RTL, а не документация; по этому правилу в проекте нашлось уже несколько расхождений, и все они оказались в пользу кода. Короткий вывод главы: плата и инструменты здесь не фон, а полноценный участник событий. Половина ошибок серии — ошибки не в логике, а в договорённостях платформы, и знание того, где проходит граница PS/PL и кто включает согласователи уровней, стоит дороже, чем ещё одна конструкция Verilog.

4.1. Три жанра под одной обложкой

Текст серии намеренно смешан из трёх жанров, и полезно уметь их различать: от жанра зависит, как читать конкретный абзац.

Теория отвечает на вопрос «почему так устроено». Зачем во всём дизайне ровно один тактовый домен и что было бы, если бы их стало два; что такое рукопожатие на шине AXI; почему SCLK у нас не второй тактовый сигнал, а обычный выход триггера, и как это сделало перенос тривиальным. Теорию читают медленно и один раз, она не устаревает ни при каких условиях.

Лаборатория отвечает на вопрос «что делать руками»: собрать такой-то командой, прочитать идентификатор, поставить джампер сюда, повесить щуп туда, ожидать вот такой результат. Эти фрагменты стоит не читать, а выполнять — и записывать то, что получилось, даже если не получилось «как в тексте».

Кладбище ошибок отвечает на вопрос «а что, если не работает». Это истории о том, как всё пошло не так, изложенные без сокращений: с ложными гипотезами, потерянным временем и решениями, которые оказались неверными. Их в серии больше всего, и это не случайность. Без третьего жанра техническая статья незаметно превращается в рекламный буклет: «перенесли ядро, добавили AXI, собрали Linux, вывели картинку» — всё правда и всё бесполезно, потому что читатель получает список результатов и ни одного инструмента для получения своих.

4.2. Шаблон разбора ошибки и почему первая гипотеза врёт

Каждый нетривиальный разбор построен по одному каркасу из шести вопросов. Мы не оформляем его таблицей: таблица создаёт ложное ощущение бюрократической процедуры, тогда как это способ думать.

Начинается всё с **симптома**, и записать его надо буквально: что вы увидели глазами, в логах, на экране осциллографа. Не «DMA не работает», а «верхняя треть кадра цветная, ниже мусор, линия выбора поднимается через 40 мкс после начала кадра, хотя кадр длится миллисекунды». Первая формулировка — уже вывод, причём чужой; вторая — факт, с которым можно работать. Симптом, записанный как вывод, задаёт направление поиска раньше, чем поиск начался.

Второй вопрос — **что казалось логичным**: честно зафиксировать первую гипотезу вместе с её обоснованием. Зачем записывать заведомо неверную версию? Затем, что она почти никогда не бывает глупой, и это самое важное наблюдение главы. В примере с дебагом искаженного кадра логичной казалась мысль «процессор не успевает подкладывать данные, надо поднять приоритет процесса» — рассуждение разумное, любой сказал бы то же самое. Именно потому, что гипотеза разумна, на неё можно потратить неделю.

Третий вопрос — **как проверяли**, и он главный. Хороший эксперимент не подтверждает вашу версию, а **делит пространство гипотез пополам**: даёт разный результат в зависимости от того, кто прав. Пример из части V: в автономном режиме есть диагностическая сборка, подающая на выводы меандры прямо со счётчика, минуя всё SPI-ядро, причём три разные частоты на трёх выводах, чтобы опознать их одним взглядом. Меандры видны — физика в порядке, ищите в логике; не видны — отлаживать RTL бессмысленно, проблема в выводах, банке, щупе или питании. Одна пересборка, и половина версий отпала.

Четвёртый — **корневая причина**, сформулированная так, чтобы из неё следовало предсказание: «линия выбора привязана к условию „передающий буфер не пуст“, а его глубина восемь слов при кадре в 110 килобайт» — это причина, сразу видно, что ускорение процессора не поможет; «что-то не так с DMA» — нет.

Пятый — **исправление**, с указанием, что именно изменилось: строка в коде, в ограничениях, бит в

реестре, абзац в документации.

И шестой, единственный обязательный, — **правило на будущее**: одна фраза, применимая в следующем проекте, где не будет ни этого IP, ни этой платы. «Регистрируемый производитель данных требует запаса по заполнению буфера, равного глубине конвейера, а не сырого признака „полон“» — вот это правило; «мы добавили `almost_full`» — запись в журнале.

Откуда берётся закономерность про врущую первую гипотезу, стоит сказать прямо, иначе шаблон выглядит суеверием. Гипотеза формируется из того, что вы недавно трогали, и из той части системы, которую понимаете лучше всего; оба источника искажения тянут в одну сторону. Правила драйвер — значит, ошибка в драйвере; хорошо знаете C и хуже знаете AXI DMA — значит, объяснение на стороне C. В результате гипотеза оказывается не самой вероятной, а самой доступной. Помогает приём: прежде чем проверять первую версию, вслух назовите вторую и третью из другой области системы. Ошибка в кадре может жить в приложении, драйвере, описании железа, блок-дизайне, RTL, ограничениях, кэше синтезатора или в самом дисплее — восемь этажей, и если все ваши версии с одного, вы ещё не начали расследование.

И последнее: записывайте улику, а не вывод. Улика — измеренное значение, снимок с анализатора, строка из журнала, время изменения файла; вывод — ваша интерпретация, и он меняется по три раза за вечер. К концу дня вы не вспомните, было ли «CS поднялся рано» измерением или предположением, а это ровно та разница, которая определяет, зря вы потратили день или нет.

4.3. Что за расследования вас ждут

Чтобы не читать серию как справочник, полезно знать заранее, где в ней интересные места. Ниже анонсы без спойлеров: симптом назван, разгадка нет.

В **части II** живут дефекты самого ядра, унаследованные с Altera. Главный: делитель, установленный в единицу, портит приём — отправляете `0x3C`, получаете `0x1E`, причём передача при этом идеальна. Второй: бит регистра состояния, по документации сообщающий о переполнении приёмного буфера, не выставляется никогда — и обнаружился он не тестом, а безобидным предупреждением синтезатора про неподключённый порт.

В **части IV** выясняется, что аккуратное перенесённое из Altera объявление `SCLK` производным тактовым сигналом делает временной анализ не пессимистичным, а оптимистичным, то есть хуже, чем его отсутствие; там же банк выводов, чьё напряжение зависит от выставленного значения.

Часть V — первое включение и весь букет проблем начинающего: дизайн, который «мёртв», потому что не включена половина кристалла; линия, выглядящая пустой из-за скажности в сотые доли процента; паттерн `0xA5`, который кажется случайным, но читается одинаково в обе стороны и потому пропускает ошибку порядка бит насквозь.

Часть VII — самая длинная. Цепочка загрузки собиралась с кодом инициализации от отладочной платы ZC702: тот же кристалл `xc7z020`, но другая память и другие выводы, и обнаружилось это поиском конкретной константы в двоичном файле загрузчика. Там же Ethernet, который «Link Up», но не пингуется: описание железа от чужой платы ищет физический уровень по неправильному адресу на служебной шине и перепрограммирует выводы в другой электрический стандарт поверх правильной настройки загрузчика. И ядро Linux, зависающее сразу после инициализации выводов, потому что подсистема тактирования выключила «неиспользуемый» такт, на котором работает вся логика. Мой косяк.

Часть VIII — дисплей: панель физически 172 на 320 точек, а часы хочется в ландшафте; команда смены ориентации меняет систему координат, но не содержимое памяти панели, и от старого

кадра остаются призраки, плюс смещение начала кадра по вертикали ровно на 34 точки — число, которое нельзя вывести, можно только узнать.

Часть IX — главное кладбище багов. Передающий буфер на восемь слов против кадра в 110 килобайт: линия выбора снимается посреди кадра, и ускорение программы этого не меняет, потому что решение принимает конечный автомат, а не процессор. Сброс контроллера DMA перед каждым кадром, при включённом потоковом режиме порождающий ложный признак конца передачи и двоящуюся картинку. Гонка на один такт между регистрируемым сигналом записи и признаком «буфер полон», из-за которой байты пропадают молча, а шрифт оказывается сжат по горизонтали. И под конец мета-ошибка, самая обидная в курсе: исправление уже в репозитории, новый битстрим собран, а на плате поведение прежнее, потому что синтезатор переиспользовал свой прошлый результат для этого блока. Общее у всех историй: ни одна не про Verilog. Это и есть тезис главы 1, рассказанный перечнем симптомов.

4.4. Что вы должны уметь после частей 0–III

Первые четыре файла — фундамент, и у него есть измеримый результат. Проверьте себя перед тем, как идти в Vivado.

1. Объяснить своими словами, почему перенос SPI на Zynq — это в первую очередь системная интеграция, а не переписывание конечного автомата.
2. Провести аудит проекта на вендорские примитивы и не растеряться, когда поиск не нашёл ничего.
3. Нарисовать по памяти дерево модулей: что копируется байт-в-байт, что оборачивается, что остаётся дома и почему.
4. Связать ограничение «делитель не меньше двух» с задержкой синхронизатора на входе MISO и посчитать из него максимальную частоту SPI.
5. Объяснить, зачем параллельная шина превращается в AXI4-Lite, какие четыре отклонения от учебного варианта мы себе позволили и чем за них платим.
6. Сказать, что делает драйвер, прочитав по смещению θ_{x3C} значение θ , и почему это правильное поведение.

Если хотя бы один пункт вызывает затруднение — соответствующая часть не прочитана, а пролистана. Дальше будет только плотнее.

4.5. Темп чтения для первокурсника

И последнее — про то, как распределить силы: серия длинная, и сдаться на середине проще, чем кажется. Если Verilog ещё вызывает напряжение, не идите строго по порядку: прочитайте части 0 и I полностью, часть II — до места, где начинаются номера строк исходников, а затем прыгайте в части VIII и IX. Там мало RTL и много зримых причинно-следственных связей, а мотивация от увиденной картинки на дисплее стоит дороже строгой последовательности; к AXI и блок-дизайну вернётесь, когда захочется понять, откуда берутся адреса. Если вы уже собирали Altera-версию, части I и II можно читать по диагонали, но разборы дефектов A-3 и A-5 прочитайте целиком: первый определяет максимальную частоту, которую вы вправе заявить в документации, второй — то, как драйвер обнаруживает потерю данных. Обе вещи всплывут в части VII, когда дойдёт до кода драйвера.

Не пропускайте mermaid-схемы: они сжимают страницу текста в одну картинку, которую удобно держать в голове, стоя у осциллографа. И держите открытой карту регистров: когда в тексте мелькает смещение θ_{x3C} или девятый бит регистра состояния, она играет роль словаря. Наконец, полезная привычка на всю серию — после каждой части выписывать одно правило,

которое унесёте с собой. Не конспект, не пересказ, а одну фразу, применимую в проекте, где не будет ни Zynq, ни SPI. К концу наберётся десяток таких фраз, и именно они, а не рабочий битстрим, окажутся главным результатом.

Часть I. Зачем переносить и что показал аудит

Глава 5. Vendor-neutral RTL vs vendor glue

5.1. Что мы хотели понять до того, как что-то трогать

Перед тем как открыть Vivado, нам нужно было ответить на один вопрос: сколько именно кода придётся переписать. Ответ «перепишем половину» и ответ «поменяем два атрибута» ведут к принципиально разным проектам. В первом случае мы делаем новое устройство и обязаны заново доказывать, что оно работает; во втором — переносим уже доказанное и обязаны доказать только то, что ничего не сломали. Разница вся в масштабе доказательства, а не в объёме.

Наивное ожидание звучит так: «перенести с Altera на Xilinx» — значит «переписать Verilog». Оно почти всегда неверно. Verilog — стандартизованный язык описания аппаратуры (IEEE 1364), и синтезаторы обоих вендоров понимают одно и то же его подмножество. Строчка `always @(posedge clk) q <= d;` означает «триггер, защёлкивающий d по фронту clk», и это верно на Cyclone IV, на Artix-7 и на бумаге. Переносить тут нечего: триггеры есть у всех (ха-ха).

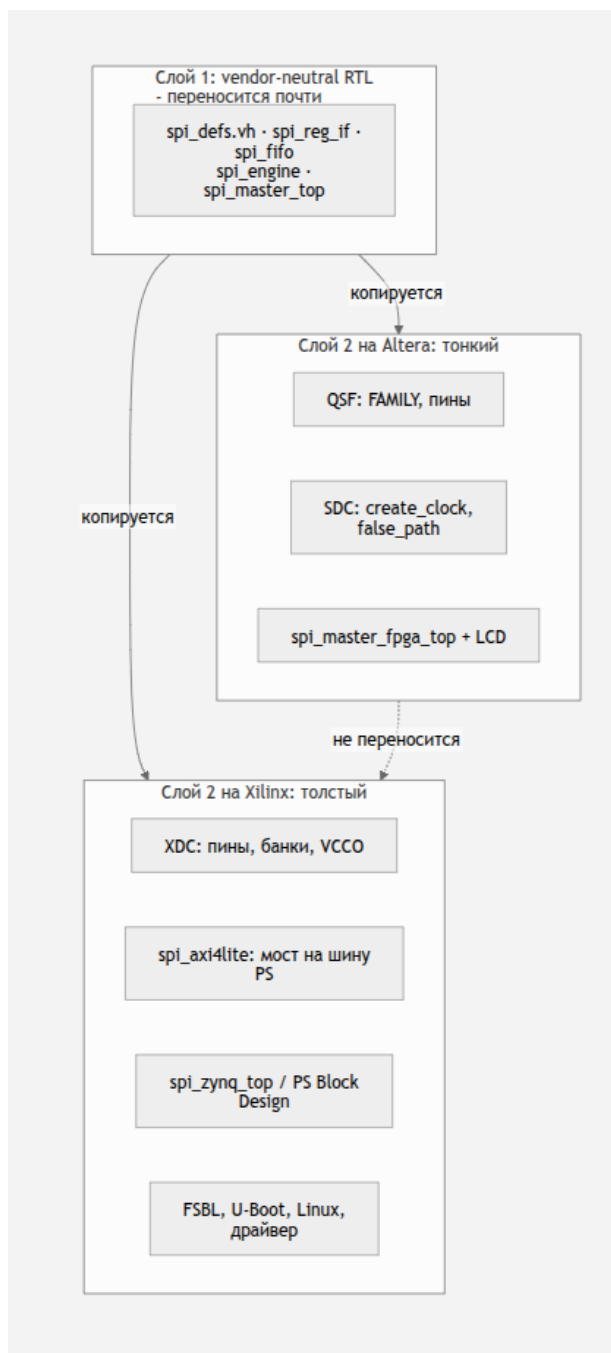
Тогда откуда берётся vendor lock — привязка к конкретному производителю? Не из языка, а из того, что вы в этом языке **позвали по имени**. Как только вы пишете имя модуля, которого нет в стандарте, — вы позвали деталь из фирменного набора конкретного вендора. И вот эта деталь не переносится.

5.2. Два слоя проекта: чертёж и фирменные детали

Любой FPGA-проект состоит из двух слоёв. Первый — **vendor-neutral RTL**: поведенческое описание логики. Регистры, конечные автоматы на `localparam`, счётчики, массивы `reg`, арифметика, комбинационные выражения. Синтезатор читает это описание и сам решает, какими физическими ячейками кристалла его реализовать. Вы описали *что* должно происходить, инструмент выбрал *чем*. Такое описание переносится между вендорами практически даром: меняется только исполнитель.

Второй слой — **vendor glue**, «клей» к конкретной экосистеме. Сюда попадает всё, что имеет смысл только внутри одного инструмента или одной платы: назначения выводов, файлы временных ограничений, настройки проекта, интеграция с шиной процессорной системы, цепочка загрузки, драйвер. И сюда же — вендорские примитивы, если вы их использовали. Этот слой не переносится по определению: он описывает не логику, а её стыковку с внешним миром, а внешний мир у нас теперь другой.

Ядро нашего SPI почти целиком живёт в первом слое. На Altera второй слой был тонким: пины-заглушки в QSF, небольшой SDC, топ с LCD-дашбордом. На Zynq второй слой становится заметно толще — появляются AXI, Block Design процессорной системы, XDC с банками ввода-вывода, FSBL, Device Tree, Buildroot. Отсюда и сквозной тезис всей серии, заявленный в части 0: работа переноса — не «переделать SPI», а **переклеить его к другой экосистеме**.



5.3. Что такое вендорский примитив на живых примерах

Абстракция «фирменная деталь» станет понятнее на конкретных именах — все они встречались бы нам в исходном коде, пойдя автор Altera-версии обычным путём.

altp1l (в старых проектах — ALTPLL) — это PLL, схема фазовой автоподстройки частоты. Физически внутри кристалла есть отдельный аппаратный блок, который умеет из входных 50 МГц сделать 100, 125 или 33.3 МГц с нужной фазой. Вы не описываете его поведенчески — вы **инстанцируете** его, то есть создаёте экземпляр модуля с именем altp1l и передаёте параметрами коэффициенты умножения и деления. На Xilinx аналогичный блок называется MMCM или PLLE2, у него другое имя, другие параметры, другие ограничения. Модуля altp1l в библиотеке Vivado не существует — синтез просто скажет, что такого модуля нет.

Остальные имена устроены так же. altsyncram — блочная память (ячейки M9K у Cyclone IV, BRAM у 7-series). scfifo и dcfifo — готовые FIFO, одно- и двухтактный, построенные поверх

той же блочной памяти. Семейство `lpm_*` — библиотека параметризуемых счётчиков и умножителей, исторически «стандартная», а на практике реализованная у каждого вендора по-своему. `altddio_in / altddio_out` — регистры, защёлкивающие данные по обоим фронтам. `alt_iobuf` — явный буфер ввода-вывода с управлением третьим состоянием.

Есть ещё две категории, о которых новички забывают. Первая — **атрибуты синтеза**: пометки вида `/* synthesis keep /, ( altera_attribute = "...") , / synthesis translate_off */`. Они не меняют логику, но управляют поведением инструмента, и каждый вендор понимает свой набор. Вторая — **файлы IP-каталога**: `.qip` в Quartus, `.xci` в Vivado, проекты Platform Designer / Qsys — сгенерированные описания настроенных блоков, привязанные к версии инструмента ещё крепче, чем к вендору.

Общее правило стоит запомнить дословно: **поведенческий Verilog описывает намерение, вендорский примитив называет деталь**. Намерение переводится на любую платформу, потому что триггер есть у всех. Название детали не переводится: у другого вендора она называется иначе, ведёт себя не совсем так и настраивается другими параметрами.

Мегафункция Altera	Что это	Аналог у Xilinx	Что делает наш проект
<code>altpll</code>	PLL, синтез частот	MMCM / PLLE2	нет: один блок, деление счётчиком
<code>altsyncram</code>	блочная память M9K	BRAM (RAMB18/36)	нет: FIFO — массив <code>reg</code>
<code>scfifo</code>	синхронный FIFO	<code>xpm_fifo_sync</code>	написан руками
<code>dcfifo</code>	асинхронный FIFO	<code>xpm_fifo_async</code>	нет: второго домена нет
<code>lpm_*</code>	счётчики, умножители	инференс из выражений	нет: обычная арифметика
<code>altddio_in/out</code>	DDR-регистры I/O	IDDR / ODDR	нет: DDR I/O не нужен
<code>alt_iobuf</code>	буфер I/O	IBUF / OBUF / IOBUF	нет: буферы выводит синтезатор

5.4. Инференс против инстанса: почему это архитектурное решение

Теперь самое важное понятие главы. У памяти внутри FPGA есть два способа появиться в вашем проекте. Первый — **инстанс примитива**: вы явно создаёте экземпляр `altsyncram` или `RAMB36E1`, задаёте разрядность, глубину, режим чтения, и получаете ровно ту аппаратную ячейку, которую заказали. Это как заказать деталь по каталожному номеру: вы точно знаете, что придет, но придет оно только с одного склада.

Второй — **инференс** (от английского *inference*, «вывод по признакам»): вы описываете память как обычный массив регистров, а синтезатор сам догадывается, что это память, и выбирает, чем её реализовать. Это как написать в чертеже «здесь нужна ёмкость на восемь слов по 32 бита» и позволить производству решить, собирать её из триггеров, из распределённой памяти в таблицах истинности или из блока памяти. Инференс работает по шаблонам: синтезатор умеет узнавать несколько канонических способов записать память на Verilog.

Наш FIFO написан ровно во втором стиле — вот его хранилище и чтение целиком (`../.. /rtl/ spi_fifo.v`):

```

reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
reg [ADDR_WIDTH:0]   wr_ptr;
reg [ADDR_WIDTH:0]   rd_ptr;

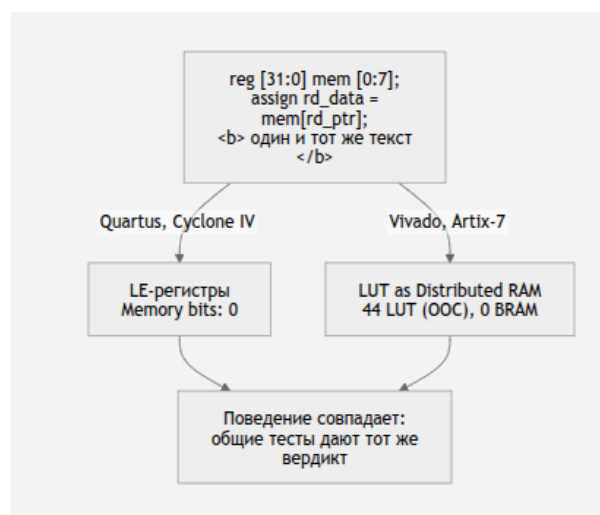
assign rd_data = mem[rd_ptr[ADDR_WIDTH-1:0]];
assign count   = wr_ptr - rd_ptr;
assign full    = (count == DEPTH[ADDR_WIDTH:0]);
assign empty   = (count == { (ADDR_WIDTH+1){1'b0} });

```

Объяснить с 

Ни одного вендорского имени. Массив, два указателя, разность указателей как счётчик занятости и **комбинационное** чтение: `rd_data` — просто провод от головы очереди, без промежуточного регистра. Указатели на один бит шире адреса затем, чтобы их разность отличала «пусто» от «полно» без дополнительного флага.

И вот самое поучительное место: один и тот же текст два инструмента реализуют разными ячейками. Quartus на Cyclone IV собрал хранилище из логических элементов и триггеров — сводка фиттера показывает `Memory bits: 0`. Vivado на 7-series увидел тот же шаблон и вывел **распределённую память**, LUTRAM: таблицы истинности, переключённые в режим маленького ОЗУ. В отчёте это 44 LUT категории «LUT as Distributed RAM» для сборки IP с AXI-обвязкой и 24 для автономной. Логика и поведение идентичны, физическая реализация разная, и для Xilinx она выгоднее.



Почему инференс — архитектурное решение, а не лень автора? У него есть и цена, и выигрыш, и их выбирают осознанно. Выигрыш: переносимость, отсутствие привязки к версии инструмента, читаемый код, который моделируется любым симулятором. Цена: вы **не управляете точно** тем, что получится. Если бы FIFO был глубиной в тысячи слов, разница между «блок памяти» и «тысячи триггеров» стала бы разницей между «помещается» и «не помещается». Именно поэтому в шапке `spi_fifo.v` стоит честная оговорка автора: для глубин от 32 записей стоит перейти на вариант с синхронным чтением, дружественный к блочной памяти, а для используемых 4–16 слов инференс в регистры уместен. Практическое правило: инференс хорош, пока вам всё равно, в какую из подходящих ячеек попадёт описание. Как только конкретная ячейка становится требованием, придётся либо переписать код под её шаблон, либо инстанцировать примитив и потерять переносимость.

5.5. CDC: единственное место, где вендор всё-таки просочился в код

Теперь про единственное изменение, которое перенос всё же внёс в существующий RTL. Чтобы

понять, зачем оно, нужно разобраться с двумя понятиями.

Метастабильность. Триггер защёлкивает данные по фронту сигнала, но требует, чтобы вход был стабилен некоторое время до фронта (setup) и после него (hold). Если сигнал меняется ровно в этом окне, триггер попадает в неопределённое состояние: выход не 0 и не 1, а «где-то посередине», и он может оставаться там долго, прежде чем свалится в одну из сторон. Это физика бистабильной ячейки, и полностью устранить эффект нельзя.

CDC, Clock Domain Crossing — пересечение тактовых доменов, то есть передача сигнала из области, тактируемой одним клоком, в область, тактируемую другим. Если клоки не связаны, момент смены сигнала относительно фронта приёмника непредсказуем, и запретное окно рано или поздно будет задето. То же относится к любому асинхронному входу извне: он тоже меняется когда хочет.

Лекарство — **синхронизатор из двух триггеров**. Первый может уйти в метастабильное состояние, но у него есть целый такт, чтобы из него выйти; второй защёлкивает уже устоявшееся значение. Вероятность того, что метастабильность переживёт обе ступени, падает на порядки — до значений, при которых среднее время между сбоями измеряется годами. Так устроен

spi_reset_sync :

```
(* ASYNC_REG = "TRUE" *)
reg [STAGES-1:0] sync_chain;

always @(posedge clk or negedge rst_n_async) begin
    if (!rst_n_async)
        sync_chain <= {STAGES{1'b0}};
    else
        sync_chain <= {sync_chain[STAGES-2:0], 1'b1};
    end

    assign rst_n_sync = sync_chain[STAGES-1];
```

Объяснить с 

Сама цепочка — чистый vendor-neutral код: сдвиговой регистр из двух разрядов. А вот строка `(* ASYNC_REG = "TRUE" *)` — уже метаданные для Vivado, и именно она составляет всё изменение RTL при переносе. Что она обещает инструменту, записано в шапке файла: держать триггеры цепочки в одном слайсе (чем короче связь между ступенями, тем больше времени на разрешение метастабильности), не применять к ним ретайминг и не сворачивать их в сдвиговой регистр SRL, а также пометить цепочку для отчётов `report_cdc` и `report_synchronizer`, чтобы пересечение было **распознано**, а не отнесено к небезопасным путям.

Ключевое слово — «распознано». Vivado не обязан догадываться, что два триггера подряд образуют синхронизатор, а не случайную задержку. Без атрибута он вправе оптимизировать их как удобно и классифицировать пересечение как непроверенное. С атрибутом отчёт после трассировки даёт «All paths are Safely Timed». Quartus делал тот же вывод из самой структуры асинхронного сброса, поэтому исходнику атрибут не требовался.

Вариантов было три. Первый — не трогать код и надеяться, что Vivado догадается: отвергнут, «надеяться» — не инженерный аргумент, да и отчёт CDC остался бы не в нашу пользу. Второй — инстанцировать готовый макрос `xpm_cdc_single` из библиотеки Xilinx: он делает то же самое, но завёл бы в vendor-neutral ядро вендорское имя, то есть создал бы собственными руками тот самый lock-in, отсутствию которого мы только что радовались. Третий, выбранный, — добавить атрибут. Логика не меняется ни на бит: то же число триггеров, те же связи, то же поведение в симуляции.

Цена третьего варианта интереснее, чем кажется. Два файла — `spi_engine.v` и `spi_reset_sync.v` — перестают быть побайтово равными оригиналу. Значит, самое простое доказательство эквивалентности («файлы совпадают — значит ведут себя одинаково») для них не работает, и нужен второй уровень: показать, что в различиях нет ничего, кроме атрибута и комментариев. Именно поэтому в скрипте проверки из главы 6 есть отдельный слой ровно для этих двух файлов. Граница проходит здесь: **логика остаётся vendor-neutral, метаданные становятся vendor-aware**. Это нормальное состояние переносимого IP.

5.6. Чем мы заплатили за vendor-neutral стиль

Красивая история про «ядро переносится даром» была бы нечестной без второй половины: за переносимость платят, и счёт приходит позже.

Готовый `scfifo` из каталога Quartus или FIFO Generator из каталога Vivado — это не просто «массив с указателями». Из коробки там `programmable full` и `programmable empty` (пороги, при которых поднимаются флаги «скоро заполнится» и «скоро опустеет»), регистр занятости, защита от переполнения, выбор между распределённой и блочной памятью одним переключателем, готовый асинхронный вариант с корректными счётчиками Грея, часто и контроль чётности.

Написав FIFO руками, мы получили ровно то, что написали: `full`, `empty`, `overflow`, `underflow`, `count`. Ни одного порога. И когда проект дошёл до потоковой передачи через AXI DMA, выяснилось, что порога не хватает. Продюсер `spi_axis_tx` **регистрирует** сигнал записи: решение «пишем» принимается по состоянию `full` на такте N, а сама запись приходит на такте N+1. FIFO принимает слово только при `wr_en && !full`. У самой границы заполнения — а под DMA полноэкранный кадр живёт именно у границы — открывается окно в один такт, в котором запись **молча** пропадает. На экране ST7789 это выглядело как горизонтально сжатая картинка: часть байтов не доехала.

Лечение — тот самый порог, которого не было:

```
/*
 * Producers that register wr_en (spi_axis_tx) sample full one cycle late.
 * Backpressure two entries early so a delayed write cannot land on "full"
 * and be discarded by the wr_en & ~full gate — that drop showed up as a
 * horizontally compressed ST7789 image under AXI DMA (FIFO stays near full).
 */

assign almost_full = (count >= (DEPTH - 2));
```

Объяснить с 

Полный разбор гонки — глава 41 в части IX; здесь она важна как иллюстрация цены. Если бы FIFO был взят из каталога, `almost_full` с настраиваемым порогом лежал бы в нём с самого начала, и целого детектива не случилось бы. Мы обменяли готовую функцию на переносимость и заплатили одним багом, найденным на живом железе.

Стоила ли сделка того? Да. Баг мы нашли, поняли и исправили одной строкой в собственном коде — его можно прочитать целиком за пять минут. Если бы каталожный FIFO повёл себя неожиданно, мы бы отлаживали чужой сгенерированный netlist. И главное: без vendor-neutral стиля этой статьи не было бы вовсе.

5.7. Вывод главы

Vendor lock живёт не в языке, а в именах, которые вы позвали. Чистый поведенческий Verilog переносится между вендорами, потому что описывает намерение; мегафункции не переносятся,

потому что называют деталь. Инференс — осознанный выбор в пользу переносимости ценой контроля над реализацией, и его последствия видны в отчётах о ресурсах: тот же массив стал LE-регистрами у Quartus и LUTRAM у Vivado. Единственная щель, через которую вендор всё-таки попал в наш RTL, — атрибут `ASYNC_REG` на цепочках синхронизаторов; это метаданные, а не логика. А за отказ от каталога мы заплатили отсутствующим `almost_full`, счёт за который пришёл в части IX. Осталось проверить, что всё сказанное — правда, а не самоуверенность. Этим занимается аудит.

Глава 6. Аудит: поиск `altpll` / `scfifo` — пустой результат

6.1. Чего мы хотели от аудита и почему «я помню» не подходит

Задача аудита формулируется в одну строку: получить список всего, что придётся переделать, **до** того как начнётся переделка. Не «примерно понять объём», а именно список, который можно предъявить, оспорить и по которому потом отчитаться.

Соблазнительный вариант — положиться на память автора: «я точно не ставил PLL, FIFO писал руками, IP-каталогом не пользовался». Мы его отвергли, и не из недоверия. Память отвечает за то, что человек делал сознательно, и не отвечает за то, что просочилось само: за строку, добавленную год назад при отладке, за настройку, автоматически дописанную инструментом в файл проекта, за подключённый чужой модуль. Кроме того, память не оставляет следа: через полгода ответ «я помню, что `dcfifo` там нет» ничего не стоит, а строка в отчёте с шаблоном поиска и списком просмотренных файлов — стоит. Поэтому аудит у нас — механическая процедура с чек-листом, результат которой записывается независимо от того, нашлось что-то или нет. Второе важнее первого: отрицательный результат тоже надо зафиксировать, иначе он неотличим от непроведённой проверки.

6.2. Как именно искали

Поиск шёл по всем файлам с расширениями `.v`, `.vh`, `.qsf`, `.qpf`, `.sdc`, `.tcl`, `*.qip` — то есть не только по RTL, но и по файлам проекта Quartus, по временным ограничениям и по скриптам сборки. Это принципиально: вендорская зависимость может сидеть не в коде, а в настройке проекта. Шаблон поиска покрывал весь список искомых сущностей одним регулярным выражением:

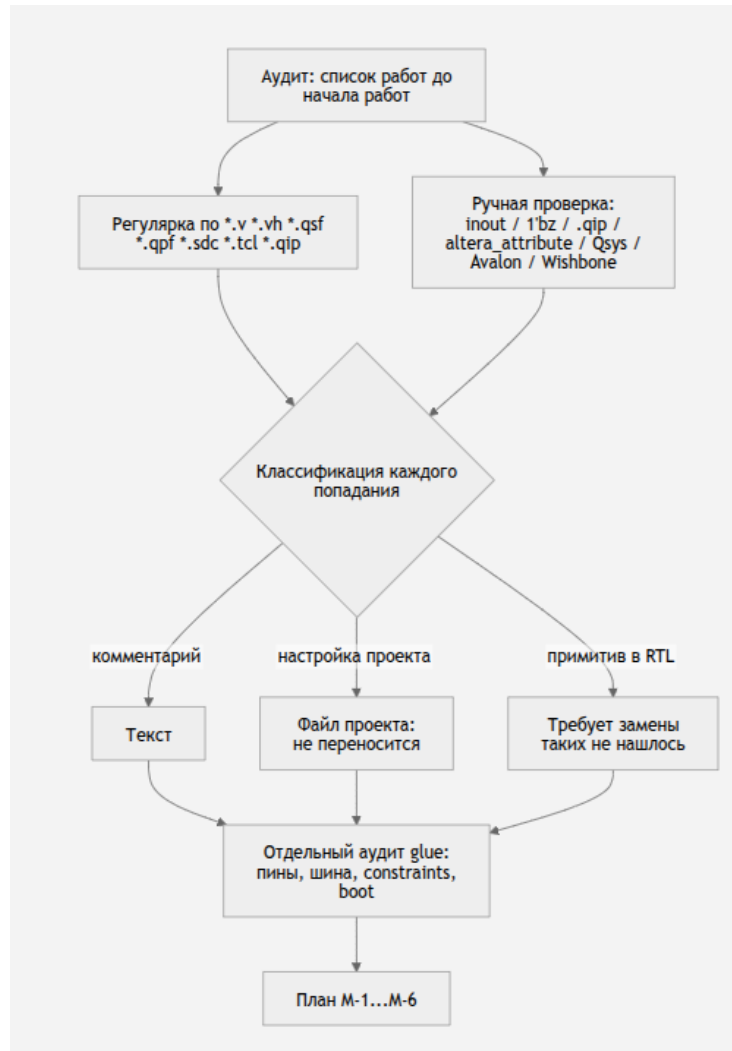
```
alt(pll|syncram|ddio|_iobuf|era_)|scfifo|dcfifo|lpm_|cyclone|arria|
megafunction|altera|synthesis (translate|keep|noprun)|DEVICE_FAMILY
```

Объяснить с 

Прочитаем его вслух — в нём видна логика чек-листа. Первая группа ловит семейство `alt* : PLL`, блочную память, DDR-регистры, буферы ввода-вывода и любое имя, начинающееся с `altera_`. Дальше оба FIFO из каталога и семейство `lpm_*`. Слова `cyclone` и `arria` ловят не примитивы, а привязку к устройству, где бы она ни лежала; `megafunction` и `altera` — широкий невод на комментарии, заголовки и настройки. Группа `synthesis (translate|keep|noprun)` ищет прагмы синтеза в стиле Quartus, а `DEVICE_FAMILY` — параметр, который Quartus подставляет в сгенерированные обёртки.

К автоматическому поиску добавилась ручная проверка того, что регуляркой не ловится: порты `inout` и присваивания `1'bz` (признак логики с третьим состоянием внутри кристалла), файлы `.qip`, атрибуты `(* altera_attribute)`, проекты *Platform Designer / Qsys* (`.qsys`),

*_hw.tcl) и сигналы чужих шин — waitrequest , readdatavalid , stb / cys / ack , по которым опознаются Avalon и Wishbone.



6.3. Что нашлось: шесть попаданий, ноль примитивов

Регулярка сработала — но ни одно попадание не оказалось вендорским примитивом в синтезируемом коде. Всё найденное распадается на две категории: текст в комментариях и настройки уровня проекта Quartus.

Файл	Строка	Что найдено	Классификация
rtl/spi_defs.vh	4	SPI Master Controller for Altera Cyclone IV	комментарий
rtl/spi_fifo.v	9	On Cyclone IV the storage is implemented in LE registers	комментарий
rtl/spi_lcd_fpga_top.v	4	Target board : ALINX PIAX301V2	комментарий

quartus/spi_master.qsf	6	set_global_assignm ent -name FAMILY "Cyclone IV E"	настройка проекта
quartus/spi_lcd.qsf	21, 42	FAMILY , CYCLONEII_RESERVE_ NCEO_AFTER_CONFIGUR ATION	настройка проекта
quartus/spi_master.sdc	4	Target : Altera Cyclone IV EP4CE6F17C8	комментарий

Пройдёмся по чек-листу словами, чтобы отрицательные результаты были названы явно. PLL, MMCM и любых генераторов частоты в дизайне нет вообще: тактовый домен один, деление выполняется счётчиком. Блочной памяти нет — хранилище FIFO описано массивом `reg [31:0] mem [0:7]`. Ни `scfifo`, ни `dcfifo`: FIFO написан руками. DDR-регистров ввода-вывода нет. Явных буферов `IBUF` / `obuf` / `Iobuf` нет — их выводит синтезатор. Логика с третьим состоянием внутри кристалла нет: ни одного порта `inout`, ни одного присваивания `1'bz`. Модулей `lpm_*`, файлов `.qip`, сериализаторов и десериализаторов нет. Атрибутов синтеза Quartus нет ни одного: из директив используются только стандартные `default_nettype` и `include`.

Есть и косвенное подтверждение, независимое от нашего поиска. Сводка фиттера Quartus показывает `Memory bits: 0` и `PLLs: 0`. Ноль бит блочной памяти и ноль PLL означают, что мегафункцию было негде спрятать: стой где-то `altsyncram` или `altpll`, счётчики были бы ненулевыми. Подтверждение ценно тем, что приходит с другой стороны — не от чтения текста, а от результата компиляции.

6.4. Почему пустой результат — это факт

Здесь возникает странное чувство: потратили время, ничего не нашли, вроде бы зря. Ощущение мешает делать аудит как следует, поэтому разберём его.

Пустой результат поиска — полноценный инженерный факт, и у него есть цена. Он означает, что этап «заменить вендорские примитивы на аналоги» можно вычеркнуть из плана целиком. В типичном переносе это самая скучная и самая рискованная часть: у `scfifo` и `xpm_fifo_sync` разная семантика флагов, разная задержка чтения, разное поведение при одновременных чтении и записи, и подмена одного другим почти всегда тянет за собой правку логики вокруг. Мы этого не делаем — и знаем об этом заранее, а не выясняем на третьей неделе. Вторая ценность менее очевидна: записанный отрицательный результат защищает от повторной проверки. Проверка, не оставившая следа, считается непроведённой — это не бюрократия, а способ не делать одну работу дважды.

И третья, самая важная: пустой результат по примитивам **не означает** пустой список работ. Это разные множества. Вендорская зависимость нашего проекта жила не в ядре протокола, а в окружении — в пинах, в `constraints`, в способе доступа с хоста, в периферии платы. Об этом — следующий разбор.

6.5. Детектив первый: «пустой грер — это успех или провал?»

Симптом. Регулярное выражение по всему репозиторию не находит ни одного Altera-примитива в синтезируемом коде. Шесть попаданий, все — текст.

Что казалось логичным. Раз в коде нет ничего вендорского, перенос тривиален: копируем шесть файлов `.v` и `.vh` в проект Vivado, нажимаем «собрать», получаем битстрим. День работы,

максимум два.

Как проверяли. Вместо того чтобы поверить в эту оценку, посмотрели на проект целиком, а не только на RTL-ядро. Открыли топ отладочной ревизии `spi_master_fpga_top` и пересчитали выводы: параллельная шина отладки занимает порядка семидесяти контактов. Открыли `quartus/spi_master.qsf` — файл сам называет свои назначения заглушками, а часть сигналов вообще не имеет `set_location_assignment`. Прочитали `quartus/spi_master.sdc` по одному ограничению: останется ли каждое осмысленным на другой платформе. Посмотрели, на чём держится наблюдаемость: на LCD-панели 480×272 и на опросе XPT2046 — двух устройствах, которых на новой плате нет.

Корневая причина. Мы искали vendor lock не там, где он был. Ядро протокола чистое, но проект — это не только ядро. Зависимость от платформы сосредоточилась в слое glue: в распиновке, во временных ограничениях, в способе общения с хостом и в наборе периферии. Пустой greg сказал правду о первом слое и ничего — о втором.

Исправление. План миграции построен вокруг шести обязательных работ M-1...M-6, и ни одна не называется «заменить `scfifo` на `xpm_fifo`». M-1 — атрибуты `ASYNC_REG` на синхронизаторы, единственная правка существующего RTL. M-2 — замена топа с отладочной шиной на два целевых: `spi_axi4lite` для процессорной системы и `spi_zynq_top` для автономной проверки. M-3 — исключение LCD-подсистемы. M-4 — XDC с нуля по фактической схеме платы. M-5 — осмысленная переработка временных ограничений. M-6 — сохранение контракта стробов регистрового интерфейса в новом AXI-автомате. Единственная работа с высоким риском — M-4: неверно назначенный вывод способен повредить железо.

Правило на будущее. Пустой поиск примитивов — хороший знак для ядра и плохой повод расслабиться. Аудируйте glue отдельным проходом и по отдельному чек-листу: пины, шина, временные ограничения, цепочка загрузки, периферия. Объём работы определяет обвязка.

6.6. Как доказывали, что перенос эквивалентен

Отдельный вопрос: чем подтверждается утверждение «мы перенесли то же самое»? Успешный синтез таким подтверждением не является — компилируется и неправильный дизайн. Мы рассматривали три варианта. Первый — заявить, что ничего не меняли, и сослаться на аккуратность: отвергнут, это не доказательство, а обещание. Второй — прогнать тесты на перенесённой версии и показать, что все зелёные: лучше, но недостаточно, потому что тест проверяет то, что в нём написано, и молчит обо всём остальном. Третий, выбранный, — построить доказательство слоями, от сильного к слабому, и автоматизировать проверку так, чтобы она падала при любом отклонении. Он реализован в `../scripts/check_equivalence.sh`.

Слой первый — **идентичность исходников**. Для каждого файла, который отчёт объявляет неизменным, сравниваются контрольные суммы MD5 оригинала из `../rtl/` и копии из `../rtl/`. Это самое сильное доказательство, какое возможно: идентичный текст не может вести себя по-разному, потому что это буквально один и тот же текст. На момент миграции слой давал четыре совпадения — `spi_defs.vh` (`aa11188644468a63...`), `spi_fifo.v` (`90b2f8da0fad37b5...`), `spi_reg_if.v` (`8adb7e0203941592...`) и `spi_master_top.v` (`e268a993d4b702d4...`).

Слой второй — **объявленные различия**. Для двух файлов, объявленных изменёнными, скрипт берёт `diff`, выбрасывает пустые строки, строки комментариев и строку самого атрибута `ASYNC_REG` — и требует, чтобы остаток был **пустым**. Если после фильтрации остаётся хоть одна строка, значит изменилось выражение, порт, параметр или состояние автомата, и заявление «изменение только атрибутивное» ложно. Доказательство слабее первого, но структурное: оно не зависит от того, насколько хороши наши тесты.

Слой третий — **поведенческое сравнение**. Эталонный тестбенч Altera гоняется против оригинального RTL, тестбенч AXI — против перенесённого, затем из обоих логов извлекаются вердикты PASS / FAIL по именам тестов, берётся пересечение имён и сравниваются вердикты. Скрипт специально проверяет, что пересечение не пустое: если общих тестов не нашлось, это не «всё сошлось», а ошибка разбора логов, и она считается провалом. Тонкость важная — она защищает от самого опасного вида зелёного результата, полученного из-за того, что проверка просто не выполнялась. Скрипт возвращает ненулевой код, если провалился любой слой, и вызывается одной командой `make equiv`. Ключевая мысль его шапки: проверка **самополицейская** — случайная правка «замороженного» файла превращается не в тихое расхождение, а в жёсткий отказ.

6.7. Как этот механизм сработал на нас самих

Проверим утверждения слоя 1 на текущем состоянии репозитория — не по отчёту, а пересчётом контрольных сумм. Результат поучителен: сегодня совпадают только `spi_defs.vh` и `spi_reg_if.v`, а `spi_fifo.v`, `spi_master_top.v` и `spi_engine.v` от оригинала отличаются.

Это не опровержение аудита и не ошибка в отчёте, который фиксирует состояние на момент миграции, и на тот момент четыре файла действительно были побайтово равны, а `spi_engine.v` отличался только атрибутом. Разошлись они позже и осознанно, в части IX, когда появился поток через AXI DMA: в `spi_fifo.v` добавился порог `almost_full`, в `spi_master_top.v` — сквозные сигналы потокового режима и второй источник записи в TX FIFO, в `spi_engine.v` — режимы `stream_en` и `tx_only` вместе с новым состоянием автомата `S_WAIT_TX`.

Именно так механизм и должен работать. Пока файл объявлен замороженным, любая правка в нём — случайная или намеренная — немедленно становится видимой: контрольная сумма расходится, слой 1 падает, и вопрос «а вы точно ничего не трогали?» получает честный ответ. Когда изменение осознанное, меняется не код проверки, а объявление: файл переходит из «неизменных» в «изменённые с доказанным перечнем отличий». Плохо не то, что файл изменился, — плохо, когда он изменился, а документ об этом молчит. Отсюда правило главы: **доказательство сильнее уверенности, а автоматическое доказательство сильнее разового**. Проверка, запускаемая одной командой, переживёт и вашу память, и вашу аккуратность.

6.8. Детектив второй: предупреждение, которое оказалось дефектом

Теперь обещанная история про то, почему «просто скопировать и собрать» не проходит с первого раза. Формально Vivado принял скопированные файлы: `elaboration` прошла с нулём ошибок и нулём `critical warnings`. Но по пути инструмент выдал предупреждения — 55 на синтезе и 52 на имплементации, — и одно из них оказалось не шумом.

Симптом. При `elaboration` Vivado сообщил:

```
WARNING: [Synth 8-7129] Port eng_err_rx_ovf in module spi_reg_if is eitherunconnected or has no
Объяснить с
```

Что казалось логичным. Не обращать внимания. Предупреждений этого же кода в сборке сорок шесть, и большинство безобидны: зарезервированные разряды `delay_cfg[31:24]`, неиспользуемые по осознанным отклонениям `awprot / arprot / wstrb` (D-1 и D-3 в части III), младшие биты байтового адреса. Логика «предупреждение о неподключённом порте — косметика» выглядит разумно и в девяти случаях из десяти верна.

Как проверяли. Вместо того чтобы отмахнуться, прошли по цепочке от бита статуса до его источника — три звена, каждое проверяемо по источнику. Первое: `spi_reg_if.v:99` формирует

девятый бит регистра STATUS (флаг ERR_RX_OVF) из сигнала rx_fifo_overflow — собственного флага FIFO, который выставляется по условию wr_en && full (spi_fifo.v:74). Второе: spi_engine.v:340-345 показывает, что при заполненном приёмном FIFO движок **не** поднимает rx_wr_en, а выставляет свой флаг err_rx_overflow. Значит условие wr_en && full со стороны движка невозможно и флаг FIFO недостижим. Третье: флаг движка приходит на порт spi_reg_if.eng_err_rx_ovf — и в теле модуля не читается ни одним выражением. Ровно про этот порт и говорило предупреждение.

Затем предположение проверили экспериментом. Тест 26 в sim/tb_spi_axi4lite.v устраивает настоящее переполнение: принимает шестнадцать слов в восьмисловный FIFO и не вычитывает их. Лог:

```
A-5 EVIDENCE: STATUS=0000007e RX_FULL=1 ERR_RX_OVF=0 IRQ_ERROR=1
```

Объяснить с 

Переполнение произошло — RX_FULL=1. Бит ERR_RX_OVF остался нулём. При этом условие видно через прерывание: IRQ_ERROR=1.

Корневая причина. Бит 9 регистра STATUS не может быть установлен ни при каких условиях: он подключён к источнику, недостижимому в этой схеме, а достижимый источник никуда не подключён. Это дефект **A-5**, и он структурный — живёт в неизменных файлах spi_reg_if.v и spi_engine.v, значит присутствует в Altera-версии в том же виде. Мы его не привезли с собой, мы его нашли. Ошибка при этом не теряется полностью: spi_master_top.v:69-70 включает флаг движка в источник события ошибки, поэтому переполнение видно через IRQ_STATUS.ERROR. Недоступно оно именно через STATUS.

Исправление. Дефект **не исправлен**, и это осознанное решение. Лечится он одной строкой — заменить в spi_reg_if.v:99 источник бита 9 на rx_fifo_overflow | eng_err_rx_ovf. Но правка меняет наблюдаемое поведение регистра: там, где был ноль, появится единица, а это разрушает аргумент эквивалентности. Поэтому дефект задокументирован, зафиксирован тестом, описан в ../register_map.md с обходным путём (детектировать переполнение через IRQ_STATUS.ERROR) и вынесен в список необязательных улучшений под идентификатором O-1. Полный разбор — в части II.

Правило на будущее. Не глушите предупреждения массово. Проведите перепись: сгруппируйте по коду, посчитайте и по каждой группе напишите одну строку — почему она допустима. В нашей сборке из семи классов пять существуют исключительно из-за режима out-of-context (портов-падов в нём нет, поэтому и претензии к ним неуместны), один — намеренное следствие отклонения D-3 (константные коды ответа AXI), и ровно один потребовал внимания по существу. Именно он и привёл к находке. Стоимость переписи — час; стоимость пропущенного дефекта — драйвер, который не замечает потерю принятых данных.

6.9. Что ещё не переносится автоматически: constraints

Кроме кода есть временные ограничения, и с ними ситуация обратная: SDC от Quartus и XDC от Vivado похожи настолько, что провоцируют на механический перевод. Это ловушка: часть ограничений исходного проекта **семантически неверна** на новой платформе, а некоторые были сомнительны и на старой.

create_clock на входные 50 МГц переносится по смыслу — это объявление реального внешнего блока, осмысленное везде. А вот create_generated_clock на spi_sclk не переносится вовсе; отклонение зафиксировано под идентификатором **C-2**. Причин три. Коэффициент деления не константа: CLK_DIV — регистр, программируемый во время работы, а в SDC стоит

фиксированное `-divide_by 10`. Сам `spi_sclk` не тактирует ни одного триггера — это обычный регистровый выход, и объявить его клоком значит создать фиктивный домен, а потом закрывать порождённые им межклоковые пути исключениями. Наконец, заявленный как пессимистичный делитель 10 на деле оптимистичен: при `CLK_DIV = 2` реальный делитель равен шести.

`set_input_delay` на `spi_miso` заменён объявлением входа асинхронным — отклонение **C-3**. Исходный SDC трактует MISO как source-synchronous вход, привязанный к SCLK, но архитектура IP сознательно отказалась от этой привязки в пользу синхронизатора. Объявлять соотношение, которым дизайн не пользуется, — значит получать бессмысленные отчёты временного анализа. Цена честности названа прямо: анализатор не проверяет тракт MISO, и корректность приёма гарантируется правилом `CLK_DIV >= 2`, а не ограничением. Потому предел A-3 и зафиксирован тестом 25.

Общее правило: ни один `false_path` и ни один `multicycle` не копируется «потому что так было». Каждое исключение обязано заново заслужить право на существование, а необоснованное исключение опаснее его отсутствия — оно даёт ложное закрытие временных требований, то есть отчёт, в котором всё зелёное, а железо не работает. Подробный разбор перевода QSF и SDC в XDC — в части IV.

6.10. Вывод главы

Аудит — механическая процедура с записанным результатом, а не воспоминание. Он дал шесть попаданий, ни одно из которых не оказалось примитивом в синтезируемом коде, и это полноценный факт: целый этап работ вычеркнут заранее. Отрицательный результат по ядру ничего не говорит об обвязке, поэтому список работ M-1...M-6 вырос из разбора пинов, шины, ограничений и периферии. Эквивалентность доказана тремя слоями и механизм оказался достаточно строгим, чтобы позже честно показать: три файла перестали быть идентичными, и произошло это в части IX. Одно предупреждение из пятидесяти пяти привело к находке дефекта A-5 — достаточная причина никогда не глушить предупреждения.

Глава 7. Дерево модулей: что копируем, что добавляем

7.1. Что было в Altera-проекте

Исходное дерево `../.../rtl/` содержит десять файлов верхнего уровня плюс каталог `lcd/` ещё из девяти. Не всё из этого — «ядро SPI»: проект жил в двух ревизиях Quartus. Ревизия `spi_master` — минимальная, только контроллер: топ отладочной платы, интеграционный модуль, регистровый интерфейс, движок протокола, FIFO в двух экземплярах, синхронизатор сброса и заголовок с картой регистров. Ревизия `spi_lcd` добавляла инициализатор `demo_init`, опросчик touch-контроллера `spi_probe`, топ `spi_lcd_fpga_top` и подсистему `rtl/lcd/*` — девять модулей вывода на RGB-панель (тайминги развёртки, знакогенератор 5×7, индикаторы, метки, генератор узоров).

Модуль Altera	Роль	Vendor-specific	Судьба при переносе
<code>spi_master_top</code>	сборка <code>reg_if</code> + 2×FIFO + engine	нет	байт-в-байт
<code>spi_reg_if</code>	регистры, sticky-флаги, IRQ	нет	байт-в-байт

spi_fifo	синхронный FIFO с флешем	нет	байт-в-байт
spi_engine	FSM протокола: SCLK, MOSI, CS, MISO	нет	+ ASYNC_REG (M-1)
spi_reset_sync	2FF: async assert, sync deassert	нет	+ ASYNC_REG (M-1)
spi_defs.vh	карта битов и смещений	нет	байт-в-байт
spi_master_fpga_top	топ платы + debug-шина ~70 выводов	нет	заменён (M-2)
spi_lcd_fpga_top , rtl/lcd/*	дашборд 480×272, 9 модулей	нет	не переносится (M-3)
demo_init	ROM-инициализация регистров	нет	идея → spi_selftest
spi_probe	опрос XPT2046, вывод координат	нет	идея → spi_selftest

Колонка «Судьба» описывает состояние на момент переноса; о том, как три файла разошлись позже под поток и DMA, сказано в главе 6.7. А вот колонка «Vendor-specific» пустая вся: ни один модуль, включая LCD-подсистему, не содержит вендорских примитивов. Значит, ничто из исключённого не выброшено «потому что технически не переносится» — решения принимались по другим причинам, и их стоит упомянуть.

7.2. Что копируем и почему байт-в-байт — принципиально

Четыре файла ядра перенесены копированием без единой правки, ещё два — с добавлением одного атрибута и комментариев. Резонный вопрос: почему такая педантичность? Разве нельзя было заодно поправить мелочи — привести комментарии в порядок, исправить найденные дефекты, переименовать пару сигналов? Нельзя.

Утверждение «мы перенесли то же самое» — самое ценное, что даёт весь перенос, и его сила прямо зависит от того, насколько дёшево его можно проверить. Пока файл идентичен, проверка стоит одну команду и не требует доверия ни к кому. Как только появляется правка — любая, даже безусловно улучшающая — проверка перестаёт быть механической и превращается в рассуждение: «мы поменяли вот это, вот почему это не влияет на поведение, вот тесты». Рассуждение можно оспорить, в нём можно ошибиться, его надо перепроверять после каждого изменения.

Отсюда жёсткое правило: **сначала доказуемо эквивалентная версия, потом отдельным шагом улучшения**. Именно поэтому все найденные дефекты — A-1 и A-2 (недопустимые part-select при экзотических значениях параметров), A-3 (сдвиг принятого слова при `CLK_DIV = 1`), A-4 (молчаливый отказ записи нуля в делитель), A-5 (мёртвый бит переполнения) и A-6 (неполные назначения выводов в QSF) — описаны, зафиксированы тестами и **не исправлены**. Их исправления вынесены в отдельный список O-1...O-5 с пометкой, какое из них меняет наблюдаемое поведение. Смешать перенос с рефакторингом — значит потерять возможность сказать, что именно сломалось.

Второе решение того же ряда: интерфейс `spi_master_top` сохранён без изменений — те же `wr_en`, `rd_en`, `addr`, `wr_data`, `rd_data`, `ready`, `irq`. AXI строится **над** ним, а не вместо него. Datapath SPI не переписывался ради смены системной шины, и результат измерим: вся AXI-обязка стоит 12 таблиц истинности и 79 триггеров против 405 и 406 у всего IP целиком. Тонкая

прослойка — ровно как и задумывал, когда делал хост-интерфейс собственным.

7.3. Что добавляем: четыре новых модуля

Вся новизна переноса сосредоточена в четырёх файлах.

Новый модуль	Роль	Что заменил или добавил
<code>spi_axi4lite</code>	AXI4-Lite slave → стробы parallel MM	заменил debug-шину как доступ с хоста
<code>spi_zynq_top</code>	PL-only топ для bring-up	заменил <code>spi_master_fpga_top</code>
<code>spi_selftest</code>	автономный «мини-хост» без софта	заменил <code>demo_init</code> + <code>spi_probe</code> + LCD
<code>spi_axis_tx</code>	AXI4-Stream → слова TX FIFO	добавлен позже, для потока и DMA

`spi_axi4lite` — единственная новая шинная логика во всём переносе. Задача звучит просто: принять транзакцию AXI4-Lite и превратить её в один такт строба для регистрового интерфейса. Сложность в слове «один». Регистровый интерфейс ядра — не универсальный файл регистров, у него три чувствительных к времени свойства, которые небрежный адаптер ломает молча: стробы `wr_en` и `rd_en` должны длиться ровно такт (удержание на всю длину рукопожатия AXI выполнит доступ несколько раз), `rd_data` комбинационный и действителен только в такт с поднятым `rd_en`, а чтение `RX_DATA` извлекает слово из FIFO. Автомат обёртки поэтому обслуживает одну транзакцию за раз и формирует стробы декодированием из состояния, так что свойство «ровно один строб на доступ» доказуемо по построению. Тесты 19–21 всё равно написаны. Это работа M-6, подробно — в части III.

`spi_zynq_top` — топ для первого включения: ни процессорной системы, ни AXI, ни единой зависимости от софта. Он опирается на три подтверждённых факта о плате. Свободно бегущий осциллятор 50 МГц на выводе W17, причём вывод clock-capable — поэтому Vivado сам вставит глобальный буфер, и ни MMCM, ни ручной BUFG не нужны. Частота ровно та же, что в референсе Altera, поэтому значения `CLK_DIV` из исходного проекта переносятся без пересчёта. И два светодиода PL — жёсткий предел платы, а не предпочтение. Отдельно перенесён режим `DIAG_PINWALK`, который гонит SCLK, MOSI и CS прямо от свободного счётчика в обход ядра. В Altera-проекте именно этот эксперимент закончил недельную охоту за «пропавшими» сигналами: если выводы шевелятся, физика в порядке (пины, питание банка, преобразователь уровней, место щупа) и виновата логика; если нет — виновата физика. Один пересбор делит пространство поиска пополам.

`spi_selftest` ведёт себя как программа, которой ещё нет: сначала прогоняет фазу конфигурации из ROM (как `demo_init`), потом уходит в бесконечный цикл транзакций (как `spi_probe`), но не с командами XPT2046, а с обобщённым бегущим узором — никакого конкретного слейва мы не предполагаем. Состояние выводится на светодиоды и на сигналы, удобные для внутреннего логического анализатора ILA: `led[0]` — heartbeat, доказывающий, что клок и сброс живы, `led[1]` — эвристический признак ответа на MISO. Эвристика намеренно слабая и честно названа слабой: без слейва MISO стоит на уровне подтягивающего резистора, поэтому принятый байт будет `0x00` или `0xFF`, и признак «пришло что-то другое» свидетельствует об отвечающем устройстве, но не доказывает корректность данных. Важно, что `spi_selftest` работает через **тот же самый** параллельный интерфейс, которым пользовался хост на Altera: если под selftest ядро работает, а под AXI нет — виновата обёртка.

`spi_axis_tx` появился позже остальных: на момент миграции DMA был вынесен за рамки (пункт O-4 — при восьмисловных FIFO и SCLK не выше 8.33 МГц узкого места нет). Когда дошло до

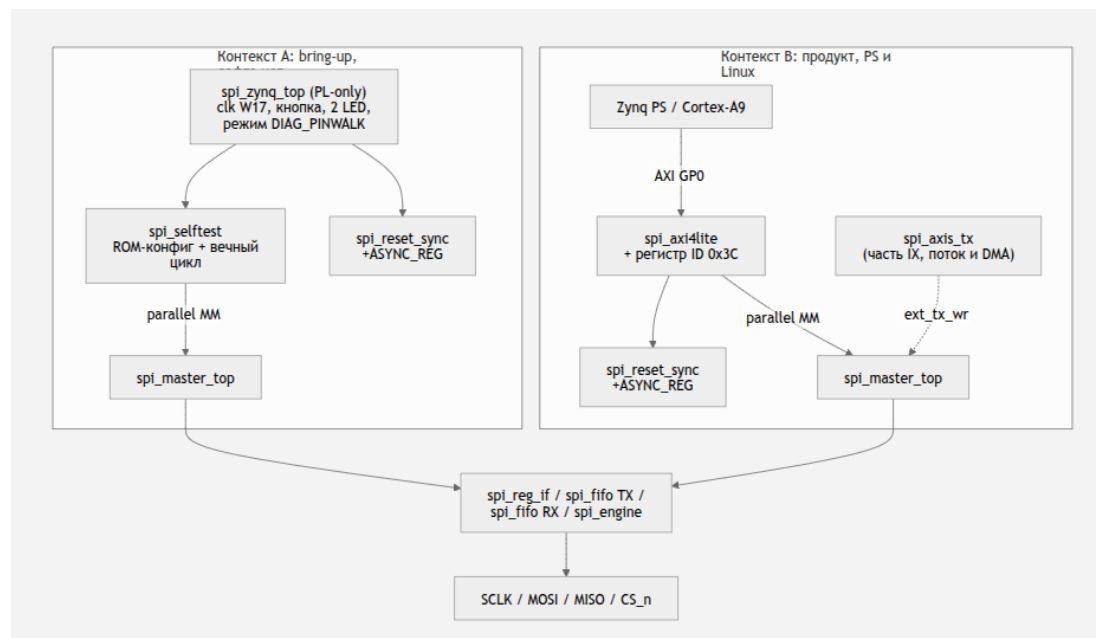
полноэкранный отрисовки на ST7789, узкое место появилось. Модуль принимает биты AXI4-Stream по 32 разряда, раскладывает на байты начиная с младшего, отправляет каждый в TX FIFO отдельным словом с нулевым расширением, а `TLAST` превращает в флаг конца буфера. Именно его регистрируемый сигнал записи породил гонку из главы 41 части IX, лечение которой потребовало `almost_full`.

7.4. Почему два топа, а не один универсальный

Развилка настоящая, и похожий выбор встретится вам в любом проекте с несколькими сценариями использования. Вариант первый — один топ с параметром вроде `USE_AXI`, переключающим источник обращений: либо внешняя шина, либо внутренний selftest; экономно, один файл и одна точка правки. Вариант второй — два независимых топа: `spi_zynq_top` для автономной проверки и `spi_axi4lite` как модуль внутри Block Design с процессорной системой. Выбрали его.

Причина в том, что это не два режима одного изделия, а **два контекста проверки**, и смысл каждого — в том, чего в нём нет. Ценность `spi_zynq_top` целиком в отсутствии AXI, процессорной системы и софта: когда после загрузки битстрима на плате тишина, вы хотите исключить из подозреваемых как можно больше, а не рассуждать, правильно ли встал параметр. Ценность `spi_axi4lite` — в том, что это чистый slave без единой строки отладочной логики. Параметр `USE_AXI` оставил бы в каждой сборке мёртвый код второй ветки: в автономной — неиспользуемые порты AXI, в продуктовой — генератор транзакций, который никто не запускает, но который синтезатор обязан обработать и который выстрелит при ошибке в условии.

Порядок работ получается естественный: сначала вы убеждаетесь, что клочок жив, CS дёргается, SCLK есть, MOSI несёт данные, — и только потом добавляете к списку подозреваемых Block Design, карту адресов, `IRQ_F2P`, драйвер и Device Tree. Цена решения названа прямо: два топа надо поддерживать, и они могут разойтись. Если завтра в ядро добавится порт, подключать его придётся в обоих местах, и забывчивость обнаружится на сборке. Обмен приемлемый: риск расхождения виден при компиляции, а риск запутаться в режимах универсального топа — только на плате.



7.5. Что исключили и почему это не вкусовщина

Из переноса исключили всю подсистему, привязанную к оборудованию старой платы:

`spi_lcd_fpga_top`, девять модулей `rtl/lcd/*`, `demo_init` и `spi_probe`. LCD-модули были

предназначены для RGB-панели 480x272, а `spi_probe` работал с touch-контроллером XPT2046. На TZT RK-ZYNQ7020-F ни такой панели, ни XPT2046 нет, поэтому переносить этот код не имело смысла.

Для автономной проверки SPI вместо старой демонстрационной логики появился `spi_selftest`. Он сам настраивает контроллер и выполняет тестовые транзакции, а состояние можно наблюдать по светодиодам и через ILA. Позже для практической проверки передачи используется штатный дисплей ST7789, подключенный по SPI.

При этом ST7789 не подходит для полной проверки контроллера: у него нет линии MISO, поэтому через него можно проверить передачу по `SCLK`, `MOSI` и `CS`, но нельзя проверить прием данных. Поэтому для первоначального bring-up используется отдельная петля `MOSI` -> `MISO`.

7.6. Почему старый top не переносится

Симптом. Ядро уже было собрано в `spi_master_fpga_top`, поэтому сначала кажется, что достаточно добавить этот файл в проект Vivado и заменить QSF на XDC.

Что оказалось. Само ядро для этого действительно менять не требуется, но старый top описывает другой способ подключения хоста. Он выводит наружу внутренний параллельный регистровый интерфейс `addr`, `wr_data`, `rd_data`, `wr_en`, `rd_en` и `ready`. На Zynq этот интерфейс наружу не нужен: управляющим хостом становится Cortex-A9 в Processing System, а доступ из PS к пользовательской логике выполняется через AXI.

Поэтому `spi_master_top` сохраняется, а верхний уровень заменяется. В рабочей системе между PS и ядром появляется `spi_axi4lite`, преобразующий транзакции AXI4-Lite в исходные однократные обращения к регистровому интерфейсу. Для автономного bring-up ту же роль хоста выполняет `spi_selftest`.

При аудите старого проекта дополнительно обнаружился дефект A-6: в placeholder-QSF для 65 top-level сигналов был задан `IO_STANDARD`, но отсутствовал `set_location_assignment`. Quartus поэтому мог автоматически назначить им свободные выводы корпуса. Практического влияния дефект не имел, поскольку эта ревизия не предназначалась для загрузки на реальную плату, но использовать ее QSF как источник распиновки новой платы тем более нельзя.

Что сделали. `spi_master_top` и его внутренний регистровый интерфейс сохранили. Старый `spi_master_fpga_top` заменили двумя вариантами интеграции:

- `spi_axi4lite` - управление от Cortex-A9 через AXI4-Lite;
- `spi_selftest` + Zynq top - автономная проверка без PS и Linux.

Распиновка внешних сигналов SPI, такта, сброса и диагностики для новой платы сформирована заново.

Правило на будущее. При переносе FPGA IP следует разделять ядро и board-level top. Внутренний интерфейс IP можно сохранить без изменений, тогда как верхний уровень определяется архитектурой конкретной системы и способом подключения хоста.

7.7. Порядок работ при миграции IP: как повторить у себя

Соберём опыт этой части в последовательность, применимую к любому переносу между вендорами. Порядок не декоративный: каждый шаг опирается на результат предыдущего, и перестановка шагов — источник потерянного времени.

1. **Зафиксируйте эталон.** Исходное дерево лежит рядом и не трогается: всё доказательство эквивалентности строится на возможности сравнить с ним.
2. **Проведите аудит зависимостей.** Регулярка по всем файлам, включая проектные и constraints, плюс ручной чек-лист по тому, что регуляркой не ловится.
3. **Классифицируйте каждое попадание** — комментарий, настройка проекта, примитив — и запишите результат, даже если он отрицательный.
4. **Отдельно аудируйте glue.** Пины, шина к хосту, временные ограничения, периферия платы, цепочка загрузки. Именно здесь окажется объём работ.
5. **Составьте план с идентификаторами и рисками.** Всё, что меняет наблюдаемое поведение, выносите в необязательное — иначе оно разрушит эквивалентность.
6. **Перенесите ядро копированием.** Минимум правок, каждая объявленная: атрибуты синтеза допустимы, изменения логики — нет.
7. **Постройте доказательство слоями:** контрольные суммы, фильтрованный diff, сравнение вердиктов тестов. Одна команда, ненулевой код при провале.
8. **Напишите обвязку и топ под новую плату** — отдельно для bring-up, отдельно для интеграции.
9. **Constraints — с нуля.** Ни одного исключения по инерции; каждое обязано заново заслужить право на существование.
10. **Проведите перепись предупреждений.** По строке обоснования на класс: здесь находятся дефекты, которые не находят тесты.

Дальше — часть II: как устроено ядро изнутри, почему один тактовый домен упрощает всё, и почему `CLK_DIV = 1` и мёртвый бит `ERR_RX_OVF` заслуживают отдельных детективов.

Часть II. Архитектура ядра SPI

В части I мы выяснили главное: ядро SPI почти целиком *vendor-neutral*, и весь перенос RTL уместился в две строки атрибутов. Вывод приятный, но ничего не объясняющий. Почему ядро оказалось таким переносимым? Что в нём устроено правильно, а что — унаследованно криво? Эта часть отвечает на оба вопроса изнутри: тактирование и сброс, четыре модуля и их контракты, карта регистров и, в финале, шесть дефектов, найденных при аудите. Читать стоит с открытым исходником в соседнем окне: всё, о чём мы говорим, лежит в `../..//rtl/`.

Глава 8. Один тактовый домен, SCLK как enable

8.1. Слово, которое решает судьбу переноса

Начнём с термина, без которого дальше нельзя. **Тактовый домен** (clock domain) — это множество триггеров, которые тактируются от одного и того же сигнала. Триггер (flip-flop) запоминает значение на входе в момент фронта тактового сигнала и держит его до следующего фронта. Если два триггера тактируются одним `clk`, они шагают в ногу: инструмент может посчитать, успевает ли сигнал добежать от одного к другому за период. Если источники разные, «успевает» становится бессмысленным словом — фронты плывут друг относительно друга, и никакой расчёт не спасает.

В нашем IP тактовый домен ровно **один** — системный `clk`. На Zynq он называется `s_axi_aclk` и приходит из PS как `FCLK_CLK0`: 50 МГц в PL-only сборке, 100 МГц в сборке с DMA. Все без

исключения триггеры дизайна тактируются от него.

А что же `spi_sclk` — сигнал, который на разъёме выглядит как «часы SPI»? Он тоже живёт в этом домене, но не как клок, а как **обычный регистровый выход**. Посмотрите объявление и присваивание в `../rtl/spi_engine.v`:

```
output reg          spi_sclk,
...
always @(posedge clk or negedge rst_n) begin
    ...
    sclk_r    <= ~sclk_r;
    spi_sclk <= ~sclk_r;
```

Объяснить с 

Присваивание стоит внутри `always @(posedge clk)`, значит `spi_sclk` — выход триггера, переключаемый системным клоком. Для внешнего устройства он выглядит как тактовый сигнал; для временного анализа внутри кристалла он такой же сигнал данных, как `spi_mosi` или `spi_cs_n`.

Проверить это можно механически: поиск `posedge sclk` и `negedge sclk` по каталогу `../rtl/` не находит ничего, совпадения есть только в `../sim/tb_spi_slave_ref.v` — в поведенческой модели слейва, которая в железо не попадает. Запомните фразу целиком: «ни один синтезируемый триггер не тактируется от `spi_sclk`». Она спасает от половины ошибок в `constraints` и от всех попыток «поставить BUFG на SCLK».

8.2. Метастабильность и CDC: почему одного домена мало не бывает

У триггера есть два требования по времени: сигнал на входе должен установиться за некоторое время **до** фронта клона (`setup time`) и не меняться некоторое время **после** фронта (`hold time`). Если оба выполнены, триггер надёжно защёлкивает ноль или единицу. Если сигнал переключился ровно в запретном окне, триггер попадает в **метастабильное** состояние: его выход зависает между логическим нулём и единицей и разрешается в одну из сторон через непредсказуемое время. Аналогия — подброшенная монета: почти всегда она ляжет орлом или решкой, но существует ненулевая вероятность, что она встанет на ребро и будет качаться. Запретить ей это нельзя, можно только подождать, пока упадёт, и не принимать решение раньше.

Внутри одного домена метастабильности не бывает: все сигналы меняются сразу после фронта и успевают устояться к следующему. Проблема возникает там, где сигнал приходит от источника, никак с нашим клоком не связанного. Такое пересечение называется **CDC** (Clock Domain Crossing — переход между тактовыми доменами), и лечится оно **синхронизатором**: цепочкой из двух триггеров подряд. Первый может уйти в метастабильность, но у него есть целый период, чтобы «упасть с ребра»; второй захватывает уже устоявшееся значение.

Ключевой вывод для переноса: внутри нашего IP пересечений доменов **нет** — ни между AXI и движком SPI, ни между «часами SPI» и логикой, потому что часов SPI внутри кристалла не существует. Это подтверждено полным поиском: нет `pulse-CDC`, нет многобитных шин между доменами, нет асинхронных FIFO, не нужны Gray-счётчики. Асинхронных сигналов всего два, и оба — **входы**, а не пересечения: `spi_miso` и `reset_n` (раздел 8.6).

8.3. Откуда тогда берётся частота SPI

Если SCLK — просто регистровый выход, кто-то должен решать, когда его переключать. Этим занят счётчик полупериода `half_cnt` в `../rtl/spi_engine.v`; логика умещается в несколько строк состояния `S_XFER`:

```

if (half_cnt >= clk_div) begin
    half_cnt <= 16'd0;
    sclk_r    <= ~sclk_r;
    spi_sclk <= ~sclk_r;
    edge_cnt <= edge_cnt + 7'd1;
    ...
end else begin
    half_cnt <= half_cnt + 16'd1;
end

```

Объяснить с 

Прочитаем по тактам. После очередного фронта SCLK счётчик обнулён; дальше он сравнивается с `clk_div` и, пока меньше, инкрементируется. Как только сравнение выполнилось, происходит переключение: `sclk_r` инвертируется, `edge_cnt` увеличивается, счётчик снова обнуляется. При `CLK_DIV = 2` картина по тактам системного блока такая:

такт	1	2	3	4	5	6	7	8	9
half_cnt	0	1	2	0	1	2	0	1	2
фронт	.	.	X	.	.	X	.	.	X
SCLK									

|<-- 3 такта -->|

Объяснить с 

Между двумя фронтами проходит `CLK_DIV + 1` тактов — это и есть полупериод. Полный период SCLK вдвое длиннее, откуда формула, которую надо выучить:

$$f_{SCLK} = f_{clk} / (2 \cdot (CLK_DIV + 1))$$
Объяснить с 

При 50 МГц и `CLK_DIV = 2` получаем 8.33 МГц. При `CLK_DIV = 4` — ровно 5 МГц. При `CLK_DIV = 24` — 1 МГц, удобное значение для логического анализатора, и именно оно стоит по умолчанию в `spi_selftest`. Нижний край задаёт разрядность поля: `clk_div` шестнадцатитрёхбитный, значит максимум 65535, что при 50 МГц даёт около 381 Гц.

Обратите внимание на пару `sclk_r / spi_sclk`. Это не дублирование ради красоты: `sclk_r` — внутренняя копия уровня, по которой вычисляется следующий, а `spi_sclk` — то, что уходит на вывод. Оба обновляются в одном такте одним и тем же выражением, поэтому расхождения нет, зато выходной сигнал гарантированно снят с триггера и не имеет комбинационного хвоста. Из-за этого глитчи (короткие паразитные импульсы при переключении комбинационной логики) на SCLK невозможны по построению. То же верно для `spi_cs_n` и `spi_mosi` — все три объявлены как `output reg`.

8.4. Clock enable style: что выигрываем и чем платим

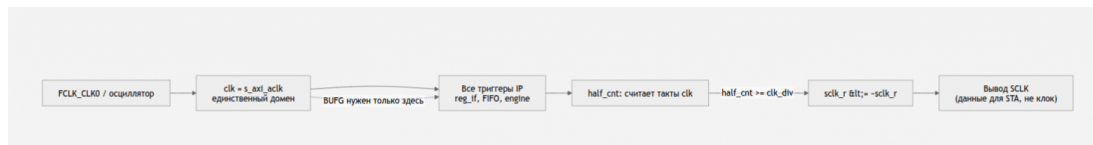
Приём, при котором вся логика сидит на быстром клоке, а «медленные» события разрешаются счётчиком, называется **clock enable style**. Альтернатива — сделать SCLK настоящим клоком: пропустить его через глобальный тактовый буфер и тактовать им часть триггеров. Второй путь выглядит естественнее для новичка («это же часы, пусть тактуют»), но на переносе он обходится дороже.

Что даёт нам первый путь. Не нужен глобальный буфер: на 7-series сигнал, попадающий на

тактовые входы триггеров, обязан идти через BUFG/BUFR, иначе перекося по кристаллу окажется неконтролируемым. Наш SCLK на тактовые входы не попадает, и в отчёте `report_clock_utilization` клоковых примитивов, кроме буфера системного клока, нет вовсе. Не нужен второй домен, а значит не нужны ни синхронизаторы между «SPI clock» и AXI, ни асинхронные FIFO. Не нужен `create_generated_clock` в constraints — и это не мелочь: коэффициент деления программируемый, диапазон 1...65535, так что любое фиксированное `divide_by N` было бы заведомо неверной моделью.

Чем платим. Расплата ровно одна, зато существенная: **SCLK не может быть быстрее, чем позволяет системный клок и счётчик**. Минимальный делитель, который регистровый интерфейс вообще принимает, равен единице (ноль отвергается — см. дефект A-4), значит физический потолок — $f_{clk} / 4$, то есть 12.5 МГц при 50 МГц. Но и его использовать нельзя: приём на делителе 1 испорчен из-за задержки синхронизатора MISO. Практический потолок — $f_{clk} / 6$, то есть **8.33 МГц**. Это дефект A-3, главная история этой части, и разбирается он в главе 11.

Второй, менее очевидный платёж — разрешение по частоте. Делитель целочисленный, поэтому доступен только ряд $f_{clk} / 6$, $f_{clk} / 8$, $f_{clk} / 10$ и так далее. Драйвер обязан округлять делитель **вверх** (то есть частоту вниз): просить 4 МГц и получить 4.17 значит превысить запрошенную скорость, чего подсистема SPI в Linux не допускает.



8.5. Сброс: асинхронная установка, синхронное снятие

Сброс (reset) переводит все триггеры в известное состояние. В нашем IP он активен низким уровнем и называется `rst_n` — черта в имени по традиции означает инверсию. Тип сброса выбран такой: **асинхронная установка, синхронное снятие** (async assert / sync deassert). Асинхронная установка означает, что сброс срабатывает немедленно, не дожидаясь фронта клока; в Verilog это записывается добавлением сигнала в список чувствительности:

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state    <= S_IDLE;
        spi_cs_n <= {NUM_CS{1'b1}};
    end
end
```

Объяснить с

Это полезно, когда клок ещё не запущен или пропал: схема всё равно окажется в предсказуемом состоянии. Проблема начинается при **снятии** сброса. Если `rst_n` отпустить в произвольный момент, часть триггеров успеет выйти из сброса по ближайшему фронту, а часть не успеет, и автомат стартует наполовину сброшенным. Формально это нарушение времён **recovery/removal** — тех же setup/hold, только для входа сброса.

Лечение стандартное и живёт в `../rtl/spi_reset_sync.v` целиком:

```
(* ASYNC_REG = "TRUE" *)
reg [STAGES-1:0] sync_chain;

always @(posedge clk or negedge rst_n_async) begin
    if (!rst_n_async)
```

```

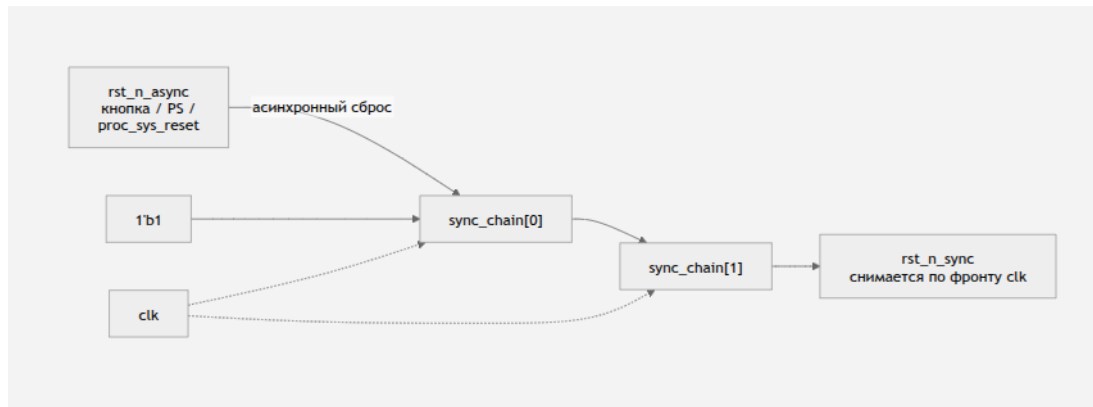
    sync_chain <= {STAGES{1'b0}};
  else
    sync_chain <= {sync_chain[STAGES-2:0], 1'b1};
  end

  assign rst_n_sync = sync_chain[STAGES-1];

```

[Объяснить с](#)

Читается так. Пока внешний сброс активен, вся цепочка держится в нуле — асинхронно, мгновенно, независимо от клок. Как только сброс отпущен, по цепочке начинает продвигаться единица: один такт до первого триггера, ещё один до второго. Наружу выходит `sync_chain[STAGES-1]`, то есть снятие сброса происходит строго по фронту клок и одновременно для всех потребителей. Экземпляр модуля нужен по одному на тактовый домен; у нас домен один.



Теперь про решение, которое мы **не** приняли. Xilinx рекомендует синхронный сброс: он лучше упаковывается в slice, потому что sync/async задаётся на весь slice целиком. Соблазн «сделать как рекомендует вендор» велик, особенно когда правишь чужой код. Мы отказались по двум причинам. Первая: перевод сброса в синхронный — это **изменение поведения**, а задача переноса состояла в том, чтобы получить доказуемо эквивалентную версию; любая правка семантики обнуляет аргумент эквивалентности. Вторая: выигрыш от упаковки здесь нулевой — дизайн занимает 0.76 % кристалла (405 LUT из 53 200 в ООС-сборке).

Опасение, что «на 7-series асинхронный сброс плохо ложится», проверено, а не принято на веру. Конструкция отображается на примитивы напрямую: FDCE (асинхронный сброс) для триггеров, сбрасываемых в ноль, и FDPE (асинхронная установка) для тех, кто сбрасывается в единицу. Смешивать их можно, потому что значение сброса задаётся на каждый триггер, а sync/async — на slice. Отчёт имплементации это подтверждает: 399 триггеров с асинхронным Reset и 7 с асинхронным Set — последние это `spi_cs_n[3:0]` и компания, ведь CS активен низким уровнем и в сбросе должен быть в единице.

8.6. Ровно два асинхронных входа

Асинхронных входов у IP два, и это исчерпывающий список: `spi_miso` и внешний `reset_n`. Почему именно два — видно из архитектуры. Все остальные входы приходят либо от AXI-обвязки, либо от `spi_selftest`, а оба источника сидят на том же `s_axi_aclk`. MISO приходит от внешней микросхемы со своим кварцем. Линия сброса вообще не имеет отношения к клок.

Оба входа пропущены через двухступенчатые цепочки. Для MISO это `miso_sync` в `../rtl/spi_engine.v`, для сброса — `sync_chain` в `spi_reset_sync.v`. Оба массива помечены атрибутом:

```
(* ASYNC_REG = "TRUE" *)
reg [MISO_SYNC_STAGES-1:0] miso_sync;
wire miso_synced = miso_sync[MISO_SYNC_STAGES-1];
```

Объяснить с 

Атрибут ничего не меняет в логике — Icarus его вообще игнорирует. Он меняет обещание, которое RTL даёт инструменту: не разносит эти триггеры по разным slice, не применяет retiming, не сворачивает цепочку в сдвиговый регистр SRL и учит её как синхронизатор в отчётах. Vivado не обязан распознавать 2FF-цепочку по одной структуре, поэтому без атрибута `report_cdc` классифицировал бы путь иначе. С атрибутом отчёт после разводки говорит «All paths are Safely Timed». Это и есть весь перенос RTL: две строки атрибутов, ноль изменений логики.

И здесь же — цена честного синхронизатора. Он спасает от метастабильности, но **съедает время**: сигнал доходит от вывода до `miso_synced` за два такта, а до точки, где его можно защёлкнуть, за три. Пока полупериод SCLK длиннее этой задержки, всё в порядке; как только вы попытаетесь разогнать SPI до $f_{clk} / 4$, приём начнёт врать. Это дефект A-3 из главы 11.

Глава 9. Блоки: `spi_reg_if` / FIFO / `spi_engine` / `spi_master_top`

9.1. Почему четыре модуля, а не один

Контроллер можно было написать одним файлом на восемьсот строк. Он бы даже заработал — в этом и коварство. Исходный автор разделил его на четыре модуля, и это решение стоит разобрать, потому что именно оно сделало перенос недельной, а не месячной работой.

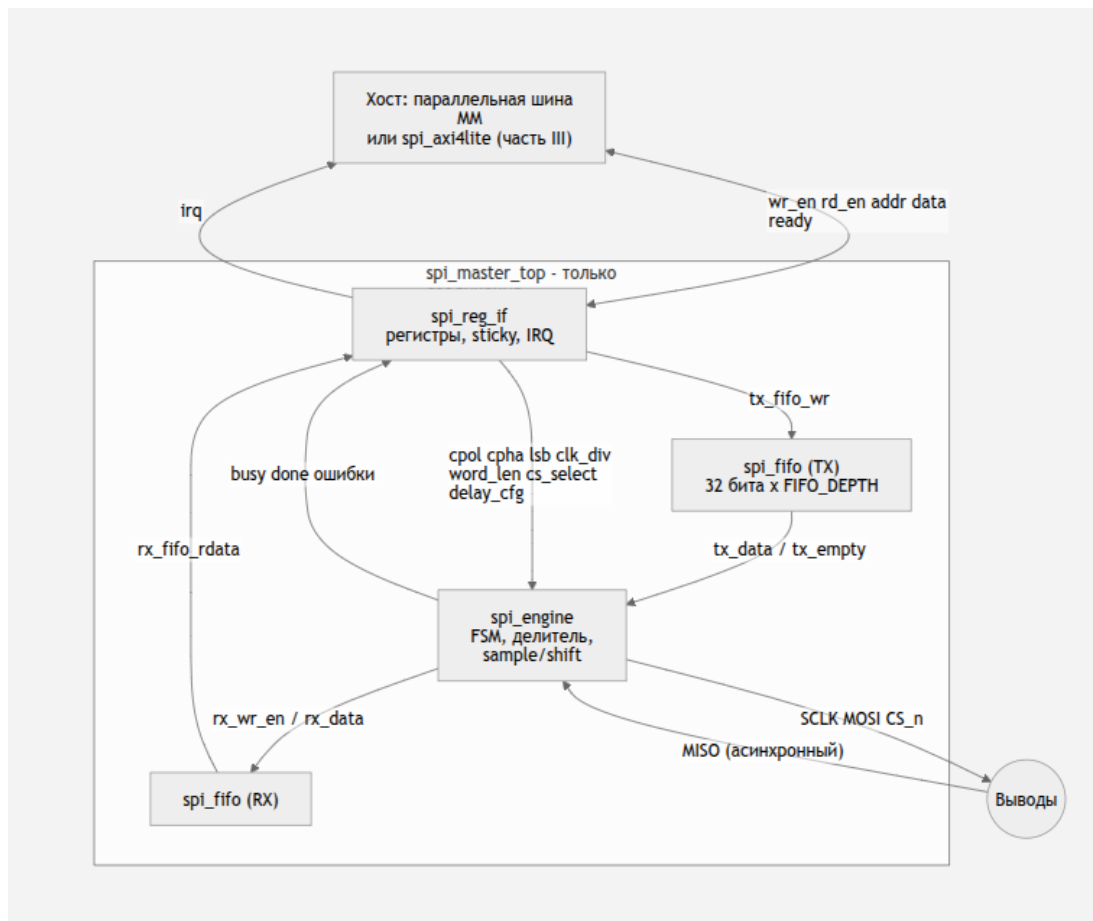
Граница проводится там, где интерфейс тоньше, а семантика по сторонам — различнее.

`spi_reg_if` живёт в мире «адрес — данные — такт»: он разговаривает с хостом, декодирует адреса, помнит sticky-флаги. `spi_engine` живёт в мире «фронты — фазы — задержки»: он не знает, по какому адресу лежит CONTROL, ему на входы приходят готовые уровни `cpol`, `cpha`, `clk_div` и одиночный строб `start`. Между ними проходит десяток проводов конфигурации и три stroba — тоньше в этом дизайне не бывает. `spi_fifo` не знает ни про хоста, ни про SPI, только про свои указатели, и поэтому один и тот же файл встаёт и в TX-, и в RX-тракт без единой правки. `spi_master_top` не содержит логики вовсе.

Что дало бы объединение? Экономия нескольких строк в топе — и потерю трёх вещей сразу.

Первое: тестируемости. Движок с уровневыми входами гоняется в тестбенче напрямую, без эмуляции шинных транзакций, а регистровый файл проверяется чтением-записью без единой SPI-передачи; когда падает тест, вы сразу знаете, в каком из миров искать. Второе: возможности заменить шину. Именно на этой границе выросла AXI-обвязка (часть III) — она надета **поверх** `spi_master_top`, а не вместо него, и внутренний datapath не переписывался вообще. Третье: локальности сложности. Отлаживая CPHA, вам не нужно держать в голове декодирование адресов.

Цифры реальной имплементации подтверждают, что «клей» действительно тонкий. По иерархии `spi_engine` — 203 LUT и 165 триггеров, `spi_reg_if` — 102 LUT и 142 триггера, RX FIFO — 56 LUT, 22 LUTRAM и 9 триггеров, а собственная логика AXI-обвязки поверх всего этого занимает 12 LUT и 79 триггеров. Двенадцать LUT — вот сколько стоит переезд с параллельной шины на AXI4-Lite, когда архитектура нарезана правильно.



9.2. spi_reg_if — дипломат

Этот модуль — единственный, кто разговаривает с хостом. Он хранит конфигурационные регистры, собирает статус со всего контроллера в одно слово и превращает события в прерывание. Про SPI он не знает ничего. Приёмы, которые здесь используются, типовые для любого регистрового файла, и их стоит узнать в лицо.

Комбинационное чтение. Мультиплексор чтения собран в блоке `always @(*)`, то есть значение появляется на `rd_data` в том же такте, когда поднят `rd_en`, без единого триггера на пути. Это даёт нулевую задержку чтения для процессора. Плата — жёсткое требование к мосту: данные надо захватить именно в том такте, когда строб активен. Если `rd_en` не поднят, `rd_data` равен нулю, а не последнему прочитанному значению.

Строб длиной ровно в один такт. И `wr_en`, и `rd_en` поднимаются на один такт на одно обращение. Строб (strobe) — импульс-событие, в отличие от уровня, который держится. Разница принципиальна: уровень можно поддержать «на всякий случай», строб — нельзя, потому что каждый такт активного стога считается отдельным обращением.

Sticky-флаг. Флаг называется `sticky` («липкий»), если он взводится событием и держится до явной очистки, даже если само событие давно прошло. Аналогия — сработавший автомат в электрощитке: ток уже не течёт, но рычажок остался внизу. Так устроены `stat_done`, `sticky_tx_ovf` и все биты `irq_status`. Без `sticky` однократное событие вроде `done_pulse` хост просто не увидит: он читает регистры раз в тысячи тактов.

W1C. Способ очистки `sticky`-флагов: `write-1-to-clear`, «запиши единицу, чтобы сбросить». Хост пишет в регистр маску, и те биты, где стоит единица, сбрасываются, а остальные не трогаются. В нашем IP это `ERROR_CLR`:

```
`SPI_ADDR_ERROR_CLR: begin
    stat_done    <= 1'b0;
    irq_status   <= irq_status & ~wr_data;
    sticky_tx_ovf <= 1'b0;
    clr_errors   <= 1'b1;
end
```

Объяснить с 

Почему не обычная запись нуля? Потому что между чтением статуса и записью нуля может прийти новое событие, и оно потеряется. W1C сбрасывает ровно то, что хост уже видел.

Self-clearing строб. Бит `START` в `CONTROL` не хранится: хост пишет единицу, а сбрасывается бит сам, когда движок событие заметил:

```
if (eng_busy || eng_done_pulse)
    ctrl_start <= 1'b0;
```

Объяснить с 

Строб держится, пока движок либо поднимет `busy`, либо выдаст `done` (второе случается, когда `START` отвергнут из-за ошибки — например, при пустом TX FIFO). Никакой защёлки `start_seen` внутри движка нет: однократность обеспечивает регистровый интерфейс. Отсюда приятная мелочь: одна запись в `CONTROL` может одновременно включить контроллер и запустить транзакцию, потому что `START` принимается, если `EN` уже стоит **или** устанавливается этой же записью.

Прерывание. Линия `irq` — регистровая, то есть поднимается на такт позже события:

```
irq_out <= ctrl_irq_en & (|irq_active);
```

Объяснить с 

где `irq_active = irq_status & irq_mask`. Три источника: `DONE`, `RX_VALID` и `ERROR`. Семантика уровневая, а не импульсная: линия держится, пока есть неквитированные разрешённые события. Для драйвера это важно — уровневое прерывание, которое нельзя быстро квитировать, требует маскирования прямо в обработчике, иначе получится interrupt storm (../hw_sw_contract.md , F-2).

9.3. spi_fifo — склад

FIFO (First In, First Out) — очередь: слова выходят в том же порядке, в котором вошли. Наш вариант одноканальный (запись и чтение в одном домене), глубина задаётся параметром `ADDR_WIDTH` и всегда есть степень двойки: `FIFO_DEPTH = 8` в PIO-сборке и обычно 1024 в сборке с DMA (часть IX).

Устройство простое до изящества. Есть массив `mem`, указатель записи `wr_ptr` и указатель чтения `rd_ptr`, и оба они **на один бит шире**, чем нужно для адресации массива. Этот лишний бит — весь секрет:

```
assign count    = wr_ptr - rd_ptr;
assign full     = (count == DEPTH[ADDR_WIDTH:0]);
assign empty    = (count == { (ADDR_WIDTH+1){1'b0} } );
```

[Объяснить с](#) 

Если бы указатели были обычной ширины, состояния «пусто» и «полно» выглядели бы одинаково: `wr_ptr == rd_ptr`. Лишний разряд позволяет их различить — разность указателей даёт настоящее количество слов, от нуля до `DEPTH`.

Выход данных **комбинационный**: `rd_data = mem[rd_ptr]`, то есть на выходе всегда стоит голова очереди, без строба и без задержки. Это часть контракта, а не деталь реализации: движок в состоянии `S_LOAD` защёлкивает `tx_data` напрямую, ожидая, что там уже лежит нужное слово. FIFO с регистровым выходом формально тоже FIFO, но с этим движком не соединится. Побочный эффект идиомы: Vivado отображает такое хранилище на распределённую память LUTRAM (44 LUT в ООС-сборке), тогда как Quartus выводил обычные регистры; RTL идентичен, отличается только инференс.

Флаги ошибок тоже sticky: `overflow` взводится при попытке записи в полный буфер, `underflow` — при чтении из пустого, и оба держатся до `flush` или сброса. Сама запись при этом подавлена гейтом `write_now = wr_en && !full`: слово теряется, но очередь не портится. Отдельного упоминания заслуживает `almost_full`:

```
assign almost_full = (count >= (DEPTH - 2));
```

[Объяснить с](#) 

Это не украшение, а заплатка на реальную гонку. Производитель, который регистрирует свой `wr_en` (так делает AXIS-адаптер `spi_axis_tx`), видит флаг `full` на такт позже, чем тот стал истиной, и его отложенная запись приходит в уже полный буфер, где её отбрасывает тот же гейт. Внешне это выглядело как горизонтально сжатая картинка на ST7789 при работе через AXI DMA. Лечение — тормозить внешнего производителя на две позиции раньше, поэтому `spi_master_top` выводит наружу именно `almost_full`. История целиком — в [части IX](#).

9.4. spi_engine — работага

Здесь живёт весь протокол. Разберём модуль по слоям: данные, классификация фронтов, автомат.

Как хранится слово. Обычно SPI-мастер пишут на сдвиговом регистре: слово сдвигается влево, наружу уходит старший бит. Наш движок держит слово неподвижно и адресует нужный бит функцией:

```
function automatic [4:0] bit_pos;
    input [5:0] cnt;
    input [5:0] total;
    input      lsb;
    begin
        if (lsb) bit_pos = cnt[4:0];
        else    bit_pos = total[4:0] - 5'd1 - cnt[4:0];
    end
endfunction
```

[Объяснить с](#) 

Для MSB-first нулевой по счёту бит — старший (`total-1`), для LSB-first — младший. MOSI берётся как `tx_word[bit_pos(bit_cnt, ...)]`, принятый бит кладётся в `rx_word[bit_pos(bit_cnt, ...)]`. Выигрыш в том, что исчезает классическая беда сдвиговых конструкций — рассогласование выравнивания при смене порядка бит и при слове короче

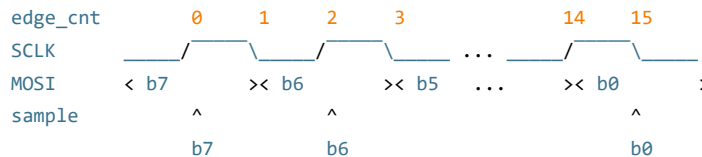
регистра. Приём дополнительно выравнивается вправо функцией `pack_rx`. Допустимые длины слова — только 8, 16, 24 и 32; проверка записана явным перечислением в `word_len_ok`, и попытка задать иное поднимает sticky-флаг `ERR_WORD_LEN` вместо выдачи мусора в эфир.

Как классифицируются фронты. Счётчик `edge_cnt` считает фронты SCLK внутри слова: их ровно $2 \cdot \text{WORD_LEN}$. Дальше — самая изящная строчка модуля:

```
wire do_sample = (edge_cnt[0] == cpha);
wire do_shift  = (edge_cnt[0] != cpha);
```

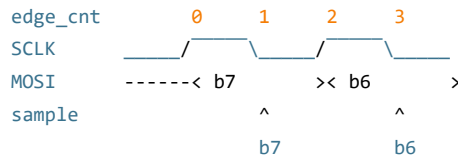
Объяснить с 

Чётность номера фронта прямо кодирует, ведущий это фронт (leading — уход от уровня покоя CPOL) или замыкающий (trailing — возврат к нему). А выбор между «сэмплировать» и «выдвигать» по стандарту SPI зависит ровно от CPHA. Поэтому вся таблица четырёх режимов сворачивается в одно сравнение, без `case` и дублирования веток. Как это выглядит во времени в режиме 0 (CPOL = 0, CPHA = 0), 8 бит, MSB-first:



Объяснить с 

Первый бит MOSI выставлен **до** первого фронта — в `S_LOAD`, строкой `spi_mosi <= tx_data[first_bit_idx]`. Так и должен вести себя CPHA = 0: слейв защёлкнет данные по ведущему фронту, значит к этому моменту они обязаны быть готовы. Режим 1 (CPOL = 0, CPHA = 1) отличается сдвигом на полпериода:



Объяснить с 

Здесь в `S_LOAD` MOSI выставляется в ноль, первый бит выдвигается по первому ведущему фронту, а сэмплирование идёт по замыкающим. CPOL меняет только уровень покоя: при CPOL = 1 картинка та же вверх ногами. Независимо от режима слово заканчивается после последнего замыкающего фронта, и SCLK возвращается к уровню CPOL. Между словами в burst он тоже стоит на CPOL — непрерывного блока IP не выдаёт.

Мелочь, объясняющая одну строчку с виду избыточного кода:

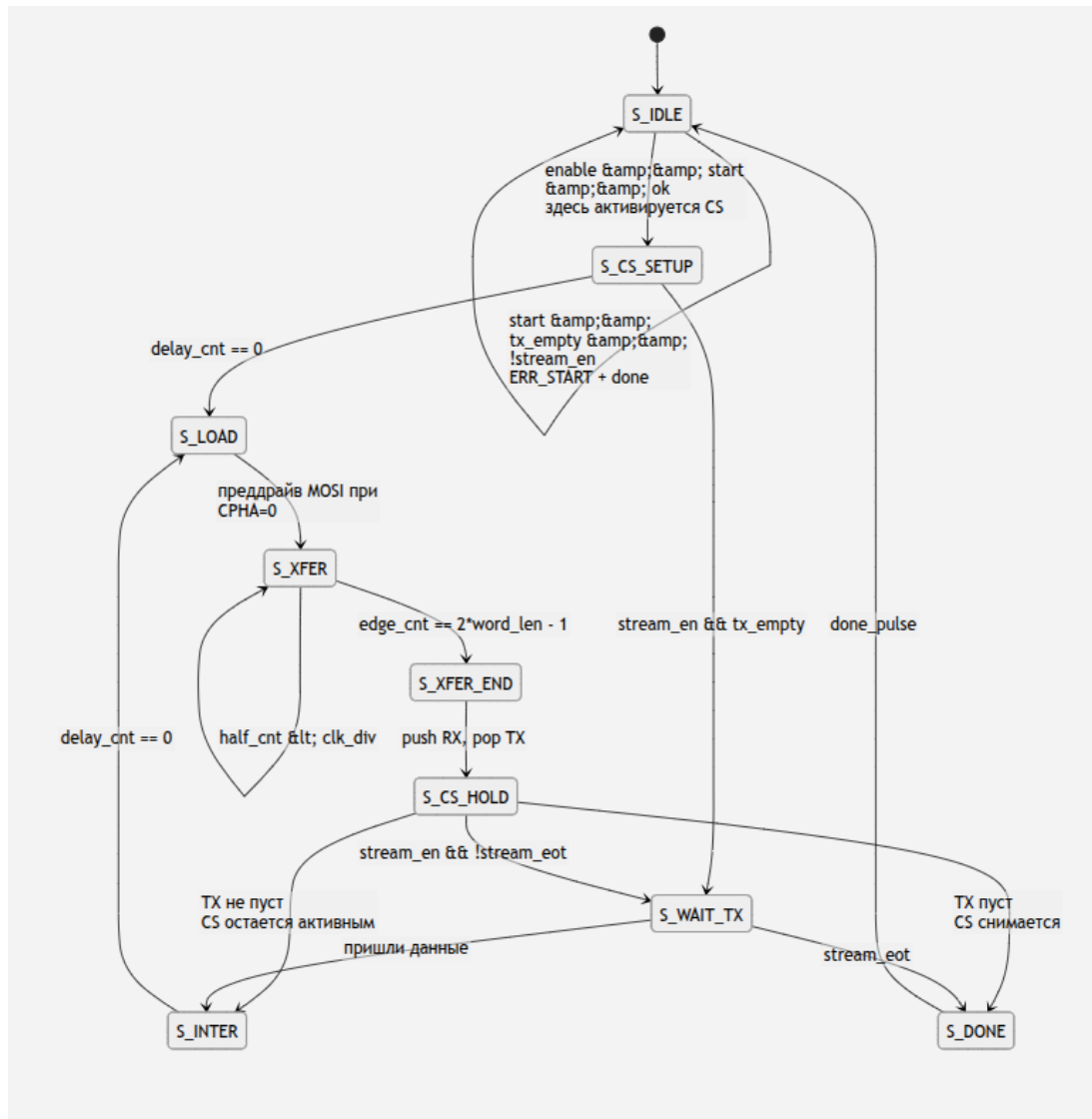
```
if (do_shift && (bit_cnt < bits_total)) begin
```

Объяснить с 

На последнем замыкающем фронте все биты уже сосчитаны, `bit_cnt == bits_total`, и выдвигать нечего; без охраны функция `bit_pos` посчитала бы позицию за границей слова. Такие

условия — признак кода, который писали, глядя на волны.

Автомат. Девять состояний, четырёхбитное кодирование, обязательная ветка `default`, возвращающая в `S_IDLE`.



Пройдём обмен по шагам. В `S_IDLE` движок держит холостые уровни: SCLK на CPOL, все CS в единице, `busy` снят. Приходит `start` при поднятом `enable` — и сразу выполняются две проверки. Если длина слова недопустима, взводится `err_word_len` и выдаётся `done_pulse`; передачи не будет. Если TX FIFO пуст (и это не потоковый режим), взводится `err_start_empty` и снова `done_pulse`. Оба отказа дают завершение — именно поэтому строб `START` снимается и по `done`, а не только по `busy`. Если проверки пройдены, движок поднимает `busy`, запоминает длину слова, активирует выбранный CS (если индекс в диапазоне) и уходит в `S_CS_SETUP`.

В `S_CS_SETUP` досчитывается пауза «CS опущен → первый фронт SCLK», её величина берётся из младшего байта `DELAY_CFG` в тактах системного блока. Затем `S_LOAD`: слово из головы TX FIFO защёлкивается в `tx_word`, обнуляются счётчики, при `CPHA = 0` предвыставляется первый бит MOSI. Дальше `S_XFER` — цикл, разобранный выше по тактам. Выход из него — по последнему фронту.

`S_XFER_END` — состояние на один такт, но самое насыщенное:

```

if (!tx_only) begin
    if (!rx_full) begin
        rx_wr_en <= 1'b1;
        rx_data <= pack_rx(rx_word, bits_total);
    end else begin
        err_rx_overflow <= 1'b1;
    end
end
end
tx_rd_en <= 1'b1;

```

[Объяснить с](#) 

Здесь принятое слово выравнивается и толкается в RX FIFO, а отправленное выталкивается из TX FIFO. Отдельное состояние нужно потому, что последний сэмпл присваивается неблокирующим оператором и становится виден только в следующем такте. Запомните ветку `else` с `err_rx_overflow` — из неё вырастет дефект A-5. Флаг `tx_only` просто выключает приём: писать в восьмисловный FIFO сотни тысяч пикселей кадра всё равно бессмысленно.

Дальше `S_CS_HOLD` выдерживает паузу «последний фронт → снятие CS» и решает судьбу транзакции: если в TX FIFO ещё что-то есть, CS **не снимается**, и через `S_INTER` (межсловная пауза) движок уходит на новое слово. Если пусто — CS поднимается, и `S_DONE` выдаёт однократный `done_pulse`. Состояние `S_WAIT_TX` относится только к потоковому режиму: оно удерживает CS низким, пока DMA не подвезёт следующую порцию или не придёт признак конца кадра.

Про мягкий сброс. Ветка `soft_rst` возвращает автомат в `S_IDLE`, ставит SCLK на CPOL, снимает CS и обнуляет счётчики — но **не трогает** sticky-ошибки. Мягкий сброс означает «прекрати и вернись в исходное», а не «забудь, что было»; забывание — это отдельный импульс `clr_errors` из `ERROR_CLR`.

9.5. spi_master_top — клей, в котором спрятаны три важные строки

Модуль не содержит собственной логики, но три выражения в нём стоят того, чтобы прочитать их глазами.

```

wire irq_err_evt = eng_err_start | eng_err_rx_ovf
                  | eng_err_word_len | tx_overflow;
assign fifo_flush = clr_errors | ctrl_soft_rst;
wire tx_wr_any = (tx_wr | ext_tx_wr) & ~tx_full;

```

[Объяснить с](#) 

Первое собирает все ошибки движка в одно событие прерывания. Заметьте: `eng_err_rx_ovf` здесь **есть**. Именно поэтому переполнение RX всё-таки видно хосту — но через IRQ, а не через STATUS. Это половина дефекта A-5.

Второе объединяет два источника очистки FIFO. Из-за него запись в `ERROR_CLR` не просто квитирует ошибки, а ещё и **уничтожает непрочитанные данные** в обоих буферах. Для драйвера это жёсткое ограничение порядка операций: сначала вычитать `RX_DATA`, только потом квитиловать (`../hw_sw_contract.md`, F-2).

Третье разрешает писать в TX FIFO двум источникам сразу — регистровому интерфейсу и AXIS-адаптеру — и никогда не пишет в полный буфер. Рядом живёт `eng_clk_div = use_clk_override ? clk_div_override : reg_clk_div`: побочный вход, позволяющий потоковому режиму задать делитель мимо регистра. Обе конструкции появились ради DMA и в

PIO-сборке подвязаны к нулю.

9.6. Контракт параллельной шины, который нельзя сломать мостом

Ядро наружу смотрит собственным memory-mapped интерфейсом: `wr_en`, `rd_en`, четырёхбитный `addr`, `wr_data`, `rd_data`, `ready`, `irq`. Ни Avalon, ни Wishbone, ни AXI — сознательный выбор автора исходного IP ради переносимости, и именно им воспользовался наш перенос. Но у простоты есть обратная сторона: контракт короткий, зато его пункты неочевидны. Стробы `wr_en` и `rd_en` активны ровно один такт; мост, который держит `rd_en` два такта, дважды вытолкнет слово из RX FIFO, и второе чтение потеряется молча. Адрес — **индекс слова**, четыре бита, а не байтовый адрес AXI; преобразование `offset = index * 4` делает обвязка. Выход `rd_data` комбинационный и действителен только в такте активного строба, в остальное время там ноль. Чтение по адресу `RX_DATA` имеет **побочный эффект**: оно вытаскивает слово из очереди, поэтому отладчик со спекулятивными чтениями по этому адресу незаметно испортит вам приём. Сигнал `ready` — константная единица, он существует только для мостов, которым нужен `waitrequest`. И наконец `irq` регистровый, то есть виден на линии на такт позже, чем в статусе.

Из-за этих пунктов AXI-обвязка получила отдельные тесты: 19 — «ровно один pop на чтение», 20 — «ровно один push на запись», 21 — чтение всех регистров. Риск M-6 в `../migration.md` описан именно так: ядро не тронута, но небрежный адаптер способен сломать его снаружи. Как этот контракт переложен на AXI4-Lite и какие четыре отклонения при этом приняты — в [части III](#).

Глава 10. Карта регистров CONTROL...ERROR_CLR

10.1. Что такое memory-mapped регистр

Управление периферией в SoC устроено так: блок занимает кусок адресного пространства процессора, и обращение по адресу из этого куска попадает не в память, а в железо. Записали слово по адресу «база + 0x14» — оно ушло в TX FIFO. Прочитали «база + 0x04» — получили состояние контроллера. Специальных инструкций не нужно, обычные `load` и `store`. Наш IP занимает **64 байта**, данные 32-битные, регистров одиннадцать плюс четыре в обвязке. Внутри ядра адрес — индекс слова (те самые четыре бита `addr`), на шине AXI — байтовое смещение, связаны они умножением на четыре. Семантика регистров при переносе **не менялась**: `spi_reg_if.v` байт-идентичен Altera-оригиналу, что подтверждено контрольной суммой в `../verification.md`.

10.2. Карта

Смещение	Регистр	Доступ	Назначение
0x00	CONTROL	R/W	EN, START, SOFT_RST, CPOL, CPHA, LSB_FIRST, IRQ_EN
0x04	STATUS	RO	BUSY, DONE, флаги FIFO, sticky-ошибки
0x08	CLK_DIV	R/W	Полупериод SCLK = CLK_DIV+1 тактов; сброс 4
0x0C	CS_SELECT	R/W	Индекс активного CS, не маска ; сброс 0

0x10	WORD_LEN	R/W	Длина слова: только 8, 16, 24, 32; сброс 8
0x14	TX_DATA	WO	Запись = push в TX FIFO
0x18	RX_DATA	RO	Чтение = pop из RX FIFO (побочный эффект)
0x1C	DELAY_CFG	R/W	CS_SETUP / CS_HOLD / INTER, в тактах clk
0x20	IRQ_STATUS	RO	Sticky-причины: DONE, RX_VALID, ERROR
0x24	IRQ_MASK	R/W	Маска разрешённых причин
0x28	ERROR_CLR	WO	W1C по IRQ_STATUS + сброс DONE/TX_OVF + flush FIFO
0x2C	—	—	Не занят, читается нулём
0x30	STREAM_CTRL	R/W	v2: STREAM_EN, TX_ONLY, TX_FAST, CLR_EOT
0x34	STREAM_STATUS	RO	v2: BUSY, EOT
0x38	FAST_DIV	R/W	v2: делитель при TX_FAST
0x3C	ID_VERSION	RO	0x53500100 или 0x53500200 — паспорт IP

Смещения 0x30 ... 0x3C живут **не в ядре**, а в AXI-обвязке `spi_axi4lite.v`; `spi_reg_if` про них не знает. В PIO-сборке их может не быть вовсе, и тогда чтение вернёт ноль. Полная версия таблицы со всеми битовыми полями — в `../register_map.md`; здесь мы разбираем только то, что нужно понять архитектуру.

10.3. CONTROL: три обычных бита и два необычных

Биты 0, 3, 4, 5 и 6 — обычные хранимые: EN разрешает контроллер, CPOL и CPHA задают режим SPI, LSB_FIRST меняет порядок бит, IRQ_EN глобально разрешает прерывание; записали — прочитали то же самое. Биты 1 (START) и 2 (SOFT_RST) устроены иначе, это **self-clearing** стробы. Записанная единица порождает однократный импульс и не сохраняется, поэтому чтение CONTROL всегда возвращает в этих позициях ноль. Отсюда безопасность приёма «прочитал — изменил — записал»: START вы случайно не повторите, он читается нулём, а EN, CPOL и остальные сохраняются правильно. И ещё одна деталь из кода: START принимается, если EN уже установлен **или** устанавливается той же записью, поэтому запуск одной транзакцией — законный приём.

10.4. STATUS: живые биты, sticky-биты и один мёртвый

Бит	Имя	Тип	Смысл
0	BUSY	живой	Транзакция выполняется
1	DONE	sticky	Транзакция завершена; квитируется через ERROR_CLR

2	RX_VALID	живой	RX FIFO не пуст
3	TX_READY	живой	TX FIFO не полон
4	ENABLED	живой	Зеркало CONTROL.EN
5	TX_EMPTY	живой	TX FIFO пуст
6	RX_FULL	живой	RX FIFO полон
8	ERR_TX_OVF	sticky	Запись TX_DATA при полном FIFO
9	ERR_RX_OVF	sticky	Не выставляется никогда — дефект A-5
10	ERR_START	sticky	START при пустом TX FIFO
11	ERR_WORD_LEN	sticky	Недопустимый WORD_LEN на момент START

Разница между «живым» и sticky-битом принципиальна для того, как вы пишете опрос. Живой бит отражает текущее состояние: увидели BUSY = 0, значит прямо сейчас передачи нет. Sticky-бит отражает историю: DONE, однажды взведённый, останется единицей до квитирования, даже если после прошло десять транзакций. Поэтому классическое условие завершения — не «DONE = 1», а «DONE = 1 и BUSY = 0», и именно так написан цикл ожидания в `../..sim/tb_clkdiv_golden.v`.

Бит 9 заслуживает отдельного предупреждения, и повторяется оно в трёх местах проекта не случайно. ERR_RX_OVF описан в документации исходного проекта как рабочий sticky-флаг переполнения приёмного буфера. Он не работает — не «иногда не срабатывает», а не выставляется ни при каких условиях, и до исправления единственный способ узнать о потере данных — бит ERROR в IRQ_STATUS (дефект A-5, глава 11).

10.5. CLK_DIV и два подводных камня

Регистр шестнадцатитбитный, значение по умолчанию после сброса — 4, то есть 5 МГц при системных 50 МГц. Соответствие делителя и частоты стоит держать перед глазами:

CLK_DIV	f_SCLK при 50 МГц	Пригодность
0	—	Запись игнорируется молча (дефект A-4)
1	12.5 МГц	Не использовать: приём сдвинут на бит (дефект A-3)
2	8.33 МГц	Минимально допустимое, оно же максимальная рабочая частота
4	5 МГц	Номинал, значение после сброса
24	1 МГц	Удобно для логического анализатора
249	100 кГц	Медленный край
65535	≈ 381 Гц	Абсолютный минимум

Первый камень: запись нуля отвергается, и никакой обратной связи об этом нет — ни флага, ни ошибки на шине. Хост уверен, что настроил делитель, а работает предыдущее значение. Второй:

единица принимается, хотя она нерабочая. Оба разбираются в главе 11 (A-4 и A-3 соответственно). Практический вывод для любого кода, который трогает этот регистр: писать не меньше двух и **читать обратно** после записи.

10.6. TX_DATA, RX_DATA и побочный эффект чтения

Два регистра данных выглядят симметрично, но ведут себя по-разному. TX_DATA — только на запись, каждая запись это push в очередь; данные выравниваются вправо, то есть для восьмибитного слова значащими будут младшие восемь бит. Запись в полный буфер поднимает sticky ERR_TX_OVF, и слово теряется.

RX_DATA — только на чтение, и это чтение **изменяет состояние железа**: слово извлекается из очереди. Чтение пустого буфера возвращает ноль и рор не выполняет, тут RTL аккуратен, но повторное чтение непустого буфера вытащит следующее слово, а не то же самое. Отсюда правило, которое стоит написать на стене лаборатории: по адресу RX_DATA не читают «на всякий случай» и не ставят точку наблюдения в отладчике.

10.7. DELAY_CFG, IRQ и ERROR_CLR

DELAY_CFG упаковывает три задержки в одно слово: биты 7:0 — пауза между опусканием CS и первым фронтом SCLK, биты 15:8 — между последним фронтом и подъёмом CS, биты 23:16 — межсловная пауза внутри burst. Единица везде одна: такт системного блока, не такт SPI. Старший байт зарезервирован и читается нулём; кстати, именно он даёт часть безобидных предупреждений «port has no load» при синтезе. IRQ_STATUS хранит три sticky-причины, IRQ_MASK разрешает нужные, итоговая линия равна $IRQ_EN \& |(IRQ_STATUS \& IRQ_MASK)$; замаскированное событие всё равно фиксируется.

ERROR_CLR — самый опасный регистр карты, потому что делает четыре вещи сразу: сбрасывает по маске биты IRQ_STATUS, сбрасывает sticky DONE, сбрасывает sticky TX overflow и **флашит оба FIFO**. Последнее не следует ни из названия, ни из семантики регистра; узнать об этом можно только из RTL или из ../register_map.md. Следствие: квитирование прерывания уничтожает непрочитанные данные, поэтому порядок «сначала прочитать RX, потом квитировать» — не рекомендация, а требование.

10.8. ID_VERSION: зачем железу паспорт

Регистр 0x3C возвращает 0x53500100 или 0x53500200. Старшие 16 бит — 0x5350, это ASCII-коды букв «S» и «P». Дальше major и minor: 01.00 — классическая PIO-сборка, 02.00 — сборка со stream и AXI DMA.

Смысл регистра в том, что программа не должна доверять адресу. Драйвер в probe() читает 0x3C первым делом: если там не 0x5350, значит либо в FPGA залит не тот битстрим, либо в Device Tree указан не тот адрес, и правильная реакция — отказаться привязываться, а не пытаться работать. Битстримы, собранные до появления регистра, вернут ноль и трактуются так же (../hw_sw_contract.md, раздел 9; сам драйвер — в части VII).

10.9. Типовая последовательность одного обмена

Соберём всё вместе. Порядок обращений тот же, что в тестбенче ../sim/tb_clkdiv_golden.v и в примере на C из ../register_map.md.

1. Записать 0xFFFFFFFF в ERROR_CLR — снять все sticky-флаги и очистить оба FIFO перед началом работы.
2. Настроить CLK_DIV (не меньше 2), CS_SELECT, WORD_LEN и DELAY_CFG.
3. Записать слово в TX_DATA — оно ложится в TX FIFO. Для burst записать несколько слов

поряд, но не больше глубины FIFO.

4. Записать в `CONTROL` единицы в биты `EN` и `START` — одной транзакцией.
5. Ждать в цикле, пока в `STATUS` не окажется `DONE = 1` при `BUSY = 0`.
6. Прочитать `RX_DATA` столько раз, сколько слов было отправлено.
7. Только теперь квитировать: записать в `ERROR_CLR` маску обработанных событий. Раньше — нельзя, иначе шаг 6 останется без данных.

Глава 11. Дефекты A-1...A-6 как мини-детективы

Прежде чем открывать дела, зафиксируем политику, без которой они выглядят странно. Ни один из шести дефектов в этом переносе **не исправлен**. Это не лень и не небрежность, а прямое следствие задачи: сначала нужно получить доказуемо эквивалентную версию ядра, и только потом чинить. Любая правка логики обнуляет аргумент эквивалентности — а он у нас построен на контрольных суммах и на построчном сравнении вердиктов тестов (`../verification.md`). Поэтому дефекты задокументированы, покрыты тестами, а исправления вынесены в отдельный список O-1...O-3 в `../migration.md` .

Более того, два теста в `../sim/tb_spi_axi4lite.v` **утверждают** дефектное поведение как ожидаемое. Приём звучит дико («тест проверяет, что баг на месте»), но он единственный честный: если кто-то поправит синхронизатор или источник бита `STATUS`, тест упадёт и потребует объяснений, вместо того чтобы молча позеленеть и оставить документацию врущей. Все шесть историй ниже идут в одном формате: что видели, какая версия казалась убедительной, чем проверяли, какая улика перевернула картину, где корень, что сделали и какое правило из этого следует. Две — A-3 и A-5 — стоит выучить наизусть: первая определяет заявленную частоту SPI, вторая — как писать драйвер.

A-1. `MISO_SYNC_STAGES = 1` ломает `part-select`

История начинается там, где обычно и начинаются такие истории: в `README`, где написано, как сделать лучше. Параметр `MISO_SYNC_STAGES` управляет глубиной цепочки синхронизации входа `MISO`, по умолчанию он равен двум. Комментарий у объявления порта в `../rtl/spi_engine.v` гласит `FF chain on spi_miso (>= 1)` , а `README` исходного проекта идёт дальше и прямо предлагает поставить единицу «для ускорения». Соблазн огромен, особенно после знакомства с дефектом A-3: раз задержка синхронизатора ограничивает частоту SPI, самое очевидное решение — укоротить синхронизатор. Логика безупречна, документация разрешает, параметр существует именно для этого.

Что мы увидели, попробовав. `Elaboration` падает: не работает медленнее, не теряет биты, а просто не проходит стадию разбора исходника, и ошибка указывает на выражение сдвига цепочки. Как проверяли: подставили `MISO_SYNC_STAGES = 1` в неизменённый исходник и посмотрели, во что превращается строка:

```
miso_sync <= { miso_sync[MISO_SYNC_STAGES-2:0], spi_miso };
```

Объяснить с 

При единице `MISO_SYNC_STAGES-2` равно минус единице, и `part-select` превращается в `miso_sync[-1:0]` . Такого диапазона в Verilog не существует, ни один инструмент его не примет. При двойке выражение даёт корректный однобитный срез `miso_sync[0:0]` , при большем

значении всё тоже закономерно. Настоящий минимум параметра — два, а не один.

Улика, которая расставила всё по местам, нашлась в соседнем файле. В `../rtl/spi_reset_sync.v` живёт точно такая же цепочка с точно таким же выражением `sync_chain[STAGES-2:0]` — и там комментарий у параметра честно говорит `number of synchroniser stages (>= 2)`. Один и тот же автор, один и тот же приём, два разных комментария. Значит дело не в коде, который «не поддерживает» единицу; дело в том, что в одном месте документация написана по факту, а в другом — по намерению.

Корневая причина: выражение сдвига в `../rtl/spi_engine.v`, массив `miso_sync`, написано в предположении минимум двух ступеней, а комментарий порта и README заявляют минимум одну. Врёт документация, код честен.

Что сделали: ничего. Значение по умолчанию — два, перенос его не меняет, на эксплуатацию дефект не влияет вовсе, проявляется только при попытке последовать совету README. Правильное исправление вынесено в список необязательных (O-3) и состоит из выбора: либо привести комментарий и README в соответствие с реальностью, либо добавить `generate`-ветку для одной ступени. Первое дешевле и ничего не меняет в железе. Второе, кстати, было бы вредно по существу: одна ступень синхронизации — это отказ от защиты от метастабильности ради одного такта, и выигрыш в частоте SPI не окупает такой размен.

Правило на будущее: параметр «для ускорения», описанный в README, проверяйте синтезом на граничных значениях, а не доверяем к тексту. И если один и тот же приём в проекте прокомментирован по-разному — верьте более строгому комментарию.

A-2. NUM_CS = 1 ломает part-select

Второе дело — близнец первого, но с другой мотивацией. Параметр `NUM_CS` задаёт количество линий выбора кристалла, по умолчанию их четыре. Ситуация, в которой хочется поставить единицу, возникает постоянно: к контроллеру подключено ровно одно устройство. Дисплей ST7789 в нашем проекте — как раз такой случай. Зачем держать четыре линии, если нужна одна? Экономия выглядит бесплатной.

Что увидели: то же самое, что в A-1 — отказ на стадии разбора исходника.

Как проверяли. Смотрели на строку активации CS в `../rtl/spi_engine.v`:

```
if (cs_select < NUM_CS)
    spi_cs_n[ cs_select[$clog2(NUM_CS)-1:0] ] <= 1'b0;
```

Объяснить с 

Функция `$clog2(N)` возвращает количество бит, необходимое для адресации `N` значений. Для четырёх это два, для двух — один, а для единицы — **ноль**, потому что для адресации одного элемента адрес не нужен вовсе. Формально всё верно, но `$clog2(1)-1` равно минус единице, и мы снова получаем `cs_select[-1:0]`. При `NUM_CS = 2` выражение даёт `cs_select[0:0]` и работает. То есть дефект — точечный, ровно в одном значении параметра.

Улика, объясняющая, почему это не заметили раньше: дефолт проекта равен четырём, и ни одна собираемая конфигурация единицу не использует. Более того, в целевой системе на Zynq потребность в единственном CS решается вообще иначе — через `CS_SELECT`, выставленный **вне диапазона**. Регистр `CS_SELECT` хранит индекс, а не маску, и если индекс больше или равен `NUM_CS`, охранный условие `cs_select < NUM_CS` не выполняется, ни одна нативная линия не активируется, а транзакция всё равно идёт: SCLK и MOSI работают. Ровно этим пользуется

драйвер Linux, когда CS управляется через GPIO: он программирует `CS_SELECT = NUM_CS` и отдаёт удержание CS ядру ОС (`../hw_sw_contract.md` , ограничение F-1). Получается, что «сэкономить на `NUM_CS`» не нужно даже практически.

Корневая причина: ширина индекса в `../rtl/spi_engine.v` посчитана формулой `$clog2(NUM_CS)` , которая вырождается при `NUM_CS = 1` . Классический случай формулы, корректной на всём диапазоне кроме граничной точки.

Что сделали: не трогали. Для всех рабочих конфигураций (два и больше) код безопасен, дефолт четыре, влияние на перенос нулевое. Исправление — та же `generate` -ветка или явно задокументированный минимум — лежит в O-3.

Правило на будущее: любая формула вида `$clog2(N)` в объявлении ширины обязана иметь либо ветку для `N == 1` , либо защитный `localparam` , берущий максимум из результата и единицы. Проверяйте параметризованные модули на границах диапазона параметров, а не только на дефолтах — граница почти всегда там, где величина обращается в ноль или в единицу.

A-3. Предел `CLK_DIV >= 2`: когда передача идеальна, а приём врёт

Главное дело части: как слово «recommended» в README оказалось замаскированным «required».

Что видели. Тест гоняет один байт в режиме 0: мастер отправляет `0xC3` , слейв отвечает `0x3C` . При делителе 2 и выше принятое слово равно `0x3C` , при делителе 1 — `0x1E` . Присмотритесь: `0x3C` это `0011_1100` , а `0x1E` это `0001_1110` . То же самое слово, сдвинутое вправо на бит. Не искажённое, не случайное — сдвинутое. Передача при этом безупречна, слейв видит ровно `0xC3` . Симптом коварен именно этим: на осциллографе MOSI и SCLK красивые, CS опускается и поднимается когда надо, а софт уже читает мусор.

Что казалось логичным. Версий было четыре. Первая: «делитель 1 законен, RTL отвергает только ноль — значит единица обязана работать»; возразить трудно, она опирается на код. Вторая: «README пишет *recommended* ≥ 2, значит на единице просто меньше запаса». Третья: «сломался MOSI, или выставлен не тот режим, или петля соединена не так» — самая соблазнительная, потому что предлагает искать там, где удобно. Четвёртая, самая опасная: «это баг переноса», то есть мы сами что-то сломали.

Как проверяли. Сначала разделили передачу и приём. Тестбенч выводит не только принятое мастером слово, но и то, что зафиксировал слейв на линии MOSI, — колонка `mosi_seen` . Она оставалась равной `0xC3` при всех делителях, значит тракт передачи цел и ломается только сэмплирование MISO.

Потом проверили, наш ли это грех. Тестбенч `../sim/tb_clkdiv_golden.v` не касается ничего портированного: он инстанцирует **неизменённое Altera-ядро** из исходного дерева и общается с ним через родную параллельную шину, без AXI-обвязки. Прогон `make sim_clkdiv` даёт свип:

```
GOLDEN_RESULT: CLK_DIV_req=0 CLK_DIV_eff=4 rx=3c mosi_seen=c3
GOLDEN_RESULT: CLK_DIV_req=1 CLK_DIV_eff=1 rx=1e mosi_seen=c3    <-- дефект
GOLDEN_RESULT: CLK_DIV_req=2 CLK_DIV_eff=2 rx=3c mosi_seen=c3
GOLDEN_RESULT: CLK_DIV_req=3 CLK_DIV_eff=3 rx=3c mosi_seen=c3
```

Объяснить с 

Дефект воспроизвёлся на оригинале байт в байт, и версия «это баг переноса» закрылась. Строка с `req=0` — не аномалия: запись нуля отвергается регистровым интерфейсом, и эффективным остаётся прежнее значение 4, что показывает колонка `CLK_DIV_eff` . Молчаливый отказ при нуле

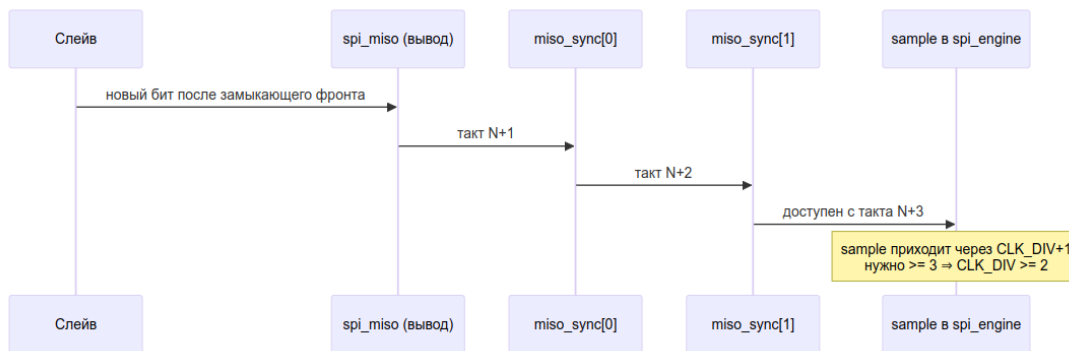
— уже дефект A-4, следующее дело.

Третий шаг — вывести механизм из RTL, а не угадать. Проследим путь принятого бита по тактам системного блока. Пусть на такте N движок формирует замыкающий фронт SCLK; так как `spi_sclk` снимается с триггера, на выводе он меняется сразу после этого фронта, и слейв, увидев его, обновляет MISO. На такте N+1 новое значение защёлкивается первой ступенью `miso_sync[0]`, на такте N+2 переходит во вторую, и только теперь `miso_synced` равен новому биту. Значит самый ранний фронт, на котором движок может прочитать бит в `rx_word`, — такт N+3. А сэмплирует он через полупериод, то есть через `CLK_DIV + 1` тактов после предыдущего фронта:

$$\text{CLK_DIV} + 1 \geq 3 \Rightarrow \text{CLK_DIV} \geq 2$$

Объяснить с 

При `CLK_DIV = 1` сэмпл приходит на такте N+2, когда `miso_synced` ещё держит **предыдущий** бит. Отсюда и сдвиг ровно на одну позицию — не порча, а запаздывание на бит.



Корневая причина. Двухступенчатый синхронизатор `miso_sync` в `../rtl/spi_engine.v` корректно защищает от метастабильности, но тратит на это время, а счётчик `half_cnt` в том же файле позволяет задать полупериод короче этой задержки. Ни одна из частей не ошибочна по отдельности; ошибочно отсутствие связи между ними. Документация же классифицировала жёсткое требование как совет, а RTL проверяет только `CLK_DIV != 0` и молча принимает опасную единицу.

Что это значит для софта. Формула $f_{\text{SCLK}} = f_{\text{clk}} / (2 \cdot (\text{CLK_DIV} + 1))$ при делителе 2 даёт $f_{\text{clk}} / 6$. Именно $/6$, а не $/4$, который получился бы при делителе 1, и тем более не $/2$ при нуле. При системных 50 МГц честный максимум SPI — **8.33 МГц**. Драйвер обязан объявлять $\text{max_speed_hz} = f_{\text{clk}} / 6$ и брать f_{clk} из `clk_get_rate()`, а не из константы: в сборке с DMA системный блок часто 100 МГц, и потолок поднимается до 16.7 МГц (`../hw_sw_contract.md`, раздел 6).

Что сделали. Поведение сохранили. Тест 25 в `../sim/tb_spi_axi4lite.v` **ожидает** `0x1E` при делителе 1 — чтобы случайное «улучшение» синхронизатора не прошло незамеченным. Исправление (O-2) — запретить запись `CLK_DIV < 2` по аналогии с запретом нуля: одна строка, но она меняет наблюдаемое поведение, поэтому в перенос не вошла. Есть у дефекта и след в `constraints`: раз схема не полагается на соотношение SCLK и MISO, в XDC стоит `set_false_path` на вход MISO, и корректность приёма гарантирует правило `CLK_DIV >= 2`, а не STA (`../migration.md`, обоснование C-3).

Правило на будущее. Синхронизатор на входе данных проверяйте свипом частоты, а не одним

удачным прогоном. Слово «recommended» в чужом README считайте подозреваемым: чаще всего это замаскированное «required». Максимальную частоту выводите из задержки CDC, а не из формулы делителя. И запомните признак: если передача идеальна, а приём сдвинут, первым подозреваемым делайте путь сэмплирования.

A-4. Запись CLK_DIV = 0 уходит в никуда

Дело маленькое, но поучительное — из тех, что съедают вечер отладки не сложностью, а неожиданностью.

Что видели. Хост записывает в CLK_DIV ноль, читает регистр обратно и получает старое значение. Ошибки нет: шина ответила OKAY, прерывание не взведено, ни один sticky-флаг не изменился. Впервые мы заметили это не в отладке, а в логе свипа из дела A-3: строка CLK_DIV_req=0 CLK_DIV_eff=4 показывает запрошенный ноль и эффективную четвёрку — значение, оставшееся с прошлой итерации.

Что казалось логичным. Первая мысль всегда одна: «запись потерялась». Дальше версия ветвится по вкусу — не тот адрес, не сработал wstrb, мост проглотил транзакцию, кэш не сбросился. Все разумны, все ведут в сторону от истины. Вторая мысль, чуть более изощрённая: «регистр read-only» — тоже мимо.

Как проверяли. Ключевое наблюдение сделано без единого инструмента: в той же последовательности записей CS_SELECT, WORD_LEN и DELAY_CFG установились нормально. Значит шина исправна, адрес верен, мост работает — проблема касается **одного регистра при одном конкретном значении**. Такое сужение почти всегда означает намеренную проверку в коде, и она нашлась в ../rtl/spi_reg_if.v :

```
`SPI_ADDR_CLK_DIV: begin
    // Refuse divisor of zero (would freeze SCLK gen).
    if (wr_data[15:0] != 16'd0)
        reg_clk_div <= wr_data[15:0];
end
```

Объяснить с 

Вот и разгадка: запись нуля просто не выполняется. Никакого флага, никакого ответа SLVERR — обвязка отвечает OKAY на всё, это её задокументированное отклонение D-3.

Мотивировку в комментарии стоит прочитать критически. Он утверждает, что ноль «заморозил бы генерацию SCLK». Но если следовать тексту счётчика, условие half_cnt >= clk_div при нуле выполняется каждый такт, то есть SCLK переключался бы на каждом фронте системного клона — не замирал, а разгонялся до f_clk / 2. Практической разницы нет: и то и другое одинаково непригодно, ведь даже вдвое меньшая частота уже нарушает предел A-3. Но факт полезен как напоминание: комментарий — это мнение автора о коде, а не сам код.

Корневая причина. Защитная проверка в ../rtl/spi_reg_if.v, ветка записи CLK_DIV, реализована как молчаливый отказ: без sticky-флага, без отражения в STATUS, без ошибки на шине. Строго говоря, это не баг: молчаливый отказ безопаснее заведомо неработающей конфигурации. Баг в том, что механизм не был описан в карте регистров и поэтому выглядел как потеря записи.

Что сделали. Поведение оставили, а недостающее звено — документацию — добавили в ../register_map.md. Дополнительно дефект оказался практически недостижим: драйвер по правилу A-3 никогда не пишет значение меньше двух, так что до нуля дело не доходит. Опциональный

sticky-флаг «отвергнутая запись» в перенос не входит.

Правило на будущее. Любой молчаливый отказ железа обязан быть в карте регистров крупным шрифтом, иначе он превращается в час отладки для каждого следующего человека. А со стороны софта правило универсальное: после записи важного конфигурационного регистра читайте его обратно и сверяйте.

A-5. Мёртвый бит: STATUS.ERR_RX_OVF не выставляется никогда

Второе большое дело. Это история о том, как одно предупреждение синтеза оказалось важнее красивой строки в документации, и почему драйвер, честно опрашивающий регистр состояния, не заметит потерю данных.

Что видели. Приёмный буфер переполнен: STATUS.RX_FULL равен единице, принятые слова действительно потеряны. А бит 9 того же регистра, ERR_RX_OVF, описанный в документации исходного проекта как sticky-флаг переполнения приёма, остаётся нулём. Хост, читающий STATUS, получает картину «буфер полон, ошибок нет» — и делает из неё неверный вывод: раз ошибок нет, ничего не потеряно.

Что казалось логичным. Первая версия — самая естественная: «переполнения не было, иначе бит бы встал». Вторая: «флаг sticky, наверное, его кто-то сбросил — надо квитиловать ERROR_CLR и перечитать». Третья: «баг в AXI-обвязке или в Linux-драйвере, ядро-то не трогали». Четвёртая, и вот она опаснее всех: «предупреждение Synth 8-7129 про неподключённый порт можно игнорировать, их там десятки».

Как проверяли. Отправной точкой стало именно то предупреждение, которое хотелось проигнорировать. Vivado при elaboration выдал:

```
WARNING: [Synth 8-7129] Port eng_err_rx_ovf in module spi_reg_if is either
unconnected or has no load
```

Объяснить с 

Всего таких предупреждений в сборке 46. Сорок пять из них безобидны: это зарезервированные биты delay_cfg[31:24], неиспользуемые по отклонениям D-1 и D-3 сигналы awprot, arprot, wstrb, младшие биты байтового адреса. Одно — про сигнал с именем, в котором стоит слово err. Порт ошибки, у которого нет нагрузки, — это не стилистика, это функциональный риск, и разбирать его надо первым.

Дальше — чтение исходников по цепочке из трёх звеньев. Звено первое: ../../rtl/spi_reg_if.v, сборка слова STATUS. Бит 9 берётся из сигнала rx_fifo_overflow — то есть из флага **самого FIFO**. Звено второе: ../../rtl/spi_fifo.v, где этот флаг взводится условием wr_en && full. Звено третье, решающее: ../../rtl/spi_engine.v, состояние S_XFER_END. При полном приёмном буфере движок **не поднимает** rx_wr_en, а вместо этого выставляет собственный флаг:

```
if (!rx_full) begin
    rx_wr_en <= 1'b1;
    rx_data <= pack_rx(rx_word, bits_total);
end else begin
    err_rx_overflow <= 1'b1;
end
```

Объяснить с 

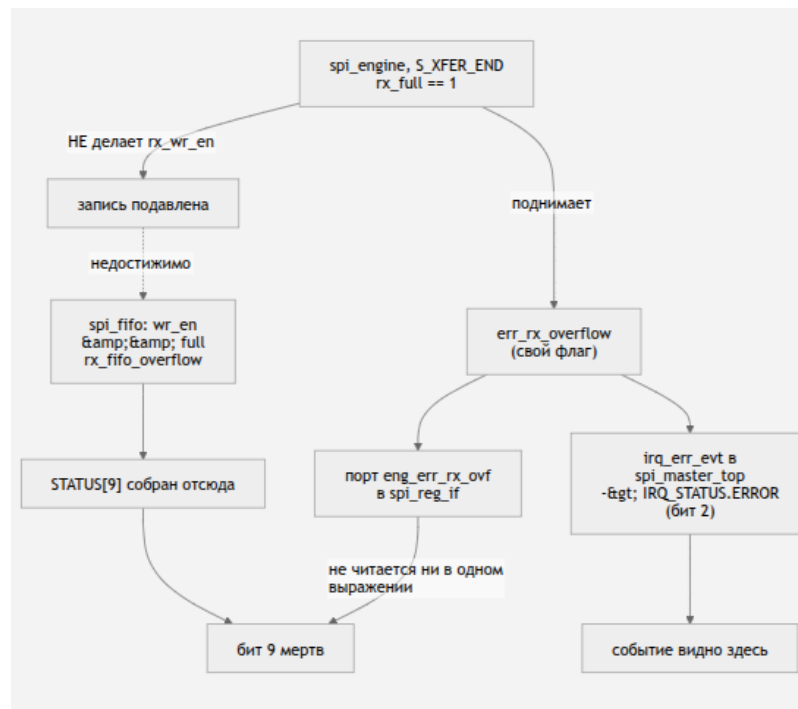
Вот и вся ловушка. Условие `wr_en && full` со стороны движка **недостижимо в принципе**: движок никогда не пишет в полный буфер, он вежливо воздерживается. Значит флаг FIFO от переполнения движком не встанет никогда, а бит 9 STATUS собран именно из него. Собственный же флаг движка приходит на порт `eng_err_rx_ovf` модуля `spi_reg_if` — и там не используется ни в одном выражении. Порт объявлен, подключён сверху, и на этом всё. Отсюда и предупреждение синтезатора.

Улику предъявили экспериментом. Тест 26 в `.././sim/tb_spi_axi4lite.v` создаёт настоящее переполнение: шестнадцать принятых слов при глубине буфера восемь, без единого чтения со стороны хоста. Результат:

A-5 EVIDENCE: STATUS=0000007e RX_FULL=1 ERR_RX_OVF=0 IRQ_ERROR=1

Объяснить с 

Переполнение состоялось, `RX_FULL` поднят, бит 9 — ноль, а вот `IRQ_ERROR` единица. Последнее и есть спасительная деталь.



Корневая причина. Три звена, каждое проверяемо по исходнику: `STATUS[9]` питается от флага FIFO (`.././rtl/spi_reg_if.v`, сборка `status_word`); условие этого флага недостижимо со стороны движка (`.././rtl/spi_fifo.v`, `ovf_flag`); собственный флаг движка (`.././rtl/spi_engine.v`, `err_rx_overflow`) до `STATUS` не доходит. Ошибка не в одном месте, а в стыке трёх — поэтому её и не видно при чтении отдельного файла.

Что это значит для драйвера. Ошибка не теряется полностью: `spi_master_top` включает `eng_err_rx_ovf` в `irq_err_evt`, поэтому событие попадает в `IRQ_STATUS.ERROR`, бит 2. Пропадает именно удобный бит `STATUS` — тот самый, на который естественнее всего сядет и драйвер, и лабораторный скрипт с `devmem`. Следствие сформулировано в `../hw_sw_contract.md`, раздел 8, и оно категорично: до исправления переполнение приёма детектируется **только** через `IRQ_STATUS.ERROR`, опрос `STATUS[9]` бесполезен. Драйвер, который живёт опросом и не читает `IRQ_STATUS`, будет молча терять данные и рапортовать об успехе.

Что сделали. Не исправили: исправление меняет наблюдаемое поведение регистра, а это выходит за рамки переноса (O-1 в `./migration.md`). Сам фикс тривиален — источником бита 9 сделать `rx_fifo_overflow | eng_err_rx_ovf`. Вместо фикса тест 26, который утверждает нулевое значение бита как ожидаемое, и предупреждение в трёх документах.

Правило на будущее. Предупреждение «port has no load» на сигнале с именем, содержащим `err`, считайте задачей высшего приоритета, пока не доказано обратное. Именно поэтому глушить предупреждения пачками запрещено — сорок пять безобидных штук в этой сборке прятали одно настоящее. Sticky-бит, описанный в документации, но не покрытый тестом на реальное событие, — это гипотеза, а не факт. И общее правило миграции: не чинить молча; сначала эквивалентность и тест, который фиксирует дефект, потом согласованное исправление.

A-6. Неполные назначения выводов в QSF

Последнее дело относится не к RTL, а к файлам проекта — и тем оно интереснее, потому что показывает класс ошибок, который на SoC становится опаснее.

Что видели. В Altera-ревизии файл `quartus/spi_master.qsf` задаёт электрический стандарт `IO_STANDARD` для шин `reg_wdata[*]`, `reg_rdata[*]` и сигнала `reg_ready` — и **не задаёт** для них `set_location_assignment`, то есть физическое расположение вывода. Всего без координаты остались 65 сигналов. Quartus в такой ситуации не ругается: он молча раскидывает их по свободным ножкам корпуса.

Что казалось логичным. «Стандарт задан — значит пины назначены». Это интуитивно очень убедительно: строка про сигнал в файле есть, значит про сигнал подумали. Ошибка в том, что две настройки отвечают за разное: `IO_STANDARD` описывает электрику (уровни, ток, терминацию), а `set_location_assignment` — географию (на какой физический вывод корпуса выйдет сигнал). Одно без другого — половина работы.

Как проверяли. Простым сопоставлением: выписали все сигналы, для которых задан стандарт, и все, для которых задано расположение, и сравнили списки. Разница — 65 позиций. Дополнительно сам файл содержит в шапке честное признание, что назначения выводов заглушечные, а ревизия `spi_master` под железо никогда не собиралась.

Улика, придающая делу вкус: в документации того же исходного проекта есть целая глава, посвящённая ровно этому классу ошибок, — про устаревшие пины в QSF и про то, как Quartus молча раскидал сигналы. Из неё даже выведено правило: сверять QSF `grep`ом после каждого изменения портов верхнего модуля. В ревизии `spi_master` собственное правило не применили. Так бывает чаще, чем хочется: правило записано, а привычки нет.

Корневая причина. Путаница между электрическим стандартом и координатой вывода, усугублённая размером интерфейса: параллельная `debug`-шина раздувала верхний модуль примерно до семидесяти линий, уследить за ними вручную нереально.

Что сделали в переносе. Два механизма, оба архитектурные, а не косметические. Первый: параллельная `debug`-шина ликвидирована как класс — её роль выполняют AXI4-Lite для процессорной системы и модуль `spi_selftest` для автономной проверки, переносить стало нечего. Второй: сборочный скрипт в режиме `standalone` **отказывается** генерировать битстрим, если в XDC остался хотя бы один маркер `TBD`. Отказ сделан фатальным, а не предупреждением, сознательно: подача битстрима с неверной распиновкой способна повредить плату. В отчёте о размещении выводов реальной сборки все выводы имеют статус `FIXED`, то есть назначены явно, с указанием банка и стандарта.

На SoC цена такой ошибки выше, чем на обычной FPGA. Банк ввода-вывода питается своим

напряжением VCCO, и объявленный в XDC стандарт обязан ему соответствовать. Объявить LVCMOS33 в банке, переключенном на плате на 1.8 V, — это не «не заработает», это риск для железа; разбор для нашей платы — в части IV.

Правило на будущее. После любого изменения портов верхнего модуля сверяйте файл назначений автоматически — грег'ом или отчётом инструмента, — а не верой в то, что «файл рядом лежит и раньше работал». И держите в голове разницу между электрикой и географией: наличие одной строчки ничего не говорит о наличии другой.

Сводка: что из этих шести дел уносить с собой

Подведём итог так, как он выглядит с точки зрения человека, который завтра сядет писать драйвер или проводить лабораторную работу. Дефекты A-1 и A-2 сейчас не мешают ничему: они срабатывают только на экстремальных значениях параметров, которых в рабочих конфигурациях нет, и вывод из них методологический — не крутите параметры наугад, а если крутите, проверяйте синтезом. A-4 не ломает работу, но путает отладку, и лечится одной строчкой в софте: после записи делителя прочитайте его обратно. A-6 на Zynq снят архитектурой — семидесятибитной GPIO-шины больше нет, XDC пишется с нуля, сборка защищена от маркеров TBD, и требуется только не возвращать удалённое.

А вот два дела влияют на эксплуатацию прямо сейчас, и их надо помнить наизусть:

1. **Делитель не меньше двух.** $\text{max_speed_hz} = \text{f_clk} / 6$, не $\text{f_clk} / 4$. При 50 МГц это 8.33 МГц, при 100 МГц — 16.7 МГц. Частоту системного блока берите из системы, а не из константы.
2. **Переполнение приёма ищите в `IRQ_STATUS.ERROR`, а не в `STATUS[9]`.** Девятый бит мёртв, и никакое квитирование его не оживит.

На этом внутреннее устройство ядра закончилось. Мы знаем, что у него один тактовый домен и почему это подарок для переноса; из каких четырёх модулей оно собрано и какие контракты действуют на их границах; знаем карту регистров и шесть мест, где реальность расходится с документацией. Следующий шаг — надеть на этот контракт системную шину так, чтобы ни один его пункт не пострадал: как превратить параллельный memory-mapped интерфейс в AXI4-Lite, какие четыре отклонения пришлось принять и почему на обвязку написаны отдельные тесты, разбирается в [части III](#). Тем, кто хочет сразу увидеть это железо живым, стоит заглянуть в [часть V](#): там ядро впервые запускается на плате без всякого Linux.

Часть III. Parallel MM → AXI4-Lite

В части II мы разобрали ядро изнутри и закончили главу 9 неприятным обещанием: у параллельной шины `spi_reg_if` есть короткий, но коварный контракт — одноктактовые стробы, комбинационное чтение, побочный эффект при чтении RX. Пока ядром управлял внутренний автомат или лабораторный стенд, контракт соблюдался сам собой. Теперь хозяином становится процессор, и между его инструкцией `store` и сигналом `wr_en` внутри регистрового файла появляется целый протокол. Эта часть — про то, как мы построили мост через эту пропасть, почему он получился именно таким, за что мы заплатили и в каком месте счёт был предъявлен позже, уже в Linux. Если вы никогда не видели шинных протоколов, не пытайтесь запомнить имена сигналов в главе 13 — запомните одну идею рукопожатия, всё остальное выводится из неё.

Глава 12. Зачем SoC нужна шина

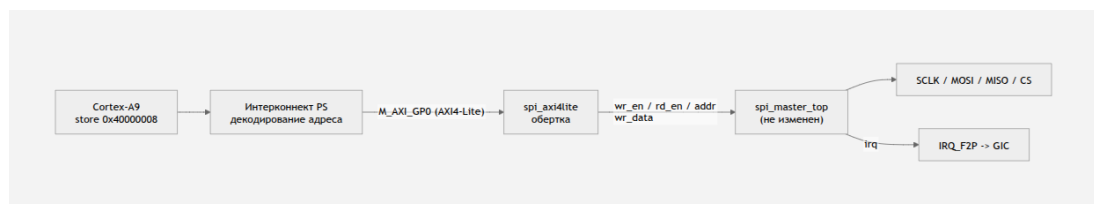
12.1. Проблема, которой не было на «голой» FPGA

На Cyclone IV в отладочной ревизии проекта всё было просто до неприличия. Хост — внешний контроллер на проводах или внутренний автомат в той же ПЛИС — просто дёргал провода. Хочешь записать регистр: выставь `addr`, выставь `wr_data`, подними `wr_en` на один такт. Хочешь прочитать: выставь `addr`, подними `rd_en`, в том же такте заведи `rd_data`. Никакого протокола, никаких согласований, никаких ответов. Провод и такт.

На Zynq так нельзя, и причина не в капризах Xilinx. Zynq — это **SoC**, System-on-Chip: на одном кристалле живут две очень разные половины. **PS** (Processing System) — это готовый, зашитый в кремний блок с двумя ядрами ARM Cortex-A9, контроллером DDR, USB, Ethernet и прочей периферией. **PL** (Programmable Logic) — это собственно ПЛИС, та самая матрица, куда попадает наш Verilog. Программа выполняется в PS. Наше ядро живёт в PL. И вот тут возникает вопрос, которого не было на Cyclone IV: а как именно инструкция, выполняемая ядром ARM, доберётся до провода `wr_en` внутри `spi_reg_if`?

Процессор умеет ровно две вещи в отношении внешнего мира: `load` (прочитать слово по адресу) и `store` (записать слово по адресу). Всё. У него нет инструкции «дёрни провод». Поэтому единственный способ управлять железом — сделать так, чтобы часть адресного пространства процессора вела не в память, а в наш блок. Это и называется **MMIO**, memory-mapped I/O, отображение устройства в память. Мы уже пользовались этим словом в главе 10, теперь объясним его буквально: адрес `0x4000_0008` не соответствует никакой ячейке DDR; когда процессор пишет туда слово, адрес и данные уезжают по внутренней шине в PL, где наш блок их узнаёт и превращает в запись регистра `CLK_DIV`. Обратно тем же путём: `load` по `0x4000_0008` возвращает не содержимое памяти, а константу `0x53500200`, которую наша логика выставляет на шину чтения.

Слово «уезжают по внутренней шине» и есть всё содержание этой части. Между физическим адресом в инструкции и стробом `wr_en` лежит **интерконнект PS** — коммутатор, который смотрит на адрес, решает, какому получателю его отдать, и транспортирует запрос вместе с данными. А чтобы получатель и коммутатор понимали друг друга, им нужен общий язык. Этот язык — протокол шины.



Обратите внимание на границу переноса. Всё правее `spi_axi4lite` приехало с Altera без единого изменения логики, всё левее — готовое железо Xilinx. Наш вклад здесь ровно один прямоугольник, и он единственный говорит на двух языках сразу.

12.2. Почему AXI4-Lite, а не что-нибудь другое

Выбор протокола выглядит формальностью — «Xilinx же, значит AXI» — но за ним стоят отброшенные альтернативы, и полезно проговорить, почему они отброшены.

AXI4 full — старший брат нашего протокола. Его главное отличие в поддержке **burst-пакетов**: одна транзакция несёт адрес и признак «дальше идут N слов подряд», после чего данные летят потоком без повторной передачи адреса. Для памяти это бесценно, для одиннадцати управляющих регистров — бессмысленно. Регистры не лежат «подряд» в смысле данных: записать `CONTROL` и следом `STATUS` одним пакетом невозможно, потому что `STATUS` вообще только для чтения. Burst добавил бы в обёртку счётчик длины, обработку типа `INCR / WRAP`, идентификаторы транзакций и

логику ответов на каждый beat — десятки строк, ни одна из которых никогда не выполнялась бы в реальной работе. И, что важнее, каждая из них была бы новым местом, где можно случайно сгенерировать два stroba rd_en там, где нужен один. Полноценный AXI4 нам всё-таки понадобится — но не для регистров, а для потока данных в DMA, и там он появится в виде готового IP от Xilinx, см. часть IX.

Wishbone — открытый протокол, знакомый по миру OpenCores, технически простой и вполне достаточный для регистровой периферии. Проблема не в нём, а в окружении: PS Zynq выдаёт наружу порты M_AXI_GP0/GP1, говорящие на AXI, и никакого другого языка не знает. Использовать Wishbone означало бы написать дополнительный мост AXI→Wishbone, то есть добавить прослойку ради прослойки. Vivado, кроме того, умеет автоматически подключать AXI-slave в Block Design и раздавать ему адреса через Address Editor; Wishbone-блок пришлось бы соединять руками. Мы бы заплатили лишним кодом и потерей инструментальной поддержки, а получили бы ровно ноль.

EMIO GPIO — самый экзотический из рассмотренных вариантов, и упомянуть его стоит, потому что новичкам он часто приходит в голову первым. PS умеет вывести десятки своих GPIO-линий прямо в PL (это и называется EMIO). Технически можно взять шестьдесят с лишним линий, объявить их шиной addr / wr_data / wr_en и буквально воспроизвести параллельный интерфейс Altera, «дёргая провода» из программы. Сблэзн понятен: ноль нового RTL, полная эквивалентность. Дальше начинается расплата. Каждое переключение GPIO из процессора — это отдельная запись в регистр контроллера GPIO, то есть один такт wr_en превращается в несколько десятков инструкций и сотни наносекунд. Никакого DMA к такой шине не подключить. Прерывание пришлось бы заводить отдельной линией. И главное — на этой конструкции нельзя построить нормальный Linux-драйвер: ядро оперирует ioremap и регистрами, а не последовательностями «подними бит 17, подожди, опусти бит 17».

Остаётся **AXI4-Lite**: подмножество AXI без burst, без внеочередного выполнения, с транзакциями фиксированной ширины в одно слово. Ровно то, для чего он придуман, — «десяток управляющих регистров». Цена решения: наш IP теперь формально обязан соблюдать спецификацию AMBA, а мы, как выяснится в главе 14, соблюдаем её с четырьмя оговорками. Это честная цена, и она задокументирована.

12.3. Почему не переписали spi_reg_if «сразу на AXI»

Второй развилкой был вопрос, где провести шов. Можно было выбросить параллельную шину и переделать сам регистровый файл так, чтобы он говорил на AXI напрямую. Так делают многие, и получается компактнее: один модуль вместо двух, нет промежуточного контракта, нет лишних тактов на переключивание.

Мы сознательно этого не сделали, и у решения три опоры.

Первая — **эквивалентность**. Весь смысл переноса, заявленный в части I, состоял в том, что ядро приезжает с Altera неизменным, а значит его поведение можно сравнивать с оригиналом побайтово. Файл spi_reg_if.v байт-идентичен исходнику, и это проверяемое утверждение, а не декларация. Стоит переписать в нём интерфейс — и аргумент рассыпается: любое расхождение поведения теперь придётся доказывать заново, потому что изменилось не только окружение, но и сам код.

Вторая — **переиспользование**. Тот же самый параллельный интерфейс кормит spi_selftest — автономный автомат, который гоняет ядро без всякого процессора. Если бы ядро умело только AXI, для автономного bring-up пришлось бы писать AXI-мастера на Verilog, то есть создавать вторую сложную сущность там, где сейчас хватает счётчика и таблицы конфигураций. Подробнее об этом в главе 15 и в части V.

Третья — **разделение ответственности**. Когда шов проходит по границе модулей, дефекты локализуются. Сломался handshake на шине — ищите в обёртке, там сто с небольшим строк. Врёт SPI на осциллографе — ищите в `spi_engine`, обёртка тут ни при чём. Если бы протокол шины и протокол SPI жили в одном файле, каждый баг начинался бы с вопроса «а это вообще чьё?». В отчёте о переносе этот приём записан как *bus wrapper around neutral core*: параллельная шина остаётся внутренним контрактом ядра, AXI — платформенным фасадом.

12.4. Что обязан сохранить мост

Теперь самое важное место главы. Регистровый файл ядра — не абстрактный «набор ячеек»: у него есть три поведения, зависящие от тактов, и невнимательный адаптер ломает их молча — так, что дизайн собирается, а данные пропадают.

Первое поведение: `wr_en` и `rd_en` — это **стробы длиной ровно один такт**, а не уровни. «Строб» здесь значит «импульс-событие»: подняли на такт — произошло ровно одно действие. Если мост поддержит `rd_en` три такта, потому что AXI столько согласовывал ответ, регистровый файл выполнит три чтения. Второе: `rd_data` **комбинационный** и открывается только тем тактом, в котором активен `rd_en`; в остальное время там ноль. Значит захватить его нужно точно в этом такте — тактом позже вы запишете ноль и будете долго думать, почему регистры «не читаются». Третье, самое опасное: чтение `RX_DATA` имеет **побочный эффект** — оно вынимает слово из очереди. Побочный эффект (side effect) — это когда операция не только возвращает значение, но и меняет состояние системы; читать такой регистр «на всякий случай» нельзя, потому что «всякий случай» стоит одно принятое слово.

Свойство ядра	Что произойдёт, если мост его нарушит
стробы ровно 1 такт	двойной pop RX или двойной push TX — данные теряются молча
<code>rd_data</code> комбинационный	прочитаете ноль или значение прошлого доступа
pop-on-read у <code>RX_DATA</code>	«посмотрю регистр ещё раз» съедает слово
слово 32 бита, byte-enable нет	частичная запись исказит весь регистр

В плане миграции это записано как риск **M-6**, и закрыт он не обещанием, а конструкцией: автомат обёртки обслуживает одну транзакцию за раз, а стробы декодируются комбинационно из **регистрового** состояния автомата. Раз состояние живёт ровно один такт, строб тоже живёт ровно один такт — по построению, а не «потому что мы понадеялись на длительность READY». Проверяется это тестами 19 (ровно один pop на чтение), 20 (ровно один push на запись) и 21 (чтение всех регистров) в `../sim/tb_spi_axi4lite.v`.

12.5. Сколько стоит мост

Приятная часть. По иерархическому отчёту об использовании ресурсов из ООС-сборки Vivado (`../migration.md`, раздел про ресурсы) весь блок `spi_axi4lite` занимает 405 LUT и 406 триггеров, из которых **собственная логика AXI — 12 LUT и 79 триггеров**; остальное — ядро и его FIFO. Эти два числа опровергают распространённый страх «шина — это дорого». Мост на системную шину для регистровой периферии действительно пишется тонкой прослойкой: семь состояний автомата, пара регистров адреса и данных, защёлка для прочитанного слова. Всё, что дороже, обычно означает, что в мост заползла логика, которой там не место.

Глава 13. AXI4-Lite за 40 минут

13.1. Рукопожатие: одна идея, из которой следует всё остальное

Если вы не знаете про AXI ничего, начните отсюда и не торопитесь. Весь протокол построен на одном приёме, который называется **рукопожатие** **VALID/READY** (handshake).

Представьте, что вы передаёте коллеге кружку с горячим чаем. Плохой способ: разжать пальцы в тот момент, когда вам удобно, и надеяться, что коллега уже подставил руку. Хороший способ: вы говорите «держи» и не отпускаете, пока коллега не скажет «беру». Обмен состоялся не тогда, когда захотел отдающий, и не тогда, когда захотел принимающий, а в момент, когда прозвучали **обе** фразы.

В AXI «держи» называется **VALID** и его выставляет источник данных; «беру» называется **READY** и его выставляет приёмник. Передача одной порции информации происходит в тот такт, когда оба сигнала подняты одновременно. Всё остальное — следствия. Источник не имеет права снять **VALID** или изменить данные, пока не дождался **READY**: нельзя передумать, уже сказав «держи». Приёмник имеет право держать **READY** внизу сколько угодно: если он занят, отправитель просто ждёт. Ни одна из сторон не обязана угадывать скорость другой, и именно поэтому AXI одинаково работает между блоками, различающимися по быстродействию в сто раз.

Сравните это с параллельной шиной ядра, где `wr_en` — просто импульс: отправитель объявляет событие, а получатель обязан быть готов всегда. Такая шина проще и быстрее, но она требует, чтобы обе стороны заранее договорились о темпе. Шина процессора этого позволить не может — она соединяет блоки, о которых ничего не знает.

Отсюда же берётся первое практическое правило отладки: если транзакция «зависла», смотрите, какая половина рукопожатия не состоялась. Либо мастер не поднял **VALID** (не туда обратились, не тот адрес декодировался), либо slave не поднял **READY** (наш автомат застрял в состоянии). Третьего не бывает.

13.2. Пять каналов

Второй кирпич: в AXI не одна шина, а несколько независимых **каналов**, и в каждом работает своё отдельное рукопожатие. Канал — это группа проводов с собственной парой **VALID / READY**. Записи нужны три канала, чтению — два.

Канал	Куда идёт (для slave)	Что несёт	Ключевые сигналы
AW	вход	адрес записи	<code>awaddr</code> , <code>awprot</code> , <code>awvalid</code> , <code>awready</code>
W	вход	данные записи	<code>wdata</code> , <code>wstrb</code> , <code>wvalid</code> , <code>wready</code>
B	выход	ответ на запись	<code>bresp</code> , <code>bvalid</code> , <code>bready</code>
AR	вход	адрес чтения	<code>araddr</code> , <code>arprot</code> , <code>arvalid</code> , <code>arready</code>
R	выход	данные чтения	<code>rdata</code> , <code>rresp</code> , <code>rvalid</code> , <code>rready</code>

Разделение адреса и данных записи на два канала (AW и W) поначалу кажется избыточным: зачем два рукопожатия там, где можно одно? Затем, что в большой системе адрес и данные могут прийти в разном темпе — адрес уже известен, а данные ещё считаются, или наоборот. AXI разрешает им приезжать в любом порядке и с любым разрывом. Наша обёртка этой свободой не пользуется: она ждёт, когда `awvalid` и `wvalid` окажутся подняты одновременно, и только тогда начинает работу.

Для AXI4-Lite это законно — спес требует лишь, чтобы slave корректно обслуживал любой порядок, а «корректно» включает «дождаться второго канала».

Канал В нужен, чтобы запись имела **завершение**. Без ответа мастер не знал бы, когда операция действительно выполнена, и не смог бы гарантировать порядок между записью и последующим чтением. Именно поэтому в драйверах существует понятие barrier: без ответа В невозможно сказать «эта запись точно дошла».

Обратите внимание на сигналы `awprot / arprot` — трёхбитные признаки уровня привилегий и типа доступа. Наша обёртка их не смотрит: защита доступа к периферии на этом уровне нам не нужна, а на уровне системы её обеспечивает MMU. В коде они явно заведены в «мусорный» провод `unusedok`, чтобы линтер жаловался на «не используется», а не на «не подключено», — намерение видно при ревью.

13.3. Запись CONTROL по тактам

Хватит абстракций, посмотрим на настоящий обмен. Возьмём типовое действие: процессор пишет в CONTROL (смещение `0x00`) значение `0x3` — одной записью включает контроллер (EN) и запускает транзакцию (START). Ниже — что при этом происходит в автомате обёртки; имена состояний взяты из `../rtl/spi_axi4lite.v`.

```
localparam [2:0] ST_IDLE   = 3'd0,
                ST_WACC    = 3'd1,
                ST_WPULSE   = 3'd2,
                ST_WRESP    = 3'd3,
                ST_RACC     = 3'd4,
                ST_RCAP     = 3'd5,
                ST_RRESP    = 3'd6;
```

Объяснить с 

	ТАКТ 0	ТАКТ 1	ТАКТ 2	ТАКТ 3	ТАКТ 4
state	ST_IDLE	ST_WACC	ST_WPULSE	ST_WRESP	ST_IDLE
s_axi_awvalid	1	1	0	0	0
s_axi_wvalid	1	1	0	0	0
s_axi_awready	0	1	0	0	0
s_axi_wready	0	1	0	0	0
wr_addr_r	-	-	0x0	0x0	0x0
wr_data_r	-	-	0x3	0x3	0x3
core_wr_en	0	0	1	0	0
s_axi_bvalid	0	0	0	1	0
s_axi_bready	1	1	1	1	1

Объяснить с 

Разберём по шагам. В такте 0 автомат сидит в `ST_IDLE` и видит, что мастер поднял `awvalid` и `wvalid` одновременно. Он решает обслужить запись и по фронту, завершающему такт 0, поднимает оба `ready`. В такте 1 (`ST_WACC`) оба `ready` подняты, значит рукопожатия обоих каналов состоялись именно сейчас — адрес и данные приняты. Спецификация гарантирует, что `awaddr` и `wdata` стабильны, пока `valid` подняты, поэтому защёлкивать их в этом такте безопасно. Автомат берёт `s_axi_awaddr[5:2]` в `wr_addr_r`, `s_axi_wdata` в `wr_data_r` и снимает `ready`.

Такт 2 — сердце всей конструкции. Состояние равно `ST_WPULSE`, и строб для ядра собран одной комбинационной строкой:

```

wire core_wr_en = (state == ST_WPULSE) &&
    (wr_addr_r != ADDR_STREAM_CTRL) &&
    (wr_addr_r != ADDR_STREAM_STATUS) &&
    (wr_addr_r != ADDR_FAST_DIV) &&
    (wr_addr_r != ADDR_ID);

```

Объяснить с 

Состояние — регистр, оно держится ровно один такт, значит `core_wr_en` держится ровно один такт. Это и есть «доказуемость по построению»: чтобы строб растянулся на два такта, автомату пришлось бы дважды подряд оказаться в `ST_WPULSE`, а переход из него безусловный. Проверять нечего — свойство видно глазами. Четыре условия неравенства отсекают адреса, принадлежащие самой обёртке (`STREAM_CTRL`, `STREAM_STATUS`, `FAST_DIV`, `ID`): их запись обрабатывается здесь же, но ядру строб не отдаётся, иначе `spi_reg_if` получил бы обращение по индексу, о котором ничего не знает.

Такт 3 — `ST_WRESP`: поднят `bvalid`, обёртка сообщает «записано, всё в порядке». Мастер подтверждает своим `breaddy`, и автомат возвращается в `ST_IDLE`. Итого четыре такта на транзакцию записи. При 50 МГц в PL-only сборке это 80 нс; при 100 МГц в сборке с DMA — 40 нс.

13.4. Чтение RX_DATA по тактам

Чтение устроено симметрично, но в нём есть такт, за который отвечать придётся всю оставшуюся часть статьи. Читаем `RX_DATA` по смещению `0x18`.

	такт 0	такт 1	такт 2	такт 3	такт 4
state	ST_IDLE	ST_RACC	ST_RCAP	ST_RRESP	ST_IDLE
s_axi_arvalid	1	1	0	0	0
s_axi_arready	0	1	0	0	0
rd_addr_r	-	-	0x6	0x6	0x6
core_rd_en	0	0	1	0	0
core_rd_data	0	0	0x5A	0	0
rdata_r	?	?	?	0x5A	0x5A
pop RX FIFO	-	-	по фронту в конце такта 2		
s_axi_rvalid	0	0	0	1	0
s_axi_rready	1	1	1	1	1

Объяснить с 

В такте 0 автомат видит `arvalid` (и, что важно, не видит одновременной записи) и решает обслужить чтение. Такт 1 (`ST_RACC`) — рукопожатие канала AR, адрес `s_axi_araddr[5:2]` защёлкивается в `rd_addr_r`, `arready` снимается.

Такт 2 (`ST_RCAP`) — тот самый. Здесь одновременно происходят три вещи. Во-первых, поднят `core_rd_en`, и ядро выставляет комбинационный `rd_data` — слово `0x5A` из головы очереди приёма. Во-вторых, обёртка выбирает, что именно попадёт в ответ: для адресов, принадлежащих ей самой, подставляется собственное значение, для всех прочих — `core_rd_data`.

```

ST_RCAP: begin
    case (rd_addr_r)
        ADDR_ID:
            rdata_r <= {IP_ID, IP_VERSION};
        ...
        default:

```

```

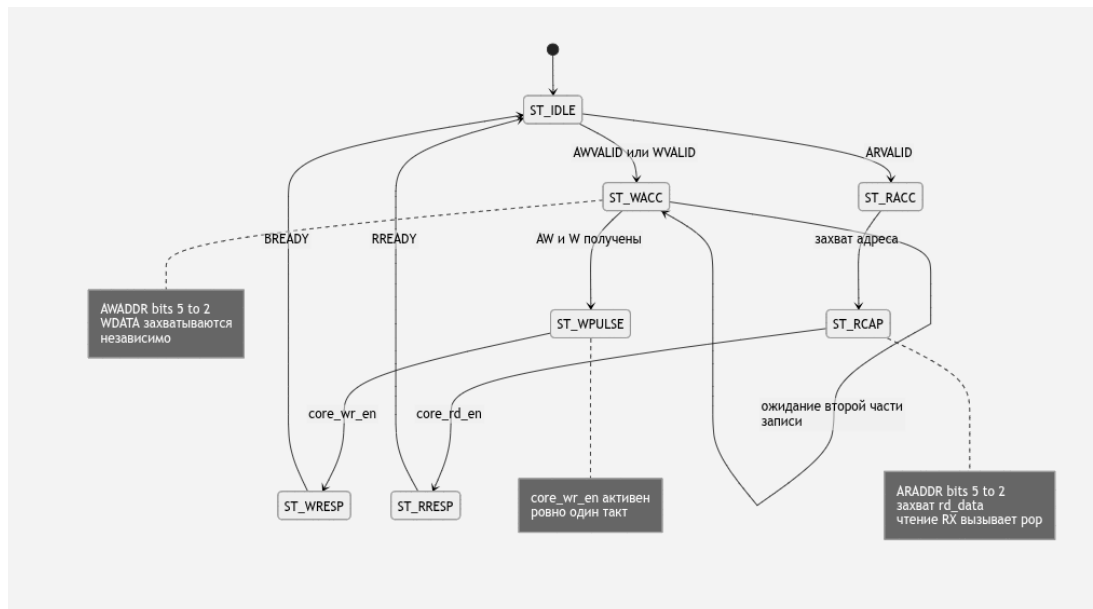
        rdata_r <= core_rd_data;
    endcase
    rvalid_r <= 1'b1;
    state <= ST_RRESP;
end

```

Объяснить с 

В-третьих — и это главное — тот же самый фронт, который защёлкивает `rdata_r`, фиксирует внутри `spi_reg_if` извлечение слова из очереди. Ровно один такт `rd_en` — ровно один поп. Не «примерно один», не «один, если интерконнект не задержит `rready`»: длительность `ST_RCAP` не зависит ни от чего внешнего. Ожидание `rready` вынесено в отдельное состояние `ST_RRESP`, где строб уже снят, и мастер может тянуть с подтверждением сколько угодно — данные уже лежат в защёлке.

Тест 19 в `../sim/tb_spi_axi4lite.v` проверяет это самым прямым способом: принимает три слова, читает `RX_DATA` трижды и смотрит на `STATUS` между чтениями. После двух чтений `RX_VALID` обязан ещё быть поднят, после третьего — упасть. Если бы обёртка выдавала два строба на одно чтение, флаг упал бы раньше, а одно из слов исчезло бы бесследно.



щё одна деталь, заметная только на схеме состояний: из `ST_IDLE` нет пути, ведущего одновременно и в запись, и в чтение. Ветка `if (awvalid && wvalid)` проверяется первой, `else if (arvalid)` — второй. Отсюда и приоритет записи, и всё отклонение D-2, к которому мы придём в следующей главе.

13.5. Адресация: байты снаружи, слова внутри

Процессор оперирует байтовыми адресами. Ядро ждёт **индекс регистра** — четыре бита, ноль для `CONTROL`, шесть для `RX_DATA`, пятнадцать для `ID`. Между ними простое соотношение: смещение = индекс × 4, потому что каждый регистр занимает четыре байта. Преобразование делает обёртка, отбрасывая два младших бита адреса и беря следующие четыре: `s_axi_awaddr[5:2]` для записи и `s_axi_araddr[5:2]` для чтения.

Почему именно `[5:2]`. Биты `[1:0]` — это положение байта внутри слова; они нам не нужны, потому что обращения всегда пословные (см. D-1). Биты `[5:2]` дают шестнадцать значений, то есть шестнадцать 32-битных регистров, то есть **окно в 64 байта**. Одиннадцать регистров ядра

плюс четыре регистра обёртки плюс одна дырка на $0x2C$ — ровно шестнадцать, всё поместилось. Ширина адресного порта задана параметром прямо в объявлении модуля:

```
parameter integer C_S_AXI_ADDR_WIDTH = 6, // 64-byte aperture (16 regs)
```

[Объяснить с](#) 

Отдельного упоминания заслуживает регистр идентификации по смещению $0x3C$ (индекс $0xF$). Он живёт **не в ядре, а в обёртке**, и это осознанно. Драйверу Linux нужно уметь отказаться от привязки к несовместимому битстриму, а для этого железо должно назвать себя. Добавить регистр в `spi_reg_if.v` значило бы нарушить байт-идентичность файла ядра и вместе с ней весь аргумент эквивалентности переноса. Обёртка же — новый код по определению, поэтому перехват одного ранее неиспользуемого адреса добавляет версионирование бесплатно. Значение читается как $0x53500100$ для PIO-сборки v1.0 и $0x53500200$ для v2.0 со stream/DMA: старшие шестнадцать бит — $0x5350$, то есть ASCII «SP», дальше major и minor.

Практическое следствие, которое стоит записать на стену: **первое обращение любого софта — чтение ID**. Если по `base+0x3C` пришло не $0x5350xxxx$, дальше отлаживать автомат SPI бессмысленно, вы смотрите не туда. Тест 27 в тестбенче проверяет этот регистр отдельно именно потому, что на нём держится вся диагностика на живой плате.

13.6. Что AXI умеет, а нам не нужно

Три возможности полного AXI мы не реализуем, и стоит объяснить каждую, потому что их отсутствие потом всплывёт в отклонениях.

Burst — пакет из нескольких слов на один адрес. AXI4-Lite его и не обещает, а регистрам он не нужен.

Outstanding transactions — способность мастера отправить второй запрос, не дождавшись ответа на первый; запросы «висят» в железе одновременно, и slave отвечает на них по мере готовности. Для памяти это способ спрятать задержку: пока едет ответ на первое чтение, уже летит второе. Для нас это яд. Пока в воздухе висят два чтения `RX_DATA`, кто-то должен помнить, какое из них уже сделало рор, а какое ещё нет. Мы отказываемся от outstanding целиком: в любой момент времени обслуживается ровно одна транзакция.

Out-of-order completion — ответы в порядке, отличном от порядка запросов (в полном AXI это разрешено благодаря идентификаторам транзакций). Для регистров с побочными эффектами перестановка чтений означала бы перестановку данных в очереди приёма. В AXI4-Lite идентификаторов нет, а у нас нет и параллелизма, так что порядок сохраняется тождественно.

Соблазн сделать «правильный, учебный» AXI-slave на все случаи жизни здесь был бы прямым врагом задачи. Каждая неиспользуемая возможность — это лишний путь в автомате, и любой из этих путей способен породить второй строб. Честные, пронумерованные и записанные в контракт отклонения лучше, чем «почти настоящий AXI», который однажды тихо съест принятое слово.

13.7. Шпаргалка для первой проверки

Когда битстрим загружен и Linux поднялся, первая проверка делается утилитой `devmem`, которая умеет читать и писать физические адреса через `/dev/mem`. База берётся из Address Editor Vivado; для актуального Block Design она равна `0x40000000` (`scripts/build_ps_axi.tcl`).

```
# 1. Паспорт IP. Всё остальное имеет смысл только после этой строки.
```

```
devmem 0x4000003C 32      # ожидаем 0x53500100 или 0x53500200

# 2. Состояние после сброса.
devmem 0x40000004 32      # STATUS, ожидаем 0x0000002A

# 3. Настроить делитель и убедиться, что он записался.
devmem 0x40000008 32 4      # CLK_DIV = 4 → 5 МГц при 50 МГц PL-клока
devmem 0x40000008 32      # обязательно прочитать обратно!
```

Объяснить с 

Три замечания к этим строкам. Ширина доступа везде **32** — прямое следствие D-1, и байтовая запись здесь не «менее удобна», а неверна. Чтение CLK_DIV обратно — не паранойя: запись нуля в этот регистр молча отвергается (дефект A-4 из [части II](#)), и хост будет уверен, что настроил делитель. И наконец, здесь демонстративно нет RX_DATA: читать его «просто посмотреть» нельзя, и почему — в разделе 14.8.

Глава 14. Обёртка spi_axi4lite и отклонения D-1...D-4

14.1. Что внутри

Модуль `../rtl/spi_axi4lite.v` состоит из четырёх смысловых частей, и ни одна из них не знает ничего о протоколе SPI. Первая — синхронизатор сброса `spi_reset_sync` на две ступени, превращающий `s_axi_aresetn` в `rst_n_sync`. Формально для сброса, приходящего из PS, он избыточен: тот уже синхронен `s_axi_aclk`. Его оставили по двум причинам, и обе записаны в комментарии к коду: контракт сброса ядра сформулирован именно в терминах «асинхронная установка, синхронное снятие», и тот же IP должен корректно работать от сырого асинхронного сброса — например, от кнопки на плате в PS-less сборке. Цена избыточности: два триггера и два лишних такта сброса.

Вторая часть — автомат из семи состояний, который мы уже разобрали по тактам, с регистрами `wr_addr_r`, `wr_data_r`, `rd_addr_r`, `rdata_r` и защёлками `awready_r` / `wready_r` / `bvalid_r` / `arready_r` / `rvalid_r`. Третья — четыре собственных регистра обёртки (`0x30` `STREAM_CTRL`, `0x34` `STREAM_STATUS`, `0x38` `FAST_DIV`, `0x3C` `ID`) и мост `spi_axis_tx`, подающий данные из AXI-Stream прямо в TX FIFO для DMA; это тема [части IX](#). Четвёртая — инстанс `spi_master_top` с параметрами `NUM_CS`, `FIFO_DEPTH`, `ADDR_WIDTH = 4` и `MISO_SYNC_STAGES`.

Показательна одна строка присваивания адреса ядру:

```
wire [3:0] core_addr = core_wr_en ? wr_addr_r : rd_addr_r;
```

Объяснить с 

У регистрового файла **один** адресный порт. Записи и чтения делят его, и выбирает между ними мультиплексор, управляемый стробом записи. Пока в системе одновременно не бывает записи и чтения, эта строка однозначна. Стоит разрешить перекрытие — и она немедленно становится источником неопределённости. Из одной строчки Verilog вырастает целое отклонение D-2, о котором ниже.

И ещё одно наблюдение, которое стоит сформулировать вслух: **ядро не знает, что снаружи AXI, а**

`spi_selftest` не знает, что AXI вообще бывает. Оба видят одну и ту же параллельную шину — ради этого в 12.3 мы и отказались переписывать регистровый файл.

14.2. Что такое «отклонение» и почему их не стыдно иметь

Дальше идут четыре пункта D-1...D-4. Прежде чем разбирать их поодиночке, нужно договориться о жанре. Отклонение — это не баг и не «мы не успели». Это место, где спецификация допускает или прямо разрешает несколько вариантов поведения, а мы выбрали не самый полный, зато самый безопасный для нашей задачи, и записали выбор так, чтобы он был виден и в коде, и в документации, и драйверу.

Разница между отклонением и багом ровно одна: **отклонение задокументировано**. Все четыре пункта живут в трёх местах сразу — в шапке `../rtl/spi_axi4lite.v`, в разделе про отклонения `../migration.md` и в таблице последствий для драйвера в `../register_map.md`. Если завтра кто-то подключит к нашему slave стандартный AXI Verification IP и получит нарекания, он сначала найдёт ответ в документе, а не заведёт дефект.

14.3. Отклонение D-1: `wstrb` игнорируется

Начнём с самого простого по формулировке и самого коварного по последствиям. Спецификация AMBA описывает канал W так: вместе с данными `wdata` мастер передаёт **строб байтов** `wstrb` — по одному биту на каждый байт слова. Бит, равный единице, означает «этот байт нужно записать»; ноль — «этот байт не трогай, оставь как был». Для 32-битной шины `wstrb` четырёхбитный: `0xF` — записать всё слово, `0x1` — только младший байт, `0xC` — только старшую половину. Механизм существует ради того, чтобы `store` одного байта в память не переписывал соседние три. Slave обязан его уважать.

Мы его не уважаем. Сигнал `s_axi_wstrb` заведён в модуль, но в автомате не используется вовсе; в такте `ST_WACC` защёлкивается `s_axi_wdata` целиком, без всякой маскировки. Чтобы это не выглядело оплошностью, он явно упомянут в «мусорном» проводе в конце файла:

```
// s_axi_awprot / s_axi_arprot / s_axi_wstrb are intentionally unused
// (D-1, D-3). Referenced here so lint reports "unused" rather than
// "undriven", and so the intent is visible in review.
wire_unused_ok = &{1'b0, s_axi_awprot, s_axi_arprot, s_axi_wstrb,
                  core_ready, 1'b0};
```

Объяснить с 

Почему не сделали «как в учебнике». Реализация `byte-enable` выглядит тривиально: собрать новое значение из старого и нового по маске и записать результат. Но для этого нужно **старое значение**, а его в общем случае нет. Регистр `CONTROL` содержит биты `START` и `SOFT_RST`, которые самоочищаются и читаются как ноль: собрать «старое слово» чтением и дописать в него байт означало бы каждый раз терять или ложно взводить эти биты. `TX_DATA` вообще не регистр, а вход очереди — «частичная запись в очередь» не имеет смысла. `RX_DATA` доступен только на чтение, и это чтение с побочным эффектом; выполнять его ради `read-modify-write` внутри записи было бы катастрофой. То есть `byte-enable` здесь не «не успели добавить», а **не определён семантически**: у регистрового файла нет такого понятия, и изобрести его — значит поменять контракт регистров и потерять эквивалентность с Altera.

Чем платим. Драйвер обязан обращаться к IP только 32-битными операциями. В Linux это означает `readl / writel` и категорический запрет на `writew` и `writel` по этим адресам. В `devmem` это означает всегда указывать ширину `32`. В коде на C это означает не пытаться «оптимизировать» запись одного бита `CONTROL` через указатель на `uint8_t`.

Как отклонение проявляется в жизни. Никакого исключения при байтовой записи не возникнет — интерконнект пропустит транзакцию с `wstrb = 0x1`, обёртка примет её и запишет в регистр **всё** слово `wdata`. Три оставшихся байта, которые мастер, скорее всего, оставил неопределёнными или нулевыми, попадут в регистр. Симптом получается издевательский: команда вроде бы «поставила один бит», а вместе с ним сбросила все остальные поля регистра. В `CS_SELECT` это переключит устройство, в `WORD_LEN` — сделает длину слова недопустимой и взведёт `sticky ERR_WORD_LEN` на ближайшем `START`, а в `CONTROL` — снимет `EN` посреди работы. И ни один из этих симптомов не указывает на настоящую причину.

Правило, которое мы из этого вынесли: **мост не должен быть умнее периферии**. Если под ним лежит пословный регистровый файл, не притворяйтесь байтовым ради галочки в спецификации. Отказ, записанный в документ, честнее, чем поддержка, работающая «в основном правильно».

14.4. Отклонение D-2: одна транзакция за раз, приоритет у записи

Спецификация AXI разрешает мастеру и slave работать по каналам чтения и записи независимо. В идеальном slave запись и чтение могут идти одновременно: пока обрабатывается запись, уже принят адрес чтения; пока формируется ответ на чтение, принята следующая запись. На этом строится конвейер, дающий в пределе одну транзакцию за такт.

Наша обёртка так не умеет и не пытается. Автомат в `ST_IDLE` смотрит сначала на пару `awvalid && wvalid` и только потом, в ветке `else if`, на `arvalid`. Если запись и чтение приходят в один такт, первой обслуживается запись, чтение ждёт своей очереди в `ST_IDLE` (сигнал `arvalid` мастер обязан держать поднятым, пока не получит `arready`, поэтому запрос не потеряется). Пока идёт любая транзакция, второй не начинается — состояние одно, и оно занято.

Почему так. Причина уже показана в 14.1: адресный порт регистрового файла единственный, и `core_addr` выбирается мультиплексором. При перекрытии чтения и записи потребовалось бы решить, чей адрес считать текущим, то есть добавить арбитраж или второй порт. Второй порт означает переписывание `spi_reg_if` — запрещено по соображениям эквивалентности. Арбитраж означает, что стробы `wr_en` и `rd_en` перестают быть однозначной функцией состояния и становятся функцией состояния и приоритетов, а вместе с этим исчезает главное свойство, ради которого всё затевалось: «один такт — один побочный эффект» перестаёт читаться глазами и начинает требовать доказательства. Мы покупаем проверяемость. AXI4-Lite это прямо позволяет — спес не требует поддержки `outstanding` и не требует перекрытия каналов.

Чем платим. Каждая транзакция занимает четыре такта, и они не перекрываются: при 50 МГц это 80 нс на доступ, то есть около 12,5 миллиона обращений в секунду в идеале. Конвейерный slave в теории мог бы выдать в несколько раз больше. Но посчитаем, где на самом деле узкое место. Максимально допустимая частота SPI в нашем ядре — 8,33 МГц при `CLK_DIV = 2` (ниже нельзя, дефект A-3 из [части II](#)). Одно восьмибитное слово при этой частоте едет почти микросекунду. На передачу и приём одного слова в режиме PIO нужно два обращения к шине — `push` в `TX_DATA` и `pop` из `RX_DATA`, — то есть 160 нс из без малого тысячи. Шина здесь не ограничитель; ограничитель — сам SPI.

Как отклонение проявилось в жизни. Ровно так, как и должно: **никак**, пока мы оставались в PIO. Драйвер работает через `readl / writel`, каждая из которых и так синхронная; перекрытие ему нечем воспользоваться. Проблема пропускной способности пришла с другой стороны — с дисплеем ST7789, где кадр это поток из многих тысяч слов, и на каждое слово приходится обращение процессора к шине плюс проверка состояния FIFO. И здесь важно понять, что D-2 не был исправлен, а был **обойдён структурно**: поток данных перестал ходить через регистровый интерфейс вообще. AXI DMA выкладывает слова в AXI-Stream, мост `spi_axis_tx` внутри обёртки заводит их прямо в TX FIFO сигналом `ext_tx_wr`, минуя и AXI4-Lite, и автомат, и всякую сериализацию. Регистры остались тем, чем должны быть, — каналом управления, а данные пошли

своей дорогой. Подробности — в [части IX](#).

Правило на будущее: для регистров с побочными эффектами сериализация — не недоработка, а страховка. Прежде чем оптимизировать шину, посчитайте, где действительно теряется время; чаще всего оказывается, что оптимизировать надо не протокол, а маршрут данных.

14.5. Отклонение D-3: всегда OKAY

В каналах B и R есть двухбитные поля ответа: `bresp` для записи и `rresp` для чтения. Значение `0b00` (OKAY) означает «выполнено», `0b10` (SLVERR) — «slave сообщает об ошибке», `0b11` (DECERR) — «такого адреса нет, ответить некому». В учебной схеме slave обязан вернуть SLVERR при обращении по незнающему смещению, и на лекциях это подают как признак приличия.

Мы возвращаем OKAY всегда, двумя строками, в которых даже нет логики:

```
assign s_axi_bresp = 2'b00;          // OKAY (D-3)
...
assign s_axi_rresp = 2'b00;          // OKAY (D-3)
```

[Объяснить с](#) 

Причина — снова эквивалентность. Параллельная шина ядра не имела понятия «ошибка адреса»: обращение по незнающему индексу читалось нулём, запись игнорировалась, и ничто об этом не сообщало. Так вело себя железо на Altera, и так же должно вести себя железо на Zynq, иначе софт, написанный под старый контракт, начнёт получать исключения там, где раньше молча работал. Ужесточать контракт в середине переноса — верный способ смешать в одну кучу дефекты переноса и дефекты старого софта.

Побочный технический эффект: Vivado при синтезе выдаёт восемь предупреждений `Synth 8-3917` — «`bresp / rresp` подключены к константе 0». В отчёте `../migration.md` этот класс предупреждений классифицирован как **допустимый** с явной пометкой «намеренно, отклонение D-3». Это, кстати, отдельный маленький урок: предупреждение синтезатора не отменяется молчанием, оно отменяется записью в документе. Иначе через полгода кто-то «починит» его, вернув SLVERR, и сломает контракт.

Чем платим и как это проявляется в жизни. Обращение по пустому смещению `0x2C` не даст ни исключения, ни признака ошибки: чтение вернёт ноль, запись уйдёт в никуда, ответ будет OKAY. Значит опечатка в константе смещения не будет поймана железом. Драйвер, который случайно читает не тот адрес, получит правдоподобный ноль и продолжит работу; в Linux это не приведёт ни к SIGBUS, ни к сообщению в `dmesg`. Особенно неприятно это выглядит в связке с диагностикой: ноль в STATUS выглядит как «контроллер мёртв», хотя на самом деле вы читаете дырку в карте. Отсюда практический вывод — проверка ID по `0x3C` приобретает двойную ценность: она отвечает не только «жив ли IP», но и «туда ли вообще указывает мой базовый адрес».

Важно не перепутать зоны ответственности. То, что мы всегда отвечаем OKAY, относится только к обращениям, которые до нас **доехали**. Если базовый адрес указан неверно и запрос вообще не попал в нашу апертуру, судьбу такого доступа решает интерконнект PS, а не наш slave, и наблюдаемый симптом будет уже другим. Ошибка адресации — свойство системы, а не периферии.

Правило на будущее: не «ужесточайте» мир в одностороннем порядке. Если старый хост никогда не видел SLVERR, новый мост не должен внезапно начать его выдавать без согласованной миграции софта. Строгость хороша, когда она запланирована.

14.6. Отклонение D-4: апертюра 64 байта и зеркалирование

Последнее отклонение самое тихое и потому самое интересное. Речь о том, сколько бит адреса мы вообще смотрим.

Адресный порт обёртки объявлен шириной шесть бит (`C_S_AXI_ADDR_WIDTH = 6`), и внутри используются биты `[5:2]` . Это значит, что блок различает шестнадцать слов, то есть **64 байта**, и ни одного бита выше пятого не существует ни физически, ни логически. Всё, что старше, интерконнект отбрасывает при маршрутизации: он уже решил, что адрес принадлежит нам, и передаёт только смещение внутри выделенного диапазона.

Проблема в том, что выделенный диапазон почти всегда больше 64 байт. Address Editor в Vivado обычно назначает AXI-периферии окно в 4 КБ — это минимум, принятый в экосистеме Xilinx, и он же кратен странице MMU, что удобно для `ioremap` . Получается несоответствие: система считает, что нам принадлежит 4096 байт, а различаем мы 64. Остальные адреса не «не существуют» — они **зеркалируются**, то есть повторяют ту же карту снова и снова, шестьдесят четыре раза подряд.

Арифметика простая, и её стоит проделать руками хотя бы раз. Возьмём смещение `0x7C` . В двоичном виде это `0b1111100` ; биты `[5:2]` дают `0b1111 = 0xF` , то есть индекс регистра ID. Читаем `base+0x7C` — получаем `0x53500200` , ровно как по `base+0x3C` . Теперь возьмём `0x54 = 0b1010100` , биты `[5:2]` дают `0b0101 = 0x5` — это TX_DATA. Запись по `base+0x54` **положит слово в очередь передачи**, хотя в карте регистров такого адреса нет. И, наконец, самое неприятное: `0x58 = 0b1011000` , биты `[5:2] = 0b0110 = 0x6` — это RX_DATA. Чтение по `base+0x58` вытолкнет слово из очереди приёма со всеми последствиями из следующего раздела.

Почему не сделали полный декодер. Потому что он не добавлял ничего, кроме кода. Полный декод означал бы сравнение всех бит адреса и выдачу DECERR за пределами карты — но DECERR мы всё равно не выдаём по D-3, значит вне карты пришлось бы возвращать ноль и игнорировать запись, то есть ровно то же поведение, что у дырки `0x2C` . Разница между «зеркало» и «нули» проявилась бы только при обращении по адресу, по которому обращаться никто не должен. За эту разницу пришлось бы заплатить сравнением широкого адреса на пути критического сигнала и лишними строками в автомате.

Чем платим и что обязан знать драйвер. Правило формулируется в одну фразу: **программный доступ разрешён только к окну `[base, base+0x40)`** . Драйвер должен считать смещения от базы и никогда не адресоваться выше `0x3C` . На практике опасность создают не осмысленные обращения, а механические: цикл, который «дампит регистры» с шагом четыре байта и не останавливается на шестнадцатой итерации, пройдёт по зеркалу второй раз и снова наткнётся на RX_DATA. Скрипт, который сравнивает содержимое всего 4-килобайтного окна «до и после», делает шестьдесят четыре рор-а из очереди приёма. Отладчик, показывающий память страницей, — то же самое. Все эти сценарии выглядят безобидно и все они разрушительны.

Правило на будущее: размер декодера — это размер полезной карты плюс явно принятый запас; окно Address Editor и фактический декод IP — **разные числа**, и знать нужно оба. Если вы видите в дизайне «регистры повторяются» — это не глюк шины, это чей-то `addr[N:2]` .

14.7. Сводка отклонений

Теперь, когда каждое отклонение разобрано, таблицу можно использовать по назначению — как памятку, а не как объяснение.

ID	Отклонение	Что обязан знать драйвер
----	------------	--------------------------

D-1	wstrb игнорируется, запись целым словом	только 32-битные доступы, writeb нельзя
D-2	Чтение и запись не перекрываются	прозрачно, влияет лишь на пропускную способность
D-3	Все ответы OKAY , дырки читаются нулём	ошибочный адрес не даст исключения
D-4	Апертура 64 байта по awaddr[5:2]	работать только в окне [base, base+0x40)

Формальные формулировки — в ../migration.md , раздел про отклонения AXI-обвязки; последствия для софта — в ../register_map.md и ../hw_sw_contract.md .

14.8. Детектив: «драйвер теряет данные»

Эта история заслуживает отдельного раздела, потому что она про то, как ломается не код, а привычка.

Симптом. На стенде с петлёй MOSI→MISO драйвер отправляет пачку из восьми слов и должен принять восемь. Принимает шесть. Иногда семь. Ровно то, что теряется, — из середины и конца пачки; первое слово всегда на месте. При повторе тест «иногда проходит». Ничто в dmesg не ругается, ошибок в STATUS нет, IRQ приходит.

Первая гипотеза, самая логичная. Драйвер теряет данные. Пропуски в конце пачки, плавающая воспроизводимость, отсутствие ошибок — классический почерк гонки между обработчиком прерывания и потоком чтения. Проверяется быстро: блокировки, порядок квитирования, повторный вход в обработчик. Ничего не находится.

Вторая гипотеза. Переполняется очередь приёма. Она правдоподобна вдвойне, потому что в части II описан дефект A-5: бит STATUS.ERR_RX_OVF никогда не выставляется, то есть переполнение RX принципиально не видно в STATUS. Значит «нет флага» ничего не доказывает. Проверяется через IRQ_STATUS.ERROR — единственный работающий индикатор. Бит чист. Переполнения нет.

Третья гипотеза. Виновата обёртка: где-то генерируется лишний строб rd_en . Гипотеза правильная по направлению и неправильная по адресу. В симуляции тест 19 проходит: три слова принято, RX_VALID держится после двух чтений и падает после третьего — ровно один рор на транзакцию. Автомат чист.

Что оказалось на самом деле. Данные забирал не тот, кто их ждал. В драйвере на время отладки была включена функция, печатающая содержимое всех регистров блока при каждом прерывании — совершенно обычная вещь, полстроки цикла: прочитать шестнадцать слов подряд от базы и вывести в лог. Одно из этих шестнадцати слов — RX_DATA по смещению 0x18 . Каждый вызов диагностики незаметно вынимал слово из очереди приёма. Сколько прерываний успело прийти за пачку, столько слов и пропало; отсюда и плавающее число потерянных слов, и зависимость от того, включён ли отладочный вывод.

Почему это так трудно поймать. Потому что нарушен не код, а модель мира. Все мы носим в голове аксиому «чтение — операция безвредная». На ней держатся привычки: посмотреть регистр ещё раз, чтобы убедиться; вывести дампы; поставить watch в отладчике; открыть окно памяти. Для RX_DATA каждая из этих привычек стоит одно слово. Инструменты усугубляют: окно памяти в отладчике обновляется само, по таймеру, без вашего участия; XSCT-команда mrd с длиной больше единицы пройдёт по RX_DATA как по обычной ячейке; hexdump на /dev/mem прочтёт

диапазон целиком. А в связке с D-4 картина становится ещё веселее: дампы 4-килобайтного окна попадают на зеркало RX_DATA шестьдесят четыре раза.

Как такое ловится. Последовательность, которая сработала и работает вообще для любого регистра с побочным эффектом:

1. посчитать, сколько слов **должно** было прийти, и сколько чтений RX_DATA выполнил драйвер по своей воле; расхождение — уже половина ответа;
2. выключить всю отладочную печать и повторить тест «вслепую», записывая результат в память, а не в лог;
3. если данные появились — включать диагностику по одной и смотреть, какая именно возвращает потери;
4. подтвердить в железе: завести `core_rd_en` и `rd_addr_r` на ILA и посчитать стробы по адресу `0x6` за одну пачку — их должно быть ровно столько, сколько слов принято.

Настоящая причина лежит не в драйвере и не в RTL, а в **контракте**. Ядро честно предупреждает: чтение RX_DATA — это рор. Обёртка честно реализует ровно один рор на транзакцию. Драйвер честно читает регистры. Каждый участник прав по отдельности, а вместе получается потеря данных, потому что никто не проверил, совместимы ли привычки отладки с семантикой регистра.

Отсюда три правила, которые с тех пор действуют в проекте. Функция дампа регистров обязана **пропускать** RX_DATA — не читать и печатать «skipped». Точки останова и окна памяти не наводятся на область IP. И — уже за пределами этой истории, но из той же оперы — квитиование через ERROR_CLR делается **после** вычитывания данных, потому что запись в ERROR_CLR флашит оба FIFO и уничтожает всё непрочитанное (`../register_map.md`).

Стоило ли вообще делать рор по факту чтения? Альтернатива очевидна: неразрушающее чтение плюс отдельная команда «сдвинуть очередь». Такой вариант дружелюбен к отладчикам, но он удваивает число обращений к шине на каждое принятое слово и добавляет состояние, которое можно рассинхронизировать — прочитать, забыть сдвинуть, прочитать то же самое ещё раз. Рор-on-read — стандартная практика для приёмных регистров, ровно так устроен приёмный буфер классического UART 16550 и большинство FIFO в SoC. Наш IP унаследовал эту семантику от Altera, и сохранить её было обязательным условием эквивалентности. Так что выбор был сделан не нами и не в этом переносе — нашей задачей было не сломать его и громко записать в контракт.

14.9. Обёртка не лечит дефекты ядра

Соблазн «подправить» ядро из моста возникает всегда, и ему надо сопротивляться. Наша обёртка не пытается ни исправлять, ни маскировать унаследованные дефекты, разобранные в части II. Запись `CLK_DIV = 1` пройдёт через AXI без единого возражения, хотя это значение портит приём (A-3): мост не знает и не должен знать, какие значения делителя осмысленны. Бит `STATUS[9]` (`ERR_RX_OVF`) останется мёртвым (A-5): обёртка отдаёт то, что выдал регистровый файл, и не «дорисовывает» флаг из своих наблюдений за очередью. Запись нуля в `CLK_DIV` так же молча отвергнется (A-4), и никакого `SLVERR` в ответ не будет — по D-3.

Это не лень, а разделение ответственности, доведённое до конца. Если бы мост начал фильтровать значения, поведение системы перестало бы совпадать с поведением ядра, и вопрос «где именно баг» снова стал бы неразрешимым. Дефекты чинят там, где они живут, а до тех пор — документируют.

Тестбенч `../sim/tb_spi_axi4lite.v` защищает это свойство явно: тест 25 утверждает документированные пределы вместо того, чтобы их обходить, а тест 26 характеризует мёртвый `ERR_RX_OVF`, то есть фиксирует дефект как ожидаемое поведение. Если однажды кто-то

«улучшит» мост, спрятав дефект за красивым ответом шины, тест немедленно покраснеет.

Глава 15. Два топа: PL-only selftest vs PS Block Design

15.1. Зачем два входа в одно ядро

К этому моменту у нас есть ядро и мост к процессору. Логично было бы собрать одну систему и начать её отлаживать. Мы держим **две**, и это отдельное архитектурное решение со своей ценой.

Топ / контекст	Кто хост	Когда используется
<code>spi_zynq_top</code> + <code>spi_selftest</code>	автомат по parallel MM	bring-up: пины, петля, ILA, без Linux
Block Design + <code>spi_axi4lite</code>	Cortex-A9 через AXI GP0	драйвер, devmem, дисплей, DMA



Путь А отвечает на вопрос «живо ли железо и верны ли назначения выводов». Путь В — на вопрос «видит ли процессор тот же самый IP по правильному адресу». Это разные вопросы, и смешивать их дорого. Если начинать сразу с Linux, то любая из пяти-шести независимых причин — неверный XDC, невыполненный `ps7_init`, неправильный адрес в Address Editor, ошибка в Device Tree, баг в драйвере, несовместимый битстрим — даёт **один и тот же** симптом: «SPI не работает». Разделение на два топа превращает одну неразрешимую задачу в две решаемые.

Обратите внимание, что оба пути соединяются с ядром через одну и ту же параллельную шину. Это прямое следствие решения из 12.3: раз ядро не знает про AXI, ему всё равно, кто им управляет.

15.2. spi_selftest как замена LCD-дашборда

В Altera-версии наблюдаемость обеспечивал экран: на панели 480×272 отображались регистры, счётчики и строка состояния. При переносе эта подсистема исключена (пункт M-3 в `../migration.md`) — такой панели на плате TZT RK-ZYNQ7020-F нет, а `spi_probe` был жёстко привязан к тачскрину XPT2046, которого тоже нет. Потеря реальная, и её пришлось компенсировать.

Компенсировали автоматом `../rtl/spi_selftest.v`. Он сам программирует регистры ядра — `CLK_DIV`, `WORD_LEN`, `CS_SELECT`, `DELAY_CFG`, — сам кладёт слова в очередь передачи, сам запускает транзакции и сам квитирует ошибки, перебирая пять конфигураций подряд. Значения по умолчанию выбраны из соображений удобства приборов: `CLK_DIV_VAL = 24` даёт 1 МГц при 50 МГц системного блока — частота, на которой любой логический анализатор читает сигнал без напряжения. Параметр `CONTINUOUS` заставляет автомат гонять транзакции без пауз, и это не эстетика: осциллограф гораздо охотнее показывает устойчивую картинку, чем «иголку раз в

секунду».

Верхний уровень `../rtl/spi_zynq_top.v` добавляет к этому два индикатора: `led[0]` — heartbeat примерно 1,5 Гц от обычного счётчика (он мигает всегда, пока жив блок и снят сброс, и именно поэтому он полезен), `led[1]` — эвристический признак присутствия слейва. Там же живёт параметр `DIAG_PINWALK`: при его включении `SCLK`, `MOSI` и `CS` перестают приходить от движка и подключаются напрямую к разным разрядам свободнобегущего счётчика.

```
assign spi_sclk = DIAG_PINWALK ? diag_cnt[4] : eng_sclk;
assign spi_mosi = DIAG_PINWALK ? diag_cnt[6] : eng_mosi;
assign spi_cs_n = DIAG_PINWALK ? {NUM_CS{diag_cnt[8]}} : eng_cs_n;
```

Объяснить с 

Смысл режима — отделить вопрос «правильно ли назначены выводы» от вопроса «правильно ли работает протокол». Каждый вывод машет своей, заведомо различной частотой, и щупа хватает, чтобы за минуту сопоставить имена в XDC с контактами разъёма. Если здесь не сходится, идти в протокол незачем; процедура описана в `../bringup.md` и части V.

Важно понимать статус этой конструкции: `spi_selftest` — не «второй хост» и не продуктовая функция, а **измерительный прибор**. Когда пины подтверждены, он уступает место AXI и Linux. Наблюдаемость вернётся позже в виде дисплея ST7789 (часть VIII), но уже как приложение поверх работающего стека, а не как костыль отладочного верхнего уровня.

15.3. Мини-детектив: «AXI не читается, значит ядро мертво»

Симптом. Linux загрузился, `devmem` по базовому адресу возвращает нули или мусор, ID по `base+0x3C` не равен `0x5350xxxx`.

Что кажется логичным. «Сломали `spi_engine` при переносе» или «забыли атрибуты `ASYNC_REG` на синхронизаторе, и всё поехало». Обе гипотезы притягательны, потому что в них есть работа: можно открыть RTL и искать.

Почему они почти наверняка неверны. Между инструкцией `load` и регистровым файлом лежит длинная цепочка: битстрим должен быть загружен в PL, `ps7_init` должен настроить тактирование и уровни, интерконнект должен декодировать адрес, Device Tree должен описывать тот же адрес, драйвер должен сделать `ioremap` туда же. Автомат SPI в этой цепочке — последнее звено, и его поломка дала бы совсем другой симптом: ID читался бы нормально, а обмен по проводам был бы неправильным.

Как разводить гипотезы.

1. загрузить `spi_zynq_top.bit` без всякого PS и посмотреть осциллографом, есть ли `SCLK` на ожидаемом контакте — это отвечает на вопрос «жив ли PL и верен ли XDC» одним измерением;
2. открыть Address Editor и убедиться, что slave действительно попал по `0x40000000` (или по тому адресу, который использует ваш софт);
3. прочитать ID именно по `base+0x3C`, а не по «где-то в начале блока» — помните про зеркалирование из D-4 и про нули из D-3;
4. убедиться, что в PL загружен тот битстрим, что собран из текущего RTL: не устаревший DCP, не чужой `system.bit` с карты памяти.

Типичная корневая причина. Ошибка карты адресов, не тот битстрим или непрограммированная PL. Автомат SPI — практически никогда.

Правило на будущее. На SoC всегда имейте способ дёрнуть IP **без** операционной системы. Иначе в один прекрасный день вы будете отлаживать цепочку загрузки осциллографом.

15.4. Чем нам дорого держать два верхних уровня

Честный разговор о цене решения из 15.1, потому что она не нулевая. Два верхних уровня означают два маршрута сборки: PL-only проект собирается своим скриптом, система с процессором — через Block Design (`scripts/build_ps_axi.tcl`). Каждое изменение в RTL нужно проверять дважды, и дважды же убеждаться, что оба маршрута собираются без новых предупреждений. Наборы constraints у путей частично разные: в PL-only топе есть кнопка сброса и светодиоды, в BD-варианте тактирование приходит из PS, а часть выводов уходит на LCD и DMA-периферию. Расходятся и параметры: PIO-сборка живёт с `FIFO_DEPTH = 8` и ID `0x53500100`, DMA-сборка — с `FIFO_DEPTH = 1024` и ID `0x53500200` (`../hw_sw_contract.md`). Это уже два разных изделия, и путать их битстримы очень легко — собственно, ради этого ID и появился.

Есть и риск тише: путь A может незаметно **отстать**. Если менять только продуктовую сборку, автономный топ однажды перестанет собираться, и обнаружится это в самый неподходящий момент — когда он снова понадобится для диагностики. Единственная защита — держать оба маршрута в регулярной сборке и относиться к поломке PL-only топа как к обычному дефекту, а не как к «ну это же отладочное».

Почему мы всё равно считаем цену оправданной. Потому что альтернатива — платить временем в момент, когда его нет. Один вечер, потраченный на диагностику «SPI не работает» без возможности исключить платформу, стоит дороже, чем весь год поддержки второго топа. И потому что путь A даёт то, чего Linux дать не может в принципе: детерминированную, повторяемую картинку на осциллографе без единой строчки софта между вами и железом.

15.5. Куда смотреть дальше

Сигналы AXI, автомат и стробы — в `../rtl/spi_axi4lite.v`; автономный верхний уровень и его диагностические режимы — в `../rtl/spi_zynq_top.v` и `../rtl/spi_selftest.v`; проверки контракта, включая тесты 19, 20, 21, 25, 26 и 27, — в `../sim/tb_spi_axi4lite.v`. Смещения, битовые поля и значения ID подробно расписаны в `../register_map.md`; формальные формулировки отклонений D-1...D-4 — в `../migration.md`; сводка обязательств железа перед софтом — в `../hw_sw_contract.md`; лабораторная процедура — в `../bringup.md`. Справочные материалы и глоссарий — в приложениях.

15.6. Мост к следующим частям

Части 0–III закрывают вопрос «что переносим и как с этим разговаривать». Ядро разобрано, контракт шины переложен на AXI4-Lite, четыре отклонения записаны и объяснены. Дальше серия уходит туда, где нейтральный Verilog заканчивается и начинается конкретная плата: часть IV — constraints, банки, уровни и напряжения; часть V — лабораторный bring-up по пути A; часть VI — Block Design, IRQ_F2P и границы возможностей софта; часть VII — Buildroot, Device Tree и драйвер; часть IX — поток данных, который обошёл наш регистровый интерфейс стороной.

Сквозной тезис остаётся прежним и только укрепляется: RTL ядра почти vendor-neutral, изменились ровно два атрибута `ASYNC_REG`, а вся настоящая работа — в обвязке. AXI-обёртка была первой крупной добавкой: 12 LUT, 79 триггеров, семь состояний автомата и четыре отклонения, каждое из которых позже отозвалось в софте. Проверьте себя одним вопросом: если вы можете своими словами объяснить, почему D-2 — это плата за pop-on-read и почему чтение `RX_DATA` нельзя ставить в функцию дампа регистров, — вы готовы идти в Vivado и на плату без

иллюзии, что «осталось только прошить».

Часть IV. Vivado, constraints, pinout

Часть III закончилась мостом AXI4-Lite: ядро научилось разговаривать с системной шиной, отвечать на чтение и запись, отдавать прерывание. С точки зрения языка Verilog проект закончен — все сигналы имеют имена, все модули соединены. Но у него пока нет ни одного свойства, которое можно измерить прибором: он не знает, с какой скоростью должен работать, на какой физической ножке микросхемы живёт `spi_sclk` и каким напряжением на этой ножке обозначается логическая единица. Всё это описывается отдельно от RTL — в файлах `constraints`, и вот их мы сейчас и будем писать.

Слово «constraints» переводят как «ограничения», и перевод сбивает с толку: звучит так, будто мы что-то запрещаем. На самом деле это скорее **техническое задание для компилятора**. RTL описывает, *что* схема делает; constraints описывают, *в каких условиях* она обязана это делать: с каким периодом тактов, через какие физические выводы, при каком напряжении, какие пути проверять, а какие проверять бессмысленно. Компилятор без constraints не отказывается работать — он просто принимает решения сам, молча и произвольно, и потом никаких гарантий не даёт.

Отсюда важное свойство этого слоя: **ошибка в constraints не выглядит как ошибка**. Синтезатор не подчёркивает её красным. Сборка проходит, битстрим получается, отчёт бодро сообщает «all constraints are met». Расплата приходит через неделю — молчащим осциллографом, словом, сдвинутым на бит, или греющейся микросхемой.

Поэтому четыре главы этой части — не описание файлов, а разбор решений. Глава 16 — что переносится между вендорами, а что обязано быть написано с нуля. Глава 17 — два ложных клока из исходного SDC: SCLK, объявленный тактовым сигналом, и MISO, объявленный синхронным входом. Глава 18 — единственное место во всём переносе, где ошибка может физически повредить плату. Глава 19 — особенность Zynq, из-за которой успешно загруженный битстрим может не подавать признаков жизни.

Глава 16. QSF/SDC → XDC: что переносить, а что выбросить

16.1. Зачем компилятору отдельный файл требований

Начнём с самого начала, потому что без этого дальше будет непонятно. Внутри кристалла сигнал не распространяется мгновенно. Триггер — это ячейка, которая запоминает бит в момент фронта тактового сигнала; между двумя такими ячейками лежит комбинационная логика и металлические дорожки, и путь от одной до другой занимает единицы наносекунд. Чтобы схема работала, данные должны успеть добежать от триггера до триггера **за один период такта** и ещё немножко раньше, потому что приёмнику нужно время на «устаканивание» входа перед фронтом. Это требование называется **setup**. Есть и зеркальное: данные не должны прибежать *слишком быстро*, иначе они проскочат в тот же фронт и затрут предыдущее значение, — это **hold**.

Проверкой всех таких путей занимается **статический временной анализ**, STA (Static Timing Analysis). «Статический» означает, что анализатор не запускает симуляцию, а перебирает все пути по таблицам задержек производителя и считает запас — **slack**: положительный означает «успевает с запасом столько-то наносекунд», отрицательный — «не успевает». Делает он это для тысяч путей за секунды, но у него есть слепое пятно: он не знает ваших намерений. В строчке `always @(posedge c1k)` не написано, 50 у вас мегагерц или 500; не написано, какой вход приходит извне асинхронно, а какой приколочен к тому же клоку; не написано, какие пути проверять не нужно. Всё

это сообщаете вы — в constraints.

Формат этого файла индустриальный и общий для всех: **SDC**, Synopsys Design Constraints, надстройка над языком Tcl. Именно поэтому перенос выглядит обманчиво лёгким: Quartus читает `.sdc`, Vivado читает `.xdc` (Xilinx Design Constraints), и это тот же самый SDC с теми же командами. Синтаксис почти совпадает — проблема переноса не в нём, и об этом вся глава.

16.2. Два вопроса, два файла: «куда» и «как быстро»

У Altera требования к сборке жили в двух файлах, и разделение удобно держать в голове. Файл `.qsf` (Quartus Settings File) отвечал на вопрос «куда»: какой кристалл, какие исходники, какой порт топ-модуля выходит на какую ножку корпуса и каким электрическим стандартом. Файл `.sdc` отвечал на вопрос «как быстро»: период клона, внешние задержки, исключения из анализа. У Xilinx оба вопроса задаются в одном формате — XDC, причём назначение вывода стало не отдельной командой, а установкой свойства объекта-порта: `set_property PACKAGE_PIN W17 [get_ports clk]` читается как «взять порт по имени `clk` и записать ему свойство `PACKAGE_PIN`, равное `W17`». Тайминговые команды остались прежними, и соответствие получается почти механическим:

Quartus	Vivado XDC	Что это значит	Решение по переносу
<code>.qsf :</code> <code>set_location_assignment PIN_x</code>	<code>set_property PACKAGE_PIN</code>	Номер вывода корпуса	Пишется с нуля: плата другая
<code>.qsf :</code> <code>IO_STANDARD "3.3-V LVTTL"</code>	<code>set_property IOSTANDARD LVCMOS33</code>	Электрический стандарт вывода	Пишется с нуля по XDC производителя
<code>.qsf :</code> <code>WEAK_PULL_UP_RESISTOR ON</code>	<code>set_property PULLTYPE PULLUP</code>	Внутренняя подтяжка	Переносится: на MISO нужна по той же причине
<code>.sdc :</code> <code>create_clock -period 20</code>	<code>create_clock -period 20.000</code>	Период системного такта	Переносится один в один: те же 50 МГц
<code>.sdc :</code> <code>create_generated_clock ... spi_sclk</code>	—	Производный тактовый сигнал	Не переносится , см. C-2
<code>.sdc :</code> <code>set_output_delay ... spi_mosi</code>	—	Требования tSU/tH внешнего slave	Не пишется: slave не выбран, числа выдумывать нельзя
<code>.sdc :</code> <code>set_input_delay ... spi_miso</code>	<code>set_false_path -from</code>	Задержка данных от slave	Заменяется , см. C-3
<code>.sdc :</code> <code>set_false_path -from reset_n</code>	<code>set_false_path -from reset_btn</code>	Исключить путь из анализа	Переносится: обоснование то же самое
<code>.sdc :</code> <code>set_false_path на debug-шину</code>	—	Медленная параллельная шина	Не нужен: шина удалена вместе с топом

Обратите внимание на правую колонку: из девяти пунктов один в один переносится ровно один — период клона; ещё два переносятся со сменой имени порта, два пишутся с нуля, один заменяется на другое по смыслу, три не переносятся вовсе. Это и есть главный тезис главы: **синтаксис переносится, семантика — нет**.

16.3. Почему распиновка пишется с нуля, а не транслируется

Соблазн написать скрипт-транслятор QSF → XDC понятен и техничен: регулярное выражение, полсотни строк, автоматизация. Мы этого не сделали, потому что переносить оказалось нечего. Исходный `quartus/spi_master.qsf` сам себя объявляет заглушкой — в нём стоит честный комментарий `Pin assignments (PLACEHOLDERS – replace with values from your board)`, а ревизия `spi_master` никогда не собиралась под реальное железо: у неё не было платы. Транслировать заглушки в другой формат значит получить заглушки в другом формате, но уже без предупреждающего комментария, потому что комментарии при трансляции обычно теряются. Автоматизация здесь ухудшает ситуацию: превращает честное «мы не знаем» в правдоподобно выглядящий файл.

Хуже того, при разборе того же QSF обнаружился собственный дефект исходного проекта — в аудите он проходит под номером **A-6**. Файл задавал `IO_STANDARD` для шин `reg_wdata[*]`, `reg_rdata[*]` и сигнала `reg_ready`, но **не задавал для них `set_location_assignment`**: шестьдесят пять сигналов получили электрический стандарт и не получили номера вывода. Quartus в такой ситуации не ругается — он считает, что назначения могут описывать ещё не написанный дизайн, поэтому неполнота не ошибка, — и молча раскидывает осиротевшие порты по свободным ножкам. Сборка зелёная, `.sof` готов, сигналы на случайных падах, а инженер неделю водит щупом по «правильным» контактам, на которых физически ничего нет. Ирония в том, что исходный проект посвятил этому классу ошибок отдельную главу разбора и вывел из неё правило «сверять QSF после каждой правки портов топа» — и в ревизии `spi_master` собственное правило не применил.

Поэтому распиновка для TZX RK-ZYNQ7020-F v1.1 строится заново и из двух независимых источников. Первый — **штатный проект производителя**: плата поставляется с `factory image`, и вместе с ним лежит исходный проект Vivado (`image_7020`), где распиновка заведомо рабочая, потому что именно с этой прошивкой плату отгружают. Оттуда взяты `other.xdc` (PL-кноп, светодиоды, кнопки, SPI-дисплей), `io_40pin.xdc` (разъёмы расширения) и `general.xdc` (`CFGVSS VCC0`, `CONFIG_VOLTAGE 3.3`), а из блок-дизайна — частота `clk_in1 FREQ_HZ = 50000000`. Второй источник — **база устройства самого Vivado**: скрипт `../scripts/query_pins.tcl` вызывает `link_design -part xc7z020clg484-2` и для каждого вывода спрашивает, существует ли он в корпусе `clg484`, в каком банке находится и какая у него выделенная функция. Проверка механическая, а не «посмотрел глазами на схему», и закрывает она ровно тот класс ошибок, на котором сгорела неделя в Altera-проекте. Третий рубеж — процедурный: скрипт сборки `../scripts/build.tcl` в режиме `standalone` читает `board-level XDC` как текст и, если находит в нём хотя бы одну подстроку `TBD`, **прекращает работу с ошибкой**:

```
if {[regexp {TBD} $body]} {
    puts "ERROR: Refusing to generate a bitstream from an unverified pinout."
    exit 1
}
```

Объяснить с 

Фатальной, а не предупреждением, сознательно: предупреждение — это строка среди сотен строк лога, её никто не читает. А битстрим с угаданной распиновкой способен выставить выход FPGA на контакт, которым на плате уже кто-то управляет, и получить конфликт драйверов — короткое

замыкание двух источников через микросхему. Ошибка в этом месте дешевле, чем плата.



Последнее звено цепочки — обратная сверка. После имплементации Vivado печатает отчёт `report_io`, где у каждого вывода есть колонка `Constraint`, и значение `FIXED` означает «назначен явно, человеком», а не размещён инструментом автоматически. В `../reports/standalone_io.rpt` у всех одиннадцати пользовательских выводов стоит `FIXED`:

```
| V8 | spi_sclk | IO_L2P_T0_13 | OUTPUT | LVCMOS33 | 13 | FIXED |
| W11 | spi_miso | IO_L3P_T0_DQS_13 | INPUT | LVCMOS33 | 13 | FIXED | PULLUP
```

Объяснить с

Тридцать секунд с `grep` против уймы времени с анализатором — цена дисциплины в измеримых единицах.

16.4. Честный перенос SDC: что осталось и чего сознательно нет

Тайминговая часть наших `constraints` вышла короткой. Основная строка ровно одна:

```
create_clock -name pl_clk_50m -period 20.000 [get_ports clk]
```

Объяснить с

Двадцать наносекунд — это 50 МГц. Число не подобрано и не округлено: на плате стоит собственный PL-осциллятор, и `factory`-блочдизайн производителя объявляет для него `FREQ_HZ = 50000000`. Совпадение с Altera-референсом очень удачное — все значения `CLK_DIV` из исходного проекта переносятся без пересчёта, а результаты Vivado напрямую сравнимы с отчётом Quartus. Одна эта строка покрывает анализом все внутренние пути проекта: регистровый файл, оба FIFO, автомат движка, — потому что домен ровно один. Дивиденд от решения «SCLK — это enable, а не clock», принятого в [части II](#), приходит здесь.

Исключений из анализа — ровно два, оба объяснены прямо в файле `../constraints/rk_zynq7020_f_v11.xdc`. Первое снимает проверку с `spi_miso` (глава 17), второе — с кнопки сброса `reset_btn`. Про кнопку скажем сразу, случай простой: человек нажимает её пальцем, и у этого события нет и не может быть фазового соотношения с 50-мегагерцовым осциллятором. Сигнал заходит в асинхронный вход модуля `spi_reset_sync`, вся задача которого — сделать *снятие* сброса синхронным; за это отвечает RTL, а не констрейнт. Это единственное исключение, унаследованное от Altera-SDC, и унаследовано оно потому, что там было верным по той же самой причине.

Куда интереснее то, чего в файле **нет**: нет `set_output_delay` на `spi_mosi`, `spi_sclk` и `spi_cs_n`. Эта команда сообщает анализатору, сколько времени требуется внешнему устройству — его `tSU` (за сколько данные обязаны быть стабильны до фронта) и `tH` (сколько держаться после), — и числа берутся из даташита конкретного `slave`. У нас `slave` не выбран: поставляемая проверка это петля MOSI→MISO проводом, а с проводом никакого `tSU` не существует. Написать правдоподобные «5 нс» значило бы выдумать констрейнт, а выдуманный хуже отсутствующего —

отсутствующий честно виден в отчёте. В XDC вместо чисел лежит заготовка с комментарием, что дописать, когда slave появится.

У этой честности есть видимая цена, и её нужно уметь читать. Отчёт `check_timing` в сборке `spi_zynq_top` выглядит так:

```
4. checking unconstrained_internal_endpoints (0)
5. checking no_input_delay (2) (MEDIUM) -> reset_btn, spi_miso: оба с false path
6. checking no_output_delay (8) (HIGH) -> led[0..1], spi_cs_n[0..3], mosi, sclk
7. checking multiple_clock (0)
8. checking generated_clocks (0)
```

Объяснить с 

Восемь портов без выходной задержки, и Vivado честно помечает это как HIGH. Соблазн «закрывать замечание» велик, но замечание правильное и относится не к нам: незаконстрейненным остаётся **внешнее, платное** соотношение — между нашим падом и чужим устройством, которого пока нет, — тогда как путь от триггера до пада полностью проанализирован клоком `p1_clk_50m`. Метрика, обязанная быть нулём и являющаяся нулём, — это `unconstrained_internal_endpoints`; строка 4 та, ради которой отчёт читают.

Итоговый тайминг, для полноты картины: худший запас по setup **+14.645 нс** при периоде 20 нс, худший по hold **+0.082 нс**, суммарные нарушения нулевые по обоим видам, 851 конечная точка; группа `async_default` (пути снятия асинхронного сброса) даёт +15.103 нс и +0.864 нс. Дизайн не близок к пределу нигде: поиск Fmax в ООС-режиме показывает **258.1 МГц** против требуемых 50.

16.5. Три XDC вместо одного, и почему это не бюрократия

В репозитории лежат три файла `constraints`, и каждый обслуживает свой режим сборки.

`rk_zynq7020_f_v11.xdc` — board-level, единственный файл, знающий о существовании платы: выводы, стандарты, подтяжка на MISO, оба тайминговых исключения и настройки битстрима. Его читает режим `standalone`, собирающий PL-only топ `spi_zynq_top` с `spi_selftest` внутри, — именно тот битстрим поедет на плату в [части V](#).

`../../constraints/spi_ooc.xdc` — out-of-context. Режим ООС синтезирует и разводит модуль `spi_axi4lite` сам по себе, без топа, без платы и без PS: его порты — границы модуля, а не пады кристалла. Файл содержит ровно одну содержательную строку, `create_clock` на 20 нс, и большой комментарий о том, чего в нём нет и почему. Ни назначений выводов, ни задержек — падов не существует; ни исключений — в ООС-режиме `spi_miso` обычный вход модуля, полностью проанализированный клоком захвата, и объявлять его асинхронным значило бы протаскать платное исключение в цифры, которые мы предъявляем как честные показатели самого IP. Косвенное подтверждение осмысленности разделения — заголовок ООС-отчётов: они сняты на корпусе `7z020-clg400`, и это ничему не мешает, потому что корпус в этом режиме ни на что не влияет. Наконец, `../../constraints/ps_axi_pins.xdc` — вариант для сборки с Processing System и штатным дисплеем платы: там SPI выведен не на разъём, а на встроенный ST7789V (`V18 / U19 / AA13` плюс три GPIO управления панелью), а MISO подтянут внутри блок-дизайна, потому что обратной линии у панели нет вовсе. Это территория [части VI](#) и [части VIII](#); здесь файл нужен, чтобы было видно: набор выводов — свойство не проекта, а *сборки*, и таких сборок у нас три.

Глава 17. C-2 и C-3: SCLK и MISO — два ложных клоковых мира

17.1. Что такое generated clock и почему это опасное слово

Прежде чем разбирать ошибку, надо понять, что именно предлагалось объявить. Тактовый сигнал для инструмента — не просто «провод, который дёргается». Это объект анализа: у него есть период и фронты, и относительно него STA выстраивает все проверки setup и hold для тактируемых им триггеров. Команда `create_clock` создаёт такой объект на входном порту: «снаружи сюда приходит меандр с таким периодом». Команда `create_generated_clock` создаёт **производный** клок: «на этом узле схемы живёт тактовый сигнал, полученный из другого клона таким-то преобразованием — делением на N, умножением, инверсией». Типичный честный случай — выход PLL или MMCM: там действительно из 50 МГц делается 100, и инструмент обязан знать соотношение, чтобы проверять пути между доменами.

Ключевое слово здесь — **домен**. Объявляя клок, вы не описываете сигнал, вы создаёте новую систему отсчёта: все тактуемые ею триггеры попадают в отдельную группу, а любой путь, пересекающий границу между группами, становится **межкловым** и требует отдельного разговора — либо инструмент проверяет его по правилам двух клоков сразу, либо вы объявляете клоки несвязанными через `set_clock_groups`. Ни того ни другого нам не надо.

17.2. C-2. Детектив о клоке, которого нет

Исходный SDC содержал такие строки:

```
create_generated_clock -name spi_sclk_gen -source [get_ports {clk_50m}] \
-divide_by 10 [get_ports {spi_sclk}]
```

Объяснить с 

Что видели своими глазами. Первая же попытка собрать XDC «по мотивам» SDC даёт зелёный отчёт: timing closure сходится, никаких нарушений, инструмент доволен. Насторожиться заставляет другое: в отчёте `report_clocks` появляется клок, которого мы не заказывали, а `check_timing` начинает считать пути между `pl_clk_50m` и `spi_sclk_gen` — то есть внутри дизайна, где физически один осциллятор, у инструмента откуда-то два тактовых домена. На плате это ничем не проявляется до тех пор, пока кто-нибудь не поднимет частоту: тогда петля MOSI→MISO начинает читать слово, сдвинутое на бит, при полностью зелёном тайминге. Худший вид отказа — тот, о котором отчёты молчат.

Какая гипотеза казалась логичной. Она и правда логична: «SDC на Altera уже отлажен, SCLK для внешнего устройства — самый настоящий тактовый сигнал, данные MOSI и MISO живут относительно его фронтов, значит инструмент обязан о нём знать; перепишу синтаксис в XDC и пойду дальше». В мире, где SPI-мастер работает на фиксированной частоте, это было бы верно.

Как проверяли. Сначала арифметикой, прямо по RTL: в `spi_engine.v` полупериод SCLK равен $CLK_DIV + 1$ тактов системного клона, то есть полный период — $2 \cdot (CLK_DIV + 1)$, при $CLK_DIV = 24$ это 50 тактов, ровно 1 МГц от 50 МГц, а при $CLK_DIV = 2$ — шесть тактов. В SDC же зафиксировано `-divide_by 10`, и рядом стоит комментарий автора: «Pessimistic worst-case divisor used here = 10». Второй проверкой были `report_clocks` и `check_timing` на пробной сборке: лишний домен виден в списке клоков и порождает межкловы пути. Третьей — поведенческая: собрать при $CLK_DIV = 1$ и при $CLK_DIV = 2$ и посмотреть на петлю (шаг 12 в bring-up).

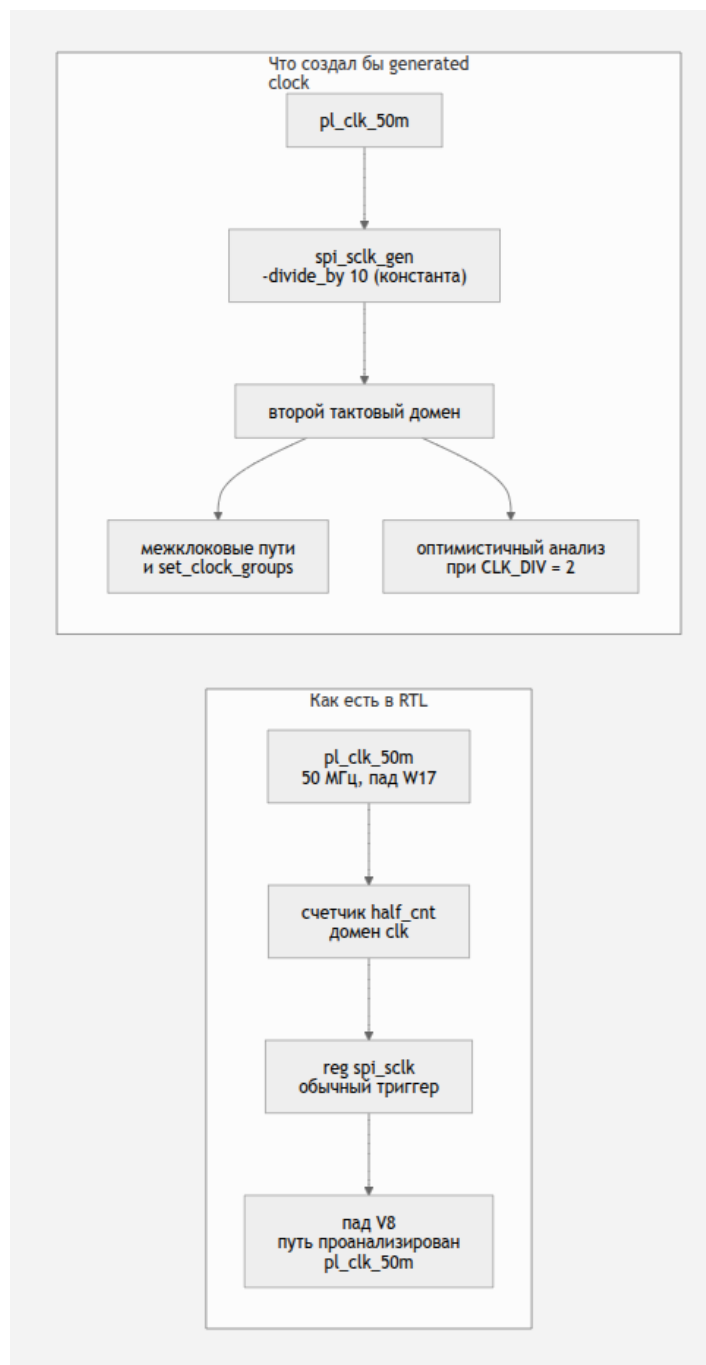
Улика, которая решила дело, — тот самый комментарий про пессимизм. Он показывает, что автор задумывался о переменном делителе и выбрал «худший случай». Но выбор сделан не в ту сторону. CLK_DIV — регистр, программируемый во время работы, диапазон 1...65535. Худший случай для тайминга — это **самый быстрый** SCLK, то есть **самый маленький** делитель, а `-divide_by 10` описывает клок *медленнее* реально достижимого. Объявленный клок с периодом больше фактического делает анализ не пессимистичным, а **оптимистичным**: STA уверенно

закрывает пути, которые на самом деле короче. Констрейнт не защищает, он усыпляет.

Корневая причина, впрочем, глубже арифметики. `spi_sclk` в этом дизайне — `output reg`, который обновляется внутри `always @(posedge clk)` по счётчику. Он не приходит ни на один тактовый вход ни одного триггера: внутри кристалла он вообще не блок, а обычный сигнал данных. Объявлять блоком имеет смысл то, что тактирует логику; здесь тактирует только `pl_clk_50m`. Всё, что даёт `create_generated_clock` в такой схеме, — фиктивный домен, межблоковые пути и необходимость закрывать `set_clock_groups` проблему, которую мы сами же секундой раньше и создали.

Исправление вышло вычитанием: в XDC нет ни одной строки `create_generated_clock`, а её отсутствие явно задокументировано в файле, чтобы читалось как решение, а не как забывчивость. SPI-выходы констрейнятся относительно системного блока — того самого, из которого они запускаются. Подтверждение берём из `check_timing : generated_clocks (0)`, `multiple_clock (0)` и, что важнее всего, `unconstrained_internal_endpoints (0)`. Ноль в первых двух строках означает, что фиктивных доменов нет; ноль в третьей — что от их отсутствия ничего не осталось непроверенным.

Правило на будущее: если сигнал не приходит на тактовый вход ни одного триггера — это не блок, как бы он ни назывался снаружи.



17.3. Что мы потеряли, отказавшись от честного клока

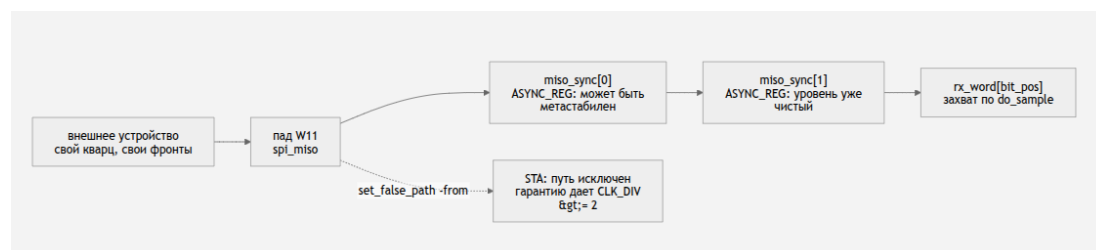
У каждого решения есть цена, и её надо назвать вслух. Отказ от `create_generated_clock` означает, что STA **не проверяет** соотношение между фронтом SCLK и данными на MOSI и CS так, как их увидит внешнее устройство: если завтра к разъёму подключат микросхему с жёстким tSU, инструмент не предупредит, он проверит только участок «триггер → пад» в домене `pl_clk_50m`. Это осознанный размен: альтернатива — объявить клок и получить *неправильную* проверку относительно константного периода там, где реальный период задаётся регистром и меняется на ходу, а неправильная проверка хуже отсутствующей, потому что выглядит как проверка. Когда slave появится, правильный путь известен и записан прямо в XDC: добавить `set_output_delay` относительно `pl_clk_50m` с числами tSU и tH из его даташита — и участок «наш триггер → пад → его вход» станет одним связным бюджетом, без промежуточного домена.

17.4. Метастабильность: почему асинхронный вход нельзя просто прочитать

Второй сюжет главы требует ещё одного понятия, и объяснить его лучше через физику, чем через формулы. Триггер запоминает вход по фронту клокa, но не мгновенно: внутри он устроен как пара инверторов, замкнутых в кольцо, и это кольцо должно «свалиться» в одно из двух устойчивых состояний. Чтобы падение произошло надёжно, вход обязан быть стабильным немного до фронта (setup) и немного после (hold). Если сигнал переключился ровно в это окно, кольцо получает толчок посередине и повисает между нулём и единицей — на выходе оказывается промежуточное напряжение, которое медленно расползается в ту или другую сторону. Это состояние называется **метастабильностью**. Аналогия — карандаш, поставленный на острие: он обязательно упадёт, но неизвестно куда и неизвестно когда, а «когда» здесь измеряется наносекундами и сверху в принципе не ограничено.

Внешний сигнал вроде MISO приходит от чужого устройства, которое живёт по своему кварцу и о наших фронтах не знает: попадание в окно неизбежно, вопрос только частоты событий. Классическое лекарство — **два триггера подряд**, 2FF-синхронизатор. Первый ловит асинхронный сигнал и имеет право провести такт в метастабильном состоянии; за целый период клокa кольцо почти наверняка успеет свалиться, и второй триггер видит уже нормальный уровень. «Почти наверняка» — не фигура речи, а вероятностная оценка: среднее время между сбоями растёт с добавленным временем экспоненциально и для двух ступеней на 50 МГц измеряется астрономическими сроками.

В нашем IP такая цепочка есть с самого начала — она была и в Altera-версии. Единственное, что добавил перенос, — атрибут `(* ASYNC_REG = "TRUE" *)` над объявлением `reg [MISO_SYNC_STAGES-1:0] miso_sync`. Это указание инструменту: «эти триггеры образуют синхронизатор». Vivado, увидев его, не станет растаскивать их по разным углам кристалла при размещении (чем короче путь между ступенями, тем больше времени остаётся на разрешение метастабильности) и не попытается их «улучшить» — вынести общую логику, продублировать регистр, слить с соседним. Это единственное отличие файла `spi_engine.v` от Altera-оригинала: логика, порты, автомат и протокол не тронуты.



17.5. С-3. Детектив о входе, который никому ничего не обещает

Исходный SDC объявлял для MISO входную задержку относительно фиктивного клокa:

```
set_input_delay -clock spi_sclk_gen -max 15.0 [get_ports {spi_miso}]
set_input_delay -clock spi_sclk_gen -min 2.0 [get_ports {spi_miso}]
```

Объяснить с

Что видели своими глазами. Симптом отделён от причины неделями. На рабочей частоте 1 МГц петля MOSI→MISO собирается и читает то, что передано, — всё хорошо. Начинаем поднимать частоту по шагу 12 процедуры: 100 кГц, 1 МГц, 5 МГц, 8.33 МГц — петля живёт. При `CLK_DIV_VAL = 1`, то есть 12.5 МГц, принятое слово оказывается **сдвинуто ровно на один бит**. При этом отчёт STA всё так же зелёный: ни одного нарушения, все требования выполнены. Констрейнт, который по идее должен был поймать именно такую ситуацию, не сказал ни слова.

Какая гипотеза казалась логичной. Первая мысль честная и почти правильная: «MISO — вход

source-synchronous, то есть данные приходят от slave вместе с тем же тактом, который мы ему и выдали; значит, надо описать задержку `set_input_delay`, и STA посчитает, влезает ли мы». Вторая мысль, когда стало ясно, что сдвиг зависит от `CLK_DIV`: «задержка задана оптимистично, увеличим числа». Обе исходят из предположения, что тракт MISO тайминговый, то есть измеряется в наносекундах.

Как проверяли. Сначала посмотрели, что говорит инструмент о самих CDC-переходах:

`report_cdc` по OOC-сборке отвечает коротко — All paths are Safely Timed, то есть механизм пересечения доменов он считает корректным. Потом посчитали по RTL. Захват MISO в `spi_engine` идёт через две ступени `miso_sync`, то есть данные с пада попадают в `rx_word` **на два такта системного блока позже**, чем появились на ноге, а полупериод SCLK равен `CLK_DIV + 1` тактов: при `CLK_DIV = 2` это три такта, и двухтактная задержка ещё умещается внутри, при `CLK_DIV = 1` — два такта, и момент сэмплирования уезжает за границу бита. Дальше это подтвердили в симуляции (тест 25 фиксирует предел как обязательное свойство) и на железе — таблицей частот шага 12.

Улика, решившая дело, — единица измерения. Задержка синхронизатора выражается **в тактах системного блока**, а не в наносекундах, и она не зависит ни от длины провода, ни от температуры, ни от speed grade. Ни один констрейнт `set_input_delay` не способен её выразить, потому что эта команда описывает внешний мир — путь от фронта чужого блока до нашего пада. Мы же упёрлись в свойство собственной архитектуры.

Корневая причина: дизайн **сознательно не полагается** на временное соотношение между SCLK и MISO — он выбросил его в обмен на устойчивость, поставив синхронизатор и приняв ограничение на максимальную частоту. Объявлять `set_input_delay` в такой ситуации значит утверждать соотношение, которого RTL не использует, и получать формально аккуратные, но бессмысленные числа. Исправление — одна строка в board-level XDC:

```
set_false_path -from [get_ports spi_miso]
```

Объяснить с 

`set_false_path` переводится буквально как «ложный путь» и означает «не анализируй его». Это не затыкание отчёта, а заявление, что путь обработан **другим механизмом**, и механизм обязан существовать: здесь это цепочка `ASYNC_REG`. Цена честности названа прямо в комментарии к констрейнту — STA не проверяет тракт MISO вовсе. Гарантию корректного приёма даёт не инструмент, а правило `CLK_DIV >= 2`, и именно поэтому оно вынесено в документацию, зафиксировано тестом 25 в симуляции и отдельно проверяется на железе провалом при `CLK_DIV = 1`. Правило, которое проверяет только человек, живёт до первого невнимательного человека; правило, зафиксированное тестом, живёт дольше.

Правило на будущее: `set_false_path` — это обещание, что путь закрыт другим механизмом; если механизма нет, вы не сняли проверку, а спрятали улику.

17.6. Как это выглядит в отчёте

Честное исключение отличается от забытого констрейнта прямо в тексте отчёта. В сборке `spi_zynq_top` секция `no_input_delay` содержит `reset_btn` и `spi_miso` с формулировкой «There are 2 input ports with no input delay **but user has a false path constraint**» — инструмент видит, что мы не забыли, а решили. В OOC-сборке та же секция показывает 47 портов и отдельной строкой сообщает, что портов с false path среди них ноль: `spi_miso` стоит наравне с `s_axi_wdata[*]`, и это правильно — пада нет, порт является границей модуля.

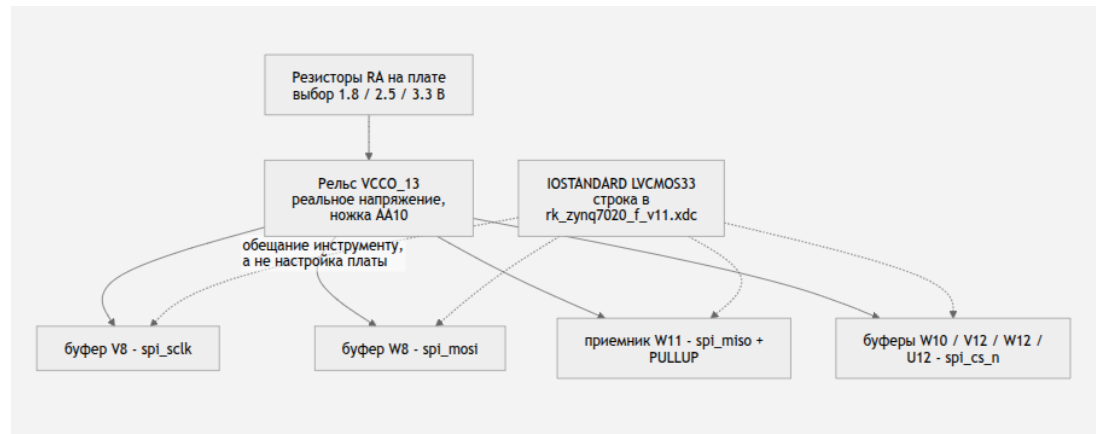
Глава 18. Банк 13 и VCCO: единственное место, где можно повредить плату

18.1. Что такое банк ввода-вывода, VCCO и IOSTANDARD

Здесь начинается физика, и без неё дальше нельзя. Выводы FPGA не равноправны и не независимы: по периметру кристалла они сгруппированы в **банки ввода-вывода**, обычно по несколько десятков выводов в банке. Банк — не логическая абстракция, а физическая группа: у всех его выводов общий рельс питания выходных буферов, и называется этот рельс **VCCO** (VCC Output). У нашего xc7z020 пользовательских банков PL четыре — 13, 33, 34 и 35 — и у каждого свой независимый VCCO; в корпусе clg484 для подачи этого питания есть отдельные ножки, например AA10 (VCCO_13) и AA20 (VCCO_33).

Что делает VCCO физически. Выходной буфер CMOS — это пара транзисторов: верхний соединяет вывод с шиной питания, нижний — с землёй. Когда буфер выдаёт логическую единицу, он **буквально подключает вывод к рельсу VCCO** — не «к условному высокому уровню», а к тому конкретному напряжению, которое подано на ножку питания банка: если VCCO 3.3 В, единица это 3.3 В, если 1.8 В — то 1.8 В. Со входами зеркально: пороги, по которым приёмник решает «ноль» или «единица», отсчитываются от VCCO. Аналогия — квартира с одной розеточной линией на комнату: нельзя запитать одну лампу от 220 В, а соседнюю от 110 В, если провод общий; нужны разные напряжения — нужны разные банки.

Теперь **IOSTANDARD**. Строка `set_property IOSTANDARD LVCMOS33 [get_ports spi_sclk]` выглядит как настройка, но настраивает она не плату. Это **объявление** инструменту: «на этом выводе используется стандарт LVCMOS33», то есть CMOS-логика с питанием 3.3 вольта. Получив его, Vivado настраивает внутренний буфер под этот стандарт (пороги, силу драйвера, скорость нарастания фронта), проверяет по DRC, что все стандарты внутри банка требуют одинакового VCCO, и записывает результат в битстрим. Чего Vivado **не делает** — не подаёт напряжение на банк и не может его измерить: питание приходит от платы. IOSTANDARD — обещание инженера о том, что на плате уже сделано, и если обещание ложное, узнать об этом от инструмента нельзя в принципе.



Сюда же относятся `set_property CFGBVS VCCO` и `set_property CONFIG_VOLTAGE 3.3` в конце того же файла: CFGBVS расшифровывается как Configuration Bank Voltage Select и вместе с CONFIG_VOLTAGE описывает напряжение банка конфигурации, через который кристалл загружается. Значения взяты из `gengr1.xdc`, чтобы наш проект объявлял плату так же, как штатная прошивка.

18.2. Куда выведен SPI и почему именно туда

Полное назначение выводов — из `../pinout.md` и factory-файлов `other.xdc` и `io_40pin.xdc`; каждый номер подтверждён `query_pins.tcl` по базе устройства:

Сигнал	Пин	Банк	IOSTANDARD	Что это на плате
clk	W17	33	LVC MOS33	PL-осциллятор 50 МГц, вывод MRCC
reset_btn	W18	33	LVC MOS33	Кнопка PL_KEY1, активный низкий
spi_sclk	V8	13	LVC MOS33	Разъём IO_40PIN1, бит 0
spi_mosi	W8	13	LVC MOS33	Разъём IO_40PIN1, бит 1
spi_miso	W11	13	LVC MOS33 + PULLUP	Разъём IO_40PIN1, бит 2
spi_cs_n[0]	W10	13	LVC MOS33	Разъём IO_40PIN1, бит 3
spi_cs_n[1]	V12	13	LVC MOS33	Разъём IO_40PIN1, бит 4
spi_cs_n[2]	W12	13	LVC MOS33	Разъём IO_40PIN1, бит 5
spi_cs_n[3]	U12	13	LVC MOS33	Разъём IO_40PIN1, бит 6
led[0]	V15	33	LVC MOS33	PL_LED1, активный высокий
led[1]	V13	33	LVC MOS33	PL_LED2, активный высокий

Разбираться в таблице удобнее по банкам. Банк 33 — «домашний»: там всё, что уже распаяно на плате и никуда не денется. Тактовый вывод W17 имеет выделенную функцию IO_L13P_T2_MRCC_33 ; буквы MRCC означают Multi-Region Clock Capable — вывод физически связан с сетью глобальной раздачи тактов. Практическое следствие: Vivado сам вставит глобальный буфер BUFG , и ни BUFG , ни PLL, ни MMCM в RTL инстанцировать не нужно — дизайн однодоменный и получает SCLK счётном, а не делением. В отчёте о таймингах это видно как W17 → IBUF → BUFGCTRL_X0Y0 → BUFG в начале каждого пути. Кнопка W18 — это IO_L13N_T2_MRCC_33 , вторая половина той же дифференциальной пары, тоже clock-capable: для обычного входа безвредно, но отметить честно стоит — один тактовый вывод банка 33 потрачен на кнопку. Светодиод led[0] сидит на IO_L19N_T3_VREF_33 , способном служить входом опорного напряжения, и это тоже не мешает: опорного напряжения не требует ни один наш стандарт. Полярности при этом не угаданы — factory-образ платы включает Linux с файлом system-user.dtsi , где PL_LED1 и PL_LED2 объявлены GPIO_ACTIVE_HIGH , а PL_KEY1 и PL_KEY2 — GPIO_ACTIVE_LOW . Заодно выясняется, что выделенной кнопки сброса PL на плате нет: аппаратный сброс заведён на PS (MIO13), а W18 — обычная пользовательская кнопка, и использование её как сброса повторяет практику штатных демо.

Банк 13 — это 40-контактный разъём расширения, и выбор в его пользу был осознанным. Альтернатива выглядела заманчиво: на плате уже распаян SPI-дисплей ST7789V (1.47 дюйма, 172×320, до 32 МГц), подключённый к V18 (SCK), U19 (MOSI) и AA13 (SS) плюс три сигнала управления панелью — W13 (DC), AA18 (RST), Y13 (подсветка). Исходный Altera-проект тоже

управлял дисплеем, так что преемственность напрашивалась. Мы отказались от него как от *основной* цели по двум причинам. Во-первых, **у него нет MISO**: производитель выводит только `io0`, что для SPI-TFT нормально, но означает, что вся приёмная половина контроллера — синхронизатор, момент сэмплирования, сборка слова, единственный известный дефект A-3 — осталась бы непроверенной. Во-вторых, линия электрически занята: к ней подключён реальный дисплей, который будет интерпретировать произвольные послышки как команды. Дисплей остаётся вторичной целью и возвращается в части VIII.

Разъём же даёт четыре по-настоящему свободных линии с настоящим MISO и открывает самую ценную проверку, не требующую никакого оборудования: **джампер с MOSI на MISO превращает контроллер в петлю**, где принятое слово обязано совпасть с переданным. Один провод проверяет весь тракт целиком — сдвиг наружу, выходной пад, соединение, входной пад, синхронизатор, момент сэмплирования, порядок бит. Поэтому MOSI (W8, бит 1) и MISO (W11, бит 2) назначены на физически соседние контакты: джампер получается коротким, и его трудно поставить не туда. Соответствие выводов контактам J1 восстановлено по именам цепей схемы — цепь `I0n` занимает контакты `2n+1` и `2n+2`, поэтому SCLK и MOSI это `I09_P` и `I09_N`, контакты 19 и 20, а MISO и `spi_cs_n[0]` — `I010_P` и `I010_N`, контакты 21 и 22. Побочное наблюдение, которое стоит записать: SCLK и MOSI попали на две половины одной дифференциальной пары, то есть клок идёт по сильно связанной трассе рядом с данными. При 1 МГц это не имеет значения; если частоту будут поднимать к потолку 8.33 МГц, дешёвая предосторожность — перенести SCLK в другую пару.

Внутренняя подтяжка `PULLTYPE PULLUP` на MISO — тоже не «на всякий случай». Без неё вход при отсутствующем slave висит в воздухе и читает наводки, то есть принятый байт — бессмысленный шум. С подтяжкой отсутствие устройства даёт устойчивый `0xFF`, а короткое замыкание на землю — устойчивый `0x00`, и ровно на этих двух «адресах отсутствия» построена эвристика `slave_present` в `spi_selftest.v`: подтяжка превращает неопределённость в диагностируемое состояние.

18.3. История «можно сжечь плату»: детектив о напряжении банка

Эта история отличается от предыдущих тем, что у неё нет симптома: её остановили до того, как что-нибудь случилось, и в этом вся суть. Чем это грозило? К моменту готовности битстрима все выводы имели `IOSTANDARD LVCMOS33`, и сомнения это не вызывало: весь XDC производителя — `LVCMOS33`, `gengral.xdc` объявляет `CONFIG_VOLTAGE 3.3`, плата собрана под 3.3 В. Затем при вычитывании руководства пользователя всплыла фраза о том, что напряжение VCC0 на 40-контактном разъёме и FMC — **не зашито навечно**, а выбирается установкой резисторов RA и может быть 1.8, 2.5 или 3.3 В. Значение 3.3 В — заводское *по умолчанию*, но плата у нас не с завода, а прошла через неизвестно чьи руки. Если банк 13 переведён на 1.8 В, а битстрим объявляет `LVCMOS33`, буферы этого банка получают команду работать в режиме, который их питание не поддерживает. Усугубляет то, что на 40-контактном разъёме, по руководству (§1.19), **нет никаких преобразователей уровней**: контакты идут прямо на выводы FPGA, между чужим 3.3-вольтовым устройством и кристаллом нет ничего.

Какая гипотеза казалась логичной. «Весь factory XDC — `LVCMOS33`, значит все банки на 3.3 В; производитель бы не стал объявлять то, чего нет». Логика почти верная, и дыра в ней ровно одна: factory XDC описывает плату **в заводской конфигурации** и не может знать, что кто-то переставил резисторы, чтобы подключить к разъёму 1.8-вольтовую периферию.

Как проверяли. Первым делом посмотрели в отчёт Vivado — и вот тут ловушка, ради которой стоило писать эту главу. В `report_io` есть колонка Voltage, и в ней у строки `VCC0_13` (ножка AA10) написано `3.30`, у `VCC0_33` (AA20) — тоже `3.30`. Выглядит как подтверждение. Это **не подтверждение**. Достаточно посмотреть чуть выше в том же файле: у `VCC0_35` (ножка A20) в той же колонке стоит `any**`. Разница между банками ровно одна — в 13 и 33 у нас назначены выводы

с LVCMOS33, а в 35 не назначено ничего. То есть отчёт печатает не измеренное напряжение платы, а **напряжение, которое требуется объявленным нами стандартам**: это зеркало, а не вольтметр, и канала, по которому Vivado мог бы узнать, что подано на ножку AA10, физически не существует.

Настоящая проверка живёт вне Vivado и делается в таком порядке:

1. Найти на схеме платы цепь VCCO_13 и посмотреть, какие резисторы RA на ней установлены (по руководству это выбор 1.8 / 2.5 / 3.3 В).
2. Подать питание, **не загружая никакого битстрима**: без конфигурации выводы PL находятся в высокоимпедансном состоянии и ничего никому не выдают.
3. Мультиметром измерить напряжение на контакте питания 40-контактного разъёма относительно земли. По схеме контакты J1 39–40 — это VCC_3V3 через 4.7 кОм, и это питание самого разъёма, а не подтяжка сигнальных линий.
4. Сверить измеренное с IOSTANDARD в секции банка 13 файла ../constraints/rk_zynq7020_f_v11.xdc — они обязаны соответствовать.
5. Только после совпадения загружать битстрим.

Улика, решившая дело, — то самое any** у неиспользуемого банка: оно доказывает, что колонка Voltage вычисляется из наших же назначений, а значит рассуждение «Vivado написал 3.30, следовательно на плате 3.3» круговое — мы объявили LVCMOS33, инструмент вывел из этого 3.30 и показал нам обратно наше же утверждение. Такие подтверждения опаснее всего: они выглядят независимыми.

Корневая причина риска формулируется одной фразой: **IOSTANDARD обязан совпадать с фактическим VCCO банка, а «значение по умолчанию» — не гарантия, а вероятность**. Что будет при рассогласовании, зависит от направления. Если банк питается ниже, чем объявлено, выходные буферы настроены под режим, которого питание не обеспечивает, и работают вне допустимого диапазона, а входные пороги смещены, так что чтение ненадёжно. Если наоборот, на выводы банка с низким VCCO приходит внешний сигнал более высокого уровня, ток идёт через защитные диоды входа в рельс питания банка — режим, не предусмотренный ни для диодов, ни для рельса. В ../pinout.md этот пункт отмечен как единственный во всей распиновке, способный привести к аппаратному повреждению.

Исправление процедурное, а не программное: подтвердить RA до первой прошивки, а при иной установке — сменить IOSTANDARD у **всех** выводов секции банка 13 разом, потому что смешивать стандарты с разным VCCO внутри банка нельзя физически. Это ограничение Vivado проверяет DRC-правилом — но только рассогласование *внутри объявленного*, не объявленного с реальностью.

Правило на будущее: перед первым битстримом на новый банк — мультиметр на VCCO; ни название платы, ни отчёт инструмента напряжения не измеряют.

18.4. Что осталось неподтверждённым и почему это не страшно

Честный список открытых вопросов короткий и, кроме VCCO, безобидный. Точный контакт внутри дифференциальной пары J1 взят из автопарсинга схемы и требует визуальной сверки перед установкой джампера; ошибка не опасна — оба контакта пары принадлежат тому же разъёму и не подключены к другой периферии, — но петля не заработает, и вы будете искать причину в логике, а она в проводе. Номинал резистора R145 на контактах J1 37–38 в netlist не указан и на SPI-выводы не влияет. Конфигурация PS — DDR, MIO, тактовые частоты — не угадывается в принципе, board file для платы в Vivado отсутствует, и единственный правильный источник здесь — preset из

штатного `design_1.bd`. Это вопрос части VI, и решается он копированием рабочей конфигурации, а не творчеством.

Глава 19. Level shifters и ps7_init: почему PL молчит

19.1. Чем Zynq отличается от обычной FPGA

Cyclone IV — это просто FPGA: подали питание, загрузили конфигурацию, логика заработала. Zynq-7000 устроен иначе, и в этой главе разница впервые становится осязаемой. На одном кристалле живут две подсистемы. **PS** (Processing System) — двухъядерный ARM Cortex-A9 со своей периферией, контроллером DDR и портами ввода-вывода MIO: полноценный процессор, способный работать вообще без программируемой логики. **PL** (Programmable Logic) — собственно FPGA: матрица, банки ввода-вывода, наши LUT и триггеры. Между ними несколько широких интерфейсов AXI и множество служебных сигналов.

Ключевая деталь в том, что PS и PL — это **разные домены питания и сброса**, и отношения между ними несимметричны. В архитектуре Zynq главный — процессор: именно он при включении выходит из сброса первым, читает загрузчик и управляет тем, в каком состоянии находится PL. На границе между доменами стоят **level shifters**, схемы-переводчики. Название буквальное: если по одну сторону границы логическая единица — одно напряжение, а по другую — другое, сигнал нельзя просто соединить проводом, нужна схема, пересчитывающая уровень. Бытовая аналогия — переходник для розетки в другой стране; разница в том, что здесь переходник встроен в кристалл и у него есть выключатель. И вот этот выключатель после подачи питания находится в положении «выкл». Включается он **программно**: при штатной загрузке это делает код инициализации PS — функция `ps7_init`, которую Vivado генерирует под конкретную конфигурацию процессорной системы, а платформа выполняет до передачи управления приложению. «Подать питание» и «разрешить fabric» на Zynq — два разных события, и второе требует, чтобы кто-то выполнил код.

19.2. Замечание DRC, которое стоит читать

Наш автономный топ `spi_zynq_top` — чисто PL-дизайн, блока PS7 в нём нет вовсе, и при проверке Vivado выдаёт ровно одно board-level замечание:

```
ZPS7-1 Warning: PS7 block required
The PS7 cell must be used in this Zynq design in order to enable correct
default configuration.
```

Объяснить с 

На «чистой» FPGA такого замечания не бывает и быть не может — там некому требовать процессорный блок. Здесь оно ожидаемо и ошибкой сборки не является: битстрим создаётся, тайминги закрыты. Но замечание не пустое, и переводится так: «в этом дизайне нет того, кто приводит кристалл в рабочее состояние по умолчанию». Логически нашему дизайну процессор не нужен — он тактируется осциллятором с W17 и сигналов от PS не использует. Электрически — PL должен быть разрешён.

19.3. История «DONE горит, а плата мертва»

Что видели своими глазами — точнее, чего бы не увидели. Плата запитана, JTAG подключён, Hardware Manager рапортует об успешной загрузке, светодиод DONE горит. Первый шаг процедуры bring-up — heartbeat: `led[0]` на выводе V15 обязан мигать с частотой примерно 1.5 Гц, потому что это просто 25-й бит счётчика от 50 МГц. Не мигает. Совсем. Другой активности тоже нет: SCLK на V8 молчит, CS на W10 молчит, петля не работает.

Какая гипотеза казалась логичной. Их сразу четыре, и все правдоподобные: битстрим повреждён или загрузился не тот; кнопка сброса залипла или её полярность перепутана; светодиод подключён иначе, чем мы думаем; нет тактового сигнала. Естественная реакция новичка — либо сразу лезть щупом, либо сразу пересобирать проект; обе стоят часов.

Как проверяли. Здесь работает принцип, за который исходный проект уже заплатил недель: **самая дешёвая проверка выполняется первой, и каждая должна убивать как можно больше гипотез**. Статус DONE в Hardware Manager стоит ноль усилий и снимает гипотезу «битстрим не загрузился». Полярности снимаются чтением factory device tree — GPIO_ACTIVE_HIGH для светодиодов, GPIO_ACTIVE_LOW для кнопок; это документ производителя, а не предположение. Более того, при ошибке полярности светодиода heartbeat всё равно мигал бы, просто в противофазе, то есть эта гипотеза не объясняет полное отсутствие активности и отпадает без всякой проверки. Клок проверяется щупом на цепи PL_CLK (осциллятор → R121 33 Ом → W17): 50 МГц на месте. Кнопку достаточно не трогать. Остаётся одна гипотеза, и она же самая неочевидная.

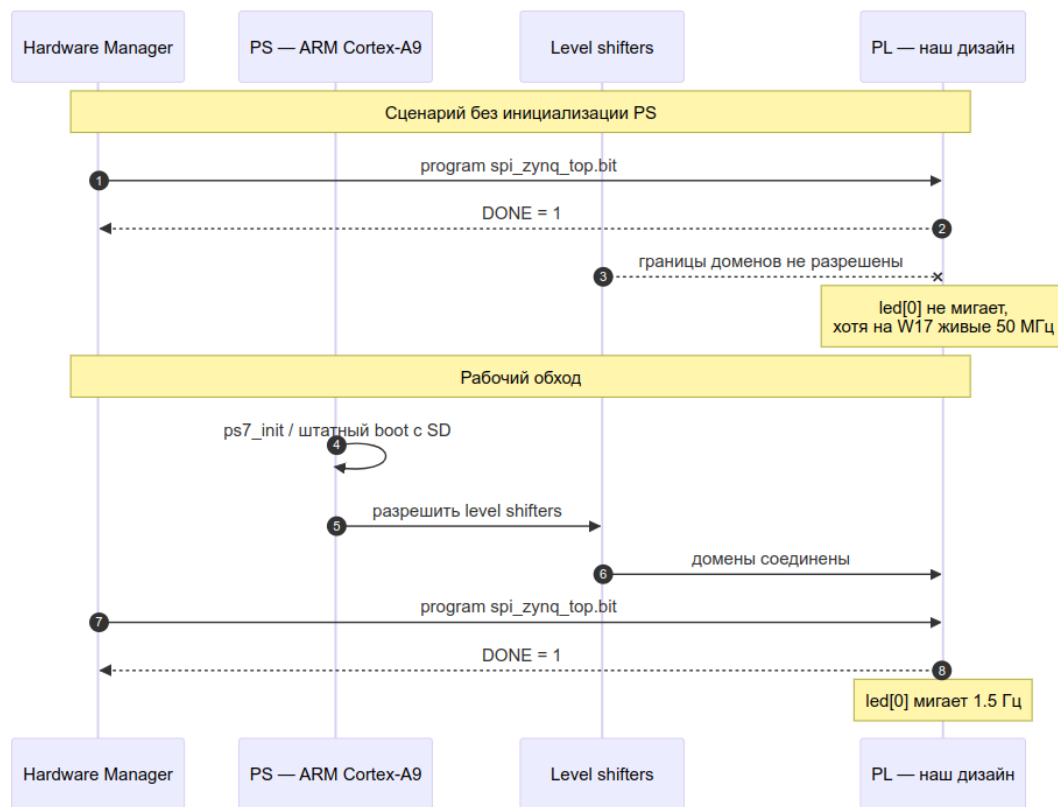
Улика, решившая дело, — сочетание «DONE горит» и «на W17 живые 50 МГц» при полном отсутствии реакции. Конфигурация загружена, тактовый сигнал подан, питание есть — и ничего не происходит. Такая комбинация не объясняется ни одной ошибкой в RTL, потому что RTL при живом блоке обязан хотя бы считать; она объясняется только тем, что fabric не выведен в рабочее состояние.

Корневая причина: в PL-only дизайне без инициализации PS уровневые преобразователи и вся процедура штатного вывода PL из сброса не выполняются, потому что выполнять их некому. Виноват не перенос RTL и не наши constraints — это свойство архитектуры Zynq, с которым на Cyclone IV столкнуться было нельзя. Исправление есть в трёх вариантах, и выбирать надо по тому, чем вы заняты:

1. Дать плате штатно загрузиться с SD или QSPI, дожидаться, пока PS отработает инициализацию и включит level shifters, и только потом загрузить PL по JTAG. Ничего не нужно собирать заново.
2. Выполнить ps7_init из XSCT или Vitis перед программированием PL. Точнее, и надёжнее, но требует установленного Vitis и конфигурации PS.
3. Перейти к сборке с блок-дизайном и Processing System — там PS7 инстанцирован в самом дизайне, и вопрос снимается по построению. Это часть VI.

Важная оговорка, без которой рассказ был бы нечестным: **на практике загрузка PL из Hardware Manager по JTAG на этой плате обычно работает сразу**. Описанный сценарий — не то, что случается всегда, а первый подозреваемый в конкретной ситуации: когда признаков жизни нет вообще никаких. Если heartbeat мигает, а не работает только SPI, ps7_init тут ни при чём и искать надо в другом месте.

Правило на будущее: на Zynq «битстрим загружен» и «PL работает» — два разных утверждения; сначала разрешите fabric, потом отлаживайте details логику.



19.4. Почему мы всё равно начали с PL-only

Резонный вопрос: если вариант с PS снимает проблему по построению, зачем было собирать автономный топ? Ответ тот же, что и во всех предыдущих частях, — **сокращение числа подозреваемых**. PL-only сборка не содержит ни PS, ни DDR, ни MIO, ни загрузчика, ни операционной системы: когда в ней не работает SPI, виноват SPI или его обвязка, больше некому. С появлением процессора к списку добавляются конфигурация DDR, адреса на шине, драйвер, устройство дерева, права доступа, и вопрос «почему на выводе нет сигнала» превращается в отладку загрузочной цепочки осциллографом. Поэтому порядок такой: сначала PL-only с `spi_selftest` в роли хоста — часть V; затем блок-дизайн с PS — часть VI; затем Linux, где полярности светодиодов и кнопок встретятся уже в нашем собственном дереве — часть VII.

Итог части IV. Constraints — не «файл для компилятора», а записанный контракт с физикой, и переписать его с одного вендора на другого механической заменой синтаксиса нельзя: из девяти разобранных пунктов QSF и SDC один в один перенёсся один. `create_generated_clock` на SCLK выброшен, потому что SCLK не тактирует ни одного триггера, а его делитель программируется во время работы. `set_input_delay` на MISO заменён честным `set_false_path`, потому что архитектура сознательно отказалась от соотношения SCLK↔MISO в пользу 2FF-синхронизатора, и цена решения — правило `CLK_DIV >= 2`, зафиксированное тестом, а не констрейнтом. Распиновка написана с нуля по factory-проекту производителя и сверена с базой устройства, а сборка отказывается делать битстрим при незакрытом TBD. Единственное место, где можно повредить железо, — соответствие IOSTANDARD фактическому VCCO банка 13, и проверяется оно мультиметром, а не отчётом Vivado, который в этой графе показывает наше же объявление. И, наконец, Zynq требует, чтобы PS разрешил fabric, даже когда наш RTL обходится без ARM.

Дальше — лаборатория: питание, JTAG, мультиметр, осциллограф и один джампер.

Часть V. Bring-up до Linux

Четыре части мы делали то, что можно сделать не вставая из-за стола, и всё это время у нас был симулятор, в котором видно каждый триггер и время можно отмотать назад. Теперь на столе появляется плата, и это другой мир: внутри кристалла по-прежнему сотни триггеров, но снаружи у него только ножки, и всё, что не выведено на ножку, наблюдению недоступно в принципе. Зависла ли FSM, не пришёл ли строб, живёт ли CS на другом выводе — изнутри неотличимо: симптом ровно один, «не работает».

Bring-up — процедура первого оживления железа: превратить единственный симптом обратно в набор различных фактов. Слово стоит запомнить вместе с четырьмя другими. **Битстрим** (файл `.bit`) — конфигурация ПЛИС, описание того, в какое положение поставить сотни тысяч внутренних переключателей, чтобы кремний повторил ваш RTL; после включения питания ПЛИС пуста, и битстрим в неё каждый раз загружают заново. **JTAG** — служебный четырёхпроводный интерфейс отладки (стандарт IEEE 1149.1), технический люк, через который в микросхему лезут снаружи: загрузить конфигурацию, остановить процессор, прочитать регистр по адресу; на нашей плате он выведен на USB-разъём. **Hardware Manager** — окно Vivado, которое говорит с платой по JTAG; сигнал **DONE** в нём означает «битстрим долит до конца и принят». **XSCT** (Xilinx Software Command-line Tool) — консольный близнец этого окна: интерпретатор Tcl, в котором те же операции пишутся скриптом, а значит воспроизводятся.

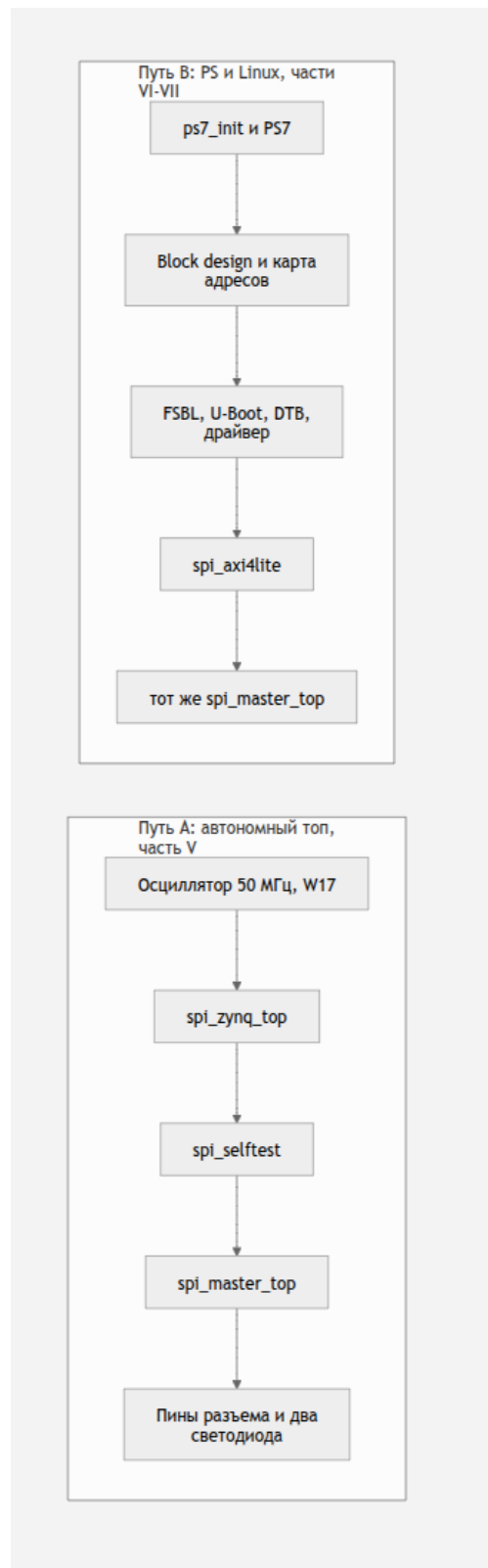
Порядок проверок не произвольный: он подчинён правилу, за которое исходный Altera-проект заплатил недель отладки, — **самая дешёвая проверка выполняется первой, и каждый шаг обязан убивать как можно больше гипотез**.

Глава 20. Лабораторные шаги 0–13

20.1. Почему отдельный топ, а не сразу PS и Linux

Первое архитектурное решение принято до того, как плата достана из коробки. У нас есть готовая обёртка `spi_axi4lite` и ясная цель: Cortex-A9 под Linux пишет регистры, драйвер их обслуживает, на дисплее появляется картинка. Логично собрать сразу это — мы так не делаем и собираем отдельный автономный верхний модуль `spi_zynq_top` со встроенным аппаратным хостом `spi_selftest`, в котором нет ни процессорной системы, ни AXI, ни строчки софта.

Причина арифметическая. Путь «сразу Linux» состоит примерно из десяти звеньев: конфигурация PS, `ps7_init`, block design, карта адресов, битстрим на карте памяти, FSBL, U-Boot, Device Tree, драйвер, программа в userspace — и только потом наш RTL и наши провода. Ошибка в любом звене даёт один и тот же внешний признак «SPI не работает», а каждая гипотеза стоит пересборки образа. Автономный топ оставляет в цепочке четыре элемента: осциллятор платы, наш RTL, констрейнты и провода. Замкнулась петля MOSI→MISO и загорелся светодиод — значит распиновка верна, банк запитан, ядро считает биты правильно, и к приходу процессора в части VI эти гипотезы уже вычеркнуты.



Платим двумя вещами, и обе честные. Появляется второй маршрут сборки: `scripts/build.tcl` - `tclargs standalone` для автономного топа и `scripts/build_ps_axi.tcl` для варианта с процессором — два набора артефактов и обязанность держать оба живыми. И `spi_selftest` — код, которого не будет в продакшене, но который придётся поддерживать. Обе цены низкие по сравнению с неделями отладки драйвера поверх неверно назначенного вывода.

20.2. Что должно лежать на столе

Список короткий и почти весь дешёвый: плата TZT RK-ZYNQ7020-F v1.1 с питанием, USB-кабель JTAG и один джампер, который понадобится на шаге 8 и стоит дороже своей цены, потому что заменяет собой целое внешнее SPI-устройство. Очень желателен мультиметр — прибор, который новички считают недостойным цифровой электроники, а он закрывает больше отказов на единицу времени, чем всё остальное вместе. Желателен осциллограф или анализатор. Известное исправное SPI-устройство — опционально: процедура намеренно построена так, чтобы без него дойти до конца.

Прибор или средство	Что им закрываем	Шаги
Отчёт <code>standalone_io.rpt</code>	Куда синтез реально посадил сигналы	0
Светодиоды на плате	Клок, сброс, факт загрузки, петля	1, 2, 8, 10, 13
Мультиметр	Уровни покоя, питание банка, обрывы	4, 8
Осциллограф или анализатор	Частоты, форму кадра, порядок бит	5–7, 9–12
ILA внутри ПЛИС	FSM, слова <code>last_tx</code> и <code>last_rx</code> , флаги	3, 8, 10, 12
Джампер	Замыкание всего тракта в петлю	8, 13

Битстрим для всей главы один и собирается так:

```
cd spi_xilinx
source /opt/xilinx/2025.2/Vivado/settings64.sh
vivado -mode batch -source scripts/query_pins.tcl
vivado -mode batch -source scripts/build.tcl -tclargs standalone
grep -E "spi_|led|clk|reset" reports/standalone_io.rpt
```

Объяснить с 

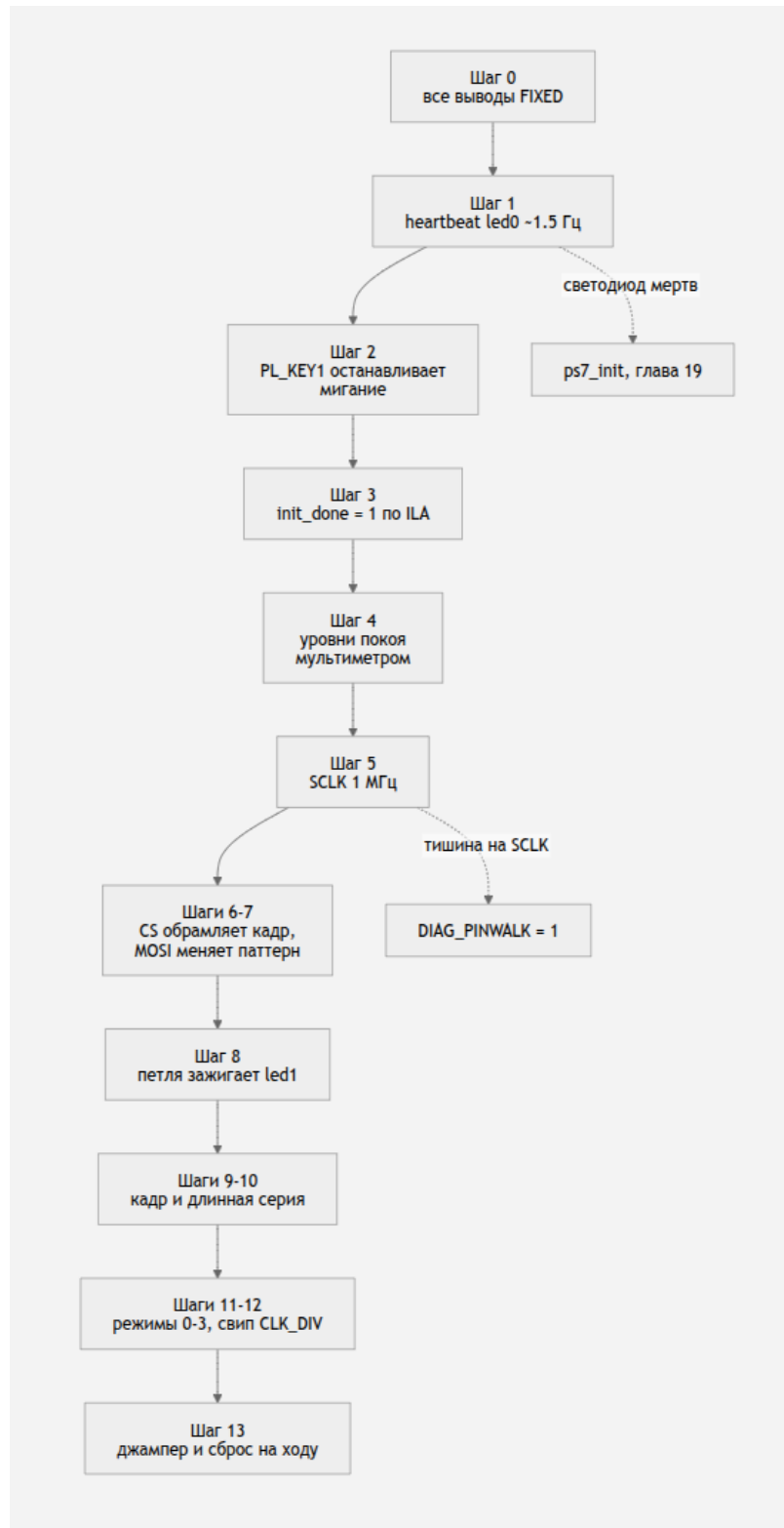
Файл ложится по пути `build/spi_zynq/spi_zynq.runs/impl_1/spi_zynq_top.bit`. Сборка **откажется** работать, если в XDC остался хотя бы один маркер `TBD`: битстрим с угаданной распиновкой способен повредить плату, поэтому ошибка сделана фатальной (глава 16 части «Vivado, constraints, pinout»).

20.3. Карта маршрута: тринадцать шагов одним взглядом

Ниже весь маршрут компактно — единственное место, где шаги перечислены списком. Дальше каждый разобран прозой: знать формулировку мало, нужно понимать, зачем шаг, что считается успехом и что делать при отказе.

1. **Шаг 0.** Сверить распиновку по отчёту, ещё не подавая питание.
2. **Шаг 1.** Загрузить битстрим: `led[0]` мигает примерно 1.5 раза в секунду.
3. **Шаг 2.** Нажать `PL_KEY1` (W18) — мигание замирает и возобновляется.
4. **Шаг 3.** Убедиться по ILA, что `init_done = 1`.
5. **Шаг 4.** Измерить мультиметром уровни покоя на `CS`, `SCLK` и `MISO`.
6. **Шаг 5.** Найти на `SCLK` (V8) пачки импульсов 1 МГц.
7. **Шаг 6.** Убедиться, что `CS` (W10) обрамляет каждый кадр.
8. **Шаг 7.** Увидеть на `MOSI` (W8) восемь бит `MSB-first` меняющимся паттерном.
9. **Шаг 8.** Поставить джампер `MOSI→MISO` и получить `led[1]`.

10. **Шаг 9.** С `CONTINUOUS = 0` поймать одиночный кадр триггером по спаду `CS`.
11. **Шаг 10.** Вернуть `CONTINUOUS = 1` : минуты работы с `err_seen = 0` .
12. **Шаг 11.** Проверить режимы 0–3: уровень покоя `SCLK` равен `CPOL`.
13. **Шаг 12.** Пройти свип `CLK_DIV` 249...1 и увидеть, где ломается приём.
14. **Шаг 13.** Снять и вернуть джампер на ходу, нажать сброс во время передачи.



20.4. Шаг 0: сверка распиновки до подачи питания

Самый скучный шаг маршрута и самый дорогой из пропущенных. Запускаем `scripts/query_pins.tcl`, который для каждого вывода спрашивает у базы устройства Vivado: существует ли такой вывод в корпусе clg484, в каком он банке, какова его выделенная функция. Затем собираем битстрим и читаем `reports/standalone_io.rpt`, где Vivado сообщает, куда в итоге сел каждый сигнал. Признак успеха конкретный: все выводы имеют статус **FIXED**, номера совпадают с `../pinout.md`, на `spi_miso` стоит `PULLUP`.

Слово **FIXED** ключевое: оно означает «назначен явно», в противоположность «размещён инструментом по своему усмотрению». Ровно на этой разнице исходный проект потерял неделю — там для десятков сигналов был задан электрический стандарт, но не координата вывода, и Quartus молча раскидал их по свободным ножкам (дефект A-6 из части «Архитектура ядра SPI»). Типичная ошибка шага — счесть его формальностью: «пины же в XDC, я их сам писал». «Прописан в XDC» и «попал в битстрим» — разные утверждения, между ними стоит синтез, который вправе выбросить логику без нагрузки. Перечитывайте отчёт I/O после **каждого** изменения портов верхнего модуля: тридцать секунд с `grep` против недели с анализатором.

20.5. Шаги 1–3: heartbeat, кнопка и `init_done`

Подаём питание, грузим битстрим по JTAG, смотрим на `led[0]` — это `PL_LED1`, вывод V15. Ожидание: мигание с частотой около 1.5 Гц. Число из простой арифметики: в `spi_zynq_top` крутится 25-битный счётчик `hb_cnt`, светодиод питается его старшим битом, и 50 МГц, делённые на 2^{25} , дают полторы вспышки в секунду.

Ценность heartbeat в том, **откуда** он взят: счётчик подключён к клоку и сбросу и не имеет отношения к SPI-ядру. Мигающий светодиод — сразу четыре доказательства: клок жив, сброс снят, битстрим загружен, банк 33 запитан. Пока шаг не пройден, остальные бессмысленны: искать причину отсутствия SCLK в дизайне, который вообще не тикает, — трата времени. Если светодиод молчит при `DONE = 1`, первым подозреваемым на Zynq является не RTL и не полярность, а разрешение фабрики со стороны процессорной системы (раздел 22.1 и глава 19 части «Vivado, constraints, pinout»); полярность подтверждена документально — заводской device tree объявляет оба PL-светодиода как `GPIO_ACTIVE_HIGH`, и даже при ошибке светодиод мигал бы в противофазе.

Шаг 2 проверяет цепь сброса: удерживаем `PL_KEY1` на выводе W18, мигание должно замереть и возобновиться после отпускания. Полярность кнопки тоже подтверждена заводским device tree (`GPIO_ACTIVE_LOW`), так что шаг проверяет цепь, а не догадку; обратное поведение лечится пересборкой с `RESET_ACTIVE_LOW = 0`. Выделенной кнопки сброса PL на плате нет вообще: аппаратный сброс заведён на процессорную систему (MIO13), а W18 — обычная пользовательская клавиша, и её использование как сброса фабрики повторяет практику штатных демо производителя.

Шаг 3 проверяет, что конфигурация ядра прошла. Отдельного шага «прочитать регистры из софта» здесь нет — некому его выполнять; проверка косвенная, но жёсткая: не прошла конфигурация — не будет и SPI-активности на шагах 5–7. Для прямого наблюдения нужен ILA: `init_done` обязан подняться примерно через пять тактов после сброса, потому что конфигурационная таблица содержит ровно пять записей. Заодно снимается вопрос про «регистр ID/version» из типовых чек-листов: в автономной сборке нашему IP предьявить нечего, исходное Altera-ядро такого регистра не имело, а добавить его означало бы разрушить доказательство эквивалентности из `../verification.md`. Пункт закрывается записью и чтением любого R/W-регистра — например, `CLK_DIV = 0x1234` с чтением обратно, как делает тест 21 в симуляции; на железе то же повторяет `jtag_verify_spi.tcl`, который пишет в `CLK_DIV` число 1234, читает обратно и следом проверяет, что запись нуля отвергается (дефект A-4). Регистр `ID` со значением `0x53500200` появляется только в AXI-сборке.

20.6. Шаг 4: мультиметр раньше осциллографа

Тот шаг, ради которого написан весь порядок. Прежде чем доставать щупы, берём мультиметр и измеряем постоянное напряжение на трёх контактах разъёма.

Сигнал	Вывод	Ожидаемо в покое
spi_cs_n[0]	W10	около 3.3 В, CS неактивен между транзакциями
spi_sclk	V8	около 0 В, потому что CPOL = 0 по умолчанию
spi_miso	W11	около 3.3 В за счёт внутреннего PULLUP

Две минуты закрывают целый класс отказов: не тот вывод, нет питания банка, вывод занят другим драйвером, обрыв до контакта разъёма. Важнее другое: каждое из трёх ожиданий — прямое следствие уже принятых решений. Ноль на SCLK следует из CPOL = 0. Единица на CS — из того, что CS активен низким уровнем и движок держит его отпущенным в покое. Единица на MISO — из подтяжки, включённой в XDC специально, чтобы отсутствие слейва читалось как `0xFF`, а не как шум.

Отсюда правило чтения результатов: любое **противоречие** между измеренным уровнем покоя и ожидаемым — улика топологии, а не логики. Ноль на CS в покое означает, что этим падом управляет не наш движок; ноль на MISO при включённой подтяжке — что подтяжка стоит не на том паде. Именно такие две улики в Altera-проекте развернули следствие в правильную сторону за две минуты — после недели охоты за фронтами. Типичная ошибка шага очень человеческая: его пропускают, потому что мультиметр кажется несерьёзным.

20.7. Шаги 5, 6 и 7: SCLK, CS и MOSI

Теперь щуп. На V8 (`spi_sclk`) при `CLK_DIV_VAL = 24` и клок 50 МГц должны идти пакки импульсов частотой 1 МГц: полупериод длится `CLK_DIV + 1` тактов, отсюда $50 / (2 \cdot 25) = 1$ МГц. Идут они практически непрерывно, потому что `CONTINUOUS` по умолчанию равен единице (раздел 21.4). При тишине на SCLK не бросайтесь читать FSM: сначала выполните бинарный эксперимент из раздела 20.10.

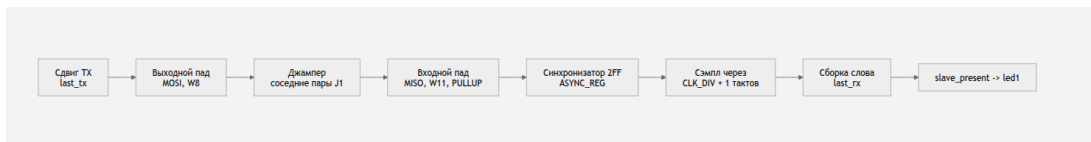
На W10 (`spi_cs_n[0]`) CS должен уходить в ноль перед пачкой SCLK и возвращаться в единицу после неё, обрамляя каждый кадр; внутри burst, между словами одной транзакции, CS остаётся низким — это нормально. Заодно проверьте, что `spi_cs_n[1..3]` на выводах V12, W12 и U12 стоят в единице: `spi_selftest` настроен на `CS_INDEX = 0`, и активность на остальных линиях означала бы ошибку маршрутизации выбора кристалла.

На W8 (`spi_mosi`), с синхронизацией по спаду CS, видно восемь бит данных, старшим вперёд, и значение обязано **меняться** от кадра к кадру. Это не эстетика: `selftest` вращает паттерн сдвиговым регистром с обратной связью `{pattern[6:0], pattern[7] ^ pattern[5]}`, начиная с `8'h01`, ради конкретного класса ошибок. Константа вроде `0xA5` выглядит случайной, но читается одинаково в обе стороны: перепутанный порядок бит проходит сквозь неё незамеченным, а залипший бит на фоне неизменного слова не отличить от нормы. В исходном проекте это правило номер десять, купленное опытом. Последняя сверка — с режимом: в режиме 0 бит на MOSI выставляется **до** первого нарастающего фронта, в состоянии `S_LOAD`; если он меняется вместе с фронтом, вы смотрите на другой режим, чем думаете (раздел 22.5).

20.8. Шаг 8: петля MOSI→MISO как первый честный тест

Петля (loopback) — соединение выхода со входом того же устройства: коротким джампером замыкаем MOSI на MISO. Мастер начинает слушать сам себя, и принятое слово **обязано** совпасть

с переданным. Проще теста не бывает, и одновременно это единственная проверка, покрывающая весь тракт целиком без единого внешнего прибора и устройства.



Каждая стрелка — отдельный подозреваемый, и петля допрашивает их всех разом: сдвиг наружу, выходной пад, провод, входной пад, двухступенчатый синхронизатор, момент сэмплирования, сборка слова, порядок бит. Совпало — весь список оправдан одним экспериментом. Именно поэтому SPI выведен на 40-контактный разъём, а не на встроенный дисплей: у разъёма есть настоящий MISO, а MOSI (W8, цепь I09_N) и MISO (W11, цепь I010_P) назначены на соседние пары контактов, примерно 19–20 и 21–22 разъёма J1, чтобы джампер был коротким. Перед установкой контакт нужно **сверить визуально**: точное распределение внутри дифференциальной пары взято из автопарсинга схемы и помечено в `../pinout.md` как неподтверждённое. Ошибка не опасна, но петля не заработает, а причину вы будете искать в логике — это самая частая ошибка шага, разобранная в разделе 22.4.

Признак успеха: загорается `led[1]`, то есть PL_LED2 на выводе V13. За ним стоит `slave_present`, и правило намеренно грубое: «принятый байт не равен ни `0x00`, ни `0xFF`». Логика физическая: без слейва и без джампера линия висит на подтяжке и читается единицами, короткое замыкание дало бы нули, всё остальное означает, что на линии кто-то живой. Это эвристика, а не проверка целостности данных: `led[1]` говорит «связь есть», но не «байт дошёл верно». Точное совпадение проверяется по ILA: `last_tx` равен младшему байту `last_rx`.

20.9. Шаги 9–13: кадр, серия, режимы, частоты, восстановление

Дальше четвёрка проверок, превращающих «однажды заработало» в «работает предсказуемо». Шаг 9 — пересборка с `CONTINUOUS = 0`: паузы возвращаются, период становится около 84 мс (2^{22} тактов при 50 МГц), кадры приходится ловить триггером по спаду CS, и кадр обязан быть идентичен наблюдаённому на шаге 7. Шаг 10 возвращает `CONTINUOUS = 1` и требует терпения: наблюдаем минуты, кадры идут непрерывно, `led[1]` горит стабильно, `xfer_count` в ILA монотонно растёт, `err_seen` остаётся нулём. Длительная серия ловит редкие отказы — одиночный удачный кадр доказывает лишь то, что удача возможна.

Шаг 11 проверяет четыре режима SPI. Тонкость реализации: режимы задаются битами CPOL и CPHA регистра CONTROL, а `spi_selftest` пишет туда только бит EN, то есть всегда работает в режиме 0. Путь два: поправить конфигурационную таблицу `S_CFG` в `spi_selftest.v`, добавив запись в CONTROL с нужными CPOL и CPHA, — или дождаться сборки с процессором и менять режим из софта. Ожидание одинаково: уровень покоя SCLK равен CPOL, а в петле принятое слово равно переданному во всех четырёх режимах (в симуляции кейсы 2–5, шпаргалка по фронтам — в приложении C).

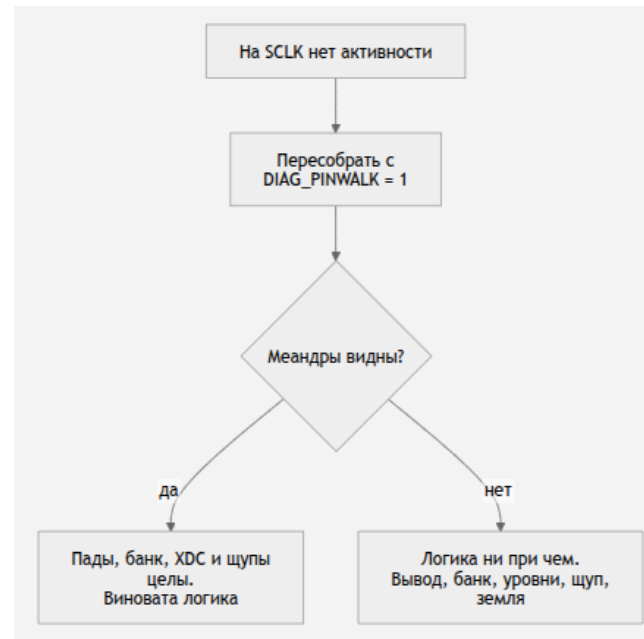
Шаг 12 — свип частоты, единственный, который **обязан** закончиться отказом. Меняя `CLK_DIV_VAL`, получаем 100 кГц при 249, один мегагерц при 24, пять мегагерц при 4 и 8.33 МГц при 2 — во всех этих точках петля работает. При значении 1 частота вырастает до 12.5 МГц, и приём портится: слово сдвигается на бит. Это дефект A-3, и проверка включена намеренно — чтобы предел был подтверждён на кремнии, а не остался особенностью симулятора; расследование — в разделе 22.6.

Шаг 13 проверяет способность приходить в себя. Вынимаем джампер на ходу: `led[1]` гаснет в течение одного цикла, потому что принят `0xFF`. Вставляем обратно — загорается снова, детекция работает в обе стороны. Затем нажимаем сброс во время активной передачи: CS немедленно

уходит в единицу, шина освобождается, после отпущания работа возобновляется без ручного вмешательства (в симуляции кейс 22). Требование не декоративное: мастер, оставляющий CS прижатым после сброса, вешает всю шину.

20.10. DIAG_PINWALK: эксперимент, делящий гипотезы пополам

Если на шаге 5 тишина, соблазн открыть `spi_engine.v` велик — и почти всегда ошибочен. Первым надо ответить на более грубый вопрос: проблема в **физике** или в **логике**? Пока он не решён, любая правка RTL — стрельба вслепую. Для этого в `spi_zynq_top` встроен параметр `DIAG_PINWALK`: при значении 1 на выводы SCLK, MOSI и CS подаются меандры прямо со свободно бегущего счётчика, минуя всё SPI-ядро — разряд 4 даёт около 1.5 МГц на SCLK, разряд 6 — около 390 кГц на MOSI, разряд 8 — около 98 кГц на CS. Частоты выбраны заметно разными специально: по одной картинке видно не только «что-то шевелится», но и какой именно вывод перед вами.



Интерпретация бинарна, и в этом вся ценность: одна пересборка делит пространство гипотез пополам, вместо того чтобы добавлять к нему ещё одну догадку. Приём перенесён из Altera-проекта, где он закрывал класс «физика или логика». Параметр обязан быть нулём в любой функциональной сборке: в этом режиме SPI-ядро физически отключено от выводов.

Глава 21. Selftest, LED и ILA

21.1. Три органа чувств вместо дисплея

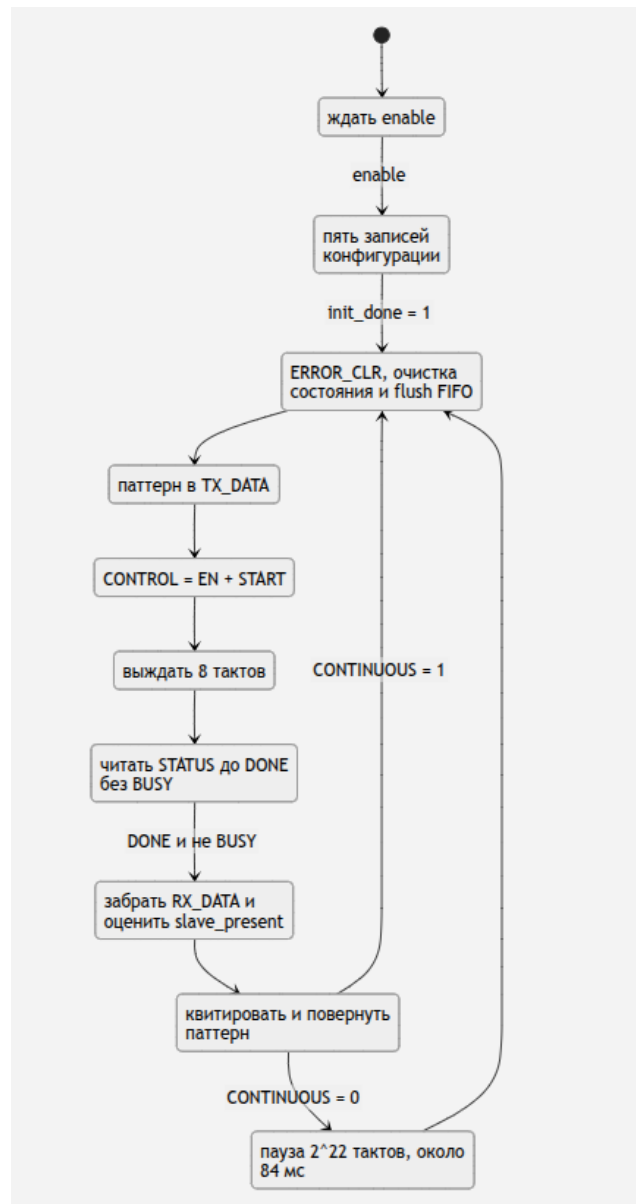
На Altera приборной панелью служил LCD 480×272: тысячи бит наблюдаемости, обновляемых шестьдесят раз в секунду, без компьютера и JTAG. Перенести панель на TZT RK-ZYNQ7020-F невозможно — параллельного RGB-дисплея на плате нет, а опросчик, привязанный к тачскрину XPT2046, потерял бы смысл вместе с отсутствующим тачем. Это пункт М-3 отчёта о миграции: подсистема исключена, и это честная потеря возможности, а не улучшение.

Соблазн заменить её встроенным SPI-дисплеем ST7789 велик. Мы отказались от этой идеи как от **основной** цели по трём независимым причинам, каждая из которых достаточна. У дисплейного разъёма нет линии MISO — производитель выводит только `io0`; без обратной линии вся приёмная половина контроллера, включая синхронизатор и дефект А-3, осталась бы непроверенной, то есть дисплей проверял бы именно ту половину, которая ломается реже. Линия

электрически занята реальным устройством, которое станет интерпретировать наши случайные посылки как команды. И чтобы дисплей ожил, нужны GPIO для DC, RST и подсветки, а значит block design, процессорная система и софт — ровно то, от чего мы избавлялись в разделе 20.1. Проверять SPI через непроверенный дисплей значит отлаживать один неизвестный интерфейс через другой. Поэтому наблюдаемость собрана из трёх дешёвых слоёв: два светодиода (ноль инфраструктуры, мгновенная реакция, но всего два бита), `spi_selftest` (аппаратный хост, заставляющий дизайн работать сам) и ILA (анализатор внутри ПЛИС, показывающий недоступное щупу). Дисплей вернётся в части «Дисплей ST7789», но уже как приложение и критерий успеха, а не как костыль отладочного топа.

21.2. Что именно делает `spi_selftest`

Это маленький автомат, повторяющий работу драйвера, только отлитый в логике. После сброса он проходит по конфигурационной таблице из пяти записей, по одной за такт: `CLK_DIV`, `CS_SELECT`, `WORD_LEN`, `DELAY_CFG` со значением `32'h0008_0808` и, последним, `CONTROL` с битом `EN`. Порядок важен ровно в одном: разрешение движка приходит последним, когда всё остальное уже запрограммировано. Затем поднимается `init_done` и начинается вечный цикл.



Два состояния существуют из-за конкретных ошибок. `S_SETTLE` выжидает восемь тактов между записью `START` и первым опросом `STATUS`: без паузы автомат прочитал бы флаг `DONE`, оставшийся от **предыдущей** транзакции, решил бы, что обмен закончен, и забрал бы из `FIFO` чужое слово. `S_CLR` перед каждой транзакцией пишет в `ERROR_CLR`, сбрасывая липкие флаги и очищая обе очереди, чтобы новая транзакция не унаследовала мусор. Обе меры — аппаратная копия дисциплины, которую в обычной системе соблюдает драйвер. И ещё одно важное свойство: `selftest` говорит с ядром по той же параллельной регистровой шине, что и Altera-хост, а не через `AXI`, — если дизайн работает под `selftest`, но не работает под `AXI`, виновата обёртка, а не ядро.

Отдельного слова заслуживает выбор того, что показывать вторым светодиодом: их на плате ровно два, первый уходит под `heartbeat`, и вариант `init_done` был рассмотрен и отвергнут. Причина в информационной ёмкости: конфигурация занимает около пяти тактов, поэтому светодиод от `init_done` загорелся бы в первое мгновение после сброса и горел бы всегда, а индикатор, который никогда не меняется, — украшение. `slave_present` меняется вместе с состоянием проводки, прямо в руках, — поэтому шаг 13 вообще является проверкой. Остальные сигналы (`init_done`, `err_seen`, `xfer_count`, `last_tx`, `last_rx`) выведены пробниками для `ILA` и удерживаются живыми через служебную цепь `unusedok`, чтобы синтезатор не выбросил их как не имеющие нагрузки.

21.3. Что смотреть в `ILA`

ILA (Integrated Logic Analyzer) — логический анализатор, который Vivado встраивает внутрь вашей же ПЛИС: он пишет выбранные внутренние сигналы в блочную память по заданному условию запуска и выгружает записанное по `JTAG`. Плюс невозможно переоценить — видно то, что не выведено ни на один вывод. Минусы честные: `ILA` занимает ресурсы и меняет разводку, поэтому в `production`-битстрим он не входит. Сигналы ядра живут под `u_spi/`, сигналы хоста — под `u_selftest/`.

Группа	Сигналы
FSM движка	<code>u_eng/state</code> , <code>edge_cnt</code> , <code>bit_cnt</code> , <code>half_cnt</code> , <code>delay_cnt</code>
Данные	<code>u_eng/tx_word</code> , <code>rx_word</code> , <code>miso_sync</code>
Выводы SPI	<code>spi_sclk</code> , <code>spi_mosi</code> , <code>spi_miso</code> , <code>spi_cs_n</code>
Состояние ядра	<code>u_reg/stat_done</code> , <code>irq_status</code> , липкие флаги ошибок
FIFO	<code>u_tx_fifo/count</code> , <code>u_rx_fifo/count</code> , <code>full</code> , <code>empty</code>
Self-test	<code>state</code> , <code>init_done</code> , <code>last_tx</code> , <code>last_rx</code> , <code>xfer_count</code> , <code>err_seen</code>

Добавляют `ILA` двумя способами: разметить цепи в RTL атрибутом (`* MARK_DEBUG = "TRUE" *`) и вызвать мастер `Set Up Debug`, либо описать ядро на Tcl перед синтезом.

```
create_debug_core u_ila_0 ila
set_property C_DATA_DEPTH 4096 [get_debug_cores u_ila_0]
# ... connect_debug_port для перечисленных выше сигналов
```

Объяснить с 

Условие запуска, с которого стоит начинать, всегда одно: **спад** `spi_cs_n[0]`. Оно ловит начало кадра, и один захват даёт полную картину транзакции. Начинать со свободного захвата бессмысленно по причине из следующего раздела.

21.4. Почему CONTINUOUS = 1 по умолчанию

Посчитаем, как выглядит наш SPI во времени в реалистичном режиме. Транзакция из восьми бит на частоте 1 МГц длится порядка десяти микросекунд, а пауза задана параметром `GAP_BITS = 22`, то есть 2^{22} тактов при 50 МГц — примерно 84 миллисекунды. Доля времени, когда на шине хоть что-то происходит:

10 мкс / 84 мс = 0.012 %

Объяснить с 

Теперь вспомним, как работает недорогой анализатор в режиме **free-run**, то есть свободного непрерывного захвата: записал буфер, отрисовал, записал следующий. Буфер на типичных настройках — единицы миллисекунд, вероятность накрыть окном десятимикросекундный всплеск — единицы процентов, а глазами на живом экране это выглядит как идеально мёртвые линии. Сигнал есть, и его не видно **никогда**. За этот урок Altera-проект заплатил половиной своей недели: там отлаживали дизайн через прибор, которому ещё не научились доверять.

Отсюда параметр `CONTINUOUS`, выбрасывающий паузу целиком: транзакции идут впритык, шина занята почти всё время, промахнуться мимо такого невозможно. По умолчанию он равен единице потому, что первый заход на плату — охота за сигналами, а не измерение реалистичного трафика; реалистичный режим включается на шаге 9. Второй путь к той же цели — триггер по спаду `CS`:

`CONTINUOUS` спасает, когда прибор простой, триггер — когда нужно измерить систему такой, какая она есть.

21.5. JTAG-скрипты: диагноз, который печатает себя сам

В каталоге `../scripts/` лежит семейство файлов `jtag_*.tcl`, и их присутствие в репозитории — тоже архитектурное решение. Все они выполняются интерпретатором `XSCT` и делают то же, что можно натывать мышью, но скрипт воспроизводим, печатает машинно-читаемые метки и живёт в системе контроля версий рядом с RTL, а не в чьей-то памяти.

Разделение обязанностей намеренное. `jtag_probe.tcl` не программирует **ничего**: его задача — безопасно ответить, виден ли кабель, что на цепочке JTAG и тот ли это XC7Z020; программировать, не убедившись в этом, — риск залить битстрим в чужой кристалл.

`jtag_load_pl.tcl` загружает битстрим и рапортует состояние до и после, честно предупреждая в шапке, что `ps7_init` он **не** запускает. `jtag_verify_spi.tcl` самый содержательный: инициализирует процессорную систему, программирует фабрику и прогоняет по регистрам полный набор проверок — от чтения ID `0x53500200` до всех четырёх режимов SPI. `jtag_recover.tcl` нужен, когда `targets` возвращает пустоту при видимой цепочке: он эскалирует воздействие от сброса через FPGA к системному сбросу и затем к POR. Прикладной `jtag_lcd_demo.tcl` поднимает панель ST7789 в режиме 3 — это территория части «Дисплей ST7789».

Ценность машинно-читаемых меток видна в разделе 22.1: скрипт не просто падает, а печатает `VERIFY_SLCR` с содержимым двух конкретных регистров и `VERIFY_FCLK` с вычисленными делителями, после чего сам ставит диагноз. Такая диагностика работает даже тогда, когда человек за пультом ещё не знает, что подозревать.

Глава 22. Типичные провалы первых измерений

Шесть разборов ниже устроены одинаково: что вы видите, какая гипотеза кажется логичной, как проверяли, какая улика решила дело, в чём корневая причина, чем лечится и какое правило остаётся на будущее. Порядок соответствует вероятности встречи, а не тяжести последствий.

22.1. Молчащий PL до ps7_init

Что видели. Кабель подключён, jtag_probe.tcl отработал чисто: цепочка видна, на ней опознан xc7z020. jtag_load_pl.tcl доложил LOAD_PROGRAM_OK и вернул состояние фабрики. Hardware Manager показывает DONE. И — ничего. led[0] не мигает, на выводах разъёма нет ни фронта. При попытке позже пойти по AXI картина ещё хуже: любое чтение памяти по адресу периферии зависает.

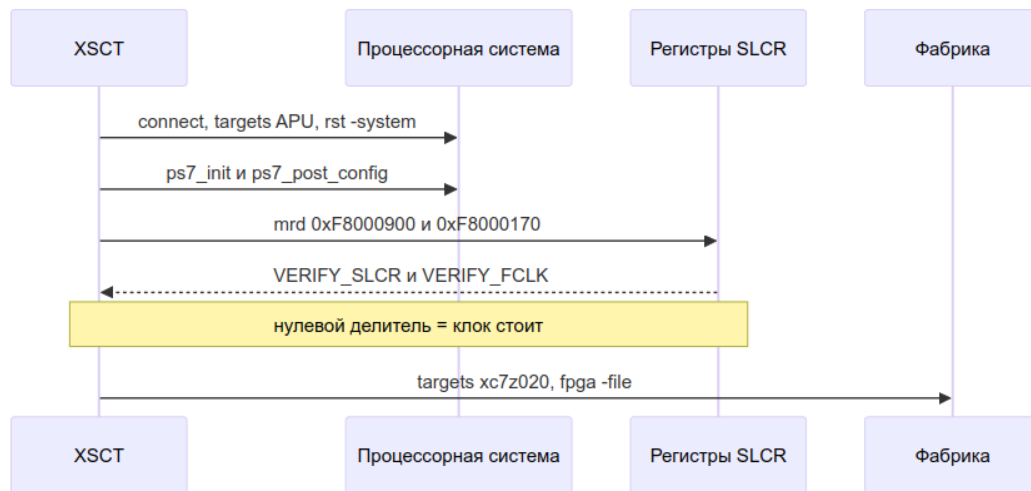
Что казалось логичным. Первые гипотезы всегда одни и те же и все разумные: битстрим повреждён; сброс залип; светодиод назначен не на тот вывод; полярность перепутана. Все четыре дороги в проверке и все четыре неверны.

Как проверяли. Сначала статус DONE: он единица, значит битстрим долит целиком, и гипотеза «повреждён» отпадает бесплатно. Дальше щуп на цепь PL-клока: осциллятор через резистор R121 номиналом 33 Ом приходит на вывод W17, и там честные 50 МГц. Полярности светодиода и кнопки подтверждены заводским device tree. Остаётся то, о чём предупреждал этап сборки: единственное замечание board-level DRC — ZPS7-1 Warning: PS7 block required, «в Zynq-дизайне обязан присутствовать блок PS7, иначе конфигурация по умолчанию некорректна». Мы читали его как безобидное. Проверяется предположение двумя чтениями регистров: 0xF8000900 — управление level shifters между процессорной системой и фабрикой, 0xF8000170 — FPGA0_CLK_CTRL, где поля делителей лежат в битах 8–13 и 20–25.

Улика. Ровно ради неё в jtag_verify_spi.tcl появились две строки печати. При первом заходе на плату level shifters оказались **выключены**, а в FPGA0_CLK_CTRL делитель DIVISOR1 был **нулём**: клок FCLK_CLK0 в сторону фабрики остановлен, потому что нулевой делитель гейтирует клок. Скрипт теперь печатает VERIFY_SLCR с обоими значениями и VERIFY_FCLK с разобранными делителями, а при нулевом делителе завершается с явным сообщением.

Корневая причина. На Zynq-7000 программируемая логика — не независимая ПЛИС, а часть системы на кристалле. Питание, вывод из сброса и, главное, разрешение преобразователей уровней между процессорной системой и фабрикой выполняются **программно**, обычно в ходе штатной загрузки, а загрузка битстрима по JTAG сама по себе ps7_init не выполняет — на это прямо указывает шапка jtag_load_pl.tcl. Кристалл сконфигурирован верно, логика внутри, возможно, работает, но наружу она не пробивается; а если разговор идёт по AXI, то ещё и клона нет. На чистой FPGA такого класса отказов нет.

Исправление. Три варианта: дать плате штатно загрузиться с SD или QSPI и затем грузить фабрику по JTAG; выполнить перед загрузкой ps7_init из XSCT — именно так поступает jtag_verify_spi.tcl, который выбирает цель APU, делает системный сброс, подгружает ps7_init.tcl производителя из linux/u-boot/, вызывает ps7_init и ps7_post_config, печатает состояние SLCR и только потом программирует фабрику; или перейти к block design с процессорной системой (часть «PS Block Design»), где вопрос снимается по построению.



Правило. На Zynq «DONE = 1» не означает «фабрика живёт»: при полном отсутствии признаков жизни первым подозреваемым делайте разрешение фабрики, а не свой RTL. И встраивайте чтение диагностических регистров прямо в скрипт — диагноз, который печатается сам, экономит тот час, за который вы успели бы испортить работающий дизайн «исправлениями».

22.2. Клока нет — или он есть, но не там

Что видели. Вариант первый: heartbeat не мигает, хотя процессорная система инициализирована и level shifters включены. Вариант второй, коварнее: heartbeat мигает, но заметно медленнее или быстрее полутора герц. Вариант третий, из сборки с процессором: всё собралось, адреса правильные, а любое обращение к регистрам зависает намертво.

Что казалось логичным. «Счётчик heartbeat написан с ошибкой», «сброс не отпускается», «неверная разрядность hb_cnt». Все три гипотезы ведут внутрь RTL — в самое дорогое для проверки и самое маловероятное для ошибки место: счётчик состоит из четырёх строк и проверен elaboration.

Как проверяли. Начинаем с арифметики, она бесплатна: heartbeat — старший бит 25-битного счётчика, 50 МГц делить на 2^{25} даёт 1.49 Гц. Мигание ровно вдвое быстрее или медленнее ожидаемого обвиняет разрядность или частоту источника; полное молчание — сам клок. Дальше щуп на цепь PL_CLK : осциллятор, резистор R121 на 33 Ом, вывод W17. Затем сверка отчёта: в reports/standalone_io.rpt вывод clk обязан быть посажен на W17 со статусом FIXED, а query_pins.tcl подтверждает функцию IO_L13P_T2_MRCC_33, то есть clock-capable вывод. **MRCC** означает, что вывод связан с глобальной сетью распределения тактов и Vivado сам вставит на него глобальный буфер; ручное инстанцирование BUFG не нужно и не выполнено. Приди клок на обычный вывод, инструмент развёл бы такт по обычным трассам с чудовищным перекосом.

Улика. В автономной сборке улика простая: на W17 либо есть 50 МГц, либо нет. В сборке с процессором улика знакома по разделу 22.1 — поле DIVISOR1 регистра FPGA0_CLK_CTRL по адресу 0xF8000170, равное нулю. Нулевой делитель — не «делить на один», а выключенный клок. Скрипт jtag_verify_spi.tcl разбирает оба поля, (fclk >> 8) & 0x3F и (fclk >> 20) & 0x3F, и отказывается продолжать при нулевом: дальше все чтения по AXI зависли бы без объяснения причин.

Корневая причина. Источников такта два, и путать их нельзя. Автономный топ тактируется собственным осциллятором платы на 50 МГц — редкая удача, потому что на многих платах Zynq фабрика тактируется только от процессорной системы, и тогда автономная сборка невозможна в принципе. Сборка с процессором работает от FCLK_CLK0, который программируется в SLCR, и в

DMA-конфигурации это уже 100 МГц. Отсюда следствие для делителя: `CLK_DIV = 24` даёт 1 МГц при 50 МГц и 2 МГц при 100 МГц. Совпадение частоты автономной сборки с Altera-референсом очень удобно: все значения `CLK_DIV` переносятся без пересчёта.

Исправление. Для автономной сборки — убедиться, что осциллятор жив и доходит до W17, а вывод объявлен как `clock-sarable`. Для сборки с процессором — выполнить `ps7_init`, который программирует делители, и убедиться печатью, что они ненулевые.

Правило. Клок — нулевая гипотеза, и проверять её надо **на входе логики**, а не по схеме. Заведите в каждом дизайне индикатор, зависящий только от клона и сброса и не зависящий от проверяемой подсистемы: это и есть смысл `heartbeat`. И помните, что на SoC «клон объявлен в дизайне» и «клон дошёл до фабрики» — разные факты, между которыми стоит регистр, который кто-то программирует.

22.3. Неверный банк и уровни

Что видели. Битстрим загружен, `heartbeat` мигает, но на выводах разъёма творится странное: логическая единица измеряется не как 3.3 В, а заметно ниже, осциллограф показывает завалы фронтов и амплитуду, не дотягивающую до порога. В худшем варианте область платы возле разъёма ощутимо греется, а поведение выводов нестабильно от включения к включению.

Что казалось логичным. «Весь заводской XDC написан в LVCMOS33, значит и банк 13 работает на 3.3 В». Рассуждение выглядит безупречно: распиновка взята из штатного проекта производителя, там везде LVCMOS33, глобальные настройки объявляют `CONFIG_VOLTAGE 3.3`, плата с этим проектом отгружается и работает. Ошибка не в логике вывода, а в том, что вывод сделан из документа, а не из измерения.

Как проверяли. Сначала — где живут наши сигналы. По `../pinout.md` и заводскому `io_40pin.xdc` весь SPI сидит в банке 13: `spi_sclk` на V8, `spi_mosi` на W8, `spi_miso` на W11, четыре линии выбора кристалла на W10, V12, W12 и U12; клон, светодиоды и кнопка — в банке 33. Дальше схема: напряжение питания банка 13 на этой плате **переключается** резисторами RA между 1.8, 2.5 и 3.3 В, и 3.3 В — заводское значение по умолчанию, но не аксиома. Финальная проверка — мультиметр на контакте питания разъёма, до первой прошивки, а не после.

Улика. Улика здесь не осциллограмма, а схема плюс одно измерение: резисторы RA задают VCCO банка 13, и если они переведены не в 3.3 В, а битстрим объявляет LVCMOS33, выходные буферы работают вне допустимого режима. Это единственный пункт всей распиновки, способный привести к **аппаратному повреждению**, и потому он вынесен в `../pinout.md` как явный риск. Прочие неопределённости — например, точный контакт внутри дифпары J1 — портят только отладку.

Корневая причина. IOSTANDARD в констрейнтах — не пожелание и не описание сигнала, а обещание инструменту, каким напряжением питается банк. Инструмент обещанию верит и настраивает буферы соответственно; если обещание расходится с реальностью, страдает не отчёт, а кремний. Вдобавок на этой плате нет преобразователей уровней на 40-контактном разъёме — подключение прямое к выводам ПЛИС, так что между ошибкой в XDC и физикой нет смягчающего звена.

Исправление. Подтвердить положение резисторов RA **до** первой прошивки. Если банк действительно на 3.3 В, ничего не меняется. Если на 1.8 или 2.5 — все IOSTANDARD секции банка 13 обязаны измениться вместе с ним, и это не косметическая правка: она меняет электрический контракт всего разъёма. Факт назначения после сборки проверяется по `reports/standalone_io.rpt`, где для каждого вывода печатается банк и стандарт.

Правило. Перед первым битстримом на новый банк — мультиметр на VCCO. Никогда не выводите

электрический стандарт из названия платы, из соседних строк чужого XDC или из того, что «по умолчанию обычно 3.3»: значение по умолчанию — не гарантия, а догадка, и в единственном случае, когда догадка может сжечь плату, вероятности недостаточно.

22.4. CS не двигается

Что видели. SCLK и MOSI живут: на щупе честные пачки импульсов и меняющийся паттерн. А линия выбора кристалла стоит колом — либо постоянно в единице, либо, что сбивает с толку сильнее, постоянно в нуле; кадры никак не обрамлены. Смежный вариант той же истории: джампер стоит, шаги 5–7 пройдены, а `led[1]` не загорается.

Что казалось логичным. «FSM движка застряла в состоянии покоя», «сломан синхронизатор», «мы всё-таки сломали ядро при переносе». Соблазн открыть `spi_engine.v` максимальный: сигнал не двигается, значит виновата логика, которая им управляет. Заметьте, насколько это похоже на правду — и насколько дорого обходится.

Как проверяли. Первый вопрос задаём не к логике, а к щупу: на **той** ли он линии? `spi_selftest` настроен на `CS_INDEX = 0`, то есть двигается только `spi_cs_n[0]` на выводе W10, а `spi_cs_n[1..3]` на V12, W12 и U12 обязаны неподвижно стоять в единице; щуп на V12 покажет ровно описанную картину, и она будет корректным поведением исправного дизайна. Второй вопрос — к ILA, и он решающий: растёт ли `xfer_count`? Счётчик увеличивается в состоянии `S_READ`, то есть только после полностью завершённой транзакции, и подделать его щупом невозможно. Третий — к режиму наблюдения: при `CONTINUOUS = 1` CS проводит в нуле почти всё время, поднимаясь на считанные такты между кадрами, и на грубой развёртке это выглядит как «CS прижат навсегда». Четвёртый — к джамперу: сверить контакты J1 визуально и убедиться мультиметром, что при снятом джампере MISO даёт около 3.3 В за счёт подтяжки, а при установленном следует за MOSI.

Улика. Развилка проходит по `xfer_count`. Растёт — транзакции реально идут, и весь вопрос в том, куда смотрит щуп и как настроен прибор. Стоит на месте — автомат застрял, и застрять он может практически только в `S_POLL`, ожидая флага DONE, которого нет: например, потому что START выдан при пустой очереди передачи и вместо завершения поднялся липкий `ERR_START`. Для петли улика ещё проще и она статическая: MISO, который при снятом джампере не показывает 3.3 В, означает, что подтяжка стоит не на том паде.

Корневая причина. В подавляющем большинстве случаев причина топологическая, а не логическая: щуп на другой линии выбора кристалла, джампер не на той паре контактов, слишком грубая развёртка. В меньшинстве случаев автомат действительно ждёт события, которое не наступит, и тогда виновата последовательность конфигурации, а не сама FSM.

Исправление. Поставить триггер по спаду `spi_cs_n[0]` вместо свободного захвата — один захват покажет весь кадр. Пересобрать с `DIAG_PINWALK = 1`, если сомнения в физике остались: там CS переключается счётчиком на 98 кГц, и спутать его с SCLK на 1.5 МГц невозможно. Сверить контакты джампера по шелкографии и таблице контактов J1.

Правило. Прежде чем винить RTL, докажите, что смотрите на тот провод. Внутренний счётчик завершённых транзакций — свидетель, показания которого нельзя подделать неправильной установкой щупа, и выводить его в ILA стоит именно поэтому. И держите в голове, что «сигнал неподвижен» иногда означает «сигнал ведёт себя правильно, а вы ожидали другого».

22.5. Режим 0 против режима 3

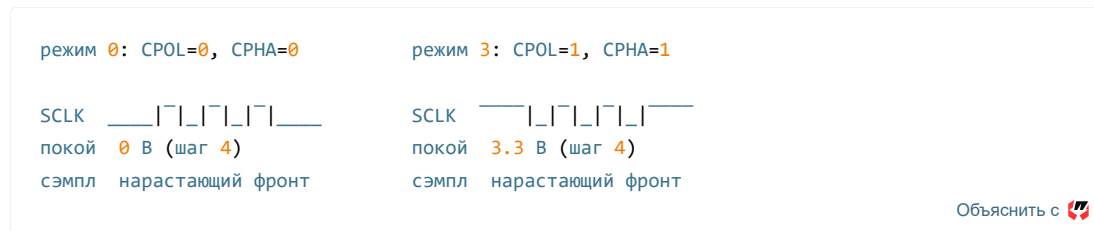
Что видели. Петля работает идеально: `led[1]` горит, ILA подтверждает совпадение `last_tx` и младшего байта `last_rx`, свип частот пройден. Затем к разъёму подключается настоящее устройство — или делается попытка поговорить со встроенной панелью ST7789 — и оно не

отвечает ничем осмысленным. Иногда ответ приходит, но сдвинут на бит. Иногда шаг 4 даёт неожиданный результат заранее: SCLK в покое стоит в нуле, а даташит слейва требует высокого уровня.

Что казалось логичным. «Раз петля сошлась, протокол реализован верно — значит виновато устройство, кабель или адресация». Рассуждение звучит солидно, но содержит скрытую ошибку, и это самая тонкая ловушка всей главы.

Как проверяли. Начали с того, что перечитали, **что именно** доказывает петля. В петле оба конца — один и тот же мастер: он сам выставляет бит и сам же его сэмплирует, пользуясь одним представлением о том, где находятся фронты. Сдвиньте фазу на полтакта, инвертируйте полярность — петля сойдётся всё равно, потому что передатчик и приёмник сдвинулись синхронно. Шаг 11 прямо утверждает, что в петле принятое слово равно переданному **во всех четырёх режимах**, а кейсы 2–5 симуляции проверяют это явно. Дальше проверили, что записано в CONTROL: `spi_selftest` кладёт туда только бит EN, то есть CPOL и CPHA остаются нулями и мастер всегда работает в режиме 0. А заводской `device tree` объявляет для ST7789 сразу `spi-cpol` и `spi-cpha`, то есть режим 3, — и `jtag_lcd_demo.tcl` соответственно формирует `CTRL_MODE3 = EN | CPOL | CPHA`, где CPOL — бит 3 (0x08), CPHA — бит 4 (0x10).

Улика. Самая дешёвая улика лежит на шаге 4 и стоит две минуты: **уровень покоя SCLK равен CPOL**. Ноль вольт на такте в покое означает CPOL = 0, то есть режим 0 или 1, независимо от того, что вы думали настроить.



Корневая причина. Режим SPI — свойство не мастера, а **пары** «мастер и слейв»; проверить его можно только против устройства, у которого есть собственное мнение о фронтах. Наш автономный стенд такого мнения не имеет по построению, поэтому единственный класс ошибок, который петля не ловит в принципе, — рассогласование режима. Обратите внимание на деталь диаграммы: в режимах 0 и 3 сэмплирование идёт по нарастающему фронту, и именно поэтому многие устройства работают в обоих; ошибка проявляется только на тех, кто чувствителен к уровню покоя такта или к моменту первого фронта.

Исправление. Для автономного стенда — путь 1 шага 11: дописать в конфигурационную таблицу `S_CFG` файла `spi_selftest.v` запись в CONTROL с нужными CPOL и CPHA, пересобрать и проверить осциллографом уровень покоя и фронт сэмплирования. Для реального устройства — путь 2: сборка с процессором, где режим меняется записью в регистр без пересборки, как в `jtag_verify_spi.tcl`.

Правило. Знайте границы своих доказательств. Зелёная петля — алиби тракта, и только тракта: о совместимости с чужим устройством она не говорит ничего. Проверяйте режим против уровня покоя такта на шаге 4 и против даташита слейва, а не против собственного эха. И записывайте режим в CONTROL явно, даже если он совпадает со сбросовым: явная запись читается как намерение, умолчание — как случайность.

22.6. Слишком агрессивный CLK_DIV

Что видели. На одном мегагерце петля работает безупречно часами. Дальше возникает

естественное желание: раз всё хорошо, давайте быстрее. `CLK_DIV_VAL` уходит с 24 на 4, потом на 2 — всё ещё хорошо. Ставим 1, то есть 12.5 МГц, и картина ломается странно: `led[1]` продолжает гореть, потому что принятый байт по-прежнему не равен ни `0x00`, ни `0xFF`, но ILA показывает, что при `last_tx = 0x3C` пришло `last_rx = 0x1E` — слово сдвинуто ровно на бит вправо. Осциллограмма при этом образцовая: SCLK чистый, MOSI несёт правильный паттерн, CS аккуратно обрамляет кадр.

Что казалось логичным. Гипотезы выстраиваются в понятный ряд, и все они о физике. «Джампер длинный, наводки». «SCLK и MOSI попали на две половины одной дифференциальной пары `I09_P` и `I09_N`, то есть такт идёт по сильно связанной трассе рядом с данными — вот и перекрёстная помеха». «Нужен BUFG на SCLK». «Не хватает согласования на разъёме». Есть и попроще: «это баг переноса».

Как проверяли. Единственная методичная проверка здесь — свип, шаг 12. Проходим весь ряд: 249 даёт 100 кГц и работает, 24 даёт 1 МГц и работает, 4 даёт 5 МГц и работает, 2 даёт 8.33 МГц и работает, 1 даёт 12.5 МГц и ломается. Затем то же прогоняется в симуляции на **неизменённом** Altera-ядре через параллельную шину командой `make sim_clkdiv`, и результат воспроизводится один в один. Наконец, отделяется передача от приёма: на MOSI слейв видит правильный паттерн, значит ломается только путь сэмплирования.

Улика. Улика — характер границы. Проблемы целостности сигнала деградируют **плавно**: сначала растёт джиттер, потом появляются редкие ошибки, потом частые. Здесь переход резкий и воспроизводимый до бита: при делителе 2 приём идеален, при делителе 1 он сдвинут ровно на один бит, всегда и на любых данных. Резкая граница на целочисленном параметре — не физика провода, а арифметика тактов. Вторая улика: дефект живёт и в исходном Altera-ядре.

Корневая причина. Дефект A-3 из части «Архитектура ядра SPI». Линия MISO — асинхронный вход, и она честно проходит через двухступенчатый синхронизатор с атрибутом `ASYNC_REG`. Слейв обновляет бит на завершающем фронте, значение доходит до точки сэмплирования через три такта системного блока (защёлка плюс две ступени синхронизатора), а мастер сэмплирует через полупериод, равный `CLK_DIV + 1` тактов. Условие корректности арифметическое: `CLK_DIV + 1 >= 3`, то есть `CLK_DIV >= 2`. При единице мастер сэмплирует раньше, чем новый бит успевает доехать, и захватывает предыдущий — отсюда ровно один бит сдвига. Синхронизатор корректен против метастабильности, но **съедает время**: в главе 17 части «Vivado, constraints, pinout» показано, что именно поэтому MISO объявлен асинхронным входом через `set_false_path`.

Исправление. Поведение сохранено намеренно: молча починить известный дефект означало бы разрушить доказательство эквивалентности с Altera-ядром. Предел зафиксирован тестом 25, который **требует** получить `0x1E` при делителе 1, — изменит кто-нибудь глубину синхронизатора, тест упадёт. Практически: делитель 1 не использовать никогда; честный потолок при 50 МГц равен 8.33 МГц, то есть `f_clk / 6`, а не 12.5 МГц из старого README; драйвер обязан обрезать по нему `max_speed_hz`. И помните про масштабирование: в сборке с процессором блок обычно 100 МГц, и тот же делитель даёт вдвое большую частоту.

Правило. Негативный тест на известный предел — часть приёмки, а не неприятный сюрприз в поле. Любой синхронизатор на входе данных протокола проверяйте свипом частоты, а не одним удачным прогоном. Если передача идеальна, а приём сдвинут — первым подозреваемым делайте путь сэмплирования, а не «наводки». И читайте «рекомендуется» в чужой документации как подозреваемое: часто это замаскированное «обязательно».

22.7. Чек-лист «можно переходить к процессору»

Минимум для перехода — пройденные шаги с нулевого по восьмой; желательно все четырнадцать.

Критерий формулируется списком, и это второй и последний список в этой части:

- heartbeat стабильно мигает, кнопка PL_KEY1 его останавливает;
- уровни покоя на CS, SCLK и MISO соответствуют таблице шага 4;
- SCLK, CS и MOSI наблюдаются на контактах разъёма, паттерн меняется;
- петля MOSI→MISO зажигает led[1], ILA даёт last_tx == last_rx[7:0];
- длительная серия идёт с err_seen = 0 и растущим xfer_count;
- предел CLK_DIV >= 2 подтверждён на железе, а не принят на веру.

Только после этого имеет смысл собирать block design и пускать Cortex-A9 к тем же регистрам: каждая невычеркнутая здесь гипотеза переключается в часть VI и смешивается там с ошибками карты адресов, ps7_init, Device Tree и драйвера, превратившись в неразличимое «SPI не работает».

Итог части V. Bring-up — дисциплина порядка, а не героизм с анализатором: сначала отчёт о распиновке, потом светодиод, потом мультиметр, и только потом щупы. Автономный топ со встроенным spi_selftest существует затем, чтобы отсечь половину переменных, пока их ещё можно отсекают по одной; два светодиода и ILA заменяют отсутствующий дисплей; петля MOSI→MISO одним проводом допрашивает весь тракт, а DIAG_PINWALK одной пересборкой делит пространство гипотез пополам. Шесть разобранных провалов объединяет одно наблюдение: почти все они живут **снаружи** ядра — в разрешении фабрики, в питании банка, в делителе клок, в положении щупа и в границах того, что доказывает ваш тест. Дальше — Cortex-A9 и та же карта регистров, но уже по AXI: часть VI.

Часть VI. PS Block Design

PL-only selftest из части V доказал, что SPI-ядро живо: клок идёт, сброс снимается, CS обрамляет кадр, петля MOSI→MISO сходится бит в бит. Всё это ядро делало **само**, без процессора: роль хоста играл конечный автомат spi_selftest, вшитый в тот же битстрим. Теперь мы сажаем рядом с ядром настоящий процессор — Cortex-A9 внутри того же кристалла XC7Z020 — и заставляем его писать регистры, читать STATUS и принимать прерывания. RTL при этом не меняется ни на строчку: вся работа происходит в обвязке.

Обвязка на Xilinx называется **Block Design** (BD), и это не «ещё один файл на Verilog». BD — это схема соединений блоков внутри кристалла, которую вы рисуете в редакторе IP Integrator примерно так же, как макетную плату собирают из готовых микросхем. Блоки — это IP-ядра: либо готовые от Xilinx (процессорная система, интерконнект, GPIO, DMA), либо ваши собственные, подключённые как **module reference** — «возьми вот этот Verilog-модуль и считай его блоком». Провода между блоками — это либо отдельные сигналы, либо целые интерфейсы (весь AXI одной линией на схеме). На выходе Vivado генерирует обычный Verilog-файл system_wrapper.v, который и становится верхним модулем проекта. То есть BD — способ не писать руками несколько сотен строк соединений.

Цена этого удобства — три новых класса ошибок, и все три коварны одинаково: проявляются они не в Vivado, а через недели, уже в Linux, и выглядят как «не работает драйвер». Первый класс — **карта адресов**: процессор пишет по адресу, которого нет, и читает нули. Второй — **маршрут прерывания**: логика дёргает провод, а ядро об этом не узнаёт. Третий — не ошибка, а **контракт**: три свойства нашего RTL (F-1...F-3), которые запрещают софту делать очевидные вещи, и если их не прочитать заранее, драйвер будет спроектирован неправильно, а всплывёт это на дисплее и на

DMA. Этим темам и посвящены главы 23–25.

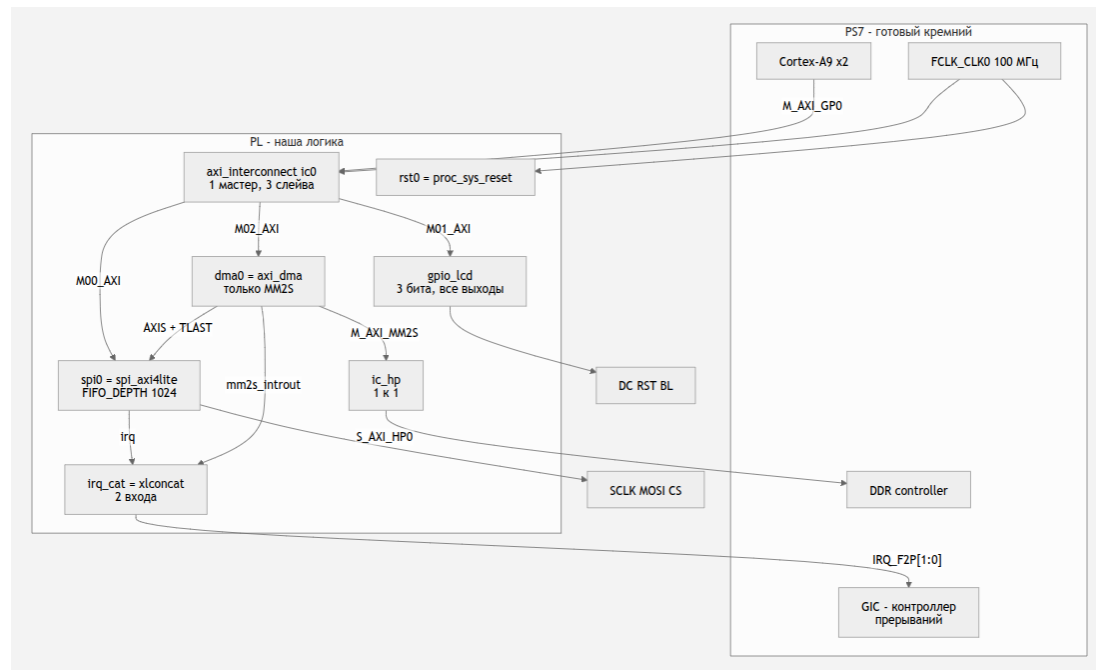
Глава 23. GP0, Address Editor и карта 0x4000_0000

23.1. Из чего собран наш Block Design

Наш BD целиком описан скриптом `../scripts/build_ps_axi.tcl` — по нему Vivado 2025.2 создаёт проект `build/ps_axi` под кристалл `xc7z020clg484-2`, собирает схему `system` и доводит её до битстрима. Прежде чем разбираться в адресах, посмотрим на население схемы.

Центральный блок — **processing_system7** (в скрипте он называется `ps7`). Это не «ещё одно IP», а представление всей аппаратной половины кристалла: двух ядер Cortex-A9, контроллера DDR, UART, SD, Ethernet и сотни настроек. Zynq устроен так, что процессорная часть (**PS**, Processing System) — физически готовый кремний, а программируемая логика (**PL**, Programmable Logic) — то, что мы конфигурируем битстримом. Блок `ps7` в BD — граница между ними: всё, что на нём торчит наружу, и есть провода, которыми PS разговаривает с нашей логикой. Включаем мы ровно пять настроек: мастер-порт `M_AXI_GP0`, слейв-порт `S_AXI_HP0` шириной 32 бита, вывод тактового сигнала `FCLK_CLK0` на 100 МГц и разрешение прерываний из фабрики (`PCW_USE_FABRIC_INTERRUPT`, `PCW_IRQ_F2P_INTR`). Остальное — DDR и `FIXED_IO` — выводит наружу автоматика `apply_bd_automation`, минуя нашу логику.

Дальше идут наши и покупные блоки. `spi0` — это `spi_axi4lite` из части III, подключённый как `module reference` с параметром `FIFO_DEPTH = 1024`. `rst0` — `proc_sys_reset`, генератор сбросов: он берёт `FCLK_RESET0_N` от PS и превращает его в синхронизированные `peripheral_aresetn` (для периферии) и `interconnect_aresetn` (для шины), потому что снимать сброс с интерконнекта нужно чуть иначе, чем с устройств за ним. `ic0` — интерконнект на три слейва; `gpio_lcd` — трёхбитный `axi_gpio` только на выход (DC, RST, подсветка дисплея, см. часть VIII); `dma0` — `axi_dma` в простейшей конфигурации: без scatter-gather, только канал MM2S (память→поток), 32 бита, размер burst'a 16. И три «клеевых» блока: `miso_tie` (константа 1 на вход MISO, у панели нет обратной линии), `cs_slice` (из четырёхбитной шины `spi_cs_n` наружу идёт только нулевой бит) и `irq_cat` — про него вся глава 24.



Обратите внимание на асимметрию, которая объясняет половину архитектуры: к нашим регистрам

ведёт **одна** тонкая линия от процессора, а поток пиксельных данных идёт **другим** путём — из DDR через `dma0` и обратно в PS через `S_AXI_HP0`.

23.2. Что такое порт GP и почему регистры висят именно на нём

AXI — это шина, по которой внутри SoC общаются блоки; подробный разбор AXI4-Lite был в части III. Здесь достаточно помнить роли: **мастер** начинает транзакцию («прочитай мне слово по адресу X»), **слейв** отвечает. Наш `spi_axi4lite` — всегда слейв, процессор — всегда мастер.

У PS Zynq-7000 есть несколько заранее разведённых AXI-портов на границе с PL, и они не взаимозаменяемы. В сторону логики смотрят два мастер-порта: **M_AXI_GP0** и **M_AXI_GP1** (GP — General Purpose, «общего назначения»), оба 32-битные. Именно и только через них процессор может дотянуться до регистра, живущего в PL. Навстречу, из логики в память, смотрят слейв-порты **S_AXI_HP0...HP3** (HP — High Performance): это входы в подсистему памяти, через которые *мастер, находящийся в PL*, читает и пишет DDR, минуя процессор.

Отсюда сразу ответ на естественный вопрос: «а нельзя ли повесить регистры SPI на быстрый HP вместо медленного GP?» Нельзя, и не потому что «не принято», а потому что направление другое. HP-порт — это **слейв на стороне PS**: он принимает транзакции от мастеров из фабрики и ведёт их в контроллер DDR и ОCM. Процессорная запись в регистр SPI туда физически не попадёт: у CPU нет пути «наружу» через HP. Мастер-порты GP — единственная дверь из процессора в нашу логику, и это не предмет выбора, а данность топологии кремния.

Цена этого решения реальна, и мы за неё заплатим в части IX. GP0 — 32-битный порт, каждое обращение к регистру идёт полным циклом AXI через интерконнект: адрес, данные, ответ. Для управления это идеально: записать `CLK_DIV`, `WORD_LEN`, дернуть `START` — единицы обращений на транзакцию, задержка неважна. Для перекачки кадра дисплея $320 \times 172 \times 2 \approx 110$ КиБ **словами через GP0** это катастрофа: сто тысяч отдельных round-trip'ов из пользовательского процесса под Linux. Поэтому в BD с самого начала предусмотрен второй, широкий путь: `dma0` сам становится мастером, читает буфер из DDR через `ic_hr` и `S_AXI_HP0`, а в SPI-ядро данные приходят потоком AXI-Stream, не проходя через процессор. Регистры на GP0, данные по HP0 — разделение труда, типовое для любого SoC-дизайна.

23.3. Интерконнект: один мастер, три слейва, и почему их два

`M_AXI_GP0` — один порт, а слейвов у нас трое. Соединить один мастер с несколькими слейвами напрямую нельзя: кто-то должен смотреть на адрес в транзакции и решать, кому её отдать. Этим занимается **интерконнект** — блок, который по адресу выбирает получателя и разруливает одновременные запросы. Аналогия — сортировочный узел на почте: конверты приходят в одну щель, а дальше расходятся по ящикам согласно индексу на конверте. В скрипте это `ic0` с `CONFIG.NUM_MI = 3`: три мастер-интерфейса наружу (`M00_AXI` → SPI, `M01_AXI` → GPIO дисплея, `M02_AXI` → регистры управления DMA) и один слейв-интерфейс `S00_AXI`, куда подключён `M_AXI_GP0`. Принцип, который стоит усвоить: **номер порта интерконнекта и адрес слейва — разные вещи**. Порт говорит «куда идёт провод», адрес — «по какому числу он выбирается», а связывает их Address Editor из следующего раздела.

Второй интерконнект, `ic_hr`, устроен вырожденно: один мастер, один слейв (`NUM_MI = 1`, `NUM_SI = 1`). Зачем блок, который ничего не выбирает? Он служит адаптером и точкой изоляции между `dma0/M_AXI_MM2S` и `ps7/S_AXI_HP0` и оставляет место для будущего второго мастера. Но важнее то, чего мы этим **не** сделали: не завели поток данных на тот же `ic0`, где живут регистры. Смешивать управляющий трафик с потоковым — верный способ получить необъяснимые задержки: запись в `CONTROL` встаёт в очередь за пачкой burst'ов кадра. Два интерконнекта стоят немного логики и покупают предсказуемость.

Всё тактируется одним сигналом `FCLK_CLK0`, и все `*_ACLK` в скрипте подключены к нему же. Это

осознанное упрощение: единый тактовый домен по всей фабрике избавляет от переходов между частотами (CDC) внутри AXI и от целого класса ошибок, которые новичок не умеет диагностировать. В PL-only сборке из части V блок был 50 МГц с осциллятора платы, здесь — 100 МГц из PS (`PCW_FPGA0_PERIPHERAL_FREQMHZ 100`). Именно поэтому драйвер обязан брать частоту из `clk_get_rate()` , а не из константы: одно и то же ядро живёт на двух разных частотах, и от них зависит расчёт делителя SCLK (см. `../hw_sw_contract.md`).

Наружу из BD выведены четыре порта — `spi_sclk` , `spi_mosi` , `spi_cs_n` и трёхбитный `lcd_gpio` . Их привязка к физическим ножкам живёт не в BD, а в `../..constraints/ps_axi_pins.xdc` : SCLK на V18, MOSI на U19, CS на AA13, DC/RST/BL на W13/AA18/Y13, все LVCMOS33. Почему банк и уровень напряжения здесь — вопрос безопасности платы, а не вкуса, разбиралось в части IV.

23.4. Address Editor: откуда берутся адреса и почему они не магические

Теперь главное. Процессор обращается к регистру SPI по адресу `0x40000000` — откуда взялось это число и почему не любое другое?

Первое, что нужно понять: у Zynq-7000 **единая карта физических адресов** на 4 ГиБ, и в ней заранее, на уровне кремния, размечено, какой диапазон куда ведёт. DDR начинается с нуля, регистры собственной периферии PS живут высоко, около `0xE0000000` , а для мастер-портов в фабрику зарезервированы два окна по 1 ГиБ: `0x4000_0000 ... 0x7FFF_FFFF` **принадлежит M_AXI_GP0**, а `0x8000_0000 ... 0xBFFF_FFFF` — **M_AXI_GP1**. Это не настройка проекта, а свойство чипа, описанное в UG585. Любой адрес, который вы назначите слейву за GP0, обязан лежать внутри первого окна — иначе транзакция не уйдёт в фабрику, её перехватит другой участок карты.

Отсюда и разгадка «магии»: `0x40000000` — это **самый первый адрес окна GP0**. Мы не выбирали красивое число, мы взяли начало доступного диапазона и дальше пошли вверх с запасом. Никакой другой семантики в нём нет. Точно так же не магический и адрес `0x43C00000` из половины учебников по Zynq: это просто значение, которое Vivado исторически присваивает первому PL-периферийному блоку при автоматическом назначении. Оба адреса — из одного окна GP0; разница лишь в том, кто их назначил — вы или автоматика.

Назначением адресов занимается **Address Editor** — вкладка в BD, где каждому слейву задаётся смещение (offset) и размер окна (range). В нашем скрипте это три явных вызова `assign_bd_address` с `-offset` и `-range 0x10000` , то есть по 64 КиБ на блок. Явное назначение вместо автоматического — тоже решение с обоснованием: адрес, написанный руками в Tcl, невозможно «случайно» получить другим при следующей пересборке, а именно это делает автоматика, когда в схеме меняется состав блоков. Карта адресов — часть контракта с Device Tree, и контракт не должен зависеть от порядка добавления IP.

Блок в BD	Базовый адрес	Размер окна	Что за ним	Где ещё это число
<code>spi0/s_axi/reg0</code>	<code>0x40000000</code> <code>00</code>	<code>0x10000</code> (64 КиБ)	CONTROL... ID_VERSION нашего SPI	DT <code>spi@40000000</code> , <code>reg = <0x40000000</code> <code>0x1000></code>
<code>gpio_lcd/S_AXI/Reg</code>	<code>0x400100</code> <code>00</code>	<code>0x10000</code> (64 КиБ)	DC / RST / подсветка дисплея	DT <code>gpio@40010000</code>

dma0/ S_AXI_LITE/ Reg	0x404000 00	0x10000 (64 КиБ)	Управление каналом MM2S	DT dma@40400000
reserved- memory в DDR	0x1f0000 00	0x100000 (1 МиБ)	Буфер кадра для DMA, no-map	DT memory@1f000000

Первые три строки — это ровно то, что печатает сам скрипт между маркерами `BD_ADDRESS_MAP_BEGIN` и `BD_ADDRESS_MAP_END`; сверять карту после сборки следует по этому выводу, а не по памяти. Четвёртая строка — из другого мира: это не PL-слейв, а участок DDR, вырезанный из-под управления ядром (no-map в reserved-memory), чтобы приложение получило непрерывный физический буфер для DMA. Подробнее — в части VII.

Два числа в этой таблице требуют пояснения, потому что на них спотыкаются. Первое: **окно 64 КиБ и апертюра 64 байта — не одно и то же**. Address Editor работает с грубой гранулярностью (минимум обычно 4 КиБ, мы взяли по 64 КиБ с запасом), а наш `spi_axi4lite` декодирует всего шесть бит адреса, `awaddr[5:2]`, то есть занимает 64 байта. Остальные 65 472 байта окна **зеркалируют** те же регистры: интерконнект пропускает транзакцию, слейв смотрит только на младшие биты. Это отклонение D-4 из `../register_map.md`, и для отладки оно значит вот что: `devmem 0x40000040` прочитает CONTROL, а не «пустоту». Второе: в Device Tree у SPI-узла стоит `reg = <0x40000000 0x1000>` — 4 КиБ, а не 64 КиБ. Это не расхождение, `reg` описывает лишь то, сколько драйвер отобразит в память ядра. Совпадать обязан **базовый адрес**, а не размер.

Есть ещё одна карта адресов, о которой легко забыть, потому что она не процессорная. `dma0` — тоже мастер, и у него **своё** адресное пространство (`dma0/Data_MM2S`): он должен знать, по каким адресам ему разрешено читать DDR. Последний `assign_bd_address` в скрипте, без `-offset`, привязывает это пространство к сегменту `ps7/S_AXI_HP0/HP0_DDR_LOW0CM`. Правило простое: **сколько в схеме мастеров, столько адресных пространств**, и каждое нужно заполнить, иначе мастер читает нули.

Полная последовательность того, что делает скрипт с адресами и почему именно в таком порядке:

1. создать блоки и соединить интерфейсы (`connect_bd_intf_net`) — до этого Address Editor не видит, что и куда можно привязывать;
2. `assign_bd_address -offset 0x40000000 -range 0x10000` для SPI, затем `0x40010000` для GPIO и `0x40400000` для DMA Lite — все в `ps7/Data`;
3. `assign_bd_address` без смещения для `dma0/Data_MM2S` → `HP0_DDR_LOW0CM` (адрес здесь диктует контроллер памяти, выбирать нечего);
4. `validate_bd_design` — проверка на неподключённые интерфейсы, пересечения окон и незаполненные адресные пространства;
5. `save_bd_design` и печать карты между `BD_ADDRESS_MAP_*` — протокол сборки, который надо сохранять вместе с отчётами;
6. `make_wrapper -top` и синтез: обёртка `system_wrapper.v` становится верхним модулем, к проекту добавляется XDC.

Первая проверка после прошивки — ещё до всякого Linux и драйвера — читает паспорт IP по адресу `база + 0x3C`:

```
devmem 0x4000003c 32 # ожидание 0x53500100 (v1 PIO) или 0x53500200 (v2 DMA)
```

Объяснить с 

Ответ `0x53500200` означает, что совпало сразу три вещи: PL сконфигурирован, адрес в карте существует, и за ним действительно наш IP нужной версии. Ответ `0` или зависание шины — разбор в главе 30 части VII.

23.5. Ошибка: «адрес как в учебнике, `0x43c00000`»

Первые черновики Device Tree в этом проекте таскали за собой `0x43c00000` — и это ровно та ошибка, которую совершает каждый, кто начинал с чужого примера. Симптом обманчиво спокоен: сборка проходит, битстрим грузится, модуль вставляется, а `devmem 0x43c0003c 32` возвращает нули (в худшем случае — зависает на обращении к несуществующему адресу), драйвер в `probe()` читает `ID = 0` и отказывается биндиться с сообщением про несовместимый bitstream. Логичным казалось объяснение «сломался наш IP» или «битстрим старый, без регистра ID» — тем более что драйвер именно так нулевой ID и трактует, и формально он прав. Версию битстрима перепроверили, пересобрали. Не помогло.

Улика нашлась при сверке двух источников, которые обязаны совпадать: вывода `BD_ADDRESS_MAP` из `build_ps_axi.tcl`, где стоял `0x40000000`, и свойства `reg` в тогдашнем черновике `../linux/dts/zynq-rk7020-spi.dts` вместе с его фрагментом `zynq-rk7020-fpga-spi.dtsi`, где всё ещё оставался `0x43c00000`. В репозитории эту правку давно внесли — сегодня оба файла описывают узел как `spi@40000000`, — но след старой карты уцелел в логах загрузки, где устройство называется `43c00000.spi`. Корневая причина оказалась не в железе и не в драйвере: **Device Tree описывал другую карту адресов, чем та, что зашита в битстрим**. Оба адреса лежат в окне GP0, оба «правильные» с точки зрения архитектуры Zynq — но слейв разведён по одному, а обращаются по другому, и интерконнект не имеет права передать такую транзакцию никому. Отсюда и нули, и вариативность симптома.

Исправление заняло одну строку в DTS (`reg = <0x40000000 0x1000>`) плюс приведение остальных узлов к тем же смещениям, что в Tcl. Но правило важнее правки: **адрес рождается в Address Editor и копируется в Device Tree, никогда наоборот и никогда «по памяти»**. Учебник Xilinx с его `0x43c00000` описывает чужой проект, где так решила автоматика; ваш проект — там, где вы написали `assign_bd_address`. И следствие для процедуры: после любого изменения состава BD первым делом смотрите распечатку карты адресов, а не логи синтеза.

23.6. Почему BD хранится как Tcl-скрипт, а не как файл `.bd` в git

Vivado умеет сохранять Block Design в собственный файл — у нас это `build/ps_axi/ps_axi.srcs/sources_1/bd/system/system.bd`. Соблазн положить его в репозиторий велик: это же «исходник схемы». Мы сознательно так не делаем — каталог `spi_xilinx/build/` целиком в `.gitignore`, а в git лежит только генерирующий скрипт. Обоснование стоит проговорить: это типовое решение для любого FPGA-проекта, который переживёт больше одного человека.

Файл `.bd` — машинно-генерируемое описание, привязанное к версии среды и набитое координатами блоков на холсте, GUID'ами и путями. Он не читается глазами: попробуйте по diff'y понять, что коллега переставил вход `concat`'а или сдвинул окно адресов на 64 КиБ. Он плохо мерджится: два человека, подвинувшие блоки в редакторе, получают неразрешимый конфликт. И он не переносится между версиями Vivado без миграции. Tcl-скрипт лишён всех трёх недостатков: `NUM_MI {3}` и `-offset 0x40000000` видно в diff'e, конфликт по строке решается как в любом коде, а несовместимость со средой проявляется явной ошибкой, а не тихой «миграцией».

Дополнительный выигрыш: скрипт заодно является документацией. Всё, что мы разбирали в этой

главе, — конфигурация PS7, состав интерконнекта, карта адресов, склейка прерываний — читается из одного файла на 200 строк, и он не может рассинхронизироваться с реальностью, потому что из него и собирается битстрим. Плюс встроенная приёмка: скрипт печатает PSAXI_WNS / PSAXI_WHS (запас по временке), пишет отчёты в reports/ и завершается маркером PSAXI_DONE, так что «собралось ли» — вопрос к grep, а не к человеку.

Цена решения дисциплинарная. Правку, сделанную мышкой в GUI, нужно **вручную перенести в Tcl**, иначе она исчезнет при следующей пересборке с нуля. Второй нюанс сложнее: BD кэширует результат синтеза module-reference-блоков в out-of-context .dcp, и правка rtl/*.v может не попасть в битстрим, даже если скрипт пересобран. Ровно для этого существует второй скрипт, ../../scripts/rebuild_ps_axi_rtl.tcl: он принудительно перечитывает RTL, валидирует BD с -force, сбрасывает OOC-run'ы, содержащие spi0, и только потом гоняет синтез и имплементацию. Эта ловушка стоила дня отладки и разобрана как мета-ошибка в главе 42 части IX.

Глава 24. IRQ_F2P и xlconcat

24.1. Как прерывание физически доходит из логики до ядра Linux

Прерывание — это способ железа сказать процессору «я закончил» вместо того, чтобы процессор постоянно спрашивал «ну что, готово?». Второй вариант называется опросом (polling), он в нашем драйвере тоже есть как резервный режим, но сжигает процессорное время в цикле чтения STATUS. Аналогия — звонок на двери против привычки каждые десять секунд проверять, не пришёл ли кто. Дальше проследим весь путь этого звонка: именно на нём теряются прерывания.

Начало пути — в нашем RTL. Регистровый блок вычисляет выходной сигнал по формуле из ../register_map.md:

```
irq_out = CONTROL.IRQ_EN & |(IRQ_STATUS & IRQ_MASK)
```

Объяснить с 

То есть линия поднимается, если глобальное разрешение включено **и** хотя бы одна из трёх причин (DONE, RX_VALID, ERROR) и случилась, и разрешена маской. Биты IRQ_STATUS — **sticky**: это флаги, которые, раз поднявшись, стоят до явного сброса, как выбитый автомат в щитке. Сами они не вернуться, даже если причина давно исчезла. Свойство ключевое, к нему мы вернёмся в 24.4 и в главе 25.

Дальше сигнал spi0/irq — один провод в фабрике, и ему нужно попасть в контроллер прерываний внутри PS. У блока ps7 для этого есть вход **IRQ_F2P** («fabric to PS», из фабрики в процессорную систему) — но это не одиночный провод, а **шина** шириной до 16 бит: PS готов принять до шестнадцати независимых источников из PL. У нас шина двухбитная, потому что источников два: spi0/irq и dma0/mm2s_introut (прерывание канала MM2S, «буфер отправлен»).

Внутри PS шина попадает в **GIC** (Generic Interrupt Controller) — стандартный для ARM блок, который принимает десятки линий от всей периферии, назначает им приоритеты и доставляет тому ядру, которое свободно. Источники GIC различает по номерам, и здесь начинается терминологическая ловушка: в мире GIC аббревиатура **SPI** означает Shared Peripheral Interrupt, «разделяемое периферийное прерывание», и к нашей шине SPI отношения не имеет. Когда в документации написано «IRQ_F2P отображается в SPI 61...68», речь о номерах прерываний. Глоссарий сокращений — в appendices.md.

Последнее звено — Linux. Ядро не знает про Vivado и не читает битстрим; всё, что у него есть, — Device Tree. Узел SPI в `zynq-rk7020-fpga-spi.dtsi` объявляет:

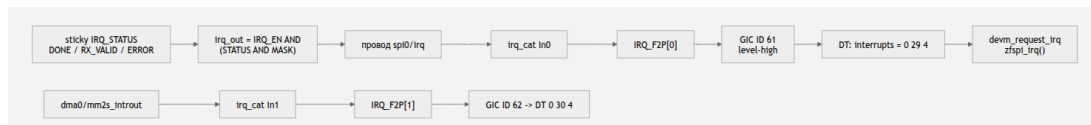
```
interrupt-parent = <&intc>; interrupts = <0 29 4>; /* IRQ_F2P[0] */
```

[Объяснить с](#) 

Три числа читаются так. Первое — тип: 0 значит SPI в смысле GIC (а 1 означало бы PPI, приватное для ядра). Второе — номер внутри этой группы; у GIC на ARM разделяемые прерывания начинаются с абсолютного идентификатора 32, поэтому 29 в Device Tree означает **GIC ID 61**. Третье — тип срабатывания: 4 — это `IRQ_TYPE_LEVEL_HIGH`, «активный уровень, высокий». Наша линия именно уровневая, и это принципиально (24.4). Для DMA соответственно `<0 30 4>`, то есть GIC ID 62.

Вход concat	Источник в BD	Бит IRQ_F2P	GIC ID	interrupts в DT
In0	spi0/irq	IRQ_F2P[0]	61	<0 29 4>
In1	dma0/mm2s_introut	IRQ_F2P[1]	62	<0 30 4>

Вся цепочка целиком, от sticky-флага до обработчика:



Семь звеньев, и каждое можно перепутать независимо от остальных. Именно поэтому симптом «драйвер не видит прерывание» из 24.3 — не одна ошибка, а целый класс.

24.2. Почему xiconcat, а не одно прерывание на всё

Вход `IRQ_F2P` — шина, а в фабрике у нас два отдельных провода. Соединить одиночный сигнал с многобитным входом нельзя, это ошибка типов; нужен блок, который «склеит» биты — **xiconcat**. Логика он не содержит вообще: чистое переименование проводов, `dout = {In1, In0}`, ноль LUT и ноль триггеров. В нашем скрипте он называется `irq_cat`, `NUM_PORTS = 2`. Аналогия — шлейф на два провода: разъём один, но вы должны решить, какой провод в какой контакт.

Альтернатива, которая приходит в голову первой, — сложить оба источника по ИЛИ и подать одну линию: «прерывание есть, а кто именно — разберёмся в обработчике». Такой разделяемый (shared) вектор в Linux легален, и драйверы это умеют: обработчик читает свой статус, и если причина не его — возвращает `IRQ_NONE`, чтобы ядро опросило следующего в цепочке. Наш обработчик, кстати, именно так и делает, если `IRQ_STATUS` пуст. Но цена такого решения складывается из четырёх пунктов, и ни один не виден сразу.

Первое: каждое прерывание любого устройства заставляет опросить регистры **всех** кандидатов, а чтение через GP0 не бесплатно. Второе: теряется наблюдаемость — в `/proc/interrupts` строка одна, и вопрос «сколько прерываний дал именно SPI» становится неотвечаемым, тогда как в `../linux_bringup.md` весь критерий приёмки построен на этом счётчике: число прерываний должно примерно равняться числу burst'ов, а в покое не расти. Третье, самое коварное: **маскирование становится общим**. Наш драйвер глушит свою линию записью `IRQ_MASK = 0` (см. 24.4) — при разделяемой линии это не помогло бы, потому что её продолжал бы держать DMA, и уровневое прерывание срабатывало бы снова и снова. Четвёртое: GIC умеет назначать разным номерам

разные приоритеты, и, объединив источники, вы отказываетесь от этой возможности навсегда.

Итого: отдельный бит на источник стоит ноль ресурсов и один блок в схеме, а покупает независимое маскирование, независимую статистику и независимые приоритеты. Плата — необходимость держать в синхроне три сущности: номер входа concat, бит IRQ_F2P и число в Device Tree. Ровно эта плата и была внесена.

Последовательность действий в BD, если вы делаете это руками в GUI (в Tcl она же, четырьмя вызовами `connect_bd_net`):

1. в настройках `ps7` включить Fabric Interrupts и внутри них — `IRQ_F2P` (в скрипте это `PCW_USE_FABRIC_INTERRUPT {1}` и `PCW_IRQ_F2P_INTR {1}`); без этого шага у блока просто не появится входного порта;
2. добавить `xlconcat` и задать `NUM_PORTS` по числу источников — здесь 2;
3. соединить `spi0/irq` с `In0` , `dma0/mm2s_introut` с `In1` , и **записать, что куда пошло**: это будущие номера в Device Tree;
4. соединить `irq_cat/dout` с `ps7/IRQ_F2P` ;
5. после `validate_bd_design` убедиться, что ширина `dout` совпала с ожидаемой шириной `IRQ_F2P` — несовпадение здесь Vivado поймает, а перепутанные входы не поймает никогда.

24.3. История: драйвер не видит прерывание

Эта история стоила примерно полдня и запомнилась тем, что все три первые гипотезы были разумными и все три были неверными.

Что видели. Битстрим с PS прошит, `devmem 0x4000003c` возвращает `0x53500200` — значит адреса верны и IP тот. Модуль загружается, `probe()` проходит, в `dmesg` печатается ровно то, что должно: `registered: AXI clock ... Hz, SCLK ..., 4 CS, 1024-word FIFO, interrupt driven` . Последние два слова важны: драйвер получил номер прерывания из Device Tree и зарегистрировал обработчик, то есть с его точки зрения всё хорошо. Но первая же транзакция через `spidev` заканчивается сообщением `burst timed out after N ms` и ошибкой `- ETIMEDOUT` . При этом на осциллографе кадр **уходит**: `SCLK` тикает, `CS` обрамляет посылку, данные на `MOSI` правильные. Железо работает, а драйвер этого не знает.

Что казалось логичным. Первая гипотеза — баг в драйвере: неверно выставлена маска, забыт бит `CONTROL.IRQ_EN` , или `complete()` вызывается не там. Вторая — «залипло железо»: sticky-флаг `DONE` не поднимается, и линия не активируется. Третья — таймаут посчитан слишком коротким. Все три проверяются дёшево, и все три отпали.

Как проверяли. По правилу локализации из `../linux_bringup.md` сначала определяем уровень, а потом правим код. Порядок был такой:

1. `grep spi /proc/interrupts` — счётчик стоит на **нуле**. Первая развилка: если бы обработчик хоть раз вызвался и вернул `IRQ_HANDLED` , счётчик бы вырос. Ноль означает, что до обработчика дело не доходило вообще, — значит гипотеза «баг внутри обработчика» мертва, искать надо до него.
2. Проверяем, поднимает ли линию сама логика. Драйвер снимаем с устройства (`unbind` , как описано в главе 35 части VIII) — у MMIO должен быть один владелец, параллельный `devmem` при живом драйвере запрещён процедурой. Повторяем транзакцию вручную и читаем `devmem 0x40000020 32 ( IRQ_STATUS )`: там **ненулевое** значение, бит `DONE` поднят. Итог: sticky-флаг есть, `IRQ_MASK` открыт, формула `irq_out` обязана давать 1.
3. ILA (порядок подключения — в `../bringup.md`) на цепи `spi0/irq` подтверждает: линия

уходит в единицу по завершении burst'a и **держится** там, никакого короткого импульса. Вся часть цепочки внутри фабрики исправна.

- Убираем свойство `interrupts` из узла Device Tree целиком. Драйвер написан так, что отсутствие прерывания — законная ситуация: `platform_get_irq_optional()` возвращает `-ENXIO`, и в лог уходит `no interrupt specified, using polled transfers`. В этом режиме **все транзакции проходят и данные корректны**. Это решающая улика: тракт данных, RTL, адреса и логика драйвера в порядке; сломан ровно и только маршрут прерывания между фабрикой и GIC.

Корневая причина. Число в Device Tree не соответствовало тому биту `IRQ_F2P`, куда физически подключён провод `spi0/irq`. Разрыв в этой цепочке возможен в трёх местах, и все три дают один симптом: в конфигурации `ps7` не включены Fabric Interrupts (тогда порта `IRQ_F2P` нет, и BD собирается без него); провод воткнут в `In1` вместо `In0` (тогда SPI «отвечает» на номере DMA, а драйвер SPI ждёт своего и не получает ничего); либо в DT записан абсолютный идентификатор GIC вместо смещения, которого ждёт биндинг, — то самое `61` вместо `29`. Последнее случается чаще всего: документация Xilinx говорит про «SPI 61», а Device Tree хочет `61 - 32 = 29`, и оба числа выглядят одинаково правдоподобно.

Исправление. `interrupts = <0 29 4>` для SPI на входе `In0` и `interrupts = <0 30 4>` для DMA на `In1` — ровно то, что сейчас записано в `zynq-rk7020-fpga-spi.dtsi` вместе с комментариями `/* IRQ_F2P[0] / u / IRQ_F2P[1] */`. Эти комментарии — не украшение: они единственное место, где связь «номер в DT ↔ вход concat» зафиксирована в читаемом виде.

Правило на будущее. Прерывание — цепочка из семи звеньев, и она проверяется от концов к середине, а не гаданием. Полезная привычка: при любом «прерывание не приходит» первым делом посмотреть `/proc/interrupts` (вызывался ли обработчик хоть раз), вторым — перевести драйвер в polled-режим (жив ли тракт данных). Эти два измерения делят пространство гипотез на четыре части за минуту. И отдельное правило про Zynq: номер в Device Tree — не тот, что в документации на чип; расхождение на 32 — норма, а не опечатка.

24.4. Уровень, а не фронт: почему обработчик первым делом маскирует

Прерывания бывают двух видов. **Фронтное** (edge-triggered) — короткий щелчок: устройство дёрнуло линию, GIC запомнил факт, дальше линия неважна. **Уровневое** (level-sensitive) — кнопка звонка, которую держат нажатой: пока уровень активен, прерывание считается запрошенным. У нас второй случай, и это записано и в железе, и в DT (`4 = IRQ_TYPE_LEVEL_HIGH`).

Следствие жёсткое. Обработчик, который просто вернулся, ничего не изменив, не решил проблему: линия всё ещё держится, GIC немедленно вызывает обработчик снова, тот снова возвращается — и получается **interrupt storm**, шторм прерываний. Внешне это выглядит не как ошибка SPI, а как «система стала неотзывчивой». Выйти из уровневого обработчика можно только двумя способами: либо **квитировать** причину (сбросить sticky-флаг, чтобы линия опустилась), либо **замаскировать** источник (запретить ему поднимать линию).

Квитировать у нас нельзя: единственный путь сбросить sticky `IRQ_STATUS` — запись в `ERROR_CLR`, а она вместе с флагами уничтожает содержимое обоих FIFO (это ограничение F-2, разбор в 25.2). Остаётся маскирование, и наш обработчик состоит ровно из этого:

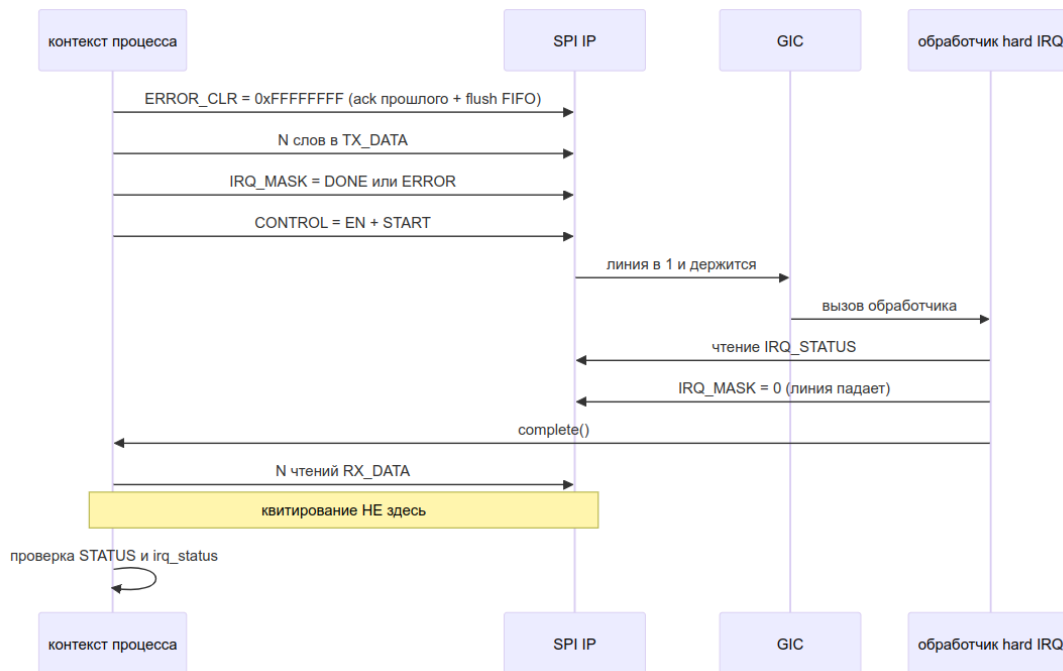
```
status = zfspi_read(zs, ZFSPI_REG_IRQ_STATUS) & ZFSPI_IRQ_ALL;
if (!status)
    return IRQ_NONE;          /* не наше прерывание */
zfspi_write(zs, ZFSPI_REG_IRQ_MASK, 0); /* заглушить линию */
zs->irq_status |= status;     /* запомнить причину */
```

```
complete(&zs->done);          /* разбудить процесс */
return IRQ_HANDLED;
```

Объяснить с 

Пять строк, но за каждой стоит свойство железа. Чтение `IRQ_STATUS` и возврат `IRQ_NONE` при нуле — правило вежливости для разделяемых линий и защита от ложного вызова. `IRQ_MASK = 0` — это не «выключить прерывания навсегда», а снять уровень: формула `irq_out` обнуляется из-за пустой маски, при этом sticky-флаги остаются стоять и данные не теряются. `complete()` будит поток, ждущий в `wait_for_completion_timeout()`. Вся медленная и разрушительная работа — вычитывание RX и квитирование — уезжает в контекст процесса.

Полная последовательность одного burst'a в драйвере выглядит так, и в ней есть один неочевидный ход:



Неочевидный ход — в первой строке. Квитирование прошлого burst'a сделано **первым действием следующего**: `ERROR_CLR = 0xFFFFFFFF` сбрасывает sticky-флаги и попутно чистит оба FIFO — а именно этого мы здесь и хотим, потому что собираемся заливать TX с нуля. Разрушительный побочный эффект превращён в полезный простым переносом момента вызова: пока следующий burst не начался, флаги стоят, но линия молчит.

Вторая деталь: в маску записывается только `DONE | ERROR`, но **никогда** `RX_VALID`. Прерывание на каждое принятое слово в железе есть, и соблазн «читать RX по мере поступления» велик, но квитировать его в середине burst'a означало бы сбросить FIFO и сломать передачу. Это снова F-2 — и снова видно, как одно свойство RTL вычёркивает целый способ написания драйвера.

Проверяется всё это одной командой из `../linux_bringup.md`:

```
grep spi /proc/interrupts;
grep spi /proc/interrupts
```

Объяснить с 

В покое счётчик обязан не измениться. Если он растёт — линия осталась активной, то есть либо маска не снята, либо нарушен порядок «сначала вычитать RX, потом ERROR_CLR ». А если под нагрузкой счётчик кратно больше числа транзакций — вы поймали шторм в мягкой форме, и это тоже дефект приёмки, а не «особенность».

Глава 25. Ограничения F-1...F-3 — железо диктует софт

Block Design мы собрали, адреса назначили, прерывание довели до обработчика. Казалось бы, дальше дело за программистом. Но прежде чем написана первая строка на C, нужно прочитать три страницы `../hw_sw_contract.md` — документа, который фиксирует аппаратную модель **до** софта. Его главный результат — три ограничения, пронумерованные F-1, F-2 и F-3; каждое выведено из чтения RTL, а не из документации, и каждое меняет проект драйвера.

Важно правильно к ним отнестись. Это не баги и не «недоделки, которые надо починить в Linux». Починить их в Linux нельзя — только в RTL, а RTL мы по условию задачи не переписываем. Это **входные данные для проектирования**, такие же обязательные, как напряжение питания. Разница между инженером и новичком лишь в том, что новичок узнаёт про F-2 через два дня отладки потерянных данных, а инженер — за двадцать минут чтения перед началом работы.

25.1. F-1. CS жёстко связан с burst'ом FIFO — нет способа удержать CS отдельно

Что в RTL. Сигнал выбора устройства CS активируется в состоянии `S_IDLE` в момент приёма команды START и снимается в состоянии `S_CS_HOLD`, когда TX FIFO опустел (`spi_engine.v:238-257`, `357-366`). Регистра или бита «активировать CS и держать его» не существует в принципе: CS — это функция состояния автомата, а не самостоятельный выход, которым можно управлять извне.

Почему так написано. Стоит проявить уважение к автору исходного Altera-проекта: для его задачи решение было правильным. Хостом ядра был конечный автомат в той же фабрике, который заливал в FIFO ровно столько слов, сколько нужно, и запускал burst. При такой модели «CS живёт, пока есть что передавать» — самое простое и безопасное поведение: невозможно забыть снять CS, невозможно оставить шину захваченной из-за зависшего софта, не нужен ни один дополнительный регистр. Ограничение появилось из другой модели использования, а не из небрежности.

Что это запрещает софту. Модель Linux устроена иначе. Единица работы там — `spi_message` из нескольких `spi_transfer`: например, «передай байт команды, потом прими четыре байта ответа», и всё это под одним удержанным CS, потому что для устройства это одна неделимая операция. Ядро управляет линией через callback `set_cs()`, который драйвер обязан реализовать. У нас реализовать его **нечем**: бита, поднимающего CS без запуска передачи, нет. Любая реализация была бы либо пустышкой, либо ложью — и то и другое хуже честного отказа.

К какой ошибке приводит нарушение. Самая неприятная — не отказ, а тихо неверная осциллограмма. Если притвориться, что `set_cs()` работает, и отдать в железо сообщение длиннее FIFO, драйвер отчитается об успехе, а на шине CS **моргнёт** в момент, когда FIFO опустеет между порциями. Slave воспримет это как конец операции и начало новой: ответ будет мусором, причём нестабильным, зависящим от планировщика.

Как это решено в драйвере. Двумя режимами, и выбор делает Device Tree. Основной путь — **GPIO CS**: драйвер объявляет `use_gpio_descriptors = true`, и если в DT задан `cs-gpios`, линию держит само ядро обычным выводом GPIO. Нативные CS при этом надо гарантированно заглушить, и делается это красиво: в `CS_SELECT` пишется значение `NUM_CS`, то есть **заведомо вне диапазона 0...NUM_CS-1**, после чего ни одна аппаратная линия CS не активируется, а транзакция всё равно выполняется. Длина сообщения тогда не ограничена ничем: между burst'ами

SCLK стоит на уровне CPOL, данные не идут, а CS держит GPIO — для устройства это неотличимо от одной длинной посылки. Второй путь — **нативный CS**, и тогда сообщение обязано целиком укладываться в FIFO. Проверка живёт в `prepare_message()`, потому что этот callback видит `spi_message` целиком: драйвер обходит все `spi_transfer`, суммирует слова и при превышении возвращает `-EINVAL` с диагностикой, прямо подсказывающей решение — «используйте `cs-gpios` для более длинных сообщений». Дополнительно объявлен `max_transfer_size`, чтобы верхние слои знали границу заранее.

Как это всплыло потом. Дважды, и оба раза на дисплее. В главе 34 части VIII — при построчной отрисовке кадра CS падал между строками, а для ST7789 подъём CS означает конец записи в память экрана: в GRAM попадала только первая полоса, остальное выглядело как призраки прошлого кадра. В главе 36 части IX — то же ограничение в максимально жёсткой форме: кадр $320 \times 172 \times 2 \approx 110$ КиБ против FIFO глубины 8 слов. Соблазн «подниму приоритет процесса, поставлю `SCHED_FIFO` и успею кормить FIFO» разбивается о то, что решение снять CS принимает автомат в фабрике по признаку «TX пуст». Настоящим ответом стали `STREAM_EN` (держат CS при кратко опустевшем FIFO до конца потока) и FIFO на 1024 слова — то есть изменение железа, а не софта.

Правило на будущее. Если в RTL нет бита управления CS — не обещайте в описании драйвера семантику CS, принятую в Linux. Проверяйте возможность удержания линии до того, как спроектируете модель обмена, и делайте невозможное явной ошибкой с подсказкой, а не молчаливо неверной осциллограммой.

25.2. F-2. Квитирование IRQ флашит оба FIFO

Что в RTL. Единственный способ сбросить sticky-биты `IRQ_STATUS` — запись в регистр `ERROR_CLR` (`spi_reg_if.v:244-252`). Но та же запись поднимает внутренний импульс `clr_errors`, а в верхнем модуле он соединён с сигналом очистки FIFO (`spi_master_top.v:73`):

```
assign fifo_flush = clr_errors | ctrl_soft_rst;
```

Объяснить с 

Одна строка на Verilog, и из неё вырастает половина архитектуры драйвера.

Почему так написано. Логика автора понятна: «сбросить ошибки» для его хоста означало «начать с чистого листа». Автомат-хост после ошибки всё равно не собирался использовать содержимое FIFO — он перезапускал обмен целиком. Совместить квитирование с очисткой было экономно: один строб делает всё нужное для восстановления. Проблема не в решении, а в том, что побочный эффект не задокументирован: карта регистров исходного проекта описывает `ERROR_CLR` как обычный W1C.

Что это запрещает софту. Три вещи сразу. Нельзя квитировать прерывание до того, как вычитан RX — данные будут уничтожены. Нельзя квитировать внутри обработчика `hard IRQ`, потому что там нет времени вычитывать FIFO по AXI, а без квитирования уровневая линия не опустится (24.4). И нельзя строить обработку «по слову»: источник `RX_VALID` срабатывает на каждое принятое слово, но квитирование его в середине `burst'a` сбросит оба FIFO и разрушит передачу. Целый привычный стиль написания драйверов — тот, по которому пишут UART, — здесь неприменим.

К какой ошибке приводит нарушение. К самой опасной из возможных: к тихой потере данных. Обработчик, написанный «как в учебнике» — сначала квитировать, потом разобраться, — отработает без единой жалобы, а пользовательский процесс получит нули или прошлое содержимое буфера. Хуже того, симптом будет непостоянным: если чтение успело произойти до квитирования, данные окажутся верными. Отдельная ловушка сверху: переполнение RX в этом IP

видно **только** через `IRQ_STATUS.ERROR`, потому что бит `STATUS[9]` мёртв (унаследованный дефект A-5, см. `../register_map.md`). То есть узнать о потере данных можно лишь из того регистра, который квитирование обнуляет.

Как это решено в драйвере. Порядок операций возведён в ранг контракта. Обработчик `hard IRQ` читает `IRQ_STATUS`, **глушит линию маскированием** (`IRQ_MASK = 0`), сохраняет причину в защищённое спинлоком поле и будит процесс. В контексте процесса драйвер читает `STATUS`, забирает сохранённый `irq_status`, вычитывает из `RX` ровно столько слов, сколько отправил (движок кладёт по одному принятому слову на каждое переданное), и только затем анализирует ошибки. Само квитирование, как мы видели в 24.4, перенесено в начало следующего `burst'a`, где очистка FIFO желательна. Ту же осторожность видно в процедуре сброса из `../hw_sw_contract.md`: сначала `IRQ_MASK = 0`, потом `SOFT_RST`, потом `ERROR_CLR = 0xFFFFFFFF`, и только затем восстановление конфигурации и включение. Порядок не косметический: `SOFT_RST` флашит FIFO, но sticky-ошибки намеренно **сохраняет**, поэтому два сброса не взаимозаменяемы.

Как это всплыло потом. В главе 39 части IX, в форме, которую заранее никто не предсказал. При потоковой передаче кадра на дисплей движок продолжает сэмплировать `MISO`, а у панели эта линия подтянута к единице — `RX FIFO` наполняется байтами `0xFF` и на кадре в десятки килобайт неизбежно переполняется. Дальше срабатывает рефлекс «есть ошибка — почистим через `ERROR_CLR`», и эта чистка **флашит TX посреди потока**, уничтожая ещё не отправленную часть кадра. Картинка рвётся, а причина выглядит как проблема дисплея. Настоящее решение — бит `TX_ONLY`, который вообще не пускает принятые слова в `RX FIFO`, то есть устраняет причину переполнения, а не последствия.

Правило на будущее. Читать документ аппаратного контракта нужно **до** написания обработчика, а не после первого странного поведения. «Квитирование с побочным эффектом» — отдельный класс ловушек, который стоит держать в чек-листе для любого незнакомого IP: выясните, что именно делает запись в регистр очистки, прежде чем поставить её в начало обработчика.

25.3. F-3. Нет прерывания по порогу TX FIFO

Что в RTL. Источников прерывания ровно три: `DONE`, `RX_VALID` и `ERROR` (`spi_defs.vh:59-61`). События «TX FIFO опустел» или «в TX осталось меньше N слов» — того, что в промышленных SPI-контроллерах называется `threshold interrupt`, — в этом IP нет ни в каком виде.

Почему так написано. Причина та же, что и у F-1: другой хост. Автомату в фабрике порог не нужен — он видит бит `TX_READY` в `STATUS` каждый такт бесплатно и подливает слово ровно тогда, когда есть место. Порог понадобился бы только медленному хосту, который не может позволить себе опрос; для железного это лишний компаратор на счётчике FIFO плюс регистр уровня. Никто не платит за то, что не нужно. Проблема возникает в момент, когда хостом становится Linux.

Что это запрещает софту. Дозаливку FIFO на ходу. Классический приём драйвера — «залил, что поместилось, по прерыванию долил остальное» — требует события, по которому доливать. События нет. Теоретически остаётся опрос `STATUS` в цикле прямо во время `burst'a`, и здесь стоит посчитать бюджет времени: это самое поучительное вычисление в главе. При максимальной для этого IP частоте 8.33 МГц (`CLK_DIV = 2`, ограничение из дефекта A-3) одно 8-битное слово уходит примерно за микросекунду. Восемь слов в FIFO версии v1 — это **восемь микросекунд** запаса, а задержка планировщика Linux, прерывание таймера или миграция потока между ядрами занимают больше. Окно принципиально не наше.

К какой ошибке приводит нарушение. К `underrun`: FIFO опустел раньше, чем подросла новая порция, и по F-1 движок немедленно снял `CS`. Сообщение разорвано, `slave` считает его законченным, данные испорчены. Код при этом выглядит абсолютно правильным, и именно

поэтому появляется искушение оптимизировать цикл записи вместо того, чтобы признать: контракт железа не позволяет. Есть и второй способ нарушить F-3 — спроектировать драйвер вокруг возможности, которой нет, и обнаружить это, когда API уже объявлено. Возможности драйвера — подмножество возможностей RTL, и это проверяемое утверждение, а не философия.

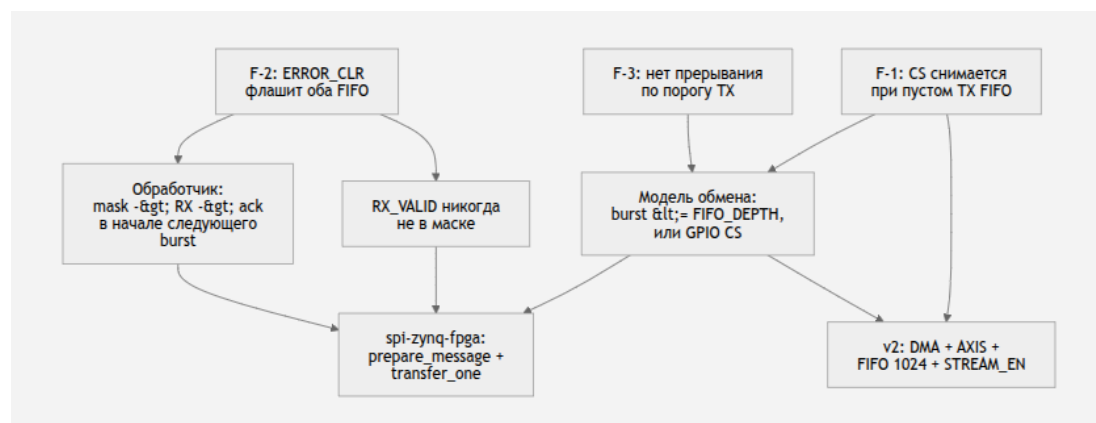
Как это решено в драйвере. Дозаливка на ходу не реализуется вообще. Модель обмена — «один burst = одна порция не больше FIFO_DEPTH слов»: драйвер в цикле нарезает передачу на порции по глубине FIFO, и каждая проходит полный цикл «очистить, залить, открыть маску, START, дождаться DONE, вычитать RX». Непрерывность CS между порциями обеспечивает не хитрое прерывание, а GPIO CS из F-1 — при нём нарезка не видна устройству. Для нативного CS нарезки не бывает по построению: `prepare_message()` уже отверг слишком длинное сообщение.

Как это всплыло потом. Самым интересным образом: F-3 в итоге закрыли не софтом и не прерыванием, а **другим трактом данных**. В главах 36–37 части IX роль «того, кто вовремя подливает в FIFO» передана железу: `axi_dma` читает буфер из DDR через HP0 и гонит его в ядро потоком AXI-Stream, а FIFO углублён до 1024 слов. Управление потоком делается не событиями, а **backpressure** — обратным давлением: приёмник говорит «стоп», когда занято, и передатчик ждёт, всё в пределах такта и без участия процессора. Отсутствующее прерывание по порогу заменил сигнал `almost_full`, поднимающийся при заполнении `DEPTH-2`; почему именно минус два и как наивная реакция на «сырой» признак `full` тихо теряла байты — история главы 41. Мораль общая: когда бюджет времени измеряется микросекундами, ответ лежит в аппаратном управлении потоком, а не в более быстром обработчике.

Правило на будущее. Не рекламируйте порог, которого нет. Перед выбором модели обмена посчитайте бюджет времени в микросекундах и сравните с реальной задержкой планировщика: если запас меньше десятков микросекунд, софтовое решение не годится в принципе, каким бы аккуратным ни был код.

25.4. Как три ограничения складываются в одну архитектуру

Поодиночке каждое ограничение выглядит мелкой неприятностью. Вместе они образуют жёсткую систему, из которой есть ровно один разумный выход — и наш драйвер в него и попал, причём попал бы гораздо позже и дороже без выписанного заранее контракта.



ID	Что запрещено софту	Обход в v1 (PIO)	Ответ в v2 (stream/DMA)
F-1	Удерживать CS между burst'ами	cs-gpios, иначе -EINVAL с подсказкой	STREAM_EN держит CS до EOT

F-2	Квитировать до чтения RX; IRQ на слово	Маска в hard IRQ, ack в начале следующего burst	TX_ONLY убирает причину overflow
F-3	Дозаливать FIFO на ходу	Нарезка на порции $\leq$ FIFO_DEPTH	DMA + almost_full, FIFO 1024

Читать таблицу следует не как «проблема и заплатка», а как эволюцию проекта. В версии v1 мы честно ограничили функциональность тем, что железо умеет гарантировать, и сделали недопустимое явной ошибкой. В v2 те же три ограничения были сняты не переписыванием драйвера, а изменением тракта: добавили путь $\text{DDR} \rightarrow \text{HP0} \rightarrow \text{DMA} \rightarrow \text{AXI-Stream} \rightarrow$ глубокое FIFO и два бита в управляющем регистре. Драйвер почти не изменился — изменился контракт, на который он опирается.

Отсюда главный вывод всей части VI. Когда что-то «не получается в софте», у задачи есть два уровня решения: переписать код или изменить условия, в которых он работает. Ограничения F-1... F-3 — это и есть условия, и они были известны **до** написания кода, потому что кто-то потратил время на чтение RTL и оформил результат документом. Это самая недооценённая работа в проекте и единственная, которая экономит недели.

Итог части VI. Block Design фиксирует три числа ($0x40000000$ / $0x40010000$ / $0x40400000$), которые не магические, а просто начало окна GP0 и шаг вверх от него; два прерывания, собранные в шину IRQ_F2P через xlconcat и превращающиеся в $\langle 0\ 29\ 4 \rangle$ и $\langle 0\ 30\ 4 \rangle$ в Device Tree; и один Tcl-скрипт, который всё это порождает и одновременно документирует. Но настоящий контракт с софтом — не адреса, а F-1...F-3: они решили, каким будет драйвер, ещё до первой строки на C, и объясняют половину ошибок, которые ждут нас на дисплее и на DMA. Дальше — Linux, Buildroot, Device Tree и сам драйвер: часть VII.

Продолжение читайте во второй части данной статьи. До встречи :)

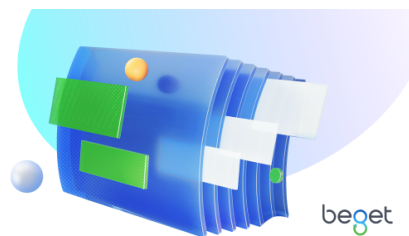
Размещайте облачную инфраструктуру и масштабируйте сервисы с надёжным облачным провайдером Beget.

Эксклюзивно для читателей Хабра мы даем бонус 10% при первом пополнении.

**Добавим 10% на баланс
к первому пополнению**

читателям нашего блога

Воспользоваться



Теги: fpga, xilinx zynq, spi controller, verilog, lcd, dma, buildroot, uboot, spi

Хэбы: Блог компании Beget, FPGA, Производство и разработка электроники, Электроника для

начинающих, DIY или Сделай сам

Если эта публикация вас вдохновила и вы хотите поддержать автора — не стесняйтесь нажать на кнопку

Задонатить



+24



24



4

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



64K+

Охват за 30 дней

Beget

[ВКонтакте](#) [Telegram](#) [Telegram](#)



16K+

Охват за 30 дней

207

Карма

Андрей [@andreyzaostrovnykh](#)

Инженер, тестировщик, радиоинженер

Подписаться



[Telegram](#) [Telegram](#) [Telegram](#)

 Комментарии 4

ЕЖЕДНЕВНЫЙ ХАБР | 15 СЕНТ 2026



 13K



Реальные истории IT-найма 2026

Привет, Хабр! (И тебе, IT-шник, который хочет узнать, что вообще происходит с наймом в 2026. Сейчас разберёмся, но легче тебе от этого не станет).
Две статьи с аналитикой позади ([раз](#), [два](#)). 106 тысяч вакансий разобрали, графики посмотрели, узнали, что...

Хабр проиграл войну с нейрослопом

Не трогая веса модели: как мы построили исследовательского агента Алисы AI и в разы сократили потребление GPU

Где HTTP/2 и HTTP/3 реально быстрее, а где не дают ничего

Ещё

0

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ



MainEI

21 час назад

Как советский компьютер «Сетунь» опередил время на 30 лет — и почему о нём забыли

Средний

8 мин

15K

Аналитика

+94

37

102



Drvmneka

22 часа назад

Зачем нужны векторные БД в эпоху LLM

12 мин

11K

+34

47

7

**interpres**

17 часов назад

Как всегда, (почти) во всём виноват Internet Explorer: древности и артефакты в HTML

Средний

14 мин

7.3K

Обзор

Перевод

+31

11

0

**MrSotnik**

23 часа назад

Реальные истории IT-найма 2026

18 мин

21K

+31

21

23

**alizar**

20 часов назад

Электричество из воздуха

Средний

6 мин

11K

Обзор

+25

9

9

**prohor33**

22 часа назад

Не трогая веса модели: как мы построили исследовательского агента Алисы AI и в разы сократили потребление GPU

10 мин

20K

+22

12

12

**beget_com**

22 часа назад

Ваш телефон здесь бесполезен. Большой обзор транкинговой связи. Часть 2

Средний

17 мин

11K

Обзор

+21

24

8

**sonikdixi**

12 часов назад

Я сисадмин, а не разработчик. Полгода я собирала симулятор ПК на телефон, чтобы научить людей отличать архив от папки

Простой

9 мин

7.8K

Кейс

Из песочницы

 +17 7 21**Dimalam**

15 часов назад

«Агат», «ДВК», «Микроша»: на чём советские гики учились программировать? Часть 1



7 мин



9.1K

Ретроспектива



+15



17



26

**ShevchikIgor**

22 часа назад

Как я встроил feedback loop в приложение и превратил пользовательские замечания в issues и доработки



Средний



14 мин



6.8K

Кейс



+15



13



0

Alice AI Search: как обучили ИИ для быстрых ответов в Поиске и выложили в опенсорс

Турбо

[Показать еще](#)

ИСТОРИИ



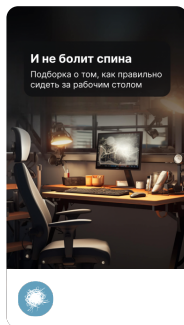
Лучшее из блогов компаний



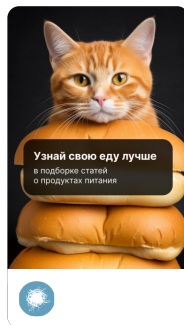
Проблемы? Внимательно вас слушаю



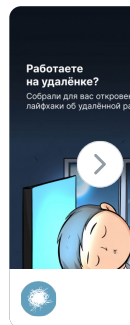
Жизнь без багов



Ваша спина очень важна для вас



Узнай свою еду лучше

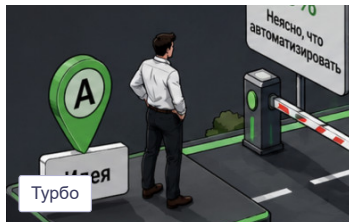


Мифы и леге удалёнки

МИНУТОЧКУ ВНИМАНИЯ



ИИ ускорил разработчиков, но не команды: что показал опрос



Работает нелегально? Выяснили, почему теневой ИИ живет



Я календарь переверну, там новые ивенты для айтишников



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr

На информационном ресурсе применяются [рекомендательные технологии](#)